

Coursera Course Transcripts

Programming Mobile Devices

Generated on June 24, 2026

Total lectures: 128

Modules / Weeks: 1

Programming Mobile Devices

Meet Your Instructor - Chandra Shekar RK Video• . Duration: 1 minute

[Notes]

Hello everyone, I am Chandra Shekar RK and I am working as an Associate Professor in Work Integrated Learning Program Division of CSS Group BITS Pilani. Before that, I have taught at academic institutions like PSIT, TJIT and BTLIT in Bangalore and gained industry experience with CALS in Bangalore and Firimax Maxis Communications in Kuala Lumpur, Malaysia. With 25 years of experience, I have had opportunity to work across both academia and IT industry. I completed my Bachelor of Engineering in Computer Science and Engineering from UVCE in Bangalore and later pursued my M.Tech in Computer Network Engineering from Department of PG Studies, VTU, Belgaum. Currently, I am a PhD Scholar at BITS Pilani KK Birla Goa campus where my research focuses on Edge and Cloud Computing and IoT. My research interests lie in Cloud Computing and IoT, which I actively explore. I also teach a range of subjects including Software Development for Portable Devices or Mobile Application Development, Cloud Computing or Cloud Native Development, Operating Systems and Cloud Infrastructure and System Software, Computer Networks, Security Protocols, Network Programming and Software Development Architecture. With my background in both industry and academia, I bring a mix of practical and theoretical knowledge in my teaching. Thank you.

Course Introductory Video Video• . Duration: 4 minutes

[Notes]

Hello everyone, I am Professor Chandra Shekar RK. I am here to introduce the course to you. This course offers a comprehensive introduction to software development for portable devices with primary focus on Android applications. It is designed to guide learners through the essential aspects of mobile application development from the basics of Android programming to advanced topics like performance optimization and security best practices. Module 1 begins with an overview of portable devices, their applications, and the hardware and software ecosystems that drive them. Learners will explore why mobile apps are essential in today's world, with a special focus on Android. In module 2, learners will dive into fundamentals of Android application development using Kotlin, the primary programming language for Android. The module also introduces Android Studio, covering how to create and run projects and provide hands-on experience in debugging and version control. Module 3 takes learners into Android UI design by focusing on layout creation using XML, key concepts such as linear and relative layouts are introduced, followed by core development topics like activities and intents, where learners will understand how to manage the activity life cycles, navigate between activities, and exchange data. Building on the previous modules, module 4 focuses on fragments, user interaction, and navigation. Learners will create and manage activities and explore communication between them using intents. The module also covers fragments and their user interaction elements such as clickable images, checkboxes, and menus. In module 5, learners will learn data storage techniques including using shared preferences, performing file operations, and managing databases with SQLite and content providers. Module 6 introduces

network-connected Android applications. Learners will explore networking basics, HTTP fundamentals, managing network states, and the use of retrofit for consuming RESTful APIs. This module also covers background processes and Android services, helping learners understand how to manage app functionality while ensuring robust network connectivity. Module 7 focuses on integrating location services and maps within Android apps. Learners will gain hands-on experience using Google Maps API and handling location data. In module 8, learners will build on their understanding of location services by diving deeper into practical applications of Google Maps API and the fused location provider client. This module emphasizes how to handle and display location data effectively with an app. Module 9 addresses performance optimization, background processing, and security best practices. Learners will study strategies for enhancing app performance and managing tasks in the background. The module concludes with security best practices, focusing on protecting sensitive data and implementing secure storage in Android applications. Finally, module 10 covers the complete process of publishing Android apps on the Google Play Store. Learners will gain insight into modern Android app development practices by leveraging Jetpack, preparing them to bring their applications to a professional level and to reach a global audience. This course is designed to provide learners with a good understanding and foundation in Android development, from basic concepts to advanced techniques, preparing them to be successful in mobile application development. This course is designed to provide learners with a good foundation in Android development, from basic concepts to advanced techniques, preparing them to be successful in mobile application development. I wish you all a very good luck in this course. Thank you.

Overview of Portable Devices Video• . Duration: 4 minutes

[Notes]

Hello everyone, welcome to this module on mobile app development. My name is Professor Chaturshekar RK and today we will be introducing you to the exciting world of software development for portable devices. We will be covering the basics of portable devices, their types, evolution, operating systems and their impact on society. By the end of this lesson, you will have a solid understanding of what portable devices are, the different types available, how they have evolved over time, the various operating systems used in these devices and how these devices have impacted our daily lives. Portable devices are small, lightweight computing gadgets that you can easily carry with you wherever you go. Some common examples include smartphones, tablets, e-readers and portable gaming consoles. These devices have significantly changed the way we communicate, work and entertain ourselves on the move. Portable devices come in various forms. The most common types include smartphones, which are essential for communication and productivity, tablets, which provide a large screen for entertainment and work, e-readers, which are designed for reading digital books and portable gaming consoles, which offer gaming experiences on the go. Portable devices have come a long way since their inception. In the early days, the devices like PDAs and the first mobile devices had basic functions like managing contacts and making calls. Today, modern portable devices are equipped with powerful processors, high-resolution displays and advanced connectivity options. As we look to the future, technologies such as foldable screens, augmented reality and 5G connectivity will further enhance the capabilities of portable devices. The operating systems used in portable devices have also evolved over time. Initially, we had simple open-source systems like early versions of Android, which were

known for their flexibility. Today, Apple's iOS is renowned for its smooth user experience and strong security features. Additionally, there are less common operating systems like Windows Mobile and specialized systems for e-readers such as Amazon's Kindle OS. Portable devices have had a profound impact on society. They have transformed the way we communicate, making it easier to stay in touch with others no matter where we are. These devices have also boosted productivity by allowing us to work from anywhere. In terms of entertainment, portable devices offer endless options for music, movies, games and more. Furthermore, they have revolutionized education by providing access to information and learning, resources anytime, anywhere. In summary, we have covered the definition and types of portable devices, their evolution, operating systems that power them and their impact on society. Portable devices are an integral part of our lives, influencing how we can communicate, work, learn and entertain ourselves. As technology continues to advance, these devices will only become more essential and capable. Thank you for your attention. I hope this lesson has provided you a clear understanding of portable devices and their significance in our modern world. Thank you.

Applications of Portable Devices Video• . Duration: 6 minutes

[Notes]

Hello everyone, welcome back to this module on mobile app development. In this session, we will explore the key applications of portable devices and how they have become an essential part of our everyday lives. By the end of this session, you will have a clear understanding of how these devices enhance our communication, productivity, entertainment, and education. Before we get started, let's take a look at what you will learn today. By the end of this video, you will understand the key features that make portable devices so popular, how these devices have transformed communication and social connectivity, the role of portable devices in boosting productivity and providing entertainment, the impact of portable devices on education and learning. These objectives will help you see the broad impact of portable devices having on various aspects of life and why they are so important in our modern world. Let's begin by discussing the key features of portable devices. What makes these devices so valuable to us? First and foremost is mobility. Portable devices like smartphones, tablets, and laptops are designed to be carried anywhere allowing us to stay connected and productive on the go. Another important feature is connectivity. Whether it is through Wi-Fi, mobile networks, or Bluetooth, these devices keep us connected to the internet enabling us to communicate, work, and access information from virtually anywhere. Finally, there is versatility. Portable devices are not limited to one function. A single device can be used for making calls, sending messages, browsing the internet, playing games, watching videos, and even learning. This combination of mobility, connectivity, and versatility is what makes portable devices an integral part of our daily lives. One of the most significant impacts of portable devices is in the realm of communication and social connectivity. Let's start with smartphones. These devices have revolutionized the way we stay in touch with others, allowing us to make calls, send texts, and connect on social media platforms from anywhere. Tablets also have a crucial role in communication, offering large screens for video calls, social media interactions, and online meetings. They are especially useful in situations where a smartphone screen might be too small to comfortably manage multiple tasks. Moreover, wearable devices like smartwatches are becoming increasingly popular for communication. These devices allow us to receive notifications, send quick

replies, and even make calls—all from our wrist. This convenience makes it easier than ever to stay connected without needing to reach for a phone or a tablet. Portable devices are not just about communication. They are also powerful tools for productivity and entertainment. Laptops and tablets have become essential for work and study, providing the flexibility to create documents, manage projects, and collaborate with others from anywhere. When it comes to entertainment, these devices shine as well. Whether it is streaming movies, playing games, or listening to music, portable devices offer endless options for entertainment, making them an essential part of our leisure time. A combination of productivity and entertainment capabilities in a single device means that we can seamlessly switch between work and play, making the most of our time when we are at home, in the office, or on the move. Portable devices have also had a profound impact on education and learning. E-learning platforms and educational apps on tablets and smartphones have made it possible for students to learn from anywhere, at any time. This flexibility is particularly valuable in today's world, where remote learning has become more common. Additionally, e-books and digital libraries provide access to a vast amount of educational content. Students can carry an entire library in their device, making it easier to study and access information whenever they need it. These resources have made learning more interactive and accessible, helping students of all ages to enhance their knowledge and skills. To summarize, portable devices are more than just gadgets. They are essential tools that have transformed our lives in many ways. We discussed their key features such as mobility, connectivity, and versatility, which make them indispensable. We also explored their role in enhancing communication, boosting productivity, providing entertainment, and revolutionizing education. Understanding these applications is crucial for anyone involved in mobile app development as it highlights the importance of creating apps that leverage these capabilities to improve users' lives. As we continue to advance in technology, the role of portable devices will only grow. Thank you for joining me in this session. I hope now you have a better understanding of the significant role portable devices play in our lives and how they serve as platforms for innovation in communication, productivity, entertainment, and education. Thank you.

Hardware for Portable Devices Video• . Duration: 8 minutes

[Notes]

Hello everyone, and welcome to this session. In this session, we are going to explore the fundamental hardware components of portable devices. Before we dive into the technical details, let's first look at the learning objectives for this session. By the end of this lesson, you should be able to understand the basic overview of hardware used in portable devices, identify different types of processors that are commonly used, explain the importance of memory and storage in portable devices, describe the different types of displays and touchscreens used in these devices. These objectives will help you grasp the essential hardware aspects that support the software and apps we use daily. Let's start with an overview of hardware in portable devices. Portable devices like smartphones and tablets, use specialized processors that are designed to be both powerful and energy efficient. The most common type of processors found in these devices are ARM-based processors. These processors are widely used because they provide a great balance between performance and battery life, making them ideal for mobile use. The hardware in these devices is specifically engineered to handle the demands of multitasking and high performance applications while consuming as little power as possible. This is why our smartphones can last for hours on a

single charge, even when running multiple apps at the same time. Now, let's delve deeper into the processors used in portable devices. The processor often referred to as the brain of the device is responsible for executing instructions and running the operating system and applications. There are two main types of processors used in portable devices. ARM processors. These are most common processors in smartphones and tablets. ARM processors are known for their risk, that is reduced instruction set computing architecture, which allows them to perform operations efficiently using less power. This makes them perfect for mobile devices where battery life is crucial. The next set of processors are x86 processors. These processors are typically found in high performance tablets and two-in-one devices. They are more powerful and offer compatibility with desktop applications making them suitable for tasks that require more computing power, such as video editing or running complex software. The key features of these processors include multi-core architecture, which enables better multitasking and improved performance and integrated graphics, which runs the visual experience by supporting high resolution displays and smooth graphics rendering. Now let us talk about memory and storage, two critical components that influence the performance of portable devices. Memory, that is RAM or random access memory, provides temporary storage for running applications. The amount of RAM in a device determines how many applications can run simultaneously and how smoothly they perform. In portable devices, RAM configurations can range from 2 GB in basic models to 12 GB or gigabyte in high end models. Storage is where all data apps and operating systems are saved. Most portable devices use flash memory for internal storage, which comes in various forms like eMMC, which is embedded multimedia card, or UFS which is universal flash storage. These types of storage offer faster data access, which enhances the overall performance of the device. Additionally, many devices offer expandable storage through microSD cards, allowing users to increase their storage capacity as needed. The key features of memory and storage include speed, which affects how quickly data can be accessed and processed, and capacity which determines how much data and how many apps can be stored on the device. Finally, let's discuss the display and touchscreen, which are most visible hardware components of any portable device. There are different types of displays used in portable devices. First one is a liquid crystal display or simply LCD. LCDs are widely used because they offer good brightness and color accuracy. They are often found in budget and mid-range devices. The next one is the OLED or organic light-emitting diode. OLED displays are known for their vibrant colors and deep blacks. They are more energy efficient than LCDs and are commonly used in high end smartphones and tablets. In addition to the display, the touchscreen is a crucial interface element that allows users to interact with device. There are two main types of touchscreens. Capacitive touchscreen, this is the most common used today. It responds to electrical properties of the human body, making it highly responsive and capable of multi-touch gestures. The next one is a resistive touch screen. This type is less common and requires pressure to register a touch. It is often used in older or more rugged devices. Key features of these displays and touchscreens include resolution, which determines the clarity and detail of the images. Durability, which ensures the screen can withstand daily use and accidental drops. To summarize, in this session, we explored the essential hardware components of portable devices. We discussed the different types of processors like ARM and x86, which provide the power and efficiency needed for mobile devices. We also covered the importance of memory and storage in ensuring smooth performance and ample space for apps and data. Finally, we learned about the various display and touchscreen technologies that enhance our interaction with these devices. Understanding these hardware elements is key to developing better mobile applications and optimizing them for the devices they run on. Thank you for your attention. I hope

this session has provided you with valuable insights into the hardware that powers our portable devices. As you continue your journey in mobile app development, remember that a deep understanding of the hardware will help you create more efficient and user friendly applications. Thank you.

Software for Portable Devices Video• . Duration: 6 minutes

[Notes]

Hello everyone, welcome to this module on mobile app development. In this session, we will be discussing the software side of portable devices. Software is the heart of any device. It controls the hardware, enables applications to run, and enhances the user experience. Before we dive in, let's look at what you will learn in this session. By the end of this session, you should be able to understand the different operating systems used in portable devices, learn the basics of application development for these devices, and also recognize how software enhances the overall user experience. The subjectives will help you grasp the essential role that software plays in making portable devices functional, versatile, and user-friendly. Let's begin by talking about operating systems, which are the backbone of any portable device. An operating system, or just OS, is what enables the device's hardware and software to work together smoothly. It manages the device's resources, runs applications, and provides a user interface. Without an OS, a device would just be a collection of hardware components with no way to interact with them. One of the most popular operating systems for portable devices is Android, developed by Google. Android is an open-source OS based on Linux kernel, which means it is highly customizable. This flexibility has made Android the preferred choice for many manufacturers, powering a wide range of devices, including smartphones, tablets, smartwatches, and even TVs. Android is also known for its large ecosystem of apps available on Google Play Store. This vast selection of apps enhances the functionality of Android devices, allowing users to tailor their experience according to their needs. Next, we have iOS, the operating system developed by Apple Inc. Unlike Android, iOS is a proprietary software, meaning it's only available on Apple devices like iPhone, iPad, iPod Touch. iOS is renowned for its smooth performance and strong security features, which are the major selling points for Apple products. One of the key strengths of iOS is its seamless integration with other Apple services and devices, creating a cohesive ecosystem that enhances the user experience. The Apple App Store, known for its strict quality control, ensures that users have access to high-quality apps that run smoothly on their devices. Now that we have covered the operating systems, let's talk about application development. Developing apps for portable devices requires a deep understanding of the operating system's capabilities and constraints. For Android, developers use programming languages like Java and Kotlin, while iOS development primarily happens using Swift and Objective-C. App development involves designing the user interface, writing code to handle different functionalities, and testing the app to ensure it works well on various devices. Developers must also consider the performance security and user experience while creating apps, as these factors significantly impact an app's success. Finally, let's discuss how software enhances the user experience on portable devices. User experience, or just UX, is all about how a person feels when interacting with a device or app. Good software design ensures that the device is easy to use, responsive, and meets the user's needs efficiently. For example, Android's customization options allow users to personalize their home screens, change themes, and use widgets, making the device

feel uniquely theirs. On the other hand, iOS focuses on simplicity and consistency, with a clean interface that is easy to navigate even for new users. In addition to the interface, features like app notifications, quick settings, and voice assistants contribute to a seamless user experience, making tasks easier and more intuitive. Ultimately, the goal of software in portable devices is to make technology more accessible and enjoyable for everyone. To summarize, software is the driving force behind portable devices, enabling them to function effectively and providing users with a rich and engaging experience. We discussed the key operating systems, that is Android and iOS, that power most portable devices today, looked at the basics of application development, and explored how software enhances user experience. As we move forward in our study of mobile app development, remember that a good understanding of both the hardware and software aspects of portable devices is essential. This knowledge will help you create apps that not only work well, but also provide users with an enjoyable experience. Thank you for your attention throughout this session. I hope now you have a clear understanding of software aspects of portable devices and how they contribute to the overall performance and user satisfaction. Thank you.

Components of Mobile Application Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. Let's begin by outlining our learning objectives for this session. By the end of this lesson, you should be able to Understand the basic components that make up a mobile application Identify the key user interface components and their roles Describe background processing components and their importance Explain the methods used for data storage and communication within mobile applications These objectives will guide us through the different aspects of mobile app development, helping you gain a strong foundational understanding on how app works under the hood. To start with, let's get an overview of components that make up a mobile application. A mobile app is not just a single entity. It is made up of several essential parts that work together to provide a seamless and interactive experience for the user. UI components ■■■ these are the elements that users interact with directly, like buttons, text fields, and images. Background processing components ■■■ these handle tasks that run in the background, such as fetching data or playing music without interrupting the user experience. Data storage and communication components ■■■ these manage how data is stored and retrieved and how the app communicates with external services. Each of these components play a vital role in making sure that app functions smoothly and efficiently. Now let's look into user interface components. These are the elements that directly interact with the user. UI components are what the user sees and touches, making them one of the most critical parts of any mobile app. Buttons ■■■ these are used to perform actions when the user taps on them. Next one, text field ■■■ allows user to input data like typing in their name or a search query. Images ■■■ these can be used to display visuals, such as a company logo or a photo gallery. The primary purpose of UI components is to enable users to interact with the app, input data, and receive information. A well-designed UI ensures that users can navigate the app easily and perform tasks without confusion. Now let's discuss about background processing components. These are the parts of the app that handle operations not directly visible to the user but are essential for app's functionality. Like fetching data. For example, an app might need to download information from internet in the background. Or it could be playing music. If you are using a music app, the background processing components

ensure that music keeps playing even if you navigate away from the app. Or performing calculations. Some apps require complex calculations to be done in the background, ensuring that the user interface remains responsive. The main goal of background processing components is to ensure that long-running tasks do not interfere with the user interface, allowing the app to run smoothly even when performing demanding operations. Next, we move on to data storage and communication components, which are crucial for storing data and enabling communication within the app and with external services. Data storage involves mechanisms to store and retrieve data. Examples include database, file storage, and shared preferences. For instance, a notes app would use data storage to save the notes you write so you can access them later. Coming to communication, this involves the components that enable data exchange, such as network requests, API calls, and broadcast receivers. For example, when you send a message through a chat app, communication components handle sending and receiving the data to and from the server. These components ensure that data is persisted across sessions and that app can interact with other apps or services on the internet. To summarize, a mobile application is made up of several key components that work together to provide a complete and seamless experience for the user. In this session, we discussed UI components. These are the elements allowing users to interact with the app. Background processing elements. These ensure that tasks running in the background do not affect the user experience. And data storage and communication components, which handle the data and communication with external services. Understanding these components is very important for developing effective and efficient mobile applications. Each component plays a vital role in the overall functionality of the user experience of the app. Thank you for your attention throughout this session. I hope now you have a better understanding of different components that make up a mobile application and how they interact to create apps we use every day. Thank you

Need for Mobile Apps Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will be discussing the importance of mobile apps in our daily lives and business. Before we dive into the details, let's outline our learning objectives for today. By the end of this lesson, you should be able to understand how mobile apps enhance our everyday lives, recognize the role of mobile apps in driving business success, discuss the growth and future trends in mobile app industry. These objectives will help you appreciate the widespread impact of mobile apps on both personal and professional levels and give you insights into the future direction of this rapidly evolving industry. Let's start by discussing how mobile apps have become integral to our daily lives. Mobile apps are everywhere, making our lives more convenient and efficient. Whether it's for communication, entertainment, finance, or health, there is an app for almost everything. For instance, messaging apps like WhatsApp have revolutionized how we communicate, allowing us to stay connected with friends and family anytime, anywhere. Similarly, apps like Amazon and Flipkart have made shopping easier, letting us buy anything we need with just a few taps on our phone. The wide range of apps available today caters to diverse needs. From fitness tracking to managing finances, these apps streamline our tasks and enhance our overall user experience. They save us time, provide us with instant access to information, and help us stay organized in our busy lives. Now let us look at how mobile apps are driving business success. In today's digital age, businesses rely heavily on

mobile apps to connect with their customers. Mobile apps offer personalized experiences, which help in increasing customer loyalty and satisfaction. For example, an app can send personalized notifications to customers based on their preferences, encouraging them to make a purchase or engage with the brand. This level of personalization builds a stronger relationship between the business and the customer. Additionally, mobile apps provide businesses with direct marketing channels. Companies can use apps to promote new products, offer discounts, and keep customers informed about the latest updates. This direct line of communication is highly effective in reaching a global audience efficiently, allowing businesses to expand their reach and grow their market share. Let's now talk about growth and future trends in the mobile app market. The mobile app industry has been rapidly growing, and this trend is expected to continue. According to market projections, the demand for mobile apps will increase significantly in coming years, with gaming, social media, and shopping apps leading the way. A key trend to watch out for is the rise of apps that utilize advanced technologies like Artificial Intelligence and Augmented Reality. These technologies are set to transform how we interact with the apps, making the user experience more immersive and personalized. Another important trend is the increasing importance of mobile apps in the business world. As more businesses recognize the benefits of having mobile presence, we will see a rise in the development of custom apps tailored to specific business needs. This will further drive innovation and growth in the mobile app industry. To summarize, mobile apps have become an essential part of both our personal lives and business world. We discussed how apps enhance our everyday lives by making tasks more convenient and efficient. We also explored how mobile apps drive business success by increasing customer engagement and providing direct marketing opportunities. Finally, we looked at the growth and future trends in the mobile app market, highlighting the potential for continued expansion and innovation in this field. Understanding the role and impact of mobile apps is crucial for anyone interested in mobile app development or business strategy. Thank you for your attention throughout this session. I hope now you have a better understanding of the significance of mobile apps in our daily lives and business.

Overview of Android Video• . Duration: 7 minutes

[Notes]

Hello everyone, welcome to this session on mobile app development. In this lesson, we will begin our journey into understanding Android, the world's most popular mobile operating system. In this session, we will explore what Android is. Before we start, let's go over the learning objectives for this session. By the end of this session, you should be able to explain what Android is and its significance in the mobile world, understand the different architecture layers of Android, and familiarize yourself with the development environment for Android app creation. These objectives will help you grasp the fundamental aspects of Android, setting a solid foundation for more advanced topics in mobile app development. So let's begin by understanding what Android is. Android is an open-source operating system developed by Google, primarily designed for touchscreen devices like smartphones and tablets. Launched in 2008, Android has quickly become the world's most popular mobile platform due to its flexibility and wide range of applications. Android is based on a modified Linux kernel, which means it inherits the stability and security features of Linux while being tailored for mobile devices. The vast majority of apps that you find on Google Play Store are designed to run on Android, making it a versatile and user-friendly platform. One of the key reasons

for Android's popularity is its extensive support for developers. Google provides a vast ecosystem of tools and libraries that make it easier to develop apps for Android catering to both beginners and experienced developers. Next, let's explore the architecture of Android. Understanding the architecture is very important because it helps you know how different components of the system work together to run apps smoothly. First one is the application layer. This is where the user-facing apps like email, SMS, and Maps reside. These are the apps you interact with directly on your device. Then we have the application framework. This layer provides APIs for managing user interface elements, resources, and more. It acts as a foundation for app development. Then we have the libraries. These are C, C++ libraries used by various components of the system. They provide essential services like data storage, graphics rendering, and web browsing. Then we have the most important ART which is Android Runtime. This is where core libraries and the Dalvik virtual machine operate, allowing apps to run efficiently on Android devices. Then we have the Linux kernel which is the base of the architecture. The Linux kernel provides a hardware abstraction layer, managing device drivers, and basic system functionalities. Each of these layers plays a vital role in ensuring that Android devices operate smoothly and efficiently, allowing for seamless app performance. Going deeper into the architecture, let's look at some core components that are integral to Android. First one is the activities. These are single screens with a user interface, such as a login screen or a main menu in an app. Then we have the services. These are background operations that do not require a user interface. For example, a service might play music in the background while you use other apps. Then we have broadcast receivers. These respond to system-wide broadcast announcements, such as when the battery is low or when the device is charging. Then we have the content providers. These manage access to structured data, such as contacts or media files, allowing apps to share data securely. Understanding these core components is essential for anyone looking to develop Android apps, as they form the building blocks of app functionality. Now let us talk about the development environment used for creating Android apps. The official IDE for Android is Android Studio, which is based on IntelliJ IDEA. Android Studio provides all the tools necessary for designing user interfaces, writing code, and testing applications. Android apps are primarily written in Java and Kotlin, with support for C++ through Android NDK, which is Native Development Kit. Java has been the traditional language for Android development, while Kotlin is now the preferred language due to its modern features and safety. The key tools of Android ecosystem are Android SDK. This includes tools and libraries needed for developing Android apps. Then we have emulators. These simulate Android devices on your computer, allowing you to test apps without needing a physical device. Then we have the ADB or Android Debug Bridge. This is a command-line tool that helps you communicate with Android devices for debugging and running apps. With these tools, developers can efficiently create, test, and debug Android applications, ensuring they work well on a variety of devices. To summarize, today we covered the fundamentals of Android, the most widely used mobile operating system. We discussed what Android is, its architecture layers, including key components like activities and services, and the development environment as well. Android understanding these basics is crucial as they form the foundation of Android app development. Thank you for your attention throughout this session. I hope now you have a solid understanding of what Android is and how it functions. Thank you.

Android Versions Video• . Duration: 8 minutes

Hello everyone, and welcome to this session on mobile app development. In this session, we are going to explore the different versions of Android. Before getting into the details, let's outline our learning objectives for this session. By the end of this session, you should be able to understand the key features of early Android versions, recognize the advancements made in mid-range Android versions, discuss the latest Android versions and their significant features. These objectives will help us track the development of Android over the years and understand the impact of each version on the mobile ecosystem. Let us begin with the early versions of Android, which laid the foundation for what we use today. The first version, Android 1.0, was released on September 23, 2008. It did not have a code name and included basic functions like Google Apps, which were quite revolutionary at the time. Soon after, Android 1.5, known as Cupcake, was released in April 2009. Cupcake introduced the on-screen keyboard, allowing users to type directly on the screen, which was a big step forward for physical keyboards. Following Cupcake, Android Donut version 1.6 came out in September 2009. Donut brought an improved user interface and universal search, making it easier for users to find information on their devices. The next major release was Android Eclair, versions 2.0 and 2.1, launched in October 2009. Eclair introduced Google Maps navigation and significantly improved the speed of the operating system. This version marked the beginning of Android's journey towards becoming a powerful tool for navigation and location-based services. The evolution continued with Android Froyo, version 2.2, released in May 2010. Froyo introduced USB tethering and Wi-Fi hotspot functionality, allowing users to share their mobile data connections with other devices. Performance optimizations also made Froyo faster and more responsive. In December 2010, Android Gingerbread was released. This version brought enhancements to the user interface, an input keyboard and support for NFC, which is near-field communication, which opened up new possibilities for mobile payments and data sharing. Moving forward to the mid-range versions of Android, Android Honeycomb, version 3.0, 3.1 and 3.2, was launched in February 2011. Honeycomb was specifically designed for tablets and introduced a holographic user interface, which was visually striking and optimized for larger screens. Next Android Ice Cream Sandwich was released in October 2011. This was version 4.0. Ice Cream Sandwich combined the features of Honeycomb with those of earlier versions, creating a unified experience for both smartphones and tablets. It introduced virtual buttons and improved the copy-paste functionality. Android Jelly Bean, versions 4.1, 4.2 and 4.3, launched in July 2012, focused on performance improvements with Project Butter, making the user interface smoother and more responsive. Jelly Bean also introduced expandable notifications and Google Now, which provided users with personalized information cards. In October 2013, Android KitKat, version 4.4, was released. KitKat improved the performance on low-end devices, introduced immersive mode for better full-screen experiences, and launched the OK Google voice search feature, which allowed users to perform tasks using voice commands. Now let us look at the latest versions of Android, starting with Android Lollipop, versions 5.0 and 5.1, which was released in November 2014. Lollipop introduced Material Design, a new visual language that emphasized clean, bold graphics and smooth animations. It also enhanced notifications, making them more interactive. Android Marshmallow, versions 6.0, followed in October 2015, bringing features like Doze mode for battery, management, and more detailed app permission system, giving users greater control over their privacy. Android Nougat, versions 7.0 and 7.1, released in August 2016, introduced multi-window support allowing users to run two apps side-by-side. This also improved Doze mode and added the Vulkan API for better graphics performance. Android Oreo, versions 8.0 and 8.1, launched in August 2017, brought features like

Picture-in-Picture mode, notification channels for better management of notifications, and Autofill's API to streamline the login process. The next version, Android Pie, version 9, released in August 2018, introduced a new navigation system based on gestures, digital well-being features, and enhanced AI for adaptive battery and brightness management. Android 10, released in September 2019, marked a significant change as it dropped the desert naming convention. It introduced a system-wide dark mode, new gesture navigation, improved privacy controls, and support for foldable displays. In September 2020, Android 11 was released, focusing on conversations in the notifications shade, built-in screen recording, and improved privacy features with one-time permissions. Android 12, released in October 2021, introduced a new material, U-Design, which allows for more personalized interfaces. It also added an enhanced privacy dashboard and improved app hibernation for faster performance. Finally, Android 13 was released in August 2022, bringing enhanced privacy controls, new notification permissions, and support for multilingual users with per-app language settings. Android 14 is expected in 2024, focusing on performance improvements, user interface refinements, and enhanced security features. To summarize, we have traced the evolution of Android from its early versions to the latest versions. Each version of Android has brought in new features and improvements, helping Android become the dominant mobile platform it is today. From basic functions in the early versions to advanced features like AI and enhanced privacy controls in the latest versions, Android has consistently evolved to meet the needs of users and developers alike. Thank you for your attention throughout this session. I hope you have now a better understanding of the evolution of Android. Thank you.

Challenges in Developing Android Apps Video• . Duration: 7 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss the challenges that developers face when creating Android apps. Before we get into the details, let's go over our learning objectives for this session. By the end of this session, you should be able to understand the challenge of device fragmentation in Android development, recognize the importance of security in Android apps, discuss the impact of continuous updates and compatibility issues, explore the complexities of user interface design for Android apps. These objectives will guide us through the critical areas where developers often face difficulties and provide insights into how to overcome these challenges. Let us start with one of the most significant challenges in Android development, that is, device fragmentation. Android operates across a vast array of devices, each with different screen size, resolutions and hardware configurations. This diversity is one of the Android's strengths, but it also creates a major challenge for developers. Ensuring that an app works well on all these different devices requires extensive testing and optimization. For example, an app that looks great on a high-end smartphone with a large screen might not function as well on the lower-end device with a smaller display. To address this challenge, developers use tools like Android Studio's emulator to simulate various devices and versions for testing. This allows them to see how their app will perform on different devices and make necessary adjustments to ensure compatibility. Despite these tools, the process, can still be time-consuming and requires careful attention to detail. Now let's discuss about security concerns which are critical in Android app development. Android apps often handle sensitive user data, such as personal information, financial details and location data. This makes robust security measures essential to protect users from

malicious attacks and data breaches. Developers face the challenge of implementing strong security features while ensuring their apps remain user-friendly. This includes using encryption to protect data, secure authentication mechanisms like biometrics or two-factor authentication, and regular security audits to identify and fix vulnerabilities. Moreover, developers must also ensure their apps comply with global regulations like GDPR that is General Data Protection Regulation and other privacy laws. This adds another layer of complexity to the development process, as failing to meet these standards can result in serious legal and financial consequences. Another major challenge in Android development is managing continuous updates and compatibility. Android frequently releases updates to its operating system which can introduce new features, security patches, and sometimes changes that affect how apps behave. Developers need to ensure their apps are compatible with latest Android versions while also maintaining support for older versions. This is particularly challenging because not all users update their devices to the latest OS version, leading to a wide range of Android versions in use at any given point of time. To overcome this, developers must regularly update their apps and test them against different versions of Android. This ongoing maintenance is essential to ensure that apps continue to function correctly across all supported devices and OS versions. Finally, let's discuss the challenges of designing a user interface for Android apps. Creating a user-friendly UI that works well across different screen sizes and resolutions is not easy. Android devices come in various shapes and sizes, from small smartphones to large tablets, and the UI needs to adapt seamlessly to all of them. Following Android's material design guidelines can help developers create a consistent and visually appealing UI. Material design provides a set of design principles that guide the look and feel of Android apps. Ensuring that they are intuitive and easy to use. In addition, developers must use responsive layout techniques to create adaptive UIs that can adjust to different screen sizes. Continuous usability testing and gathering user feedback are also crucial steps in refining the UI design and making sure it meets user needs. Before we conclude, it's important to understand why overcoming these challenges are so crucial. As of early 2024, Android holds a dominant 70.69% share of global smartphone market. This means that Android apps have a vast potential user base. And creating apps that work well across all these devices can lead to significant success. However, this also means that developers must be diligent in addressing the challenges we discussed today. Ensuring compatibility, security, and a good user experience is essential to capturing and retaining a large audience. To summarize, developing Android apps comes with several challenges including device fragmentation, security concerns, and continuous updates and compatibility issues, and complexities of UI design. Despite these challenges, the potential rewards are immense given Android's large global market share. Thank you for your attention throughout this session. I hope you have a better understanding of the challenges involved in developing Android apps. Thank you.

Need for Android Apps Video• . Duration: 7 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss the need for Android apps in today's world. Let's begin by outlining the learning objectives for this session. By the end of this session, you should be able to understand the growth and popularity of Android platform, recognize the market demand for Android apps, identify the benefits of developing Android apps. These objectives will help us understand the importance of Android in

both the consumer and business markets and why is it preferred platform for developers. So let us start by looking at the growth and popularity of Android. Android is the world's most popular mobile platform with over 1 billion active devices worldwide. It is a comprehensive open-source platform based on Linux and is championed by Google. The versatility and power of Android development framework have made it a go-to choice for developers, manufacturers alike. Android supports a mix of Java and XML for app development, making it accessible and flexible. Moreover, Android powers a wide variety of devices, including smartphones, tablets, smartwatches, and even TVs. This wide reach is one of the key reasons behind Android's dominance in the mobile market. Let's explore why Android has become so popular. One of the main reasons is its versatility and powerful development framework. Android's open-source nature allows developers to innovate and create apps with high flexibility in design and functionality. Developers have access to a wide range of tools and libraries, which makes the development process more efficient. Additionally, Android supports customizable user interfaces, allowing developers to create unique and engaging experiences for users. This flexibility in customization enhances the user experience and allows apps to stand out in the competitive market. Let's now look at the market demand for Android apps. The increasing number of smartphone and tablet users has led to a high demand for innovative and functional apps. Consumers are constantly looking for apps that can make their lives easier, whether it is entertainment, productivity, health, or other purposes. From a business perspective, the expanding market offers numerous opportunities for developers and businesses. Android apps can target various sectors, providing solutions that cater to specific needs. This opens up significant business opportunities, as apps can be developed for a wide range of industries, including entertainment, finance, healthcare, and more. Moreover, Android's extensive user base provides a global audience for app distribution. This means that an app developed by Android has the potential to reach users all over the world, making it a powerful platform for businesses to expand their reach and influence. Finally, let's discuss the benefits of developing Android apps. One of the key advantages is that Android is an open-source platform. This encourages innovation, as developers have the freedom to customize and optimize their apps according to their needs. The open-source nature of Android also means that developers have access to a wide range of tools and libraries, making the development process more efficient and cost-effective. Another significant benefit is the flexibility and customization options available on Android. Developers can create highly customized user interfaces that enhance the user experience. This flexibility allows for the development of apps that can meet the specific needs of different user groups. Moreover, Android offers seamless integration with Google services such as Google Maps, Google Drive, and more. This integration enhances the app's capabilities and provides users with a rich experience. For instance, integrating Google Maps into an app can provide users with accurate navigation and location-based services, making the app more valuable and useful. To further emphasize the importance of Android, let's look at its market share. As of early 2024, Android holds a 70.69% share of the global smartphone market, significantly surpassing iPhone's 28.58%. Android has maintained its dominance as the most used mobile operating system worldwide since it first captured over half of the market in 2015. In Q4 2023, Android smartphones accounted for 56% of all smartphone sales globally. This dominant market position means that Android apps have vast potential user base, making it an attractive platform for developers and businesses looking to reach a wide audience. To summarize, Android has established itself as a leading mobile platform with significant share of the global market. The demand for Android apps continues to grow, driven by increasing number of smartphone and tablet users. So developing apps for Android offers numerous benefits including flexibility, customization,

and seamless integration with Google services. Thank you for your attention throughout this session. I hope now you have a good understanding of the need for Android apps and why developing for this platform is so important. Thank you.

Major Components of Android Platform Video• . Duration: 6 minutes

[Notes]

Hello everyone. Welcome to this session on mobile app development. In this, we will discuss the major components of Android platform. Let's begin by going over the learning objectives for this session. By the end of this session, you should be able to identify the key components of Android platform, understand the role of data management components like content providers, explore the communication and interaction components like intents and widgets, learn how user notification keep users informed about important events. These objectives will guide us as we explore each component, helping you understand how they contribute to the overall functionality of an Android app. Let us start with the key components of Android platform. These components are the building blocks of any Android app. First one is the activities. An activity represents a single screen with a user interface. Think of it as a page or a screen in an app. Each activity manages user interactions such as tapping buttons, entering text or navigating between screens. For example, in a messaging app, the screen where you view your messages is an activity. And so is the screen where you compose a new message. Coming to the services, unlike activities, services run in the background without a user interface. They are used to perform long running operations such as playing music, downloading files or syncing data in the background. Services allow the app to continue functioning even when the user is not actively interacting with it. These components are essential for managing user interactions and ensuring that the app runs smoothly, both when it is visible to the user and when it is performing tasks in the background. Next let's discuss data management components in Android. First one among them is content provider. Content providers manage access to a structured set of data. They allow different applications to share data securely. For example, the contacts app on your phone uses a content provider to manage the contact information which can be accessed by other apps like messaging or email apps. The next one is broadcast receivers. Broadcast receivers are components that listen for and respond to system wide broadcast announcements. These announcements could be about events like phone charging, receiving a message or a change in network connectivity. When such an event occurs, the broadcast receiver triggers an appropriate action within the application. These components ensure that data is handled efficiently and the app responds to important events in real time. Communication between different components of an app is crucial for its seamless operation. In Android, this communication is facilitated by components like intents and widgets. Intents are used to facilitate communication between components such as activities and services. They allow data to be passed between these components and can also be used to initiate actions like opening a new screen or starting a service. For example, when you share a photo from your gallery to a messaging app, an intent is used to pass the photo from the gallery to the messaging app. Likewise, widgets are miniature application views that can be embedded in other applications typically on the home screen. They provide quick access to specific features of an app, such as weather update or a music player. Widgets enhance the user experience by providing essential information or controls directly on the home screen without needing to open the app. These components make sure that different parts of the app can

interact effectively and the users can access key features easily. Now let us talk about user notifications which are vital part of any Android app. Notifications inform the user about events, updates, or alerts in a non-intrusive manner. They usually appear in the status bar and provide quick access to the app. For example, when you receive a new message, a notification appears in the status bar and tapping on it takes you directly to the message. Notifications are essential for keeping users informed about important activities even when they are not actively using the app. They help engage users and encourage them to return to the app. To summarize, we have covered the major components of the Android platform including activities, services, content providers, broadcast receivers, intents, widgets, and user notifications. Each of these components play a very crucial role in the functionality and user experience of an Android app. Thank you for your attention throughout this session. I hope you now have a clear understanding of the key components of Android.

Device Compatibility in Android App Development Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile application development. In this session, we will discuss an important aspect of Android app development, which is device compatibility. Let's begin by outlining the learning objectives for this session. By the end of this session, you should be able to understand the challenges related to hardware compatibility in Android devices, recognize the issues with software version compatibility, learn about device fragmentation and how to handle it effectively. These objectives will guide us through the complexities of developing Android apps that work seamlessly across a wide range of devices. First let's talk about hardware compatibility. Android devices come with varying hardware capabilities such as different processors, amounts of RAM, and types of sensors. This variation can make it challenging to optimize app performance for all devices. For instance, an app that runs smoothly on a high-end device with a powerful processor and plenty of RAM might struggle on a lower-end device with limited resources. Additionally, different devices may have different hardware features like cameras, GPS, and sensors which can affect the app functions. To address these challenges, developers should use techniques like feature detection allows the app to check the presence of specific hardware features and adjust its functionality accordingly. Graceful degradation ensures that the app continues to work even if certain features are not available on a device. Moreover, it is important to optimize code for performance and memory usage, ensuring that app can run efficiently on devices with varying hardware specifications. Testing the app on a range of devices with different hardware capabilities is also very important to ensure compatibility and performance across the board. Next let's discuss software version compatibility. Different devices run on different versions of Android operating system from older versions to the latest versions. This diversity can create challenges when developing apps as you need to ensure that your app runs smoothly on both older and newer OS versions. One of the main challenges in handling deprecated APIs, which may no longer be supported in newer versions of Android. At the same time, developers need to introduce new features that take advantage of the latest OS capabilities. To overcome these challenges, developers can use backward compatibility libraries like Android X, which help ensure that apps can run on older versions of Android while still using the latest features. Conducting thorough testing on various OS versions is also essential to identify and resolve any compatibility issues. Additionally,

regularly updating the app to support new Android versions is important to maintain compatibility and take advantage of new features and security updates. Now let's talk about device fragmentation, a major challenge in Android app development. Device fragmentation refers to the wide variety of Android devices with different screen sizes, resolutions and hardware specifications. This diversity can make it difficult to ensure that an app works well on all devices. One of the key challenges of device fragmentation is ensuring compatibility across numerous devices. Developers need to test their apps on multiple devices to prevent bugs and performance issues that might arise due to differences in hardware or screen configurations. To handle device fragmentation effectively, developers can use Android Studio's emulator to simulate various devices and test how the app performs on different screen sizes and hardware configurations. Implementing responsive layouts and design patterns is also crucial to ensure that the app adapts the different screen sizes and orientation. By taking these steps, developers can create apps that provide a consistent user experience across a wide range of devices. To summarize, developing Android apps that are compatible with a wide range of devices involves several challenges, including hardware compatibility, software version compatibility and device fragmentation. However, by using techniques like feature detection, backward compatibility libraries and responsive design, developers can overcome these challenges and create apps that work smoothly across all devices. Understanding these challenges and the solutions available is essential for anyone involved in Android app development. Thank you for your attention. I hope now you have a better understanding of the importance of device compatibility in Android app. Thank you.

Recording of Programming Mobile Devices: Week 1 - Live Session on 26-06-06 Video• . Duration: 1 hour 7 minutes 1h 7m

[Notes]

So since the course has just started, I think it will take some time for you when the core topics will come up. And then definitely if you have questions around core topics, then we will definitely cover them in this Saturday session. But today it's just an introduction. So feel free to ask anything if you want to know about Android, if you want to know any about what industries you're expecting, anything around futuristic approach, anything about what you need to know and all. Anything concerning placements, what we will be asked and all those, maybe I can help you because I was part of the placement panel of Microsoft and we're visiting different campuses as well as off-campus placements. So that also we can discuss. So I'm just keeping it on a lighter note since it's an introduction. Maybe from next Saturday, we can take some topics or if you have any doubts because the modules will progress, we can utilize those to clear those doubts. So open to all, the forum is open, you can take this opportunity. And ask whatever you feel like. And definitely you should ask questions because these are very dedicated sessions. So you should utilize it to the fullest because you're doing it for yourself. So don't be hesitant in any way. See, I'm not sharing my video because right now the weather here is not good. It's raining a lot and to preserve the internet bandwidth, I haven't actually kept it off. But definitely we'll do the video session from next time when the weather is better. Any question, anyone? Anything you can ask, it's an open floor. Any question, anything if I'm in the mind related to Android, industry scope, whatever you want to connect. And remember, no question is a silly question. A question is always a question. Don't think that if you'll ask, I will not going to feel anything. You ask, I'll answer. If I know the answer, if I also don't know

the answer, then it's fine. How to join Microsoft without DSA? What's your ESF experience? That can help me answer. If you're a student and DSA is a must. And I don't know why people are generally very afraid of DSA. It is the easiest part of the interview because the answer is fixed. The question is also fixed. Right. So DSA is the easiest part of the interview that I can tell you. And if you're a college student, definitely you're going to have around four rounds, what we generally take. Out of these four rounds, two rounds will be definitely your DSA. And then one round can be more around system designing. And then one round can be about non-technical or can be a managerial round. So DSA will be there as a student that you have to anyways work on it. That's for sure. And don't be resistant to DSA in any way. It's a very simple thing and it's a very enjoyable aspect. So these interviews, I've seen students doing very well in DSA interviews. So that is a very easy part. Do we need background in Java to stand Portland? Yes, some background is needed. While you're all asking in chat, you can directly ask on your voice also. That's absolutely fine. Problem in chat is I can miss, right? If I'm saying something and if you're asking and if I miss, then that's an issue. But if you ask directly, then that will be better. Anything, any other question? For Android, you need to have Java basics because Android is nothing but you can think it of as a wrapper that is built on top of Java. If you're not very well aware of Java, if you're not very well aware of OOPS concepts, you may find little difficulty in understanding Android. But I won't say that you cannot understand it all. It's not that it all depends on how much effort you're putting in. But yes, Java knowledge is needed for Android. Any other question? Any, anything you want to, anything regarding placement, markets, Android, anything? See, Android is a very, very important field right now. Earlier, the apps were not much into the market. But suddenly, you know, everyone is having an app. Everybody's moving towards an app-based development. Rather, web-based developments are now what I was accusing. But app-based development is very much on cards. Each and every company is going for an app-based development. And there are many companies that they don't have a web-based product. They only have app-based products. For example, Uber, Uber, Mintra, you are already shopping in there, right? Mintra doesn't have a website. It's all app-based. Developer or an iOS developer. And Android also, with the passage of time, it is also, you know, enhancing itself a lot. The Android that you see 10 years ago, when I started working with Android, that was around 2012. So Android at that time, and Android at today, 2026, there's a huge difference, actually. So Android also has grown a lot as a platform, as an operating system. It has, Android is, by the way, a mobile operating system. So this is what is the very basic definition of Android. So it's not a programming language or something. Don't get confused with that. You have a Windows that is a desktop or a laptop operating system. Similarly, Android is nothing but a mobile operating system. So then an Android developer will be developing apps or will be developing the framework level things for this operating system that can run on the app. So this is a very basic definition of an Android. How do I get internship? Like what main topic should I focus on? Should I focus on AI, ML, mobile dev, full stack, developer? See, this totally depends on the position for which you are applying for the internship. If the internship requirement says that they're looking for a mobile development, then, of course, they're looking for your Android skills. If the internship says that they are completely focused on AI, ML, and they want you to know Python, they want you to know RG, they want you to know how to deploy a model, want you to know how you can use cloud and all, then you have to think on that direction. Or if they are completely saying that they need a full stack, so you have to think on that. So it's whatever position you are applying for, what is the requirement of that position? Based on that, only you can take a call. I cannot say that, OK, you learn Android and you can get an internship anywhere. It's not possible. What is their requirement? What specific they're looking for?

So it all depends on that. AI, ML also depends. See, AI, ML, you can learn only when you get some work around it, right? More practical hands-on, you can learn only when you get some work that requires AI, ML. Critical knowledges are everywhere. Internet is flooded with critical knowledge. But actual learning will have only when you get a live hands-on on AI and ML. Like if you yourself go and write Python, if you yourself go and deploy the models, if you yourself go and write RG implementation, if you yourself go and, you know, something called Lankin and all if you implement, then only you will know. Theoretically, you can find resources, but actual knowledge is when you actually do it. There are good courses on Udemy and all for AI development you can go to. But again, unless and until you do something on your own in AI, ML, it is difficult to get the knowledge. If you're getting internships in AI, ML, well and good. If you're getting college projects in AI, ML, well and good. But actual knowledge is completely enhanced. Same for Android development also. Whatever critical you learned or something, unless and until you do the practical, unless and until you practically develop an app that will be used by the user, you will not be able to understand or go deep into the Android. So that's the key thing. Any other question? See, why Android matters, first of all, why we are learning Android, there are actually 3 plus billion devices that are currently in the market. So imagine billions of apps are there, you will be going to Play Store and you will definitely be seeing yourself. This is a very diverse ecosystem. There are a lot of enterprises and a lot of startup demand. So that is why Android matters a lot. If I come to an industry expectation, what an industry expects from you as an Android developer, expectations are very clear. Now with AI, there have been a complete disruption in market, but the expectation stands very clear. You should be a problem solver. You should be able to write clean codes or get the clean codes written from the AI. You should have a sense of ownership that, OK, I will be handling this area. If I mean, I'll be handling, I'm completely responsible for this development. You should have a good communication. You should be able to communicate with different stakeholders and whatever you're working in. And you should be having a learning mindset that is very important. You should have that eagerness to learn, because in software industry, things are changing very fast. Whatever you're learning now, after six months, it's not guaranteed that it will not become obsolete. It can become obsolete, right? So that you need to have. You should be a very quick learner. That we definitely see in our organization, that how he's learning or how someone is performing. It's not only about writing code, writing code is now at the very bottom level of the pyramid. Right. There are a lot of things there. It's not only about writing code. If you're writing code, I'm not interested in, OK, I'm giving the input, your code is giving the output. Yeah, then you are great. No, if you are writing code, how you are writing it. What is the cleanliness coefficient in your code, how you're communicating it, right? How you're talking to different stakeholders, what is the design you're adopting? There are a lot of things out there. But that is why I'm saying writing code, just completing a code is the very bottom of the pyramid. There are a lot of things on top. Communication is very important because if you're working in the industry, you will have to interact with your team. You have to interact with your team members for any work. You have to co-work with them. Some part they will develop, some part you will develop. How will you communicate it? OK, how the two models will interact. You have to communicate with product managers. You have to communicate with leadership that, OK, this is what you are doing, what timelines you are proposing and all those things. So that all requires a very holistic picture all around. Can you suggest some good projects to gain experience? Of course, we are doing a BitTest learning course. They will definitely give you projects to gain experience. So that will be there. You should try for internships. If you get an internship, that is a very good way to get experience. Else you can go to Udemy. A lot of courses are there which you can go. But as I

said, that is courses we will be doing. Actual practical hands-on you will get only from either BitPlan is give you those projects or you do some internships where you get these projects to work on. That will actually give you a proper hands-on. OK, in terms of Android architecture, there are four very important architecture that Android has. First is app. Second is framework. Third is Android runtime. And the last one is Linux kernel. So Android will be learning on a Linux kernel. Above that, there will be an Android runtime environment that will be running. Above that, there will be a framework and above that will be the app. So whatever app you implement, that will be using a framework that Android has provided. That framework will be based on an Android runtime and that entire Android runtime or the Android JDK will be running on a Linux kernel. So this is the overall architecture of your Android thing. In terms of programming fundamentals, yes, Java and Portland are the very best languages currently for Android development. Maybe six months down the line, some new knowledge, new language can come up. OOP principles are very important because Java is an object-oriented programming language. Kotlin is also the same. And so you have to be aware of OOP fundamentals whenever you are programming in Android. You have to be very well aware of collections, concurrency, control, exception handling. So all those that are part of Java will be part of your Android programming. Question can come up that, OK, since it is based on Java, why we are moving into Kotlin, right? Why Kotlin came into, I don't know. Most of you may be knowing Kotlin, but to them, but to those who for whom Kotlin is new, Kotlin actually reduces your coding time very much. How it reduces, you will come to know when you are actually coding Kotlin. Also, null pointer exceptions and all that Kotlin provides a very safer way to handle those things. Also, Kotlin comes up with a lot of inbuilt functionality like coroutine, concise syntax, it will come to know when you will do Kotlin. All this makes Android development very safe, very fast, and very concise. That's what the beauty Kotlin brings it. And of course, it's a Google-recommended language. Kotlin is also developed by Google, so it recommends that you use it for mobile development. More importantly, it's a lightweight programming language compared to Java, so that is why it's preferred for the app development. We were said that we would get placement support at the end of third year. What's the stats on the CA? This I am not aware regarding the placement support. That you need to talk to your core or the primary mentor. I am just here to help you clear your doubts, give you industrial feedback, your assignments and all, but actual scenario in the college and how it is going, I think you should talk to your coordinator. They are the best person here to help you out. I myself am not aware much on these grounds. Topic seems to be not communicated. I think there is proper details of the topics that will be conducted each week, different modules. I think that is already communicated to you, right, what you will be studying, and when there will be exams, there will be some quizzes in between. All those, I think, are very properly in place. You already registered to the website. I think they can go and see all the list of topics, what will be covered in first week, what will be covered in second week, third week, and you will also get access to the recordings and all. So I think that should be fine. That's there. But still, if you have any question or doubt, you can reach out to your course coordinator. They are the best person to help you. See, there are certain questions I see I'm missing. That's why I'm preferring you can ask, you can clearly speak out. The question is, what's the best framework to work with? See, there's nothing best. Everybody is using the framework that is suiting their use case. But right now, if you want me to answer a single-based framework, if your question is more around the mobile context, then it's something not straightforward answer. Many companies are preferring iOS frameworks also. Many are on Android, Java. In Android also, many of them are way on a very newer version of Android. Some of them are on an older version of Android. Some have moved to Kotlin. Some are yet on Java. So it all depends

on company to company. So there's no fixed answer to these questions. Many companies trying to implement local AI model in their app. See, you cannot implement an AI model in app, right? It will be implemented in the server only. Your app will just interact with it. So that is called SML, small language, sorry, SLM, small language models. So ChargeGPT and all that, I think they are a large language model. They have a huge number of data that they do computation on billions of records that may not be relevant to you also, right? But if you want it, okay, no, do computation only on a certain fixed set of records, only these information I want the model to compute and give me the answer, then there's a concept of small language models. These are small models that work on a limited set of data. And whatever questions you ask them, they will answer based on that limited set of data only. So the processing time is also less, maintenance is also less. So that is called SLM. So now most companies are developing it based on their use cases. For example, healthcare companies will be more specific to healthcare data, right? They don't want any other data, for example, movie-related data, sports-related data. They are not interested. They're only interested in health-related data. So they can implement a model that will only work on the health-related aspects or can only answer questions related to health-related data. That is called SLM, small language models. Yes, these companies are going on because training these models are easy. They are less computation expensive. They will eat less of your computation power. This is the biggest problem. So that is why companies are moving to it. In terms of Android UI development, there are certain components that you need to know, like XML layout, Jetpack Compose, Material Design, Responsive UI. These are certain things you have to understand and know whenever it comes to adding UI. So Android has two parts, backend and UI, right? So basically what you look and feel in the app, the buttons and all, all those are part of UI. So for UI, there are certain kits that Android provides, like XML layout, Jetpack Compose, Material Design, that we use to generate those UI. There are two important things that will be part of your Android application. One will be the activity and another will be the fragment. And so you will come to know as the course progresses what is an activity, what is a fragment, or you can move from one activity to another, one fragment to another, what they basically show, when they will be used. So these are a certain part of the app lifecycle. Since you are working with mobile data, you will definitely show some data to the users in your app. So then you have to do data management also. So Android provides a lot of things, like shared preferences, room database, data store, and other offline concepts and all that Android provides, where you can save, store data, and then show those data to the user. Since your device will be interacting with server, so networking will also be part of Android. So Android will, your app will talk to different services that are hosted on different IP addresses. So you need to know REST APIs, you need to be aware of retrofit, JSON parsing, error handling, security, all these things also you have to understand as part of your Android app development. Yeah, that is what is the brief. There are certain architectural patterns, like MVC, MVP, MVVM, clean architecture, that you will know as you start developing app that what are these architectures. And actually, as an Android developer, when you sit for your placement, the focus will be here only. The different architecture patterns like MVC, MVP, what are these, how you implement, how you use an activity, how you can interact with different services in an optimized way, how you're handling concurrency, how you compute too many data and all those. This will be basically what, as an interview will be judged around regarding Android. Then testing is also a very important thing. You will be writing unit testing, UI testing, all these things. And of course, performance and security, like memory leaks, encryption, all those things as part of Android development you have to understand. Common interview topics will be mostly, since you all are college grads, so of course, if you want me to rank them, what will be your

interviews around, it will be the first thing will be definitely DSA. And if you're going for an Android developer position, then they can ask you Android internals, like whatever we discussed, all the frameworks and all those. Kotlin-related thing, system design basics is something that can be asked to you at this level. It will be basic only, not advanced because you are a fresher. Advanced level system design will come into picture once you get around five years of experience and after that, the system designing will broaden. Yeah, so that is all I wanted to convey, that this is all what Android is all about. And this is where the course should be focused and this is where you should also be focused on learning these things. Now, open to you, any question? Yeah. Anything you want to ask again? See, this session is your session, this is not my session because I'm not here to take any theoretical thing or take any, what I say, any topic and teach you. It's not meant where I'll be teaching you. Here, the whole idea is you ask me some doubts, I'll try my best to clear it or if you want any questions, you can ask. Startups, if you want to join, yes. Android is a very good thing, they will definitely give you an edge. If you want to work multinationals, that depends on the job requirements, how they are presenting it, what they're needing it. So, that's totally on their requirement, there's no rule. Placement-wise, I'm reiterating, DSA will be the core and if they are coming to recruit you for an Android development, definitely ask for Android internals. So DSA, Android internals, some problem-solving skills and some basics of system learning is what you can expect in any company, not specific to Microsoft, any company. Any questions? Any question, anyone? I hope you have started your modules. I think two modules have planned to be covered this week and if you're not started, then recordings are also shared. So, please go over them, take it seriously, understand all the concepts, do the hands-on, get the knowledge because now, interviews are very strict. It's not the same that used to happen five years ago when even if you give some close answers, you used to get selected. Right now, the interviews are very much focused. They're quite aware what they want from you, what is the answer expected. And the main part is you're competing with your batchmates. So, if the other batchmates are able to give the answers, the expectation is very clear, you should also be able to give the answers. The market has also changed, people are also learning a lot. So, yeah, you should be very well aware of the fundamentals. That's the way it's important. Companies still focus on fundamentals. Make sure your fundamentals are clear. If your fundamentals are clear, whatever question they ask, you'll be able to think, you'll be able to connect the dots and you'll be able to reach the solution. If your fundamentals are not clear, you will not be able to reach the solution, you will not be able to connect the dots. The interview, what we do in Microsoft is, if we give you a question to answer, we don't expect you to give a direct answer and a very straight answer. Right, we see how you are approaching the problem. What is your thought process? What you are thinking? What you are communicating with the interviewer? What questions you are asking with the interviewer? Are you asking the right set of information that is needed? And the interviewer is giving you hints where you are able to capitalize on those hints and able to reach the solutions or not. So these are all what we see. And once we see that, yes, you are asking the right set of questions, you are moving in the right direction in that problem. If I'm giving some hints, you are able to take those hints and make a progress. And finally, you are able to reach the solution. This is very much appreciated. So very holistically, the interviews are happening right now. So every aspect of you is what they're looking into. If it's straightforward to jump to the answer is something is not expected. So don't think that whenever they ask a question, I have to be very correct and straight. I have to jump to the answer and tell them the correct answer in one go. No, no, that is not the expectation. You can take your time, you reach it slowly. In the first hand, you can give a solution that is not proper. The interviewer may then ask you that, okay,

optimize your solution. Then you can optimize the solution. So it's a very iterative process unless you reach a perfect answer. This is how it should go about. And this is what is expected in an interviewer. So they will see your communicative skills, how you're communicating, because you have to tell the interviewer also, right, that this is what you're thinking, this is how you're planning to solve it. So as per you, this is what should be the correct way. As per you, this is not the correct way. So all these, you have to communicate to the interviewer. So the interviewer is able to understand your thought process. And then judge, okay, if you're going on the right track or not. Even if you're not going on the right track, the interviewer, in Microsoft, we generally give hints. If you're not going in the right direction, if a problem is given to you, and we see that you are stuck or you're not able to proceed, so we give you hints, that, okay, try this way, or why not think in this language? And then we see if you're able to still make progress or not after hints. So that is also being judged. So don't think that you need to directly go to the answer and it has to be a very perfect answer, no. Go slowly, go with a steady pace, give your thought process, help the interviewer understand what you are thinking. And that you can only do if your fundamentals are strong. If your fundamentals are weak, then even if the interviewer gives you 100 hints, you'll not be able to make progress. So that is the case. In terms of Android development, yes, they can ask you some fundamental questions. Definitely they can give you fundamental questions also. They can give you some scenario-based thing that, okay, this is what you want to implement in the app, how will you go about implementing it, what architectural pattern you will choose, what will your component, how will you develop the UI, how will you talk to the backend, how will you store the data, all those things they will ask. In some companies, they will ask you to implement it on some ID or something, or whether they would want you to actually go and do the implementation and show them a running app. So that also happens in a lot of interviews. And they will give you all the setups and all, and you have to actually go and code and generate the app and then see the app is running properly. So that is also there. You can do it only if your fundamentals are clear. So that's why I'm stressing more on fundamentals. And of course, you will be given ample of hints and all so that you can make progress. So this is all from my side for today, just an introduction. I'm still seeing any question anyone if you want to ask. Any question? Since there's no question, I think it's okay for us as it's our first introduction. Maybe as the week progresses, as your module progresses, when we meet next Saturday, so definitely if you have any questions by that time or if you want something to be covered, we'll try covering them. Also, I'll see if I can come across some basics of Android that I can help you all understand. So that will take up in the next session. But please, these sessions are planned to help you clear doubts. Utilize it to the fullest. Not asking doubts, even if you're having is just a waste of time that you're doing and waste of an opportunity. So this is an opportunity. Take it as an opportunity rather than a lecture session. It's not a lecture session. It's just an opportunity to you know, get your things clear, sorted out. So that's the thing. Again, I'll ask anything anyone wants to ask. Sir, what do you think about hybrid cross-platform frameworks? The new terms that are coming in, because basically, this was utilization, two developers working on two things for two different platforms, rather than the same developer working on a thing that can come to two different platforms. So that gaining traction, having fun, but still they're not on a very successful model. Because Kotlin, for example, if you take Kotlin as an example, Kotlin can run on iOS also, right? They're coming with a, they have did some enhancement in Kotlin, where if you do a Kotlin port, it can run on Android device as well as it can run on iOS device. But still, I have not seen much traction around it. But yes, it is gaining hybrid platforms. And the whole idea is that the companies want to reduce the resources. So that's the whole idea behind the hybrid cross-platform framework.

And the whole idea is that the companies want to reduce the resources. So two developers working on two platforms, rather than make it a single developer working on two platforms. That is why these are coming up. See, maybe six months down the line, we will be able to get a more clearer picture on these hybrid platforms. Okay, thank you sir. Recently, I built an app in the React Native. It's basically a notification reminder app. Where I use the, yeah, where I use the firebase, means notification things. Right now, I'm working on a gallery app, making it easy to react. But I'm facing lots of problems in making the gallery app, facing scratch problem. Some latency issue, how to handle the threads. Actually, React Native is the only, it's a JS, so it's a single thread. So it's facing some problems. See, first of all, whenever you want to implement something, or whenever you want to develop something, first you need to be very clear on what framework you want to develop it on. What is the requirement of your app that you are doing? For example, if you are developing a gallery application, how your application will be doing? Will it be needing multi-threading? It will not be needing multi-threading. So all you should first think, and then make a conscious call at, okay, what framework or what technology you will be using. For example, for gallery, you're using React Native, right? Which, if you ask me, I say it may not be needed. You can create gallery very easily from just Kotlin and Java. And in Kotlin, if you use, you can very easily handle multi-threading in coroutines. So you should always think that, okay, what you want to develop, what you want to implement, what is the use case, and then close down on what platform you will be using. Don't go with any notion that, okay, React is being used, I'll use React, or Kotlin is being used, I'll use Kotlin. No, it's not that. If you feel you want to use just simple Java-based standard application to build your application, that is perfect. Use it, if you feel so. Don't go by the names, don't go by these things. Because if you just go that, okay, I have to use React. See, for example, you are stuck in an issue, right? You're stuck in performance issue and all. It happens with React. So using React is something, is difficult. Even for an experienced developer, it is difficult. So that is why we make a call, what we want to use, and why, if React is perfectly suiting your case, that, okay, there's no other way, only React is there, and we have to only use React, then go for it, and that's absolutely fine. But if you feel that, no, React will add more complexity to my project, instead of benefit, it will provide me more cons. But if I use simple Java-Kotlin, I can do away with those cons, and I'm able to do the implementation easily, my all requirements will be met, then why not go with Java and Kotlin? I'm not saying React, you should not use, you should use, definitely, but based on the use case, if it is fitting your use case perfectly. If it's not fitting your use case, don't use that technology, simple, that's how it goes. Yes, you're right. So if you give your interviews, especially your system design interviews, so in the system design interview, the interviewer will see this thing only, what system you have chosen, why you have chosen this, and you have to convince the interviewer that, okay, this is the reason for my choice. For example, take an example for an app, you're using React Native, right? If you say this in an interview that, okay, I've been working on this app, and I've used React Native, the interviewer can question you, why did you use React Native? What was the reason that forced you to use React Native? So now you have to convince the interviewer that, okay, these are the reasons why I use React Native, these are the reasons why I don't use Java or Kotlin. And if you are not convinced with proper points, judge in your system design. I'm not telling you anything, I'm just giving an example. So always be very clear that what technology you are using and why you are using. This is what is system design. I will ask you this question. They can ask you, why are you not using databases? All these questions interviewers ask because they want to know that what is your choice and why you reached to that choice. Will you be able to persuade me or not? It's all about persuasion. If you are able to persuade the interviewer, the interview goes

very well. But contrary to this, DSA is not DSA. Interviews are fixed. Okay, this is how you have to implement. This is the time complexity, you have to implement your coding. This is the optimized version. So interviewer will look that okay, you reached that. But system designs are open ended. There is no hard and fast solution of any system design question. Nobody can say that this design is bad or able to convince the interviewer. Even if you do a very poor design proposal, but you are able to convince the interviewer that okay, this is what I'm doing is correct. And interviewer is convinced it's fine. So that makes these two rounds very different. DSA and system design. System design checks your ability to make decisions state of convincing ability, how you are able to convince the interviewer. That's it. So I took you one example, there was a system design round and it was not actually system design to mix up a system design and DSA round. So one of the students I asked a question and he used two loops, two for loops to compute some millions of data that was coming. And my question is very simple at why you're using two for loops for millions of data. So it will be millions of data into millions of data. So imagine the amount of computation that will happen. So you should have done it in a very simple way by using some data structures to make it optimized. But that student, he gave such good logics that even made me think that okay, he's using two for loop is fine. So that is the sense of, I was looking into that, why he's able to justify using two for loops or not. If I said that why was in two for loops, and he then used to say that, okay, okay, I made a mistake, I'll revert. This may not give me a good impression, right? But rather than saying that he convinced me and he gave me all the logics that why he used two for loops for millions of records. And in the end, I was like, okay, fine, you can go ahead with two for loops, because this is suiting your scenario, it is suiting your use case, whatever you're trying to do, it is getting very much fulfilled by using two for loops, even though time complexity will increase. But what we want that we are achieving properly. In case he goes for some optimization, the time complexity may reduce, but what we want to achieve that cannot be proper. So this is what the logic he gave, he gave me the complete trade off. This is what we were looking at, okay, just convince me why you made this choice. There's nothing right or wrong. Two loops is also right, not using two loops is also right. Both are right, but convince me why you chose this. That's what the simple thing. Get it? Yes, exactly. Cool. Any other question? Cool. And I think that's all we have for today's session. Keep learning your weekly sessions. And we'll meet next Saturday. Be ready with your doubts if you have any and we'll try to work together and solve them. Else, if no doubts, we'll see what we can discuss. Thank you. Thank you all. Have a nice evening. So just one announcement, information I wanted to convey. Hello, students, I'm one of the TAs for this course. For any doubts that you may encounter during the live sessions or otherwise also, you can post also on discussion forums. You will find them in the course content website. And I will post a sample post there so that you can come to know of its presence. Thank you. Fine. Thank you. Sir, I have a doubt. Right now, I know React Native. Should I shift to Kotlin now? No, I'm not telling you to shift it. If you know React Native, you want to proceed with React Native, go ahead. Learn, fix, debug, fix your things. It's good to know React Native. It's very much used. It's very much used. You learn both. What's the problem? The more you learn, the more it's good for you. Thank you. Fine, folks. Thank you, everyone. See you next Saturday. Thank you. you

Programming Language for Android (Kotlin) Overview Video• .

Duration: 6 minutes

[Notes]

Hello everyone, welcome to the new module on Basics of Application Development in Android. I am Professor Chandrasekhar RK and in this session we will be discussing Kotlin, a modern programming language that is becoming increasingly popular for Android development. Kotlin is designed to be concise, safe, and fully interoperable with Java. Let's begin by outlining the learning objectives for this session. By the end of this session, you should be able to understand the key features of Kotlin, recognize the advantages of using Kotlin for Android development, learn how to get started with Kotlin in Android Studio. These objectives will guide us through the basics of Kotlin and help you see why it's becoming the preferred language for many Android developers. Let's start with the introduction to Kotlin. Kotlin is a modern, statically typed programming language designed by JetBrains. It was officially supported by Google for Android development in 2017, making it the second official language for Android after Java. One of the most important aspects of Kotlin is that it is fully interoperable with Java. This means that you can use Kotlin in existing Java projects without having to rewrite your entire codebase. This interoperability makes it easier for developers to transition to Kotlin while still leveraging the vast ecosystem of Java libraries and tools. Kotlin is also known for its concise syntax, which reduces the amount of boilerplate code that developers need to write. This makes the code more readable and easier to maintain, leading to fewer errors and a more efficient development process. Now let us look at some of the key features that make Kotlin a powerful language for Android development. First one being the concise syntax. Kotlin syntax is designed to be concise, meaning that you can write less code to achieve the same functionality. This not only speeds up development, but also makes the code easier to understand and maintain. Kotlin includes built-in safety features to prevent common errors like null pointer exceptions. This makes your code more robust and reduces the likelihood of runtime crashes. It also has extension functions. Kotlin allows you to add new functions to existing classes without modifying their source code. This feature, known as extension functions, helps you extend the functionality of classes in a clean and organized way. Kotlin's coroutines make it easier to write asynchronous code, allowing you to perform tasks like network calls or database operations without blocking the main thread. This leads to smoother and more responsive apps. Here are some of the popular applications developed in Kotlin proving its worth as Java replacement for Android app development. Now let's discuss why many developers are choosing Kotlin over Java for Android development. While Java has been the traditional language for Android, Kotlin offers several advantages that make it a better choice for modern app development. As mentioned earlier, Kotlin's concise syntax reduces the amount of code you need to write. This leads to faster development and fewer chances for errors. And Kotlin's approach to null safety helps prevent one of the most common issues in Java, that is null pointer exception, making your apps more stable. Kotlin can easily extend the functionality of existing classes by using extension functions, which is not as straightforward in Java. Kotlin's smart cast feature automatically handles the type conversions, making your code cleaner and reducing the need for manual casting. One of the highlighting features of Kotlin is that it is fully interoperable with Java, meaning you can use both languages within the same project. This allows for smooth transition from Java to Kotlin. And Kotlin's coroutines simplify asynchronous programming, making it easier to perform tasks like fetching data from the internet without freezing the user interface. Then it also supports default and named arguments, which improve code readability and reduce the need for overloaded functions. To summarize, Kotlin is a modern programming language that offers numerous benefits for Android development. Its concise syntax, null safety, extension functions, and coroutines make it a powerful tool for creating efficient and robust Android apps. Additionally, Kotlin's interoperability with Java allows developers to seamlessly integrate it into existing projects.

By getting started with Kotlin in Android Studio, you can take advantage of these features and improve your development workflow. Thank you for your attention throughout this session. I hope you now have a better understanding of Kotlin and its benefits for Android development. Thank you

Android Studio Kotlin Video• . Duration: 6 minutes

[Notes]

Hello everyone, welcome to this module on Application Programming Development in Android. In this session, we will be discussing Android Studio and how it supports Kotlin, a modern programming language. Let's start by outlining our learning objectives for this session. By the end of this session, you should be able to understand how Android Studio supports Kotlin, explore the enhanced development experience provided by Kotlin in Android Studio, identify Kotlin's unique features that improve Android app development. These objectives will help you grasp the advantages of using Kotlin and Android Studio, making your development process smoother and more productive. Let's begin by understanding Kotlin support in Android Studio. Kotlin is natively supported in Android Studio, which means it integrates seamlessly into the development environment. This native support ensures that developers can use Kotlin without any additional setup, making it easy to start coding right away. Android Studio offers several Kotlin-specific tools and features that enhance the development experience. For example, the IDE provides intelligent code completion, which suggests code snippets as you type, helping you write code faster and with few errors. Refactoring tools allow you to make changes to your codebase quickly, and debugging features help you find and fix issues efficiently. Because of these advantages, Kotlin has become the preferred language for modern app development. Its seamless integration with Android Studio makes it an excellent choice for both new projects and existing ones that need a modern touch. So let's talk about enhanced development experience that Kotlin provides in Android Studio. One of the key aspects is the integrated tools. These tools include intelligent code completion, which helps you write code more quickly and accurately. Additionally, refactoring tools make it easy to reorganize your code, while debugging features allow you to troubleshoot issues effectively. Another important feature is interoperability. Kotlin can interoperate with Java, meaning you can use both languages in the same project. This is particularly useful if you are working on a project that started in Java, but now you want to take advantage of Kotlin's modern features. You can gradually migrate parts of your project to Kotlin without having to rewrite everything from scratch. Android Studio also provides project templates specifically for Kotlin, making it effortless to create new projects. These templates come with pre-configured settings and code snippets, allowing you to start developing apps right away without worrying about setup. Now let's look at some of the Kotlin's unique features that make it stand out in Android development. First one being null safety. Kotlin has built-in null safety features that minimize the risk of null pointer exceptions, one of the most common issues in Java. This feature helps you write safer and more reliable code. Kotlin also allows you to add new functionality to existing classes without modifying their source code. This is known as extension functions and it helps you keep your code organized and maintainable. Coroutines are a powerful feature in Kotlin that simplifies asynchronous programming. They make it easier to handle tasks like fetching data from the internet or performing database operations without blocking the main thread, resulting in smoother and more responsive apps. Then Android Studio includes built-in tools to convert Java code to Kotlin, making it easy to transition existing projects to Kotlin. This feature

allows you to take advantage of Kotlin's benefits even in projects that were originally written in Java. To summarize, Kotlin is a powerful programming language that offers several advantages for Android development. Its native support in Android Studio provides a seamless development experience with features like intelligent code completion, refactoring tools, and debugging features. Kotlin's unique features such as null safety, extension functions, and coroutines make it an ideal choice for modern Android apps. Whether you are starting a new project or updating an existing one, Kotlin in Android Studio helps you write efficient, reliable, and maintainable code. Thank you for your attention throughout this session. I hope now you have a better understanding of how Kotlin is supported in Android Studio and how it analyzes the Android app development process. Thank you.

Interface in Android Studio Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on the basics of Android application development. Now we will be discussing the user interface of Android Studio, which is the official integrated development environment. Let's start by outlining the learning objective for this session. By the end of this session, you should be able to understand the overview of Android Studio user interface, identify the key components of the interface and their functions. Learn how to customize the interface to improve productivity. These objectives will guide you through the process of mastering Android Studio, making your development process smoother and more efficient. Let's begin with an overview of Android Studio user interface. The interface is designed to be intuitive, providing you with all the tools you need to develop Android apps efficiently. First we have the project view. This section helps you organize and navigate through your project files. It displays the structure of your project, making it easy to find and manage different files and resources. Then we have the editor window. This is where you write and edit your code. The editor window comes with features like syntax highlighting and error checking, which help you write clean and error-free code. Then we also have tool window. Android Studio provides several tool windows that offer additional functionalities such as the logcat, terminal, and device file explorer. These tools help you debug your app, execute commands, and explore the file system of connected devices. Understanding these components is essential as they form the foundation of your work in Android Studio. Here we see the overview of Android Studio UI. There is a project view, editor window, logcat, device explorer, and the terminal in this screenshot. Now let's dive deeper into the key components of the Android Studio interface. First let's talk about toolbar. The toolbar provides quick access to common actions such as running your app, debugging, and syncing your project. It's a handy place to find the most frequently used commands. Then we have the navigation bar. This component allows you to navigate quickly through your project hierarchy. It's especially useful for large projects where finding files and resources can become time consuming. Then we also have the status bar. Status bar is located at the bottom of the IDE. The status bar provides real-time information about the current state of your project and development environment. It shows details such as memory usage, syncing status, and warnings. Then we have logcat. Logcat is a critical tool for debugging your application. It shows logs generated by your app, which can be filtered to help you diagnose issues and monitor the behavior of your app during runtime. These components are designed to streamline your workflow and make it easier to manage your project from a single interface. To make most of Android Studio, it's important to customize the interface to suit your personal workflow and preferences. Here are some

ways to do that. First let's talk about themes and appearance. Android Studio allows you to customize the look and feel of the IDE. You can choose from a variety of themes to reduce eye strain and improve focus. For example, switching to a dark theme can make coding more comfortable during long sessions. Then we have keyboard shortcuts. Configuring and using keyboard shortcuts can significantly speed up your workflow. Instead of relying on the mouse to perform actions, you can execute commands with a simple key press, saving time and effort. Then we have the tool window layouts. Android Studio allows you to arrange and save different tool window layouts. This means you can set up the interface to fit your workflow and quickly switch between layouts depending on the task at hand, such as coding, debugging, or designing UI. By customizing these aspects of the interface, you can create a workspace that enhances your productivity and makes your development experience more enjoyable. To summarize, mastering the interface of Android Studio is key to becoming an efficient and productive Android developer. We covered the overview of the user interface, including the project view, editor window, and tools window. We also explored key components such as toolbar, navigation bar, status bar, and logcat, which are essential for managing your project and debugging your app. Finally, we discussed how to customize the interface for better productivity by adjusting themes. Thank you for your attention throughout this session. I hope now you have a better understanding of Android Studio interface and how to customize it to enhance your productivity. Thank you.

First Android Application Overview Video• . Duration: 6 minutes

[Notes]

Hello everyone and welcome to this session on Basics of Android Application Development. Now we will be looking closer to how to create your first Android application using Android Studio. Let's begin by discussing the learning objectives for this session. By the end of this session, you should be able to understand the steps to create a new project in Android, name the project, configure the initial setup, get an overview of XML code used for the layout, explain the main activity class, view the final output of your app in Android Emulator. These objectives will help you navigate through the process of developing your first Android application, making sure you understand each step clearly. Let's start by creating a new Android project in Android Studio. The process is straightforward. First we have to open the Android Studio. Start by opening Android Studio on your computer. Then click on Start a new Android Studio project. The next step would be to choose a template. For this project, we will choose the basic activity template which provides a simple starting point with some default views. Next step is to name the project. Set the project name, for example, First Application Development, and choose the location where you want to save your project files. Finally, we finish the setup. So after naming and configuring the project settings, click Finish to create the project. Once these steps are completed, Android Studio will set up your project with the necessary files and folders and you will be ready to start coding. Now let us take a look at XML code used for layout of your app. In Android, XML is used to define the user interface, which includes the positioning and appearance of various elements. First let's talk about the root element. In the relative layout, the root element in our layout is RelativeLayout. This is a flexible layout container that allows you to position child views relative to each other or the parent layout. It provides a lot of versatility in designing the user interface. Next let us look into a child element which will be TextView. In the relative layout, we have a TextView element. A TextView is a UI component that displays text on the screen. In our

example, this `TextView` is centered in the parent layout and displays the message Hello World. This simple layout is often used as a starting point for many Android applications. Next let's discuss the `MainActivity` class, which is the backbone of your Android application. First let's talk about Package. The `MainActivity` class starts with a package declaration which defines the namespace for your class. For example, `com.example.firstApplicationDev` is the package name. Then we have the imports. The class uses imports like `AppCompatActivity` for backward compatible activity features and `Bundle` for handling the state of the activity. Then we have to define the class. `MainActivity` inherits from `AppCompatActivity` making it the entry point of your app. This is where the app begins its execution. Then we have the `onCreate` method. The `onCreate` method is the first method that gets called when the activity is created. It initializes the activity and sets the user interface layout using `setContentView` in which we pass on `r.layout.activity.main` which points to the XML layout file we discussed earlier. This method ensures that the layout is displayed when the application runs. Finally let's look at how to view the output of your app in Android emulator. First we have to connect to the emulator. Ensure that the Android emulator is running. You can start it from Android Studio or use an already active virtual device. Next we have to build the project. In order to build the project, click on the build menu in Android Studio and select make project. Or you can even use shortcut key control F9 to compile the project and build the project. After building the project, we have to run the application. In order to run the application, we have to click on the green run button or press shift F10 and choose your emulator device from the list of available virtual devices. Once the application is deployed, the emulator will start and display the hello world message on the screen. Seeing the output on the emulator is a satisfying moment as it shows that your code is working as expected. To summarize, we have gone through the process of creating your first Android application in Android Studio. We covered how to start a new project, configure the initial setup and get an overview of the XML code used for the layout. We also discussed the main activity class which is crucial for apps functionality and finally we looked at how to view the output in the Android emulator. With these steps, you now have a solid foundation to create simple Android apps and build on this knowledge for more complex projects. Thank you for your attention throughout this session. I hope you now feel more confident in starting your Android development journey. Thank you.

Running the Application on Emulator Video• . Duration: 5 minutes

[Notes]

Hello everyone and welcome to this session on Basics of Android Application Development. We will now discuss how to run an Android application using an emulator. Let's begin by discussing the learning objectives for this session. By the end of this session, you should be able to understand the need for an emulator and its role in testing Android applications. Learn the steps to set up a new virtual device using the Device Manager in Android Studio. Run your Android application on the emulator and view the output. These objectives will guide you through the process of using emulator in Android Studio, helping you test your applications effectively. Let's start by understanding the need for an emulator in Android development. An Android emulator provides a virtual environment that simulates an actual Android device, allowing developers to test their applications without needing physical hardware. This is especially useful because it enables cross-device testing, ensuring that your app works well on different types of devices with various screen sizes and specifications. Emulators are also cost-effective as they eliminate the need to purchase multiple

physical devices for testing. Furthermore, they offer accessibility, allowing developers to test apps in environments that may not be readily available on physical devices, such as older Android versions or specific screen configurations. Now that we understand the importance of an emulator, let's move on to setting up a new virtual device using the Device Manager in Android Studio. In Android Studio, we go to the Tools menu and select Device Manager. This will open the Device Manager window when you create, manage, and launch Android virtual devices or in short called as AVDs. And in order to create a new virtual device, in the Device Manager, we click on the Create Device button to start setting up a new virtual device. You can choose from a variety of device models, including different phone models, tablets, and other Android devices with various screen sizes, resolutions, and selecting the device model. After selecting the device model, click Next. You will be prompted to choose a system image which determines the version of Android the virtual device will run. Select an Android version from the list. If you don't have the desired system image downloaded, you can download it directly from this window. Click Next once you have selected the system image. API level in Android defines the version of the operating system and the set of features and APIs your app can access, ensuring compatibility across devices. ABI, which is Application Binary Interface, specifies the processor architecture, such as ARM or x86, that your app is built for, ensuring it runs efficiently on the device hardware. The target in Android development determines the overall Android version your app or virtual device will use, along with additional features like Google Play services, ensuring the app leverages the latest platform compatibilities. Continuing with the setup process. Now we have to configure the device. In order to configure the device, we have to configure the hardware specifications of your virtual device, such as RAM, internal storage, and other settings. Optionally, you can also name your AVD for easy identification. Once it is done, click Finish to complete the setup. Once your AVD is created, it will appear in the list of devices in the device manager. To start the emulator, click the Play, which is the green triangle button next to the virtual device in the list. The emulator will launch and you will see a virtual Android device on your screen ready to run your application. This setup allows you to test your app in a controlled environment that mimics a real device, ensuring that your app works as expected before deploying it to the physical devices or publishing it. To summarize, using an emulator is a crucial part of Android app development. It allows you to test your app across various device configurations without needing physical hardware. We discussed the steps to set up a new virtual device using the device manager in Android Studio, and also how to configure and launch the emulator, and how to run your Android application on the emulator to view the output. Mastering the use of emulators will enhance your development process, making it more efficient and cost-effective. Thank you for your attention throughout this session. I hope now you have a better understanding of how to use an emulator in Android Studio. Thank you.

Debugging and Views in Android Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this lesson on Debugging and Views in Android. Let's begin by outlining the learning objectives for this session. By the end of this session, you will be able to Set a breakpoint in your Android application Start debugging your application using Android Studio Inspect the application state during debugging Understand the process of debugging and execution These objectives will guide you through the critical steps of debugging, helping you identify and

resolve issues in your Android apps efficiently. Let's start with the first step in debugging. Setting a breakpoint A breakpoint is a marker you set in your code to pause the execution of the program at a specific point. This allows you to examine the state of your application at that moment, making it easier to identify where things might be going wrong. For this, you have to open your Android Studio

Navigate to the main activity.kt file or any other file where you want to debug your code Find the line of code where you want to pause the execution and set a breakpoint by clicking in the left margin next to the line number. A red dot will appear indicating the breakpoint is set. Setting breakpoints strategically allows you to monitor the behavior of your app at crucial points, giving you a clearer picture of what's happening during execution. Now that we have set a breakpoint, the next step is to start debugging your application. In Android Studio, we have to go to the Run menu and select Debug. Alternatively, you can use the shortcut by clicking the green bug icon in the toolbar. A dialog will appear asking you to choose the device or emulator on which you want to run your app. Select your preferred option and the app will start running in debug mode. When you run the app in debug mode, the execution will pause automatically when it reaches the breakpoint you set. This allows you to inspect the current state of the application, check variable values and understand the flow of execution. Once the breakpoint is hit, you can inspect the state of your application to understand what's happening at the point in the code. When the app pauses at the breakpoint, the debugger window will open at the bottom of Android Studio. Here you can see important information such as variable values, the call stack and the other relevant data. This information helps you determine whether the code is behaving as expected. You can also use the following options to control the flow of debugging. In order to move to the next line of the code, we can use stepover which is F8 without entering any functions. We can use stepinto which is F7 which enters the next method or function allowing you to debug it line by line or we can make use of stepout which is shift plus F8 which exits the current method and returns to the previous one. These tools allow you to examine the behavior of your app in detail, helping you pinpoint the exact location of any issues. After inspecting the application state, you can continue debugging by viewing the logcat output and resuming program execution. The logcat window displays logs related to your app's execution. These logs provide valuable information including any errors or warnings that occur while the app is running. You can filter the logs to show only those related to your application making it easier to diagnose issues. To resume the execution of your program, press the resume program button or press F9. The app will continue running until it either reaches the next breakpoint or completes execution. By understanding how to use these debugging tools effectively, you can quickly identify and resolve issues in your Android applications, ensuring they run smoothly and without errors. To summarize, debugging is a crucial part of Android app development that helps you identify and fix issues in your code. In this session, we learnt how to set a breakpoint, start debugging, inspect the application state and understand the flow of execution. These skills are essential for ensuring that your applications are robust and free of errors. By mastering these debugging techniques, you will be better equipped to create high-quality Android apps. Thank you for your attention throughout this session. I hope you have a better understanding of debugging process in Android Studio and how to use it effectively to troubleshoot your applications. Thank you.

Version Control Integration Video• . Duration: 5 minutes

[Notes]

Hello everyone and welcome to this lesson which is on Debugging and Views in Android. Let's begin by outlining our learning objectives for this session. By the end of this session, you should be able to understand what version control is and why it is important. Learn about version control integration in Android Studio and follow the basic workflow of using version control in your projects. These projects will help you grasp the importance of version control and how it effectively used in your Android development projects. Let's start with an introduction to version control. Version control systems or VCS in short are tools that help you track code changes over time. They allow multiple developers to work on the same project simultaneously without overwriting each other's work. This makes collaboration easier and more efficient. Version control also enables you to maintain a history of your code, which means you can go back to the previous versions if something goes wrong. It's like having an undo button for your entire project. Additionally, version control makes it easier to backup your code and recover it if your system crashes or if you accidentally delete important files. There are several popular version control systems such as Git, SVN, and Mercurial. Among these, Git is the most widely used, especially in Android development. Now let's discuss how version control is integrated into Android Studio. Android Studio comes with built-in support for Git, which allows you to perform version control operations directly within the IDE. This integration makes it convenient for developers to manage their code without needing to switch between different tools. Some of the key features of version control in Android Studio include Commit and Push Using this, you can save your changes locally with a commit and then push these changes to a remote repository such as GitHub or Bitbucket so that they are backed up and shared with your team. Then we have Branching and Merging These features allow you to work on different features and fixes independently and then merge them back into a main codebase when they are ready. This keeps your main project stable while allowing for experimentation and development in parallel. It also has an option to view history. You can inspect the evolution of your project by viewing to commit history, which shows all the changes made over time. This helps in understanding how the code has evolved and in identifying when and where bugs were introduced. Next, let's look at the basic workflow for using version control in Android Studio. This workflow involves several key steps. In that, the very first step is to initialize or clone a repository. If you are starting a new project, you can initialize a Git repository in Android Studio. If you are joining an existing project, you can clone the repository from a remote source like GitHub. The next one is make changes. As you work on your project, you will make changes to the code. These changes could be anything from fixing bugs to adding new features. Then we have the commit changes. Once you have made some progress, it's important to commit your changes. A commit records the changes you have made along with a message describing what was done. This helps in keeping the track of different stages of development. Then we have push to remote. After committing, you should push your changes to a remote repository. This ensures that your work is backed up and shared with your team. Finally, we have pull and merge. Before starting new work, it's a good idea to pull the latest changes from the remote repository. This updates your local codebase with any new changes made by your team members. If there are conflicts, you will need to merge these changes manually. To summarize, version control is an essential tool for managing code in any software development project. It helps you track changes, collaborate with others, and ensure that your code is safe and recoverable. In this session, we discussed the basics of version control, its integration in Android Studio and the basic workflow you need to follow. By mastering these skills, you can ensure that your development process is smooth, efficient and reliable. Thank you for your attention throughout this session. I hope now you have a clear understanding on how to use version control in your Android

development projects. Thank you.

Overview of Views in Android Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this lesson on views. Views are the basic building blocks of user interface in Android. Let's discuss the learning objectives for this session. By the end of this session, you should be able to understand the concept of views in Android, learn the XML syntax for creating a view, identify the most commonly used Android view classes. These objectives will help you grasp the importance of views in Android development, and give you the tools to start designing your own interfaces. First, let's understand what a view is in Android. A view is the basic building block of the UI in Android. It represents a rectangular area on the screen that can display content like text or images and can also respond to user actions like clicks or touches. In technical terms, a view refers to the `android.view.View` class, which is the superclass for all GUI components like `TextView`, `ImageView`, and `Button`. Every view occupies a rectangle on the screen. Even though this rectangle is usually invisible to the user, this rectangle defines the space that the view will occupy and how will it interact with other views on the screen. Mastering views is essential because they are the foundation of all user interfaces in Android, whether you are creating a simple app or a complex one. Mastering views will help you design interfaces that are both functional and aesthetically pleasing. Now that you know what a view is, let's learn how to create one using XML. In Android, the layout of an app's UI is typically defined using XML. XML allows you to describe the structure of your UI in a clear and organized way. When creating a view in XML, two attributes are mandatory, i.e., `android:layoutWidth` and `android:layoutHeight`. These attributes define the size of the invisible rectangle that the view occupies. You can set these attributes to specific values like 100dp for 100 density-independent pixels or match the parent or content. In addition to layout underscore width and layout underscore height, there are other commonly used attributes like gravity, layout gravity, padding, and margin. These attributes help you control the positioning and spacing of the view within its parent layout, allowing you to fine-tune the appearance and behavior of your UI elements. Using XML to define your UI is powerful because it separates the design of the interface from the logic of the app, making it easier to manage and update. Let's move on to the most commonly used Android view classes. These are the building blocks you will use to create various elements of your app's UI. Text is the simplest and the most common type, which is `TextView`. This is used to display text on the screen. Then we have `EditText`. This view allows users to input text. It's often used for forms or search bars. Then we have a common view, which is `Button`. A button is a clickable view that users can press to perform an action. Then we have the `ImageView`. This view is used to display images in your app. Then we have `ImageButton`, similar to a button, but with an image instead of text. Then we have `CheckBox`. This is a two-state button that can be either checked or unchecked, commonly used in forms. We also have `RadioButton`, used in groups where only one option can be selected at a time. Then we also have `ScrollView`, a view group that allows its content to be scrolled vertically. Then we have `ListView`, which displays a list of scrollable items, such as text or images. We also have `GridView`, displaying items in a two-dimensional scrollable grid. `DatePicker` allows users to select a date from a calendar. Then we have `Spinner`, which provides a drop-down menu for users to select an option. These view classes are the essential tools in your Android development toolkit. Learning how to use them effectively will

help you build complex and interactive user interfaces. To summarize, views are the core components of any Android user interface. They define the visual elements of your app and how users interact with it. We discussed that views are, how to create them, using XML, and most commonly used view classes in Android. By understanding and mastering these concepts, you will be well on your way to create user-friendly and visually appealing Android applications. Thank you for your attention. I hope now you have a better understanding of views in Android. Thank you.

Text View and Edit Text in Android Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this lesson on Views in Android App Development. In this session, we will be discussing two essential components of Android user interfaces, which are TextView and EditText. Let's begin by discussing the learning objectives for this session. By the end of this session, you should be able to understand what TextView is and how to create it. Understand what EditText is and how to create it. Identify the basic attributes and values associated with TextView and EditView. These objectives will help you with the basics of these UI components, which are crucial for building interactive Android applications. TextView is a user interface element used to display text to the user. It is a non-editable component, typically used for static text like labels, instructions, or any other text that does not require user interaction. TextView is one of the most commonly used components in Android because it allows developers to present information clearly and effectively. For example, you can use TextView to display a title, a message, or a label next to an input field. Some of the common properties of TextView include The text. This property sets the text that will be displayed. We can even assign the size to it. By TextSize. This property defines the size of the text. Then we have TextColor. This property is used to set the color of the text. Then we have Gravity. This property controls the alignment of text within the TextView. MaxLines. This property limits the number of lines that text can occupy. Using these properties will help you customize TextView to suit the needs of your application. Now let's look at how to create a TextView in Android. You can create a TextView either using XML in your layout file or programmatically using Java or Kotlin. In your XML layout file, you can define a TextView like this using the XML. We have to use TextView tag with various attributes. Here the AndroidId attribute assigns a unique ID to the TextView, which you can use to reference it in your Java or Kotlin code. To interact with the TextView in your Java or Kotlin code, you can use the following lines where using findViewById will get you the reference of TextView. And using this particular reference, you can use setText method to set the text to this particular TextView. This code finds the TextView by its ID and sets text to the Hello Android. By combining XML and code, you can fully customize the appearance and behavior of TextView in your application. Now let's discuss EditText. EditText is a user interface element that allows users to input and modify text. Unlike TextView, EditText is editable and commonly used for input fields like forms, search boxes, and any other scenario where user input is required. EditText is a flexible component that can handle different types of input, such as text, numbers, passwords, and more. This makes it an essential tool for creating interactive applications. Some of the common properties of EditText include Hint. This property displays a hint to the user about what they should enter. InputType. This input type property defines the type of data that can be entered, such as text or number or it could even be password. TextColor. This property sets the color of the text which is entered by the user. We can use the TextSize property which defines the size of the text. And there

is also `MaxLength` which is a property that limits the number of characters the user can enter. These properties allow you to control the behavior and appearance of `EditText` to ensure user-friendly experience. Now let's see how to create an `EditText` in Android. Similar to `TextView`, you can create a `EditText` using XML in your layout file or programmatically using Java or Kotlin. In your XML layout file, you can define a `EditText` as shown in the screenshot. Here, the `AndroidHint` attribute provides a hint to the user and the `AndroidInputType` attribute restricts the input to text. To retrieve the text entered by the user in your Java or Kotlin code, you can use the following lines as shown in the screenshot. This code finds the `EditText` by its ID and retrieves the text entered by the user as a string. By using `EditText`, you can create interactive and responsive forms and input fields in your Android application. To summarize, `TextView` and `EditText` are fundamental components in Android development that allow you to display and collect text information from users. We discussed what `TextView` is, how to create it, and its common properties. We also explored `EditText`, its creation, and its properties that makes it ideal for input fields. Understanding these components will enable you to build more interactive and user-friendly applications. Thank you for your attention. I hope now you have a better understanding of how to use `TextView` and `EditText` in your Android applications. Thank you.

Buttons in Android Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on views. Let's outline our learning objectives for this session. By the end of this session, you will be able to Set up a button in Android application Define the button layout using XML Add interactivity to the button using `onClickListener` function These objectives will guide you through the process of creating and managing buttons in Android Making your applications more interactive and user-friendly Let's start with setting up a button in Android The button is a user interface element that you can initialize by finding its reference in a XML layout file In Android, every UI component is typically defined in a XML file And each component has a unique ID For example, to set up a button in your Kotlin code You first need to reference it using the `findViewById` function As you can see in the screenshot Here, `r.id.button` refers to the button's ID defined in the XML layout file This line of code allows you to interact with the button in your Kotlin code Enabling you to customize its behavior and appearance Setting up the button correctly is the first step towards adding functionality to your app's user interface Next, let's look at how to define the button using XML The layout of a button, like other UI components, is defined in the activity underscore main dot XML layout file Or it could be in any other layout file as well in the project In the screenshot, you can see an example of how to define a button in XML Using the XML tag In this code, the `androidId` attribute assigns a unique ID to the button Which you can use to refer it in your Kotlin code The `androidLayout underscore width` and `androidLayout underscore height` attributes define the size of the button Then we have `androidTextAttribute` specifies the text displayed on the button And `androidLayoutCenterInParent` attribute centers the button within its parent layout Defining the layout in XML makes it easy to organize and manage visual structure of your app's interface Now that we have set up the button and defined its layout, let's add some interactivity Interactivity is what makes your app responsive to user actions In Android, you can add interactivity to a button using `setOnClickListener` function You can see the code snippet using Kotlin in the screenshot In this code, `setOnClickListener` is used to define the action that occurs when the button

is clicked In this example, a toast message is displayed with text button clicked when the user taps on the button The toast.makeText function creates a pop-up message and show is called to display it on the screen Adding interactivity with OnClickListener is essential for creating dynamic and engaging Android applications After implementing the button and its interactivity, it's important to test how it behaves in your application You can do this by using the Android emulator When you run the app on the emulator and click the button, you should see the button on the screen with the text Click Me When you click the button, the toast message, which is a pop-up, button clicked, will appear briefly at the bottom of the screen Testing your app on the emulator helps you verify that button functions as expected Providing the desired user experience To summarize, buttons are fundamental part of any Android application's user interface We discussed how to set up a button in Android by referencing it in Kotlin code, defining its layout using XML, and adding interactivity with OnClickListener Understanding these concepts allows you to create interactive and user-friendly applications that respond to user inputs effectively By mastering the use of buttons, you can enhance the functionality and usability of your Android apps by making them more engaging for users Thank you for your attention throughout this session I now hope you have a better understanding of how to implement and manage buttons in apps Thank you

ScrollView in Android Video• . Duration: 4 minutes

[Notes]

Hello everyone and welcome to this lesson on Views in Android App Development. Let's begin by discussing the learning objectives for this session. By the end of this session, you should be able to set up a scroll view in Android XML layout, implement scroll view functionality in Kotlin, test and verify scroll view output using an emulator. These objectives will guide you through the process of adding scrollable content to your Android apps, ensuring that your apps remain functional and user-friendly even when dealing with large amount of content. First, let's look at how to set up scroll view in XML. Scroll view is a layout container that allows you to place a single child component which can be scrolled vertically. It's particularly useful when the content inside the container exceeds the screen height. In the screenshot, you can see an example of how to create a setup scroll view in XML using the scroll view tag. Inside the scroll view tag, we have embedded linear layout tag as well, and the layout tag will have the various other UI components like text view, edit text, buttons, etc. In this code, the scroll view wraps around a linear layout that has a vertical orientation. Inside the linear layout, you can place multiple text view elements or other components that you want to make scrollable. This structure allows the content to be scrolled when exceeds the screen height, providing a smooth scrolling experience for the user. Now that we have set up scroll view in XML, let's see how to implement it in Kotlin. After defining the scroll view in the XML layout, you can access it in your Kotlin code by using findViewById. And you see the code snippet in the screenshot. If you want to scroll to the bottom, we have to use scrollView.post. In this example, we access the scroll view by its ID and then use fullScroll method to scroll to the bottom of the content. The scrollView.focus__constant ensures that the scroll view scrolls to the bottom. This method is particularly useful when you want to ensure that a specific part of the content is visible when the activity is loaded. By implementing scrollView in Kotlin, you gain control over how and when the scrolling happens, enhancing the user experience in your application. After setting up and implementing scrollView, it's important to test its functionality to ensure it works as expected. You

can do this using Android Emulator. To test the scrollView, launch the emulator and run your application. Once the app is running, try swiping up and down on the screen. Even if everything is set up correctly, the content should scroll smoothly, even if the scroll bar isn't visible. This behavior indicates that scrollView is functioning properly, allowing users to navigate through all the content on the screen. Testing on the emulator is crucial because it helps you catch any issues in the scrolling behavior before deploying your apps into real devices. To summarize, scrollView is an essential component in Android when you need to handle the content that exceeds the screen size. We discussed how to set up scrollView in XML, how to implement it in Kotlin, and how to test its functionality using Android Emulator. By mastering the use of scrollView, you can create applications that are both user-friendly and capable of handling large amounts of content efficiently. Understanding and implementing scrollView correctly is key to provide smooth and seamless experience for users. Thank you for your attention. I hope you now have a better understanding of how to implement scrollView in your Android applications. Thank you.

Introduction to Android Studio Video• . Duration: 16 minutes

[Notes]

Hello everyone, welcome to this demo. In this demo, we'll be learning about Android Studio and the different features of it, and we will also create a Hello World application. So after you have downloaded the Android Studio and installed it in your system, and for the very first time when you start, this is the window you will be presented with. So I'm going to make use of this window in order to create a new project. So as you can see, I am using Android Studio, and its version is 4.1.1. And for this to run, I will also require JDK to be installed in the system. So which I have already done. So let me create a new project. So when I click on new project, I'll be presented with a window for me to select a project template. Now there is a wide range of templates which are provided, especially for phone and tablet, you can see that starting from no activity to empty activity, and there is a full screen activity login activity and others which are presented for us, and we could make selection of any of this activity templates. But I'm going to suggest to make use of empty activity, because this gives us the freedom to learn. There are other activities as well, which you can make use of. But I would suggest that it is for advanced developers, not only for phone and tablet, there is also project templates for wearables, TV, cars, and even Android things. But we'll be concentrating on the phone and tablet. So let me select empty activity. And here, I will have to give a name to the project. So here I will name it as my first app, okay. So the package name is formed automatically. And it will also save give you the save location, okay, as to where you want to save your project. And the preference of language over here for code generation is Kotlin. Of course, there is option for Java as well. But we will be doing all our development using the new Kotlin language, which is supported by Google. And the next item which we will have to select over here from the drop down is the minimum SDK, which your app should support. I am selecting API 19, which belongs to Android 4.4, which is KitKat. So if you actually see there are other versions as well, okay. So if you say like, consider API 30, which belongs to Android 11, it says that your app will run on less than 1% of the devices because there are not many devices with this particular API support. Similarly, if you consider say like Oreo, it is around 60% of the devices are already working on Oreo. But I would also want to support certain other devices, which are working still in Android 4.4, which is KitKat. So therefore, I will be selecting the API level 19, which is supported by the version Android 4.4. And as

you can see, this particular app will run on 98.1% of devices across all the devices as such. So once I have given the configuration for this particular my project, let me click on finish. So once I click on finish, it's going to give me the window for the workspace. So it basically makes use of Gradle as a build tool, and it creates the project folder for you. And it's also going to build certain code behind and give most of the templates for you. So as you can see, it has created two files for me, one is the main activity.kt. So this is my main file over here. Okay, so this is where all my logic will go into. So this is the activity file, which basically extends AppCompatActivity. So this this particular class that is AppCompatActivity will give support to or provide compatibility to other versions of Android as well. So this is the class which is created for us. And you must be noticing that it has already created the method or overridden the method onCreate. And even setContentView method is also called. In setContentView, there is an argument which is passed on to setContentView. And this is nothing but our layout file, r.layout.activity_underscore_main. Physically, this particular file is present under res, which is for resources, under that there is a folder layout. Inside this, we have the activity_underscore_main.xml. And this is automatically created for us, okay, by Android Studio. Let us look into what is present in this particular file. So this is the layout file in which all my widgets will go in. So currently, in the view, it shows that there is one particular view, which is named as text view. So let me zoom in. So you will be able to see that it displays Hello World, okay. And here you see the palette, which gives you the different other widgets which could actually be used in your view, okay. So let me show you the code as well for this particular layout. So if you see that it is making use of the constraint layout as a view group, and within that there is one particular view, which is named as text view. And the parameters are the attributes for this particular element, the layout width, it says wrap content, height, also it says wrap content, basically this will tell the system to place this particular text view containing only what is present as content into that particular text view. And the text which is going into this text view is the Hello World. So the space this particular text view is going to take will be around this particular Hello World only. Let's go back to the design of it. And if you see that the constraint layout is provided over here, and I could basically move around this particular component to any part of the screen, okay. So once I have placed the constraints for this particular view, I can drag it to any part of the screen, okay. So let me come back to the code. If you see that there is a text view which is defined, I can even specify a ID to it. And I would say text view display, okay. So let me go back to my main activity. So you can see that the main activity class is created, it extends AppCompatActivity, then onCreate method is also created for us. And even setContentView is also defined for us, okay. And the argument which goes to the setContentView is the layout file. And it is using the setContentView. This is where the layout file will get inflated into the activity, okay. Now let us see as to how we can run this particular application, okay. Whenever you make changes or whenever you create a project, the Gradle build tool will automatically detect all the changes and it's going to build often for you, okay. Now what I'm going to do is, I'm going to run this particular project, okay. In order to run the project, there is a button for you to run the app. And there is a dropdown over here from where you can actually make a selection of different devices on which you can actually run. So currently, there is no device which is running. When I say device, basically, there is no emulator which is running on which I can run this particular application. So what I will do is, I will go to the AVD manager, I will click on AVD manager. And you can see that there are three devices in the list, okay, with different versions of API and different targets, okay. What I'm going to do is, I'm going to make use of this particular virtual device, which is named as Nexus 5 API 19, okay, which supports API 19 onwards. And the CPU is x86 based. So I'm going to start this. So this is going to start the emulator for us. Once this particular emulator has started, then

I will be able to run my application using this emulator. Now, when you first start Android Studio, there will not be any devices which are created for you. So you will have to create a virtual device. In order to create a virtual device, you can click on this button in the AVD manager in order to create a virtual device. So once you click, then it's going to give you different categories for which you need to create the emulator. So the very first step over there is to select the hardware for it. So depending upon which particular hardware you want to use, you can make a selection over here. After that, so here for this particular example, Pixel 2 is selected. Otherwise, you can even go for say Nexus 6, depending upon your need. Okay, then you can click on Next. Here, we will have to select a system image. Now the system image is the one which will be loaded into this emulator virtual machine. In which it is going to run. So each of the emulator when you create, while it is running, it is a virtual machine in itself. Because it is loading your operating system, it should have an image for it to work. Okay, so depending upon what is your requirement, you can select a system image over here, depending upon the different API levels which you want to support. Okay, so if that particular image is not already present in your system, you will get an option to download it. If it is already downloaded, then it is not going to present that option of download. So you can see that these are all the recommended images for me. And it says Q is already downloaded. But in case I would want to try out others, then I will have to download that system image. But for now, I'm going to use the one which is already downloaded. So I'm going to click on Next. So here, I can give a name to my AVD, as well as go to advanced settings and provide different other settings as such. So when I click on show advanced settings, it is going to show us to what should be the startup orientation. Generally, in mobile devices, we know the startup orientation is portrait. But if it is a tablet, then the startup orientation, it is likely to have a landscape orientation, then the camera features front, how do you want it whether it is emulated or you don't want to have it back camera, how do you want it, then the network, there are different protocols, which are actually given what is the latency, which we would want to provide. Apart from that, what is a boot option, what how much of how many CPUs this particular mobile is going to use, then what is our RAM size, which will be available in this particular mobile VM heat, which the system is going to make use of internal storage space, what is the size of SD card, okay, so all these details can be entered. And when you click on finish, it is going to create a virtual machine or a virtual device for us to run with. Since I already have a few of it, I'm not going to create a new one, but make use of the existing one. So my emulator is ready for execution. So I have my emulator as well as my Android Studio workspace running side by side. So what I'm going to do is so since if you see now there is a distinction, so it says running devices, my Nexus 5 with support to API 19 is running and the other available devices which are not running is also showing the list. So I'm going to select Nexus 5 and click on run app. So when I click on run app, okay, you will see that my Gradle build tool will build this particular project again. And it is going to make use of the ADB, which is running inside this particular Android Studio to connect to my device at install. In order to identify the device, basically, each of this particular emulator runs in a different port, and that is specified by this particular number 5554. Okay, so if you see, it has got different features, gives you the cellular information, okay, as to how the connectivity is there, battery information, then camera, phone, and different other details as well. So we'll come back to this particular extended controls later. Now, after the Gradle build tool has finished building, it even made use of the installer, that is the APK package to install the application for us. So as you can see, my application is installed in this emulator. And you can see that hello world is getting displayed over here. Okay, and the title is coming as my first app. So where is this information like my first app is coming in from? So for that, I will have to look out for the Android manifest.xml file, which is a

manifest file for this particular project. So if you actually see, let me stop running this particular application and expand this window. So if you actually see, there is an application element which is present in Android manifest.xml file. And it has a activity element defined as well. So the name of the class is passed on over here. And it also has created a intent filter with two attributes with action and category. So we have learned this in our theory sessions. So action is action.me and category is category.launcher. So this will help the Android operating system to identify that whenever user clicks on this particular applications icon, then this is the first activity which needs to be presented to the user. Okay, and you can see the label for this particular app, it is coming in from this application tag. But it is not directly present in this tag over here, it is coming from another file, which is called as the string.xml. you will see that there is a name given to the string which is app underscore name and my first app is displayed over here so let me change it to fsc okay now what i'm going to do is i will also include a text size so that the visibility of this particular text is a little better so i'm going to make it 20 sp and i don't have to explicitly say these are already automatically it gets saved as such then i'm not going to change anything in my main activity so here this label it is referring to the strings file and it is going to pick the value which is assigned to app underscore name okay so let me run this particular application again to show you the changes so when i click on run again the gradle build tool is going to build this particular application and as you can see now you have a install and start application so you can see that the hello world is the size the text size is increased i could as well give the color to this particular text and display it over here okay so that's all for this particular demo thank you

Recording of Programming Mobile Devices: Week 2 - Live Session on 26-06-14 Video• . Duration: 26 minutes 26 min Resume . Click to resume

[Notes]

Oh, sorry, I think it had it had not started yet. Fine, it's recording now. Any any question around this and you're not details of this will it will definitely be covered in your course. What are the part of app? What are the part of framework? What are the part of Android runtime? But we also will try to see if we can cover some of these aspects as the course progresses. So any question around this? Keep keep these things in mind that this is a basic structure. So Android is based on Linux. That's a very important thing. So if anybody asked you interviewed at what is the underlying operating system in Android under itself as an operating system, but what underlying it is based on. So the answer is Linux. If if we discussed it, what are the different? See Android is built in Java, right? The very core programming language that that is there in Android is Java, so it's nothing but it's a wrapper on top of Java. Rapper means Java is the base, but with some more thing added on top of it. Now that Java is sick, Kotlin is also fundamentally Java only. So two languages Java and Kotlin are not very different. Don't think that Kotlin is a very different language. Yes, the syntax is different. It provides a lot of functionalities written out there in Java, but the essence and basics of Kotlin is also Java. Right, so Android is based on Java. Kotlin is also derived from Java. So Android and Kotlin forms the core fundamental of. Sorry, Java and Kotlin forms the core fundamental of Android. Since it's it's based on Java, it has all the. Of features that Java programming languages has, for example, oops, concepts, collections, concurrency, exception handling. Memory handling garbage collection string framework everything that is there in Java is or are part of the Android programming. Rather more on top because I just told that the basic is Java, but Android is a

wrapper means something built on top of Java with some more functionalities, but the core functionalities that Java has all part of Android. So if you are good in Java, if your fundamentals of Java are strong. Definitely an Android will be working on Android will be easy for you. If you're very new in Java, you're not aware of Java fundamental, then of course you have to put in some more effort to understand the concepts of Android. But yeah, these are the core programming fundamental. Now what wrapper is there? What Android is provided that will see as the core processor like activities are there. XML based layouts are there that help you put your elements. Fragments are there. Loading is there. Satellite views are there. View components are there. Those are something that are specific to Android, but all those users Java only for programming. So just keep in mind Android is nothing but an operating system that is built on Java programming. Programming fundamental wise Java is the base, so all the characteristics of Java are characteristics of your Android app programming. Any question in here? OK, now the question come is why Kotlin because Java is also evolving. Latest version of Java is also coming in then. Why Kotlin came into the picture? This is the very fundamental answer of why Kotlin word again. This this is a part of the questions that can be asked to you. That why Kotlin is there? What are the advantages that Kotlin brings? So again, Kotlin is is also the base of the Kotlin is also Java, but Kotlin is a very lightweight language compared to Java. It provides a lot of things like very good null safety. Null safety means you can avoid null pointer exceptions. Kotlin has something called coroutines that can help you work with different threads. Easily get get the result from different threads, merge it, do on main thread, do on a background thread. So those are very well managed through coroutines. There's a concise syntax. So what this concise syntax is? Very well in the same flow of the program you can do multiple things, keeping the flow same, returning back to the same flow and all. So that is called a concise syntax. Once the course is covered, you will understand it better with live examples or we'll see if we can cover this in the Saturday sessions and it's also Google recommended language. So that is why Kotlin is nowadays getting preference for Android development. But again, don't think that it's completely a different thing. No, you can think it as a brother of Java only, but a brother that is very easy to use, very lightweight, syntactically. Syntactically it's it's it's very simple. You not even have to define int float and all for any variable type. You can simply do var and val as a concept there in JavaScript. Same way it works, but it's a very simple language. Method declaration is very simple. Declaring variables are very simple. Inheritance is very simple. All those things. So you'll see as as as the code progresses, or if you already know, then it's well and good. You can understand the difference. But remember why Kotlin? Is very much now getting preferred for Android development in Java. So these are the certain reasons. So these are the very fundamental thing that you should be aware for your interviews also, as well as if you're working anywhere, then in your organizations also should know this. Any question till now? OK, what are the different components of an Android app? So when I ask component of Android app, we are talking about the first layer, that is the app layer. So we're talking about the app layer. We're not talking about the framework or something. We're just talking about the app layer. So what are the different component of our data? So we have activities. Activities where you see the buttons and all the UI. All those sit in an activity. Fragment is also same. There's a difference between an activity and a fragment. What are the difference? You will come to know. But fragment you can also think of something on which you can see the text, the buttons and all the UIs of the app. There are services. There are broadcast receivers. There are content providers. There are many other. I cannot add all those in the PPT, but these are the very main Android app components. This is all you need to learn, that what is an activity, how to use it,

how to program an activity. Similarly, what is a fragment, how to use it, what are the different aspects of a fragment, how to program it. Every activity in a fragment has life cycle. So to understand what is the life cycle in an activity, why you should know this, what happens at every life cycle. Same for services, why we use services, what are broadcast receivers, how it helps. So all this is what you will be learning as part of your Android code, how to write this, how to implement this and all. So there's a very important components of an Android app. And all your Android development will be centered around these things only. So these are the only things that you need to master. Any question here, anyone? Any question? And then your course has not yet progressed, it's slowly progressing. So once you come to know this thing, but yeah, make sure you attend the courses. If you don't attend the courses or missing, then you're not going to learn. So whatever theoretically you are taught, please make sure you are writing code for it to actually do the implementation, then only you will learn. So Android is nothing about practice. You can't memorize thing here, it's only about practice. Okay, for UI development, what we do in Android. So UI means the screen that you see, the buttons, the text boxes, videos, images in your app. So those are all termed as part of UI. UI is nothing but the visible components of an Android app. So how we develop UI? So we use XMLs, we use XML-based layouts, we use Jetpack Compose, we use material design, we use responsive UI. So there are certain things using which you can develop a UI for your app. So you'll see how to make XML-based development, Compose-based development, what's the material design, what are different themes and all those things. So all those will become part of a UI development. Any question anyone, till now? Any doubt till now? No. Anything any question? As I mentioned, there are certain life cycles that you need to know. For example, if you are working on activity or fragment, then they have a set of life cycles. Like when the activity is created, when the activity is resumed, when the activity is destroyed. So, you have to understand these life cycles thing whenever you are working with Android for activity and fragments. Especially, you also have to do state management. State management means what happens if you move from your one screen to another screen, then if you come back to the previous screen, what all you have to manage. Similarly, if you move out of your app, means if you click any web link from your app, it goes to a browser. Now, when you come back to your app from browser, then what should the state? All those things are part of the app life cycle and as a developer, it's very important that we properly define and implement the app life cycle logics. If you don't implement it properly, your app can behave in a very weird way. You will see something or get responses that is not expected. That is why app life cycle plays a very, very important part. You come to know when you study activity about the different life cycle like onCreate, onResume, onPause, onDestroy. These are the various life cycle of an activity. Similarly, they have different life cycles of a fragment. Each life cycle gets triggered when a specific thing happens. For example, if you move your app to the background state or you press the home button in your mobile phone and all your app goes to the background state, then onPause is called. All life cycle gets triggered. Similarly, when your app comes to the foreground, you are able to see it and onResume gets triggered. So you'll come to see how all those things happen once you learn activities, fragments and everything. Configuration change is again very important. If you rotate your app in landscape mode, you rotate your app in portrait mode, how the configuration needs to be changed, any rescaling needs to happen, any resizing needs to happen. All these you need to handle as part of the configuration changes. So these are the app life cycle that plays a very important role. So that you should be aware as an Android developer. I'm just giving you all the outlines for the Android team and all the important things there. Details will definitely come up in the actual course and as in our coming Saturday sessions. In data

management, you have to store data in the app, right? So there are various tools available. Android provides a lot of ways to store those data in your app. You share preferences. It's nothing but a key value pair. We use room databases, we use data store, we use offline first concepts. A lot of things are there that Android provides which you can use to process the data and show them in a proper way to the UI to the user. These are the certain data management things that your Android application provides. Any question till now? Similar to networking, since you are creating an app, you can talk to some external services, any external server hosted thing. Then Android provides a lot of things by which you can do networking like REST APIs are there, Retrofit is there, there are JSON passing methods, there is error handling, security. So everything is there which you can very well leverage and make an external call to any API or any website to show in your app. Any question on this one? No, again, I am not. These are not the only thing. I have only jotted the important fundamental ones. There are a lot of things that you can do. But yeah, these are the certain basics. Any question? Okay, since Java is there, as I told you, there is a fundamental programming, threadings are there. So see, all your Android UI is a part of main thread. So you have to do, if you want to do any processing like making an external call, talking to a database, you have to switch to a background thread. You cannot do all those in a main thread. You will see when you actually as the course progresses. So that thread switching and all, you have to take care of concurrency, coroutines, background work. So if you're talking to some services, downloading some huge data, then work manager comes into picture. If you want to do some background computation that should not be visible on the UI, then there's a background workers that are there. So all these are part of the concurrency. So concurrency is whenever there are more than one thread involved. for some time for doing any work or your main thread is in a waiting state for some work to completed, your app will go into a non-responding state, that is called ANR, app not responding. So whenever you need to do any heavy work like a network call, a DB call, some background processing, you create a background thread and then do all those things on that background thread and then come back on the main thread. So for these things, you will be working on more than one thread. One will be your main thread, one will be your background thread. So concurrency will come into picture because there are multi-threads. So how to work on them, how to ensure concurrency, how to do the background thread, so that is where Android provides lot of things. So you will see as gradually the course progresses. So concurrency is a major factor here. There are certain Jetpack libraries that help you, that you will definitely very much use in your applications like navigation, view models, live data, state flows, work manager, room DB, all those things. View model is to how you can bind your UI to the data. So whenever there is change in data, you can pass it easily to your UI and you can show it that happens to view model. Live data is whenever there is any change in data. For example, you are calling a database and there are some changes in the data, then how you can very quickly propagate that data to the UI and user can see this. So these are the libraries that help them. Definitely, you will come to know as the course progresses, but you should know there is something called a view model, there is something called live data, state flows, work manager and all. Any questions? Anyone? Okay. These are certain architecture patterns, you know what are solid principles, you might be knowing. You also know what are different type of design patterns, like Singleton design pattern, FACAD design pattern. So in the same way, there are certain architecture patterns for Android, like MVC, Model View Controller, MVP, MVVM, Clean Architecture. So we'll discuss on this what are these architecture patterns and in which scenarios, what is preferred and what are, how components are structured when we follow these architecture patterns. So these you need to know that Android, we have MVC, MVP, MVVM. Whenever you are doing an interview,

definitely expect a question around these things, especially if you are trying to interview for Meta, Google, Microsoft, and you are going to interview for an Android developer, they will ask you what is the architecture pattern, they will give you some scenario that you have to develop this type of app, see the scenario. And then they will ask you the different components, how will you develop them, how will you get the data, how will you show the data, how, what architecture pattern you will be following. So there, they definitely will come into these things, it's a very important thing. It will check your designing ability, like how you can design it. Again, I'm telling coding is not the only part as a developer, how holistically you are thinking is also very much judged in the interviews. So how you design a system, what patterns you follow, that is, we will be very much, you know, checked in your interviews. So Meta, if you are interviewing, definitely they'll ask this, they'll give you some scenarios and ask which pattern you will follow, why will you follow this pattern. They will not ask you to define an MVC pattern, they will not ask you to define an MVP pattern. But they'll give you some scenarios and they'll ask you how will you implement an app for this, write some basic code for it, some basic logic, and then what pattern you will use, how that pattern will fit in. So that answer you can only give if you are aware of the different patterns, you are aware of how, when different patterns are employed, what are the use cases of those different patterns. That is where it becomes very important. So these are the sort of the patterns that are very much used in any Android app development. Testing also, you should know how you test your app, there are unit tests, it's based on Java, unit test is something that Java provides, so definitely that will be part of your application also. There are certain framework for UI testing that is there, that also you should be knowing as part of testing. Then there's a mocking, mocking is again part of UI testing, then there are CICD gates and all those. So that is something partly that come under the testing, there are certain components will be part of your Android app testing module. So keep this thing also in mind. There is performance and security, this comes a very important thing in your Android app because your app is running on a device. And memory is fixed, you cannot keep on having as much memory as you want. So performance becomes a very important thing, you have to ensure there are no memory leaks in your app, you have to ensure there are no ANRs as I told you, application not responding. ProGuard is something we use in our app, so what it covers is de-obfuscates, so when I say it, it means that it de-obfuscates means whatever are not needed, it will simply remove it from your app memory, not take it up, so that your memory is not bloated. So for those we use ProGuard. If you are working with data, we are storing some data in our mobile application, we have to use encryption to ensure nobody is able to steal those data from your phone. Then there is something called OWASP mobile for security and secure connection, so that also we need to know these components. So these will all come under your performance and security. Very much emphasized thing, in an interview, if you are asked to implement any Android application, develop any Android application, the interviewer will definitely ask questions on how will you ensure these things, how will you ensure there is no memory leak, how will you ensure there is no ANRs, how will you ensure your data is safe, protected, all those things. So you should be aware of these things, so it's a very important thing again. Git I think, Git is not specific to Android, Git is a different code management repository, so you should be aware of Git. I think you will be working on Git only to create your projects, like what are the branching strategies, how you raise a PR, what are the Agile ceremonies and all, but again this is not specific to Android, but yeah, Git is nowadays a very much globally adapted change management system, so you should be aware. Good practices I think is your CI, CI is continuous integration, continuous development, so once you make the code changes, your code quickly builds, get deployed, observability is monitoring, logging, metricking, all those things come

part of observability, feature flags and also there are certain practices that we should have, we will discuss this, I think we will come to this during the last hours of the course, like what are the certain practices you should follow while developing a mobile application. Commentary topics at your level will remain focused on DSA, Android internal, Scotland, system design basics, so keep this in mind. System design DSA definitely will be there at your level, at Android it will be more around Android internals, some case-based studies and all, implementing some app based on some requirement, the Kotlin fundamentals and all, this is where it will be focused. This 30-60-90 day learning plan, first try to focus on the foundation of the Android, then the projects and finally the advanced architecture, so this is how you should go about planning, so first 30 days on foundation, next 30 days, this day do not go, it is as per your focus on the course that your BITS is providing you, this is something, there is a ratio there but mistakenly the PPT is 30-60-90, it should be 1 is to 2 is to 3, so fundamental followed by projects, followed by advanced architecture, so this is how you should try the learning your entire Android course. Final advice from my side will be to build projects, contribute to open source, if you are very fond of open source and you want to contribute and there are lot of things you can do in Android, try contributing, stay curious, learn continuously, most important So, that's all I had today from the slide angle, any question anyone? Since the fundamentals are only covered, so I also just retweeted what are the different components, what are all you should know, what does what, just a basic work over, once the course progresses, you will understand the internals of everything. So, they are just overview, that's it. Any question? Any doubt? I don't think you will be needing this PPT that I shared, it was just an overview, so it's fine, but if you want this, I can share to the, I can help provide this PPT, but there is nothing that you can refer. Any question? Cool name, utilize your time, focus on whatever has been covered, do an actual hands-on, if you don't do actual hands-on, it will be difficult for you to understand Android. Android is not like C++ or C language, but yeah, every language needs hands-on, but it's not like that, you have to do extensive hands-on, then only you can, because in interviews, they will give you some problem statement and will expect you to at least do some coding. Maybe they can give you a problem statement and give you an online editor and give you one hour and ask you to complete it. So, you can do that only if you have done substantial practice, you cannot fumble in interview, right? So, yeah, please practice as much as possible. No question anyone? Cool, I think we're done with today's session. Thank you all and see you all in the next session that will happen on next Saturday. Thank you.

Overview of Android UI Components Video• . Duration: 8 minutes

[Notes]

Hello everyone, and welcome to this new module on mobile app development. I am Professor Chandrasekhar R.K. and in this session, we'll be discussing the building blocks of Android applications, that is, the UI components. Let us start by going over the learning objectives for this session. By the end of this session, you should be able to understand the basic Android UI components, identify and use common UI elements like TextView, Buttons, ImageView. Also explore different layouts in Android such as LinearLayout, RelativeLayout and ConstraintLayout. Also learn about certain advanced UI components like RecyclerView and CardView. Also understand the creation of interaction components like CheckBox, RadioButton, SeekBar and Switch. These objectives will guide you through the process of building and customizing Android

user interfaces. Let's begin by understanding what Android UI components are. UI components are the building blocks of an Android application user interface. They are used to create interactive elements like Buttons, TextFields, Images and more. These components are organized in a hierarchical structure called a view hierarchy. The most basic UI components are Views, such as TextView, ImageView and Button. Views are the elements that users interact with directly. For example, a TextView displays text, an ImageView shows images and a Button triggers actions when clicked. Another important category is View Groups, which are containers that hold and manage multiple views. Examples include LinearLayout, RelativeLayout and ConstraintLayout. These view groups help you organize your views in a structured manner, making your UI more manageable and scalable. Now that we have an overview of UI components, let's look at some of the most commonly used ones.

EditText This component allows users to input text. It's commonly used in forms where users need to enter data. Its key properties include Hint, which provides a placeholder text. InputType, which defines the type of input, like a text or a number. And we could even have TextColor, which sets the color of the input text.

TextView TextView is used to display text on the screen. You can customize it using properties like text for the content, text size for the font size and gravity to align the text within the view. A button triggers an action when clicked. You can set the button's label using the text property and handle clicks using the onClick property in XML or programmatically with an onClick listener in code.

ImageView ImageView displays images in your application. You can control the image source using the src property and scale type allows you to define how the image should be scaled within the view. Layouts are critical in defining the structure of your UI. They determine how UI components are arranged on the screen. The first one is the LinearLayout. This layout aligns its children in a single direction, either vertically or horizontally. You can control the orientation using the orientation property and adjust alignment with gravity. Second one is the RelativeLayout. This layout positions its children relative to each other or to the parent container. Properties like LayoutAlignParentTop or LayoutCenterInParent help you achieve complex positioning with ease. Then we have ConstraintLayout. ConstraintLayout is a flexible and efficient layout for creating complex UIs. It allows you to position and size UI components relative to each other using constraints. Properties like ConstraintTopToTopOf and LayoutConstraintHorizontalBias are used to define these relationships.

Moving on to Advanced UI Components. First one let's see as to what is RecyclerView. RecyclerView is typically used for displaying large lists or grids of data efficiently. It requires an adapter to bind data to the views and a view holder to hold the layout for each item. Then we have the CardView. CardView provides a card-like UI with rounded corners and shadows. It is often used in combination with RecyclerView to display data in a visually appealing way. Then we have the Toolbar. The Toolbar is a customizable action bar that can include navigation, branding, and action items. It replaces the traditional action bar and gives you more control over the app's top bar. Finally, let's discuss User Interaction Components. First one being the Checkbox. A checkbox allows users to select multiple options from a set. It's commonly used in forms where users can choose one or more preferences. Then we have the RadioButton and this component allows users to select one option from a set. RadioButtons are usually grouped in RadioGroup, ensuring that only one option can be selected at a time. Then we have the SeekBar that lets users select a value from a range by sliding a thumb along a bar. It's useful for settings like volume control. The last one is the Switch. A switch is a toggle button that represents an on-off state. It is similar to a checkbox but provides a more modern look for toggling settings. In Android UI design, themes and styles define the overall look and feel of an app, which can be applied globally or to specific UI components. Drawables are graphics like backgrounds,

buttons, or icons with types such as bitmap, shape, layer, and state. Lastly, custom views allow you to create unique UI components by extending existing view classes where you can override methods like `onDraw` and `onMeasure` to implement custom behavior, making your app's interface more dynamic and tailored to your needs. To summarize, in this session, we explored the fundamental Android UI components including basic views, different layouts, advanced UI elements like `RecyclerView` and `CardView`, and user interaction components. Understanding these components is essential for building effective and user-friendly Android applications. By mastering these elements, you'll be able to create rich and interactive UIs that enhance the user experience. Thank you for your attention throughout this session. I hope now you have a better understanding of Android UI components and how to use them in your projects. Thank you.

Using XML to Define Layouts Video• . Duration: 8 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we'll explore the basics of XML layouts, understand the structure of an XML file, and look at some of the common and advanced layout types. Let's start by outlining the learning objectives for this session. By the end of this session, you should be able to understand the basics of XML layouts in Android, describe the structure of XML layout file, identify and use commonly used layouts like linear layout and relative layout, explore advanced layout like constraint and frame layout, and finally, implement the best practices for designing XML layouts. These objectives will guide you through the fundamentals of using XML to design user interfaces, ensuring your layouts are both effective and optimized. Let's begin by understanding what XML layouts are. In Android, XML is used to define the user interface layout for your applications. It provides a clear and structured way to organize UI elements, separating the visual design from underlying business logic. Each XML file represents a single screen or a part of the user interface. By defining your UI in XML, you benefit from certain key advantages like separation of concerns, that is, the user interface is defined separately from the code, making it easier to manage and update, and reusability, that is, the components defined in XML can be reused across different parts of the application, saving time and effort. And finally, we have the readability, which is, XML layouts provide clear and easy to visualize structure, which simplifies the process of designing and modifying the UI. These benefits make XML an excellent tool in Android development, enabling you to create clean and maintainable user interfaces. Let's explore the structure of XML layout file. An XML layout file in Android is composed of several elements. The first one being the root element. This root element is the base container that defines the overall structure of the layout. Common root elements include linear layout, relative layout, and constraint layout. Then we have the view elements. These are the child elements inside the root that define the actual UI components, such as text view, image view, button, etc. Each of these elements represents a UI component that will be displayed on the screen. Then we have the attributes. Each view element in XML can have multiple attributes that define the properties such as size, color, alignment, and more. For example, we have `android:layoutWidth` equal to `BRAP underscore content`, which sets the width of the view based on its content. And we also can have another attribute, which is a text attribute, where I can assign a string to the text attribute, which will set the text of the text view. Understanding the structure of the XML layout file is crucial for creating well-organized and functional user interfaces. Let's now look at some of the most commonly used

layouts in Android. The very first one among them is the linear layout. This layout arranges its child views in a single direction, either vertically or horizontally. You can control the alignment and distribution of space using attributes like orientation, gravity, and weight sum. Then we have the relative layout. This layout allows you to position child views relative to each other or to the parent container. For example, you can use layout align parent top to align a view with the top of the parent or layout underscore below to position a view below another. These layouts are foundational in Android development, enabling you to organize your UI elements in a clear and structured manner. Now let's explore some advanced layouts that provide greater flexibility. First one is the constraint layout. This is a powerful and flexible layout that allows for complex positioning of views without the need for nested layouts. It reduces the need for deep nesting, which can improve performance. Attributes like to start off, to end off, and constraint horizontal bias allow precise control over placement and alignment of UI elements. Then we have the frame layout, which is designed to hold a single child view, allowing you to stack views on top of each other. It's commonly used for displaying a single item or overlaying views, such as placing a text label over an image. By mastering these advanced layouts, you can create sophisticated and responsive user interfaces that enhance the user experience. Finally, let's discuss some best practices for designing XML layout. First thing in this is to avoid deep nesting. Deeply nested layouts can lead to performance issues. Instead, use constraint layout or flatten the layout hierarchy to improve efficiency and maintainability. The next one is to use wrap underscore content and match parent wisely. The wrap content attribute adjusts the size of view based on its content, while match parent makes the view fill the available space of its parent. Using these attributes correctly help maintain a balanced and responsive layout. Continuing with the best practices, utilize styles and themes, which define common styles and themes to maintain consistency across your application and reduce redundancy in XML files. Then to have separate layout resources, use different XML files for different screen sizes and orientations. For example, use the folder res-layout for portrait mode and res-layout-land for landscape mode. This ensures your apps adapt well to different devices. Following these best practices will help you create efficient, scalable, and maintainable Android layouts. To summarize, XML layouts are vital part of Android development, providing a clear and structured way to design user interfaces. We discussed the structure of XML layout files, explored commonly used layouts like linear, relative layout, and delved into advanced layouts like constraint layout, frame layout. You have also covered the best practices. By applying these concepts, you should be able to create professional and user-friendly interfaces for your Android applications. Thank you for your attention throughout this session. I hope now you have a better understanding of how to use XML layouts to create effective and visually appealing user interfaces. Thank you.

Linear Layout Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss one of the most commonly used layouts in Android, that is the linear layout. Let's begin by outlining the learning objectives for this session. By the end of this session, you should be able to understand what linear layout is and why it is used in Android development. Identify the two types of linear layout orientation, vertical and horizontal. Also explore the main attributes of linear layout. And also apply these concepts to create effective and responsive user interface layouts. These

objectives will guide you through the process of mastering linear layout, ensuring that you can use it effectively in your projects. Let's start with the basics. A linear layout is a simple and commonly used layout in Android development. It organizes its child elements in a single direction, either vertically or horizontally. This means that all the child elements within a linear layout are arranged one after the another in a line. Linear layout is particularly useful when you want to stack UI components in a specific order such as placing text fields on top of each other or aligning buttons side by side. The simplicity of linear layout makes it easy to use, but it also provides flexibility through various attributes that allow you to customize the arrangement and appearance of the elements within the layout. Linear layouts can be oriented in two ways, vertically and horizontally. In vertical linear layout orientation, all child elements are arranged vertically one below the other. This is useful for creating forms, lists or any other UI elements which need to be stacked vertically. In horizontal linear layout orientation, all child elements are arranged horizontally side by side. This is ideal for creating toolbars, rows of buttons or any UI where elements need to be placed next to each other. To set the orientation of a linear layout, you can use Android orientation attribute in the XML layout file. The values for this attribute can either be vertical or horizontal depending on the desired layout. Now let's explore some of the main attributes that you can use to control the appearance and behavior of a linear layout. First one is the orientation. As mentioned earlier, the orientation attribute determines whether the child elements are arranged vertically or horizontally. Then we have the gravity attribute. The gravity attribute controls the alignment of all child elements within the linear layout. You can use the values like start, end, left, right, center, top and bottom to specify where the elements should be positioned. Then we have the weight attribute. The layout underscore weight attribute allows you to specify the relative importance of each child element within the particular linear layout. This attribute is particularly useful when you want to distribute the available space among the child elements proportionally. For example, setting layout weight to 1 for two buttons will ensure they each take up an equal amount of space in the layout. Then we have the weight sum attribute. The weight sum attribute is the total of all layout weight elements assigned to the child. It helps ensure that the space distribution is balanced according to the specified weights. If you define the layout weight for multiple elements, using the weight sum ensures that the total weight adds up correctly. To summarize, linear layout is a fundamental layout in Android that allows you to organize UI elements in a simple linear fashion. We have discussed two types of orientation, vertical and horizontal, and explored different attributes such as orientation, gravity, weight, weight sum. By learning these attributes, you can create responsive and well-structured layouts that enhance the user experience in your Android applications. Thank you for your attention throughout this session. I hope you have a better understanding of how to use linear layout in Android to create effective and organized user interfaces. Thank you.

Relative Layout Video• . Duration: 6 minutes

[Notes]

Hello everyone. Welcome to the session on mobile app development. In this session, we'll explore one of the most versatile layouts in Android Relative Layout. Let's start by discussing the learning objectives for this session. By the end of this session, you should be able to understand the basic concept of relative layout. Use key properties to align views with the parent container Position views relative to sibling views manage overlapping views effectively. These objectives will guide you

through the process of mastering relative layout, ensuring that you can use it to create dynamic and responsive layouts in your Android applications. Relative Layout is a type of ViewGroup that allows you to position its child views to relative to each other or parent container. This flexibility makes it ideal for creating complex UI designs without requiring nested layouts, which can slow down performance. One of the key advantages of Relative Layout is that it offers multiple ways to align and position views, giving you control over how elements are arranged on the screen. For example, you can align a view with the top, bottom, left or right edge of the parent container using properties like Align parent top, Align parent Bottom, Align parent Left, or Align parent Right. This allows you to create layouts that adapt to different screen sizes and orientations. Let's get into aligning views with parent container. In Relative Layout, you can use several properties to position a view relative to the edges of the parent container. Like we have alignParentTop, which aligns the view with the top edge of the parent, or alignParentBottom, which aligns the view to the bottom edge of the parent. Then we have alignParentLeft or alignParentRight, which aligns the view to the left or the right of the edge of the parent. These properties are particularly useful when you want to anchor a view to a specific edge of the screen, ensuring that it remains in place regardless of other elements which are around it. Next, let's discuss how to position views relative to each other, which is one of the core features of RelativeLayout. You can use several properties to control the positioning of a view in relation to its sibling views. Like we have layout_below, which positions a view below another view. Layout_above positions a view above another view. Or you have to left of or to start of which positions a view to the left or start of another view. Likewise, we also have to write off or to end of which positions a view to the right or to the end of another view. These properties give you flexibility to create layouts where elements are dynamically positioned based on placement of other elements. This is particularly useful in responsive designs where the layout needs to adapt to different screen sizes and orientations. Here you see the basic usage of a relative layout. Here you see relative positioning to a sibling view using layout_below or layout_above or to leftOf, etc. One of the unique features of relative layout is the ability to overlap views, which can be useful for creating layered effects or complex designs. You can overlap views by positioning them in the same space using properties like Layout_alignTop or Layout_alignBottom, or it could even be align left or Align right. For example, if you want two views to overlap partially, you can align one view with the top and left edges of the parent and then position the second view slightly offset from the same edges. This creates a visually interesting effect where the two views appear to be stacked on top of each other. Overlapping views is a powerful tactic, but it's important to use it carefully to ensure that your layout remains user friendly and accessible. To summarize, Relative Layout is a highly flexible and powerful tool in Android that allows you to position UI components relative to each other or parent container. We discussed how to align views with the parent container using various different properties. And also how to position views related to sibling using properties like layoutbelow or layouttowriteoff and how to create overlapping views for complex designs. By mastering these techniques, you can create dynamic and responsive layouts that can be both efficient and visually appealing. Thank you for your attention throughout this session. I hope you have a better understanding of using the relative layout. I hope you now have a better understanding of how to use relative layout in Android. Thank you.

Introduction to Activities Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss one of the most crucial components of Android application activities. An activity represents a single screen with a user interface and acts as an entry point for user interactions. In this session, we will explore what an activity is, the roles of activities in an Android app, the concept of the activity stack, and the different types of activities you can use. By the end of this session, you will have a solid understanding of how activities work and how they contribute to the functionality of Android application. Let's start by understanding what an activity is. In Android, an activity is a crucial component that represents a single screen with a user interface. Every time a user interacts with your app, they are working within an activity. For example, when you open an app and see the home screen, that screen is an activity. Similarly, when you navigate to the settings screen or a detailed view, each of these screens is also an activity. An activity serves as an entry point for user interactions with your app's user interface. It's where the UI is drawn and where the user can perform tasks like clicking buttons, entering text or scrolling through content. Each activity is associated with a window that displays the UI and Android manages these windows to provide a seamless experience for the user. Now let's discuss the role of activities in an Android application. Activities are the primary building blocks of any Android app, allowing users to interact with different parts of the application. For example, in a typical app, you might have a main activity that serves as a home screen, a settings activity for app settings, and a detail activity for viewing detailed information. Activities are loosely bound to each other, meaning they operate independently but can start and interact with each other. Typically, one activity in an application is specified as the main activity. This main activity is what the user sees first when the app is launched. From this main activity, the app can start other activities as needed, allowing the user to navigate through different parts of the app. The Android system manages activities using an activity stack, which functions like a stack of cards. Each time a new activity starts, it is pushed onto the top of the stack and becomes the running activity. The previous activity remains in the stack below it and will not come to the foreground again until the top activity exists. This stack mechanism is essential for managing the user's navigation through the app. For instance, when the user presses the back button, the current activity is popped off the stack and the previous activity in the stack resumes. Understanding the activity stack is crucial for managing how your app responds to user actions and system events. Let's now explore the different types of activities that you can create in Android. When starting a new project in Android Studio, you often choose a project template that includes one or more predefined activities. These templates help you quickly set up common types of activities with pre-configured settings. Some of the common types of activities include No Activity, which starts with a blank canvas where you manually add everything. Empty Activity, provides a minimal setup with just the basics allowing you to customize it fully. Basic Views Activity, includes basic UI elements like buttons and text fields suitable for simple apps. Bottom Navigation Activity, it provides a template with bottom navigation bars ideal for apps with multiple sections. Then we have Navigation Drawer Activity, which includes a navigation drawer that slides in from the side, commonly used for apps with many options or settings. Choosing the right type of activity depends on the specific needs of your app and the kind of user experience you want to create. To summarize, activities are fundamental component of Android applications, representing individual screens with which users interact. We discussed what an activity is, its role in managing the user interface, the concept of activity stack and the different types of activities you can use. By understanding and effectively managing activities, you can create smooth and intuitive navigation experiences in your Android applications. Thank you for your attention throughout this session. I

hope you now have a better understanding of activities in Android. Thank you.

Creating an Activity Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss the process of creating an activity in Android. An activity is a fundamental component in Android that represents a single screen with a user interface. Let's start by outlining the learning objectives for this session. By the end of this session, you should be able to understand what an activity is and its purpose in Android development. Learn the steps to create a new activity in Android Studio. Understand how to declare an activity in AndroidManifest.xml file. These objectives will guide you through the essential steps of creating and managing activities, ensuring that you can effectively use them in your Android projects. Let's begin by understanding what an activity is in Android again. An activity is a crucial component that represents a single screen with a user interface. It is the main entry point for user interactions within your application. Whenever you create a new project in Android Studio, a main activity is automatically generated, usually named MainActivity. The main activity is responsible for displaying content to the user and handling their interactions, such as clicking buttons or entering text. Multiple activities are used within an app to build a complete user experience, allowing the users to navigate between different screens and perform various tasks. The main activity in an Android application typically consists of two files. One is the layout file, which is named as activity-main.xml. This is the layout file where you define the user interface components, such as buttons, text fields, and images. It uses XML to structure the UI elements and determine how they appear on the screen. The next file is MainActivity.kt. This is the Kotlin file that contains the code for activity. It defines the behavior of the UI components, such as what happens when a button is clicked or when data is entered in the text field. These two files work together to create the interface and functionality of an activity, allowing the user to interact with your application. Now let's go through the steps to create a new activity in Android Studio. To create a new activity file, we have to first go to the Android Studio, right-click on the App folder, then select New, Activity, Empty Activity or Basic Activity. Then you have to name your activity by giving a name to it, and click on Finish. This will create a new activity with its associated layout file. The next step is to define the layout XML file. An XML file which is associated with the new activity is automatically generated in Android Studio. Here, you can define the layout for the new activity by adding UI components like text views, buttons, and images as needed. In the next step, we have to modify the Kotlin code. The basic code structure for the new activity is automatically generated in the activity file. You can modify this file to add functionality such as handling button clicks or retrieving user input. After creating a new activity, it's important to declare it in AndroidManifest.xml file. This file acts as a central registry for all the components in your app, such as activities, services, and broadcast receivers. In order to declare a new activity in AndroidManifest.xml file, first we have to open the AndroidManifest.xml file which is located in App, Source, Main directory. Provide the application tag and activity tag for the activity to be created. As shown in the snippet, you need to add the activity tag by giving the name to it. This declaration is necessary for Android system to recognize the activity and include it in the app. Most of the times, this declaration is not required because Android Studio automatically does this for you when you create an activity file. If the new activity is the first screen that users see when they launch your app, you need to specify it as the

launcher activity by adding an intent filter. Intent filter will help in identifying as to which activity is the main activity to start with. So you can see the intent filter which has the name `android.intent.action.main` present in the activity which will be the main activity. And it also needs to create a category tag where you need to mention it as `category.launcher` indicating that as soon as the app is launched, this is the first activity which should get displayed to the user. This ensures that your new activity is the entry point of your app. To summarize, creating an activity in Android involves several key steps. We discussed what an activity is and its role in Android application, explored the process of creating a new activity in Android Studio, and learned how to create and declare an activity in `AndroidManifest.xml` file. By understanding these steps, you can effectively manage activities in your Android projects, allowing users to interact with your app seamlessly. Thank you for your attention throughout this session. I hope you now have a better understanding of how to create and manage activities in Android. Thank you.

Introduction to Intent Video• . Duration: 7 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss a fundamental concept in Android development, which are intents. Intents are a key part of Android as they allow different components of an app to communicate with each other. Whether you are starting a new activity, a service, or sending a broadcast, intents make it possible. In this session, we will cover what an intent is, the roles of intents in Android, the structure of an intent, and the different types of intents available. By the end of this session, you will have a clear understanding of how to use intents effectively in your Android applications. Let's begin by understanding what an intent is. In Android, an intent is a messaging object used to request an action from another app component. For instance, you might use an intent to start a new activity, initiate a service, or send a broadcast to notify other components about an event. Intents act as a bridge between different components within an app and even between different apps. They make it possible for one activity to start another, for a service to be initiated in the background, or for a broadcast to be sent to the system or other applications. This flexibility makes intents one of the most powerful tools in Android development. Intents in Android are mainly used for three primary purposes. Let's look into each one of it. First one is the starting. An activity represents a screen through which a user interacts with the application. Using intents, you can start a new activity by calling the method `startActivity` with an intent object. This is one of the most common uses of intents, allowing users to navigate between different screens in an app. The next one is to start a service. A service is a process that runs in the background without a user interface. Intents can be used to start a service by passing the intent object to the `startService` method. This is useful for tasks like downloading files, playing music, or syncing data in the background. And the third is to deliver a broadcast. Broadcasts are used to notify users or other components about events such as low battery warning or change in network status. You can send broadcasts using `sendBroadcast` method with an intent object, ensuring that other components or apps are informed about the event. Now let's explore the structure of an intent. An intent primarily consists of two main pieces of information. The first one is the action. The action specifies the general task to be performed, such as `action.underscore.view`, `action.underscore.edit`, or `action.underscore.main`. It tells the system what kind of operation you want to perform, like viewing a webpage, editing a contact, launching the

main activity of an app. The next one is the data. The data component specifies what the action should be performed on, such as URI pointing to a contact in the contacts database. For example, action underscore view with content, content slash people slash one would display information about a specific contact in the system. In the examples given, intents allow various actions to be performed. For example, action underscore view with content contacts people slash one display the contacts details. Using action underscore dial with a URI like tell one two three opens the dialer with the number prefilled. Another use of action underscore view with tell one two three also opens the dialer with the number ready. Finally, action underscore edit with content contacts slash people slash one allows editing the contacts information. Now, these examples show how intents can handle different actions efficiently based on the URI provided. In addition to these primary components, intents can also include a category that is the additional information about the action to be executed, such as category underscore launcher, which we have used in the AndroidManifest.xml file. This is used to launch an app from the home screen. The next one is a type. The main type of intent data like image or a mp3 file. Component the explicit name of the component class to handle the intent. The last one is extras where a bundle of additional information that provides extended data to the component handling the intent. Finally, let's discuss the two types of intents in Android. Explicit intents and implicit intents. Explicit intents specify the exact component to start by name. They are typically used to start activities within the same application. For example, you can use an explicit intent to start a new activity within your app by specifying the class name of the activity you want to start. Implicit intents do not name a specific component, but instead declare a general action to perform. The system then matches this action with a component that can handle it. For example, to show a location on a map, you can use an implicit intent and the system will find the appropriate app like Google Maps to handle this request. Understanding the difference between explicit and implicit intents is very important for effective Android development as it determines how components within and outside of your app interact with each other. To summarize, intents are a fundamental part of Android development, allowing different components within and across apps to communicate and perform tasks. We discussed what intents are, their roles in starting activities, services, and delivering broadcasts, and the structure of an intent, and the difference between explicit and implicit intents. By mastering intents, you can create more dynamic and interactive Android applications that provide a seamless user experience. Thank you for your attention throughout this session. I hope you now have a better understanding of intents in Android and how to use them effectively in your projects. Thank you.

Implicit Intents Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will be discussing implicit intents in Android. Intents are a fundamental part of Android development, enabling different components of an app to communicate with each other and with other apps. Implicit intents are particularly useful when you want to perform an action without specifying the exact component that should handle it. In this session, we will cover what an implicit intent is, its basic structure, and common use cases such as opening a web page or sending an email. By the end of this session, you will understand how to use implicit intents effectively in your Android applications. Let's start with an understanding what an implicit intent is. An implicit intent is used

when you want to perform an action in your app without specifying the exact component, like an activity or a service, that should handle it. Instead, the Android system determines the best component to perform the action based on the intent's data and category. For example, if you want to open a web page, you don't need to specify which browser app should handle the request. The system will present the user with a choice of apps that can open web pages, allowing them to select their preferred browser. This flexibility makes implicit intents a powerful tool for integrating your app with other apps and services on the device. Implicit intents are used in a variety of scenarios where you want your app to interact with other apps or components. Some common use cases include

Opening a web page. You can use an implicit intent to open a web page in a browser without specifying which browser to use, or it could be sending an email or SMS. Implicit intents allow you to send an email or SMS through users' preferred email or messaging app, viewing a location on a map. You can use an implicit intent to show a location on a map, letting the user choose their preferred mapping app, or it could even be for sharing content. You can share content like text or images with other apps using an implicit intent, allowing the user to select how they want to share the data. These examples illustrate how implicit intents enable your app to provide a seamless user experience by leveraging the capabilities of other apps on the device. Now let's look at a basic structure of an implicit intent. Creating an implicit intent involves specifying the action you want to perform and optionally the data associated with that action. In the example, you can see that an intent is being used for action underscore view, and it also specifies as to what is the intent data where a URI is parsed, and then we call the method startActivity. In this example, action underscore view is the action that tells the system to display data and URI.parse with the URI converts the URL string to into a URI object, the startActivity method that launches the appropriate app to handle this action, which in this case is a web browser. Let's explore our first example, which is opening a web page using an implicit intent. The code for this action is in Kotlin. So you have an intent constructor being called with intent dot action underscore view. Then you have an intent data which is presented with the URL as part of the parameter. Then we call the method startActivity. When you call startActivity with the intent, the Android system will prompt the user to choose a browser to open the web page. This is a simple yet powerful way to integrate web content into your app without having to manage the browser indirectly. Now let's look at another example, sending an email using an implicit intent. The snapshot presents you the code in Kotlin, where we use the intent action underscore sendTo and the intent data is the mail ID of the person or the recipient. And we also put certain other extra elements into the intent object, like what is a subject? What is the email message? And then finally, we call the method startActivity with the intent object. In this code, intent action underscore sendTo specifies that the action is to send data to a specific recipient and the parse will have a email ID. This ensures that only email apps handle this intent. The putExtra method is used to add additional data, such as the subject, message text, etc. When the startActivity intent method is called, Android system will open the user's preferred email app with the recipient's email address, subject, and message already filled in. To summarize, implicit intents are a crucial part of Android development, allowing your app to interact with other apps on the device without specifying the exact component to handle the action. We discussed what an implicit intent is, its basic structure, and a common use case, such as opening a web page or sending an email. By mastering implicit intents, you can create more flexible and user-friendly Android applications. Thank you for your attention throughout this session. I hope you now have a better understanding of implicit intents. Thank you.

Intent filters Video• . Duration: 8 minutes

[Notes]

Hello everyone, welcome to this session on mobile app development. In this session, we will discuss a key concept in Android development, intent filters. Intent filters play an essential role in Android, allowing your app to handle specific types of intents, especially implicit intents. They enable the Android system to determine which components in your app can respond to particular actions or data. In this session, we will cover what an intent filter is, its structure, the actions, categories it supports, and how to implement it in your app. By the end of this session, you will understand how to use intent filters to make your Android applications more dynamic and interact. So let's begin by understanding what an intent filter is. An intent filter is a declaration in the `AndroidManifest.xml` file that specifies the types of intents an activity, service, or broadcast receiver can respond to. It acts as a filter that tells the Android system which components are capable of handling a particular intent, especially implicit intents. For example, if your app can handle viewing images, you can declare an intent filter that allows the system to open your app whenever a user wants to view an image. This way, your app becomes an option for users when they perform certain actions, making it more integrated with the system and other apps. Intent filters allow your app to handle specific actions, such as viewing a webpage, editing a contact, or sending a message. They define which components in your app can respond to these actions. The second one being defining which activities can be started with implicit intents. By declaring intent filters, you can make certain activities in your app available to handle implicit intents from other apps or the system. This is especially useful for creating apps that interact seamlessly with other apps on the device. Now, let's explore the structure of an intent filter. An intent filter consists of three main components. The first one being action. This element specifies the general action to be performed, such as viewing or editing data. Common actions include `intent.action.view` and `intent.action.edit`. Now let us look into the category. This element provides additional information about the type of component that should handle the intent. For example, we have `android.intent.category.default`, which is required for any activity that has to be launched with an implicit intent. Then we have the third one, which is the data. This element defines the type of data the component can handle, such as MIME types or URI schemes, hosts, paths, etc. For instance, you can specify that your app can handle `image/*`, basically indicating that it can open any type of image files. By combining these components, you can create powerful intent filters that determine how your app interacts with other apps on the Android system. Actions are a crucial part of intent filters as they define what your app can do. Some of the common actions include `intent.action.view`, which is used to view data, such as opening a web page or viewing a contact. Likewise, we have `intent.action.edit`, which allows editing of data, like modifying a contact detail. We also have `intent.action.send`, which enables sending data, such as sharing text or images with another app. Then we also have `intent.action.main`, which designates the main entry of an app typically used in launcher activities. These actions allow your app to perform specific tasks and respond to user requests effectively. Now let us look into the categories. Categories provide additional context for actions defined in the intent filters. Some common categories include `intent.category.default`. This must be included in filters for activities that can be launched with implicit intents. Then we have `intent.category.launcher`, that specifies that an activity should appear in app launcher. Then we have `intent.category.browsable`, which allows an activity to be launched from a web browser. And the final, `intent.category.alternative`, provides alternative actions that the user can take within the app. Categories help refine the types of intents your app can handle, making it more versatile and user-friendly. Now let us look into the data part, in intent

filters. Data in intent filters defines the type of data your app can handle. This can include MIME types, URI schemes, hosts, or paths. First let us look into the MIME types. MIME types define the type of data such as image-star for images or text-plane for text files. URI schemes, it specifies the URI scheme such as HTTP or TEL, indicating the type of data your app can work with. Then you have hosts or paths. Further refine the type of data by specifying the host and path in an URI. By defining data in intent filters, you can ensure that your app is capable of handling the specific types of content it is designed for. Here you see the androidmanifest.xml file, which has the intent filters defined for launching the activity and to send an email. We also see that there is a main activity.kt file, an intent object is created, and extra data is added for mail id, subject, and text. And finally, startActivity method is called with intent object, after which the android system will choose a preferred app to complete the function. Here on click of the button send, the system presents a list of apps which can handle the implicit intent. To summarize, intent filters are so powerful feature in android that allows your app to handle specific types of actions, data, and intents. We discussed the structure of a intent filter, including actions, categories, and data, and how these elements work together to define the capabilities of your app. By implementing intent filters, you can make your app more dynamic, allowing it to interact seamlessly with android system and other apps. Thank you for your attention throughout this session. I hope you now have better understanding of intent filters in android. Thank you.

Starting an Activity with Explicit Intent Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will be discussing explicit intents in Android. Explicit intents are a fundamental concept in Android development, allowing you to start a specific activity or service within your application. Whether you want to navigate between screens or pass data between one activity to another, explicit intents make it possible. In this session, we will cover the basics of explicit intents, how to create them and use them, and how to pass and retrieve data between activities. By the end of this session, you will have a solid understanding of how to use explicit intents effectively in your Android applications. Let's start by understanding what an explicit intent is. An explicit intent is used to start a specific activity or service within the same application or between known applications. Unlike implicit intents, which do not specify the target component, explicit intents directly specify the target component that should handle the intent. This makes explicit intents ideal for navigating between different screens or starting services within your app. For example, if you have two activities in your app, main activity and second activity, you can use an explicit intent to navigate from main activity to second activity. This ensures that user is taken to the exact screen you want them to see. Explicit intents are commonly used in several scenarios. Let us look into a few of them. First one being navigating between activities. When you want to move from one screen to another within your app, you use an explicit intent to start the new activity. Second, you can start a service. If your app has background services that needs to be started, you can use explicit intent in order to initiate the service. Third one is to communicate with specific component. When you need to interact with a particular component with another app that you have control over, an explicit intent is used. Now these scenarios highlight the importance of explicit intents in creating a seamless and controlled user experience within your app. Now let's take a look at how to create an explicit intent. The basic structure of an explicit intent

is straightforward. We need to create the intent object will take two parameters. One is the reference. Second one is the target class file. And then you call the start activity method with the intent object. In this example, intent this comma second activity class Java specifies that you want to start second activity from the current context. That is this. The start activity intent method is then used to initiate the second activity. The simple code separate demonstrates how you can navigate between different screens in your app using explicit intents. One of the powerful features of explicit intents is the ability to pass data between activities. This is done using put extra method, which allows you to add key value pairs to the intent that can be accessed by the target activity. Here is an example where put extra method is actually used to have a key value pair. Key is named as extra key and value is some data. And finally, we call the start method with the intent object. In this particular code, put extra method adds the data to the intent. The key, which is extra underscore key is used to retrieve this data in the target activity. This is a common method for passing information like user input settings or other data between screens. So how do we retrieve data in the target activity after passing data with the explicit intent, the target activity needs to retrieve this data. This is done using the get intent method, along with methods like get string extra to extract the data. So this is how it works. So using the intent object, we call the method get string extra. And the parameter which we pass to it is the key value. Once we do this, it retrieves the string data associated with the key extra underscore key from the intent. This allows the target activity to use the past data as needed. Understanding how to pass and retrieve data between activities is essential for creating interactive and dynamic applications. Let's consider a practical example of navigating between two activities. Imagine you have an app with main activity and a second activity. You want to navigate from main activity to second activity when a button is clicked. So here is how you would do it. First, in main activity.kt, create an intent to start second activity when the button is clicked. Then we use the put extra to pass on any necessary data. In the second activity.kt file, we retrieve the data using get intent method and display it as needed. So this example demonstrates how explicit intents allow you to manage your navigation and data flow within your app effectively. To summarize, explicit intents are vital part of Android development, allowing you to navigate between different activities, start services, and pass data between components. We discussed what an explicit intent is, how to create one, how to pass data with put extra method, and how to retrieve data in the target activity. By mastering explicit intents, you can create more interactive and user-friendly applications. Thank you for your attention throughout this session. I hope you now have a better understanding of how to use explicit intents. Thank you.

Activity Navigation Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to the session on mobile app development. In this session, we will be discussing activity navigation in Android. Activity navigation refers to the process of moving from one activity to another within an Android application. This is a fundamental aspect of creating user-friendly applications, as it allows users to interact with different screens in a seamless manner. In this session, we will cover the basics of activity navigation, how to navigate between activities using intents, and managing the back stack for better user experience. By the end of this session, you will have a solid understanding of how to implement activity navigation effectively in your Android applications. So let's get to see as to what is activity navigation. Activity navigation is the

process that allows users to move from one screen to another within an Android application. This is crucial for providing a structured user experience, as it manages the flow of tasks within an app. Navigation can be linear, where the user moves forward and backward between activities, or non-linear, where the user jumps between activities based on their actions. Understanding how to implement navigation is essential for creating applications that are intuitive and easy to use. Navigation is just not about moving between screens. It's about managing the flow of your application in a way that feels natural to the user. Proper navigation helps in guiding the user through the app, ensuring they can complete tasks efficiently. Whether it is moving from a home screen to a settings page, or from login screen to the main dashboard, effective navigation enhances the overall user experience. It's an integral part of how users interact with your app and how they perceive its usability. Now let's talk about the basic way to navigate between activities using explicit intents. Explicit intents are used when you know exactly which activity you want to start. For example, if you want to move from main activity to detail activity, you would use an explicit intent by passing on the detail activity class file as one of the parameters while creating the intent object. In this example, the intent specifies the current context, this, and the target activity, which is the class file of detail activity. When you call start activity with the intent object, the system navigates to the specified activity. Often, you will need to pass data from one activity to another during navigation. And this can be done, as we have discussed earlier, using the putExtra method, which allows you to add data to the intent that will be accessed by the target activity. In this code, putExtra method takes in two parameters, one is the key, the other one is the value, which adds a piece of data to the intent. The target activity can then retrieve this data using getIntent method and getStringExtra method by passing on the key value. This is useful for passing information like user inputs, settings, or identifiers to the next screen. Another important aspect of activity navigation is managing the back stack. The back stack is a stack of activities the user has visited, allowing them to move back to previous screens using the back button. When you navigate from one activity to another, the current activity is pushed onto the back stack. Pressing the back button pops the current activity off the stack and the previous activity resumes. In some cases, you may want to start an activity without adding it to the back stack. This is common with splash screens or login screens that should not be revisited after navigation. To achieve this, you can call finishAfterStartActivity to remove the current activity from the stack. Here is a practical example of starting an activity without adding it to the back stack. This is useful in scenarios like splash screens where you don't want the user to return to the splash screen after they have moved to the main part of the app. In this code, the finish method is called immediately after startActivity method, ensuring that the current activity is removed from back stack and the user cannot navigate back to it. To summarize, activity navigation is a crucial part of Android development, allowing users to move seamlessly between different screens within your app. We discussed the concept of activity navigation, how to use explicit intents for basic navigation, passing data between activities, and managing back stack for smooth user experience. Thank you for your attention throughout. I hope now you have better understanding of how to implement activity navigation. Thank you.

Exchanging the Data from Activities Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will be discussing a crucial aspect of Android development, that is, exchanging data between activities. In Android, it's common to pass data between different activities, whether it's user input, selections, or results from operations. This ability to transfer data ensures smooth communication and interaction within your app. So, in this session, we will cover the basics of data exchange, how to pass data using intents, and explore advanced data passing techniques. By the end of this session, you will have a solid understanding of how to effectively exchange data between activities in your Android applications. Data exchange between activities is a fundamental concept in Android development. Often, you need to pass data from one activity to another, such as when a user makes a selection or a operation produces a result that needs to be returned. Android provides several methods to facilitate this data transfer, ensuring that your activities can communicate smoothly. Understanding how to exchange data between activities is key to create interactive and dynamic applications. So let's try to answer the question, why exchange data has to happen between activities? There are several scenarios where data exchange between activities is essential. First one is passing user input. For example, if a user enters data in one activity, you might need to pass that data to another activity for further processing or display. Second case could be for returning results. Sometimes you start an activity expecting it to return a result. For instance, you might start second activity to let the user select a contact and then return that contact's information to the first activity. Or it could also be for sharing complex objects. So in more advanced scenarios, you need to pass entire objects or collection of data between activities. These examples highlight the importance of understanding how to effectively pass data between activities to ensure smooth user experience. One of the simplest ways to pass data between activities is by using intents. Intents can carry data using the `putExtra` method, which allows you to attach key-value pairs to the intent. This method supports various data types, including primitives like `int`, `boolean`, `float`, `strings`, `arrays`, and even complex objects that implement the `Serializable` or `Parcelable` interfaces. Once the data is passed to the target activity, it needs to be retrieved and used. The `getIntent` method is used in the receiving activity to access the passed data. While `putExtra` method works well for simple data types, more complex scenarios require advanced techniques. One such technique is using bundles. Bundles are useful when you need to pass multiple pieces of data together or when passing objects that implement the `Parcelable` or `Serializable` interfaces. For example, if you need to pass multiple pieces of data, we create a bundle object and we add in key-value pairs to this particular bundle. And using the `putExtra` method, we add the bundle to the intent and we call the method `startActivity` to pass on the intent to the next one. So in the target activity, you can retrieve the data from the bundle by using the reference of bundle object and the respective methods like `getString` or `getInt` depending upon what data you are retrieving. Also using bundles, it allows you to organize and pass complex data efficiently between activities, making your app more versatile and capable of handling various user interactions. To summarize, data exchange between activities is a critical aspect of Android development that ensures smooth communication within your app. We discussed the importance of data exchange, how to pass data using intents and `putExtra` and explored advanced data passing techniques using bundles. By mastering these methods, you can create more dynamic and interactive Android applications that provide a seamless user experience. Thank you for your attention throughout this session. I hope you now have a better understanding of how to exchange the data between activities in Android. Thank you.

Activity Lifecycle Video• . Duration: 9 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss a fundamental concept in Android development, that is, the activity lifecycle. The activity lifecycle is crucial for managing how an activity moves through different states from creation to destruction. Understanding this lifecycle is key to developing apps that handle user interactions and system events efficiently. Let's begin by discussing the learning objectives for this session. By the end of this session, you should be able to understand the concept of activity lifecycle in Android, identify and describe the basic lifecycle methods, learn the role of onCreate and onDestroy methods, along with onStart, onStop, onResume, and onPause, and implement the best practices for managing activity lifecycle. So let's start by answering the question, what is the activity lifecycle? Let's try to understand it. The activity lifecycle refers to the sequence of states that an Android activity goes through from the time it is created until it is destroyed. The Android system manages this lifecycle, allowing your app to respond to various user actions and system events, such as incoming calls or screen rotations. By understanding and managing the activity lifecycle, you can ensure that your app behaves consistently and efficiently, handling transitions between different states in a way that maintains a smooth user experience. Understanding the activity lifecycle is important for several reasons. One of it is resource management. Properly managing the lifecycle helps in efficiently using system resources such as memory and CPU. This is especially important for mobile devices with limited resources. It is also important for user experience. Handling the lifecycle correctly ensures that your app transitions smoothly between different states, providing a consistent and pleasant experience for the user. And it also allows to prevent memory leaks. By understanding when to release resources, you can prevent memory leaks, which can slow down or crash your app. These points highlight why mastering the activity lifecycle is very important for developing robust Android applications. Let's now look at the basic methods involved in activity lifecycle. The first one being onCreate method. This method is called when the activity is first created. It's used to initialize components, set up user interface, and prepare any resources the activity needs. Then we have onStart method. This is called when activity becomes visible to the user, but not yet interactive. It is used to initialize resources needed when the activity is visible. Then you have the third one, which is onResume. This method is called when the activity is ready to interact with the user. You can use this method to resume any paused tasks or refresh the UI. And there is an onPause method as well, which is called when the system is about to start another activity. Then this method is use this method to pause ongoing tasks, save data, and stop animations. Then we have onStop method. This method is called when activity is no longer visible to the user. It's used to release resources that should not be kept while activity is not visible. Then we have onDestroy, which is called just before the activity is destroyed. We use this method to clean up resources and finalize any pending tasks. Then we also have onRestart, which is called after the activity has been stopped, just before it starts again. These methods provide the framework for managing how your activity responds to system events and user actions. Now let us focus on two critical lifecycle methods, that is onCreate and onDestroy. onCreate method is where you should perform all initial setup for your activity such as inflating the layout, binding data to lists, and setting up any necessary resources like database. It is the first method called when the activity is created, making it essential for initializing your app's UI and functionality. Likewise, the onDestroy method is called before the activity is destroyed. This is where you should clean up the resources, unregister receivers, and release any resources that are no longer needed. Properly managing this method

helps prevent memory leaks and ensures your app does not consume unnecessary resources after the activity is destroyed. Now let's take a look at `onStart` and `onStop` methods. The `onStart` method is called when the activity is becoming visible to the user. This is where you should initialize the resources that are needed when the activity is visible such as starting animations or refreshing UI elements. The `stop` method is called when the activity is no longer visible to the user. This is where you should release resources that are not needed while activity is not visible such as stopping animations or saving persistent data. These methods are key to managing visibility and resource usage of your activity. Finally, let's discuss `onResume` and `onPause`. `onResume` this method can be used when the activity starts interacting with the user. It's the right place to resume any paused tasks such as restarting a paused animation or refreshing the UI to display updated data. Then we have the `onPause` method which is called when the system is about to start another activity. Use this method to save any unsaved changes, stop ongoing processes, or pause animations. It's important to make sure that all critical data is saved here in case activity is terminated. These methods ensure that your activity responds appropriately to user actions and system events. Now let's talk about the best practices for managing activity lifecycle. To effectively manage the lifecycle, follow these best practices. First one is avoid heavy lifting in `onCreate` method. Keep `onCreate` efficient and light by loading heavy data or starting expensive tasks in the background using coroutines or async tasks. Then you can even save critical data in `onPause` method. We have to use `onPause` method to save any user data that might be lost if the activity is unexpectedly terminated. Then we have to even look into release of resources in `onStop`. Stop animations, threads, or other resources that consume CPU or memory in `onStop` to prevent unnecessary resource usage. Then to handle edge cases in lifecycle transitions. Ensure your app handles edge cases like unexpected shutdowns or interruptions gracefully by thoroughly testing lifecycle transitions. Finally, use `viewModels` and `LiveData`. Leverage the Android's architecture components such as `viewModel` and `LiveData` to manage UI-related data in lifecycle-conscious way that survives configuration changes. Following these practices will help you create Android applications that are robust, efficient, and provide a smooth user experience. To summarize, understanding and managing activity lifecycle is essential for creating efficient, user-friendly Android applications. So we have covered the basic lifecycle methods like `onCreate`, `onDestroy`, `onStart`, `onStop`, `onResume`, `onPause`. We also discussed best practices for managing these states. By mastering the activity lifecycle, you can create apps which are both responsive and resource-efficient. Thank you for your attention throughout this session. I hope now you have a better understanding of activity lifecycle in Android and how to manage it. Thank you.

Managing Activity States Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss an essential aspect of Android development, which is managing activity states. In this session, we will explore what activity states are, why managing them is important, and how to save and restore activity states effectively. By the end of this session, you will understand how to handle activity states to ensure your app runs smoothly and preserves user data. We have already seen as to what are activity states, let's quickly reiterate it. Activity states refer to the different stages an activity goes through during its life cycle. These stages include when an activity is created, running,

paused, stopped, and eventually destroyed. Each state is part of the activity's life cycle, which the Android system manages. Understanding these states is crucial because it allows you to write code that responds appropriately to changes, such as when a user navigates away from the activity or when the system reclaims resources. Properly managing activity states is essential for several reasons.

1. Smooth application performance. By correctly handling activity states, we can ensure that the app runs smoothly without unnecessary delays or crashes. And we could even conserve resources. Properly managing states help conserve system resources such as memory, processing power, which is essential in mobile devices. Finally, let's talk about user experience. Proper state management helps preserve user data, settings, providing a better user experience, particularly during configuration changes or when the app is interrupted by other tasks. So let's explore how to save and restore an activity's state when an activity is destroyed, such as during a configuration change like screen rotation, it loses all unsaved data. To prevent this, you can use the method `onSaveInstanceState`. This method is used to save the current state before activity is destroyed. This method allows you to save UI state such as user input, scroll positions, and selections. Later, when the activity is recreated, you can restore the saved state using `onCreate` method or `onRestoreInstanceState`. This ensures that your app resumes exactly where the user left off without losing any data or progress. Let's consider a practical example to illustrate the importance of saving and restoring activity states. Imagine you have a simple counter app where a button increments a counter each time it is clicked. However, if the screen orientation changes, the system by default recreates the activity, causing the counter to reset to zero. Without using `onSaveInstanceState` and `onRestoreInstanceState`, the counter loses its previous value after orientation changes. By implementing these methods, you can save the counter's state before activity is destroyed and restore it afterwards, which ensures that the counter continues from its previous value even after the screen is rotated. In the counter app example, here is how you can save and restore the state. So we can use the method `onSaveInstanceState` to save the current counter value before the activity is destroyed. So we can put the counter value using a key inside the bundle. Then we can use the method either `onCreate` or `onRestoreInstanceState` to retrieve this counter value and proceed. By using these methods, you can ensure that your app maintains its state across configuration changes, providing a consistent user experience. To summarize, managing activity states is a crucial aspect of Android development that ensures your app runs efficiently and provides a smooth user experience. So we discussed what activity states are, importance of managing these states and how to save and restore activity states. Thank you for your attention throughout this session. I hope you now have a better understanding of managing activity states in Android and how to implement these techniques in your projects. Thank you.

Constraint Layout with Button, EditText and TextView Video• .

Duration: 10 minutes

[Notes]

In this demo, we are going to see as to how we can work with `ConstraintLayout`. And to `ConstraintLayout`, we are going to add a few widgets like `EditText`, `TextView` or `Buttons`. And in Android Tows, basically these are called as Views. So I've already created an application. And let me show you as to how I can add these views to the `ConstraintLayout`. So we have seen that this is the `ConstraintLayout` defined for us. I could present this either in the design mode or the code of it.

So first, what I'll do is, in order to place these widgets or views, I will go to the design window, and I'm going to pull the required views from the palette into this particular screen. So I will be requiring EditText. So let me go to the text. And this one is for the EditText, okay? So I have the EditText view placed here. Then I already have a TextView. This is fine. I'm going to put a button. So there are different types of buttons. I'm going to put a normal button over here. And I'm going to put one more button. I'll tell you the requirement of it. So once I have placed all these items, what I can do is, now, if I try to see this in the view, so let me run this application. Okay I've already placed certain items. Now if I run the application, let's see as to how it looks for us. So Gradle is building. So as you can see, I'm able to see only the Hello World, which is constrained to the screen, but all the others, that is the TextView and the two buttons, everything is moved to the top left hand corner of the screen. So I will have to provide the constrained elements for these views as well. So what I'm going to do is, I'm going to stop the app. I'll come back to the design window over here. I will select each one of it and I'm going to specify the constraints, okay, with respect to its parents, okay. So I can select this item and if I want to constrain it to what is the width of this particular screen, I can go to align and say horizontally in parent, okay. So it's going to place the alignment with respect to the Hello World is already constrained. I'm not going to bother it. So this, as well as I'm going to select button one and again horizontally to parent, constrain it horizontally to parent and this also will be aligned horizontally in parent. Then I'm going to pull this in alignment with Hello World, okay. And this one, I'm going to place it to the bottom of the screen, okay. So I'm going to place it here. So this is my alignment now, okay. So now if I try to display this, you will be able to see the difference. See, I have the text field, I have the text view and then there are the two buttons. Let me go and define what each of this button or the other views is going to do, okay. So before that, I'll go to the code part and add a few attributes to them. So text view, I have added the ID. So this, I'm going to rename it. So it is edit text and I'm going to give it a ID as AT name and instead of text, I will keep it as hint. So which will hint the user to say that they will have to enter the name, okay. And this button, I'm going to rename it as BT display in view, display in view. And this one, I'm going to rename the ID as BT display in toast. Toast is basically a small pop-up which provides simple feedback about a operation. So I will call this as display in toast, okay. Now let me go back to the design and you will see all the changes over here, okay. Now I'll go back to the main activity file and get references of these views to work with, okay. So my layout file gets inflated in set content view. So in order to get references of these views which we just created, we can definitely make use of find view by ID, okay. But there is another shorter way in order to access this for which what we have to do is we have to include a plugin named as woodland android extension. So after having this particular plugin installed, we'll be able to, okay. So I can go back to my main activity and I can easily refer to all my views over here. So what I want to do is, when clicked on the first button, okay. So for button, so display in view, I'm going to set a listener, okay. So what is it I would want to do when this particular button is clicked is what I will have to define here in set on click listener. So what I want is from the text view, rather from the edit text, I would want to take whatever the user has entered and I would want to append it to the text view. So this is what I want to do. What I have to do is text view display dot, I will have to set the text to this from what is defined in the edit text. So in edit text dot text. So this text is basically going to act as get text over here in this particular part of the expression, and here it is going to act as set text, okay. So when I click on the display in view, it is going to get whatever is present in the edit text view and set it to the display text view. Now let us define what should happen in the other button. So this one is for display in toast, okay. So for this also, create set on click listener, and when the user clicks on this particular button, okay, I will display the text which is set in the edit text

as a toast message for which I will have to cope with the toast, toast dot make text. First I will have to provide the context, which is this particular application. Then I will have to provide the character sequence, the character sequence is et dot name dot text, which the user has entered, then I will have to provide the duration. In duration, there are, either I can provide this particular popup for a shorter duration or a longer duration. Let me select long over here, then I will have to call the method show in order to display the toast message, okay. So in this video, what we have done, we have added multiple views to this particular screen, we have added multiple views to this particular screen, and we have written the code, okay, using which we have got the references of those respective views, and we are making the buttons functional. So let me run this particular app for you, so that we get better understanding of what we are doing. So the app is installed now, so int is provided, where I will have to give the name in the edit text field, so I am going to pass, okay. So when I click on display in view, it will replace this hello world, so let me click on display in view, you can see that whatever I have entered over here, it is replaced in the text view. So now what I will do is, I am going to change this, okay, and I am going to say, welcome to mobile application development, in short, it does not really sound good, so let me put it as mobile app dev, okay. So now I am going to click on display in toast, and you will see a small popup coming on the screen, okay, which says, welcome to mobile app dev, okay, thank you.

Activity LifeCycle and Start New Activity Video• . Duration: 8 minutes

[Notes]

In this demo, we will see as to how an activity will go through its different states in the life cycle and along the path, what are the different callback methods which it is going to call. So, I have already created a project over here and the main file is named as main activity which extends AppCompatActivity. Okay. So, just to demonstrate that when the app, this main activity gets started, it goes through this onCreate method. What I'm going to do is I'm going to lock that information in my lock tag for which I'll have to make use of the lock package and I will specify that it will have to be present in debug mode. I will have to pass on two strings over here. One is a tag which I have already created and I'm going to pass a custom message over here which will say main activity onCreate. Okay. Along with that, I also have a variable over here which will indicate with steps as to what are the different callbacks it is called in sequence. So, I'm going to call this variable over here. In order to use that along with the string, I will have to use it in this particular way. Okay. So, let me run this particular application. Of course, in the display, it is going to show start activity to add other things. Please ignore that. We'll come back to that in a while. So, let me run this application. So, you can see that my activity started with the title saying that it is first activity and I have also got a log message over here which says main activity onCreate1. Likewise, I'll show you its entire lifetime. In order to show the entire lifetime, I will have to call the method or callback method on destroy. Okay. So, here I'm going to again print the same log message or similar log message which I'll say on destroy. Okay. So, let me run the app again. You see that the app is launched and I have got the message onCreate. Okay. So, when I click on the back button, we have known that the app is going to get destroyed. Now, we will see when the app is getting destroyed, when this particular callback that is onDestroy will be called or not. So, I'll click on back button. So, you see that the app is cleared and it has come into the onDestroy method over here. Okay. Likewise, what I can do is, I can bring in the other overriding callback methods as well. Okay. So, I could have onRestart, then I

can have `onStart`, `onResume`, `onPause`, `onStop`. Okay. And I'm going to add this log message in each one of it. Okay. So, I have each of the method calling a custom log message. Now, I'm going to run this particular, let me empty the logcat and let me run this application for you. You see that the application is launched and I have three callbacks. So, it is called `onCreate`, `onStart` and `onResume`. So, this is where the application becomes visible to the user and it is running, it is present in the running state. Okay. So, once I click on the back button, you will see the other three callbacks getting called. So, I'm clicking on the back button over here. So, it paused, it stopped and finally it got destroyed. Okay. So, I'm going to extend this particular example, where, okay, I'm going to include another activity, call it as second activity. Okay. So, how will I create this new activity file is, I can go to the package manager, right click on it, go to new and either you can create a Kotlin class and extend it to `OpCompat` activity or what you can do is, you can come to activity and select a empty activity over here. Okay. So, when you come to this, it will give a window for you to configure the activity. You can give a name to it and you can select whether you need to generate a layout file or not. Okay. But I would want to generate a layout file. So, let me keep this option on and it also gives the option of whether you want to make this particular activity as launcher activity or not. Okay. So, this information, it will automatically be appended to `androidmanifest.xml` file and the source language is Kotlin. Okay. So, when you click on finish, it is going to create a new activity, which will extend `OpCompat` activity and it would have already created the `onCreate` method for you. Okay. So, since I've already done that, I'm not going to complete this. I'll click on cancel and let me show you my second activity. So, this is my second activity on which I have again, placed the log messages, okay, in the same way. So, this activity also I would want to see as to in what lifecycle or what lifecycle states it will actually go through. And in this example, I'll show you as to how we can start the second activity from first activity, for which I will require a intent. Okay. But what is the triggering element over here? Okay. In main activities layout file, I have defined a button. Let me show you the code. So, there is a text field which says first activity and I have defined a button, which is named as `btnAct2`. Okay. So, I'm going to set a listener on this. Okay. So, I will say `btnAct2.setOnClickListener`. And here, I'm going to define that I will have to go to the next activity or the next activity that the second activity needs to be started, for which I will have to create a intent. So, intent, let me name the intent. This is like, I will have to call the intent constructor. In the intent constructor, I will have to first pass the context and I will have to provide the second activity's fully qualified name. So, `secondActivity`, colon, colon, class dot java. Okay. And this, I will have to pass to the `startActivity` method, so the new intent which was created that will have to be passed on to the new activity. Okay. Now, let us run this particular example now. Let me clear the Lockcat window here. Okay. So, when I run it, it's going to start the first activity, which is the launcher activity. Okay. And let me show you the messages which I have got here. So, it says main activity on create 1, 2, 3. So, it has gone to `onResume` and now it is in running state. So, when I click on second activity, my first activity should go to the back stack and the second activity will be placed in the foreground. So, let me click on the button. Okay. So, you can see that the second activity's `onCreate`, `onStart`, `onResume` has been called. Okay. Before going to that, main activity called `onPause`. So, the first activity is paused and then the second activity will start entering into its lifecycle states. Okay. And you can see that the main activity is in stop state now. So, when I click on the back button, we know that the activity which is in the foreground will be destroyed. So, I am clicking on the back button so that the second activity gets destroyed. You can see that second activity is paused now. Once the second activity is paused, okay, main activity is restarted and then second activity is going to call `onStop` and finally it gets destroyed by calling `onDestroy`. So, when I

click on back button again, then you will see that the main activity also, it pauses, stops as well as gets destroyed.

Start New Activity with Result Video• . Duration: 10 minutes

[Notes]

In this demo, we are going to see the function of the method `ActivityResult`. So, I have create application over here, which has got two activities. One is the main activity, the other one is the color activity. The purpose of this particular application is that main activity is going to suggest me to select a color, which will lead me to second activity. And in the second activity, I am going to make a selection of the color and that value will be returned back to the main activity. So for which I have defined two activities over here. One is the main activity, which will be calling `StartActivityResult`, okay? And the other activity is color activity, in which I am going to display the different colors and I will help the user to make a selection on the color. In main activity, I have a text view, okay? So, whatever the text color is selected, this will be changed to that particular color and I have a button, on clicking of it, it will take me to another activity, that is a color activity for me to choose a color, okay? And I have created an intent over here and in the using the intent constructor, I have passed the package context and the last name of the color activity, okay? Then I have called `StartActivityResult`. So when this particular call is made, what happens is, this color activity will start getting executed. So let us see as to what is present in `colorActivity.kt` as well as what is present in its layout file. In the layout file, so this is how it looks like, okay? So there is a button and there is a radio group over here, which has got multiple radio buttons as well. So let us see in the code as to how these things are defined. So in the constraint layout, we have a radio group which is defined, it has an id with the height on a set, we have a text view which specifies the title, then each of radio button is defined, okay? So this radio button's id is red, text is red and the text color is set to red over here. Likewise, for the other buttons also, we have defined the id, the text and the text color over here. Now coming back to the `colorActivity.kt` file, so here what we will have to define is, since there is a button over here, okay? There is a button over here which is a submit button, on click of the submit button, whatever the color which is selected, that value will have to be sent back to the main activity. So I am defining a listener on the submit button. So let us see as to what is happening within the submit button. In the submit button, first from the radio group, I am basically selecting a id, okay? On which the radio button is checked, okay? If id is not selected, then it will basically be minus one, otherwise if it is selected, then we will come inside this particular if construct and we are going to find that particular radio button, whether it is green or red or blue, whatever is the case, by calling the method `findViewById`, okay? So this is going to return back the radio button which is actually selected, okay? Then based on that, I am going to get the text information which is present in that particular radio button and assign it to message, okay? And I am going to create an intent because I will have to send back the result to the main activity. So for which intent is actually required, so I am going to create an intent over here with the two arguments, one is the context, the other one is the main activity for which the result have to be passed, okay? Using the intent, I am going to put certain data in the form of key value pair. So the key for the first data item is message and the value will be whatever the color which was selected, okay? And the second one is the color code. The color code, I can get it from the radio button's current text color, okay? So here if you can see, we have defined the text color. So what is the current text color? I can get it from this particular

data, okay? So that will be assigned to the color code. So after putting in these two data values inside the intent, I am calling setResult. So this particular method is used to set the result into the intent and it is going to make a call or send this particular intent back to the main activity, okay? So when I click on finish, this particular activity is going to get destroyed on its own, okay? So that's what happens in the colorActivity.kt. So once the result arrives in main activity, let us see as to what is going to happen. Now once the result comes to the main activity, that needs to be handled in a method which is called as onActivityResult. So the result will come into this particular method and this method will have to be overwritten, okay? And this method, it comes with arguments which is the request code, the result code and the data which is part of the intent, okay? So we started the activity, that is start activity for result with intent and a request code, okay? So this particular request code is helpful for us to identify that this in fact was the activity which made the request. So I am going to demonstrate to you in case there is a send back with a different request code what is going to happen, okay? So in activity, onActivityResult, okay? First I am going to check as to with what result code it came back, okay? So if result code, that is, isActivity.resultCancelled, so for some reason, the second activity, that is color activity got destroyed without even making a selection, then it is going to return back with returnCancelled. So let us see this particular thing in action, okay? So I am going to run this app, so when I click on selectColor, it is going to call the method startActivityForResult with intent and request code, okay? So I am clicking on selectColor, it has gone to the second activity which is ActivityForResult, but for some reason, this particular activity got crashed, so when I click on the back button without making any selection, that means that activity destroyed, you can see color activity, so the toast message which I displayed, color activity did not choose a color, or I could say that color activity crashed, okay? If the result is okay, then I am going to come to the else part, and here I am going to check with the request code. So I have sent the request with code 1, and the color activity also should send back the same request code for me, different request code, that means then I am not the right activity to handle that particular result, okay? So if the request code is 1, then I am going to display a toast message or a pop-up, okay, with the result code, okay? And I am going to get the data entries which are made in the intent, okay? So from the data item, I am going to check if it is null or not, so this question mark operator is used in Kotlin to check if the data is null or not, similarly over here, this question mark against this data indicates that, okay, intent is coming, does it have any data or not, so it is going to check if data is null, if not, it is going to call the method getExtraString by passing on the key, okay? Whatever it returns, that will be the result that will be assigned to message, similarly I am going to get the selected color as well. So once I have got the selected color, so based on the selected color, I am going to place that particular string using the selected color, okay? So this is what happens, let us see the application in action now, so I have clicked on select color, I have made a choice, when I click on submit, it is going to return back 1 and you can see that the selected color is red, so this is the message which is displayed, okay? Now, let me show you the importance of request code, if I am sending the request code as 2, I would expect the color activity to send back request code 2, but here I am checking it for 1, okay, so when I am trying to do this, let me run the application, so I click on select color, I have made a choice and I have submitted, but we see nothing happens over here, because the request code with which the request was made and the request code which was sent back and here when it got compared, it was not the right request code, that is the reason my code did not go further, okay?

Implicit Intent to Send e-mail Video• . Duration: 5 minutes

[Notes]

In this demo, we are going to see an example for implicit intent, where I am going to create an email glide which will be used for sending the email. So what I have done here is, I have created an application in Android. In main activity, onCreate method is actually created, let me show you the layout file for this particular activity, in which I have defined one edit text, which is going to help me get the subject, the message and the recipient, which is the email ID. So here the input type address is text email address, and here these two are normal text. Then I have a button, which is ID as btSend. Coming back to the main method or the main activity file, so I have defined an onClick listener for btSend, which is our button. So onClick of it, what I am doing is, I am collecting the text which is entered in each of this particular field and assigning it to my local variables. Since these variables are not going to change further, that's the reason it is identified as var. If it gets changed later, then I will have to redefine this as var instead of var. Okay. And I want to create a user defined function or method named as sendEmail. So let me define that for you. So in order to create this function, I will have to provide the keyword fun and have the method name present, and I will have to provide all the arguments. So the first one, I am going to name it as recipient, okay, which is going to be of type string. The next one will be subject, which will again be of type string. And the last will be message, which will again be of type string. Okay. So when this particular method is called, it is passed with three arguments, a recipient object, subject, and the message. And that will be received over here in this particular function, which is sendMail. Okay. Now, here, what I am going to do is, I am going to define an email intent. Okay. So let me create the email intent. And this will use the constructor in order to create the intent, in which I'm going to set the action alone. As you can see, it is showing a list of options which I can make use of. For me, what is required is action underscore send. So this is the one which is required for me to define the intent. Now I have to define the data, okay. For this email intent, I am going to set the data. So this will be URI dot parse. I will have to specify BL2, which will... So I have entered the data field over here. Next I will have to specify the type. So I'm going to specify the type as text slash plain, okay. And now I will have to put in all the data inside this, okay. So that is, I will have to put in the subject email, that is the recipient's email ID, as well as the message, okay, for which I'm going to make use of the method putExtra. Email intent dot extra email. And this will be array of the recipients, okay. Next it will be email intent dot putExtra, subject, subject. Then I will have to put in the message. So once I have done that, then I will have to attempt sending this particular intent by calling in the start activity, okay. So let me have a try catch block, just to make sure that I'll be able to log in case something goes wrong. So in try, I will call start activity with this intent, okay. So if there are more than one application, then I will basically ask the Android to provide me a chooser with which I am going to select a specific application for sending this email. So in the catch, I am going to come up with a toast message to indicate that if anything has gone wrong, okay. So my application is ready now, let me run the application. So we will see that once the application starts, it will give me the layout file where I will have to enter the detail, the recipient, I will just put in a at a.com, subject is, hello, and welcome to the tutorial or implicit intents, okay. So when I click on send email, if there is more than one application, which can basically cater to my need, then the Android is going to pick those applications and show it to be this chooser box. So I can select any one of it and proceed further, okay.

Recording of Programming Mobile Devices: Week 3 - Live Session on 26-06-20 Video• . Duration: 31 minutes

[Notes]

now it started. Yeah. Okay, so any question around activities? Anyone, any doubt? Okay, so as part of today's session, what I'll do is I'll not, see, again, I'm making it very clear, the Saturday session is not for any theoretical coverage, not for any normal classes. It's just for your doubt clearance. Or if any other things that we want to cover. So I assume you you already are aware of the fundamentals of activities and all. So in today's I'll cover a very basic of something, some understanding that you should have for activities. And from the next session, next Saturday, we will start some live demos like doing some live codes around how you create a fragment and all those as your course progresses. But since your course has just started, so I'm not straight away picking the live demos because in order for you to understand the live demos and understand the live code, you should have the theoretical knowledge first. So let the course progresses. I think from next Saturday, we will do some live demos to show you how the components are there and how it will be visible on the screen and all. So today what we'll do is, we'll go through the some very basics of an activity. Again, I'm not, this is not a your original course. I'm not going to cover the details of activity or something that I presume is already covered in your weekly courses. Here we'll just reiterate through some of the concepts of an activity that you should know and what is needed from an interview perspective as well as when you land into any official work, what are the things that should keep in mind when you are developing an Android application. So these things I'll just cover. So there's a small PPT that I'll just go through for you to understand the things. But again, you must be totally relying on your regular courses to have a strong fundamental idea of the components. Saturday sessions are not for that. It's just for your doubt clearance. Any questions around anything, some live demos and some logical things covering. That is what is the idea. Uh, so let me share my screen. Uh, let me know if it's visible. Yes, sir, it's visible. Yes, sir. Okay, okay. Thanks. So what I'll do is I'll cover something called Android activity navigation and communication. So some basics of what things you should be knowing around an activity. First of all, how the activity is navigate, how you can move from one activity to another. How you can pass data between two activities to be shown on the UI. What are intents, what are backstack, what are tasks, some launch modes and what are activity results. I I I am again presuming you all would have covered activity, the fundamental and all those things. It's just something a cherry on the top is something what we'll be covering today. Again, I'm not going to cover the details of activity or something. Those are just part of your course. I'll just reiterate through some important things that you should be knowing. So the session objective basically is you should understand how activities navigate in Android. Navigation means moving from one activity to another. What are the different intent types? What are their use cases? How activities are stacked? For example, if I open activity B, now again you open activity C, how they are stacked in the operating system? What are the tasks? What are the different activity launch mode? And how you can do activity result handling? So we'll just be covering a crisp of this. The very detail and etcetera will be definitely part of your course. So this is what will be the agenda of today's session. Okay, why activity navigation matters? Real apps consist of multiple screens. You can think of activity as nothing but as a screen that you see in your mobile device. If you click login, you see some other screen comes. Once you are logged in, you see a home screen comes. So they're nothing but basically activities. And basically your Android operating system manages the transition. Transition means moving from one screen to another or moving from one activity to another. If

you're not handling the navigations properly in your code, definitely it can result to very unexpected behavior. Your app can even crash. It will not even launch. So a very good navigation is very important for a improved user experience. So as a good Android app developer, you should be very much clear with the navigation fundamentals. And you should write the code in a way that handles navigation properly and effectively. That's why the navigation matters. So basically how you move from one activity to another is something called intent in Android. So you basically use an intent, we create an object of an intent class, mention to which activity you want to go. And then Android application takes you to that activity. So intent is basically a messaging object used to request an action from Android system. So whenever you create an intent, you're asking the Android operating system that okay, I've created an intent, do this thing. It act as a bridge between components and it is a core concept of activity navigation. So whenever you move from one activity to another, you basically do it via intent. So intent is nothing but to tell the Android system that okay, I want to go to this activity. This is the intent, please do that. And your Android operating system basically will do that. So please remember what is an intent, details of intent, there are different types of intents from your, I think, examination perspective, intent is a very important concept. So intent is something you should be aware. Is what is used for moving from one activity to another. There are different types of intents, something called an explicit intent. Then there's called an implicit intent. Explicit intents are those intents in which the target is known. Means you are very clearly specifying that okay, you want to move to this particular screen or activity. The those type of intents are called explicit intent. Then there's something called an implicit intent. Implicit intent means you don't specify the target. You just create an intent object and launch it. And whatever targets themselves decide whether they need to respond on this intent or not. So those are called implicit intent. For example, broadcast receivers is an example of implicit intent where you broadcast something. And whatever activities, they themselves decide whether they need to listen to this intent and do the necessary actions. So these are the two types of intent that you should be aware. Intent resolution happens at runtime. So once that you your operating system receives the intent instruction, then they do the navigation. So this is an example of an explicit intent. So you create an intent object. You mention which activity you are currently in. For example, I'm currently in main activity. Which activity you want to go to? So I mentioned it's a detail activity. There I want to go to. And then you call this method start activity and pass the intent. So when you pass the intent, operating system will know that okay, you are currently at main activity and you want to navigate to the detail activity. So it will start the detail activity or call the detail activity. And then you will see a different screen coming on top of your existing screen. So this is what is an example of an explicit intent. See, I'm just coming, I'm just covering in crisps that you should know these things. The details and everything that will be part of your course or that would have definitely I think might have been covered also. I'm not sure there. But don't expect in-depth theoretical discussions in these sessions. It has to give a crisp understanding of components and to solve any of your doubts. So this is an explicit intent. Remember these examples. In the interviews, they can ask you that what are intents, why they are used, what are different types of intents? Can you write a code snippet for some intent? Or if they give you some problem statement to implement as an Android developer, definitely you should be knowing intents, otherwise you cannot move from one screen to another. So this is the explicit intent. And this is the implicit intent. So you are creating an intent and you are just setting the action. That okay, action is view. And you are passing a URL. That okay, this URL has to be shown. So the action is view. So here you are not mentioning any activity that where you want to go. You just leave it open. So whoever activities have that intent mentioned in their Android manifest, those activity will

respond to that intent. You are based on action, category or data, whatever. So these are called intent filters basically. So in the manifest only you mention that okay, this activity can respond to these type of intents. So whenever anyone fires those intent, that activity will automatically get launched. So all these things you do in the manifest file. For explicit intent, you don't need to mention. You just straight away mention that okay, I want to go to this activity or launch this activity and your Android application will do that. So this is the fundamental concepts around intent. Please keep this in mind. Intents are very important thing. And whatever I've given you, I've given you from the interview perspective. And the real world things when you work on activities, these things you should be knowing. These are very fundamental. Go and know these things in depth, basically in your courses and when you do the self study. But the idea is this. Now get to the very important thing. So, so if you want to pass data between two activities, how will you do that? Right? For example, you are on a particular screen. You ask some data from the user. User types those in your mobile application. Then you open a new screen and want to show the data in that screen. Then how you pass basically data from one screen to another? That is also a very important thing in Android, right? It's a very common thing. You definitely have to pass data. That also is something done through intent. So whenever you're calling an activity B, now you're launching activity B, and want some result to come back to activity A from activity B. So in the intent, there's something called put extra. It's a key value pair. So in the sending activity, you just uh, create the, add the put extra method. In that method, you mention the key and the value. And the receiving activity, you can simply using the get string extra method, you can pass that key and get the value. So this is how sending and receiving is done. So in the upper code, what the sending activity needs to do if they want to send some data as a key value is mentioned. And in the lower code, this is how the receiving activity will get those data. So that receiving activity just needs to pass the key value to the intent in the get intent and it will receive the value. It's a again a very, very simple example. Data can be complex, passing can be complex, but it depends on your use case. But my idea was to just give you an insight. So I have intentionally taken a very simple example to help you understand. So this is how basically you pass and receive data using intent as a key value. Now, value can be anything. It can be a JSON string. It can be a class object, whatever type that you can decide. But this is a very simple activity where the key is username and the value is Android user. So this is how the, these are the main uh, functionality of intents. Navigation, explicit navigation, and data passing. So if someone asks you that how you can pass data and what is the core component that is used for data passing in Android, simple answer is intents. These are the data types that it supports. Primitive data types like int, float, string, serializable objects, parcelable objects. So these are certain things that it supports. So these type of data you can pass via an intent. You can pass an integer, you can pass a float, uh, you can pass a long, you can pass a string, you can pass serializable objects, you can pass parcelable objects. Anyone aware what are parcelable objects? It's very much used in Android for passing data. Anyone aware what do you, what is a parcelable object? Have you ever used it, parcelable objects? Fine, so have a read. Again, I will not cover this. Uh, go and have a read that what are parcelable objects? How you create a parcelable objects? Serializable objects, I think definitely you would be knowing if you are aware of Java. You are also aware of what are serializable objects. So, but parcelable objects are used in Android very much. There are certain reasons why they are preferred in Android. I'll recommend go and have a look at them. That what are parcelable objects? How they are created? Why we use them in Android? And all those things. Because these are very necessary things. These are very straightforward interview questions that can be asked to you that what are parcelable objects? Simple. Why we need parcelable objects in

Android? All those things. Yes, they pass data, but what type of data and what advantages they provide? Because data passing you can do either through string also. You can do via primitive types also. You can do via serializable objects also. But why we need something called a parcelable object? So that you should be knowing. And again, in the bracket, it's mentioned recommended. Means Android should use parcelable objects for passing data. Now the question comes is why? So that you should be fully aware. In the interview or in your designing rounds, they can ask you. Okay, this is the activity backstack diagram. So for example, suppose main activity is open in your Android device. Now you move to another activity. Suppose detail activity. So in the stack, what will happen? The detail activity will come on the top of your main activity. So the detail activity will be visible to you in the mobile device. Now suppose you move to another activity by clicking any button or something. Suppose profile activity. Now that activity will come over the detail activity. So it's a stacking of activity happens in the Android operating system. And whenever you press back button, you would have seen your Android device. Whatever activity is on the top of the stack, that will get removed. So suppose I opened main activity. I clicked a button. Detail activity got opened. So it comes on the top of main activity. Now suppose again I clicked a button. Profile activity got opened. It comes over the detail activity. Now if you press the back button, what the operating system will do, the top activity in the stack, that is the profile activity, it will remove it. So now you will land back on the detail activity. So this is how it's maintained. It's a very, it's a very normal behavior you would have seen. So internally what happens, the operating system maintains an activity stack. So the most recently, the recently activity that is visible to you, opened in your device, that will be on the top of the stack. And once you press the back button, Android will remove that activity from the top. So this is how it happens. It's a very, very interesting thing. And in interview, these, they can ask these things. How activities are stacked? What will happen if you press the back button three times? What happens to the top activity? Which will be visible? To test the understanding of your Android fundamentals. So it's an activity stack is basically maintained in the Android operating system. And this is how they handle the removal on back press. You should be knowing these things. Okay, now there's something called a task. So whenever you hear the word task in Android, it is nothing but a collection of activities. Multiple activities together doing something is called a task. So the Android, if in the Android world, if you interact with anyone and they use the word task frequently, understand that they're talking about a collection of activities. Don't forget these things or don't get confused with these words. They're expected as an Android developer. They're expected that you know these things. Because in the corporate world, they will, in the offices or when they're doing the implementation, they will use these words, tasks and all. If you're not aware what is a task, it will be difficult for you to understand. As we discussed, activities are arranged in stack. Home button sends tasks to the background. Task state is preserved by OS. So whenever any activity is removed from the top or moved in the background, the state of that activity, like whatever buttons were visible in that activity, the data that are shown in that activity, that is preserved by the OS. So that next time when you call it, it will show you those things. So the Android operating system takes care of preserving the data of your activity once they are moved to the background. Background means they are not visible to you. For example, in this example, if the profile activity is opened, your main activity and detail activity are in the background. They're not visible to you. Now once this profile activity is removed, then the detail activity will be shown to you. You will see all the data still there. So how your app see those data? Because the operating system is preserving that for that activity. So that next time when it's on the top, you're able to see them. And all these groups of activity, uh, this profile, detail, main, since they are doing one job together, they are called task. Any question,

anyone? Okay, there are certain launch modes that you should be aware. I'm not sure if this covered till now in your course, but definitely should be covered. So launch modes help you control how activities are created. How activities behave in the stack. And they are generally defined in your Android manifest.xml. So launch modes are something this is how you define Android launch mode in your manifest and define whether standard or something. For example, you opened activity B. Now you opened activity, suppose there were activity A. Now you opened activity B. Now you again opened activity A. So then what will happen? It will create a new instance of the activity every time. It will not reuse the existing A's instance. So from A, if you move from an activity A to activity B, and from activity B again you come to activity A, then it will create a new instance of every activity. I think last one will also be B. It's a mistyped as A. So in this case, just a very simple thing. If the launch mode is standard, standard means whatever activity you are creating or navigating, a new instance or activity will be there. Reusing will not happen. So this is the standard mode. Single top. In single top, if you mention as the launch mode, then from A, if you move to B, and then from B again, if you want to move to A, then A's new instance will not be created. It will be reused. The older instance. So this is for the single top behavior. Then there is single task. Single task means suppose activity A is opened. Now you opened activity B. Now you opened activity C. Now if you want to open activity A again, then B and C will be cleared. Android will remove these activities. It will not keep it. So if you mention the launch mode as single task, then this behavior will happen. And again, there's a single instance. It is same as, almost same as single top. But there is a minor difference. I'll leave it up to you to do a self-read to understand what is the difference between a single instance launch mode and a single top launch mode. It's an interview question. So remember there are three, there are following types of launch modes are there, standard, single top, single task, single instance. Now what is the difference between a single instance and a single top? Just go and have a self-read. So interviewer can give you some scenarios and they can ask you that which launch mode you will prefer, whether you want a single instance, whether you want a single task, whether you want a single top, or whether you want a standard. Which will suit best? So based on the requirement, based on your use cases, you have to make a choice. So these are all whatever I'm giving you, they all part of the interviews. Okay, another thing is activity result handling. Suppose an activity A. Now you are launching activity B. And want some result to come back to activity A from activity B. Means activity B will get launched. It will perform some task. And then send the result back to activity A. So this is called activity result handling. So you can do this also in Android. You can launch a new activity. That activity will do certain result, certain computation and it will send back you the result. So this is also you can do. This is how it is done. There's something called an activity result launcher. You do a registration. You start that activity. And then that activity will pass the result with some code. You just check that okay, if this code is part of the result, then you have that result. Again, I'm just giving a very simple code snippet. I'm not going into details as this should be part of your course. So my idea is just to help you understand that you can also call an activity to do something and send back you the result. So there's something called an activity result launcher that you should be aware. And in that activity result launcher, there's something called activity result contracts that you should implement using new. And then based on the result code, you will be able to get the result from whatever activity you are launching. So keep in mind that there's something called an activity result that you can get and you should use an activity result launcher. It's a new Android API which helps you get the result from a different activity. Definitely, if you are developing an app, definitely you will get these scenarios where you want result to need to get some callback from a different activity, your result from some different activity in your main activity. So you have to

use this. So keep these in mind. Okay, coming to some common interview questions. Whatever we covered today, you should be very much aware of what is an explicit versus implicit intent. You should be very much aware of what is backstack. You should be very much aware of different launch modes and what are the difference between them with examples, you should be aware. And how to pass data between activities. These are certain common interview questions that can be asked to you from whatever we covered today. So my idea was to help you cover only those topics that are related to interviews and your fundamental understanding. Definitely, I've been reiterating again, the details of this would be covered in your courses. And if not, you should read them. Any question, anyone? Any question? What I'm not listed any questions, right? I've opened something for you to just go and have a self-read. Uh, I don't think I've opened any question for you, model answer. Just this PPT, definitely you can, you know the points that I covered. You can just, once your course is there, in your courses, if these are covered, then understand them in details. Or something that I've asked you, like parcelable, what is that? Difference between single top and single instance. Those you can go and have a self-read. I'm not listed any question. I just summarize the things that you should be knowing now. Just above your main course. Because all these are very important from your development as well as interview angle. Okay, what we covered in today's session as a summary, intent driven navigation, how you can navigate between activities, data passing between screens, how you can pass, backstack and task behavior, launch modes, impact navigation, and modern activity result handling. So these are what I wanted to just cover to give you an insight of these things because they are very, very important from development angle as well as from your interview angle. So with this, I'll wrap the session because again, I'm telling you, I'm not going to cover any theoretical. These Saturday sessions are not meant for theoretical coverage. They're meant for some extra cherries on the top. Some live demos will cover once you get a good understanding of the theoretical concepts because if I do a live demo by writing codes, which you have not yet covered, which you don't know, then it will not make sense. But I want in the live demos that you understand the codes, you understand how each is behaving. So as your course progresses, when you get a fair understanding of the Android stuff, then we'll cover some use case. We'll take some use cases and do the live demos. So till then, we'll just cover some interesting topics that you should be aware around the core topics from your interview angle as well as from your development perspective. Any question here, anyone? Any question? Okay, if no further session, sorry, if no further questions, then we can wrap up. Again, I'm asking any question, anyone? Cool, then since no question, I think we are done with today's session. It was just to add some extra thing on your course content. Be aware of these things, have a solid fundamental understanding of these things. Then only you will become a good developer. So thank you all. All this for the today's session. See you all next Saturday. Thank you.

What are Fragments Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. And I am Professor Chandrasekhar RK, and in this session, we'll be discussing a very important concept in Android development, which is fragments. Fragments are essential components in Android that allow you to create more dynamic and flexible UIs by dividing your user interface into smaller, manageable pieces. In this session, we will explore what fragments are, why they are used, and the benefits they

offer in Android app development. By the end of this session, you will have a solid understanding of how to use fragments effectively in your Android applications. Let's begin by understanding what fragments are. In Android, a fragment is a reusable portion of your application's user interface. Fragments are embedded within an activity and can be thought of as sub-activities, but they have their own lifecycle, UI, and logic. They are like building blocks that you can assemble together to create a complete user interface. For example, in a tablet layout, you might have a fragment that displays a list of items on the left side and another fragment that shows the details of the selected item on the right side. Fragments allow you to design UIs that can adapt to different screen sizes and orientations, making your app more flexible and responsive. So what is the purpose of using fragments? So now let's discuss the purpose of using fragments in Android development.

Fragments are used to create more dynamic and flexible user interfaces. They allow you to divide your app's user interface to multiple sections that can be managed independently. This modular approach makes it easier to design complex UIs that work well across various screen sizes such as phones and tablets. Additionally, fragments help in managing different parts of your UI more efficiently. For instance, you can use fragments to handle different UI components in the same activity such as `ListView` or `DetailView` without needing multiple activities. This makes your app more organized and easier to maintain. Fragments offer several benefits that make them an essential tool in Android development. First one among them is modularity. Fragments allow you to break down your UI into smaller modular components. This makes it easier to manage and update specific parts of your UI without affecting the entire activity. And they are reusable. Since fragments are modular, they can be reused across different activities. For example, you can create a fragment for a common UI component like a toolbar or a footer and reuse it in multiple activities. They are also maintainable. With fragments, you can manage your UI components more efficiently instead of handling everything in a single activity, you can distribute the workload across multiple fragments, making your codebase easier to maintain and debug. Another key benefit of using fragments is easier state management. Managing the state of an activity, especially during configuration changes like screen rotations, can be challenging. However, fragments offer built-in mechanisms to help retain state across such changes. For instance, if you have a fragment that displays a list of items, the list's scroll position can be retained automatically during the configuration changes. This ensures a smooth user experience as the app remembers where the user left off. Fragments also make it easier to manage the activity's lifecycle, allowing more efficient use of resources. Let's understand the life management of fragments. Fragments have their own lifecycle methods that work in coordination with host activity's lifecycle. These include `onCreate`, `onCreateView`, `onPause`, `onDestroyView`, and more. Managing and managing these lifecycle methods is crucial for creating fragments that respond effectively to user interactions and system events. To summarize, fragments are a powerful feature in Android development that allow you to create more dynamic, modular, and maintainable user interfaces. We discussed what fragments are, their purpose in creating flexible UIs, and the key benefits they offer, including modularity, reusability, and easier state management. We also explored how fragments help in lifecycle management, ensuring efficient use of resources. Thank you for your attention throughout this session. I hope now you have a better understanding of fragments. Thank you.

Creating Fragments Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss a critical topic in Android development that is creating fragments. Fragments are integral part of Android applications, enabling you to build dynamic and flexible user interfaces. In this session, we will learn about the basic structure of fragment, how to add them, to activity, and tools available to assist in fragment development. By the end of this session, you will understand how to create and manage fragments effectively in your Android projects. Let's start by understanding what a fragment is. A fragment in Android is a modular section of an activity, which has its own lifecycle, receives its own input events, and can add or remove while the activity is running. Fragments enable you to create more flexible and dynamic UIs by allowing different parts of your UI to be managed independently. For example, in a tablet layout, you might use one fragment to display a list of items and another fragment to show details of the selected item. Fragments allow your app to adapt to different screen sizes and orientations, providing a consistent user experience across devices. Now let's look at the basic structure of a fragment. A fragment typically consists of a class that extends `androidX.fragment.app.Fragment`. The most important method in this class is `onCreateView`, which is where you define the UI for the fragment. In the snapshot, you can see a simple example using Kotlin language for the fragment. So in the example, you see that `onCreateView` method is used in order to inflate the container, which is the layout inflater. In the example, you can see a function `onCreateView`, which is used to inflate the container, which is used for fragment. In this code, the `onView` method inflates the layout defined in `fragment underscore example dot xml`. This is where you set up the user interface for your fragment, similar to how you would have defined the UI in an activity. Now let's discuss how to add fragments to an activity. There are two main ways to do this, statically and dynamically. First let us look into the static addition. You can add a fragment to your activity's `layout.xml` and here is how to do it, as shown in the snippet, using the `fragment` tag. In the `name` attribute, you have to pass on the class name of the fragment which you have created. In this method, the fragment is added directly into the activity's layout file and is managed as a part of activity's view hierarchy. The second method is to add it dynamically. So you can also add fragment to an activity dynamically using the Kotlin code. This is very useful when you want to replace fragments during runtime. And here you can see a small Kotlin snippet in order to demonstrate the fragment replacement dynamically. In this method, you use the fragment manager to begin a transaction and add the fragment to the specified container in your activity. This approach gives you more flexibility as you can add, replace and remove fragments based on user interactions or other events. To help with fragment development, Android Studio provides several tool windows that are very useful. Some of the key tools include Logcat. This tool displays logs from your running application, which is essential for debugging. You can use Logcat to track lifecycle events, errors and other important information that helps you understand how your fragments are behaving during runtime. Then you also have Device File Explorer. This tool allows you to explore the files on your connected device or emulator. It is useful for check-in files that your fragments might be reading from or writing to. Then you also have the Terminal, that is the built-in terminal in Android Studio that gives you access to command-line tools directly from the IDE, which can be handy for running custom scripts or commands related to your fragments. Using these tools effectively can greatly improve your efficiency in developing and debugging fragments. To summarize, today we have explored how to create and manage fragments in Android. We discussed the basic structure of a fragment, including the key `onCreateView` method and learned how to add fragments to an activity both statically through XML and dynamically through Kotlin code. We also looked at some essential tool windows in Android, such as Logcat and Device File

Explorer, which aid in fragment development. We looked at some of the essential tool windows in Android Studio, such as Logcat and Device File Explorer. By mastering these techniques, you can create modular, flexible and dynamic UIs that enhance the user experience in your Android applications. Thank you for your attention throughout this session. I hope now you have a better understanding of how to create and manage fragments. Thank you.

Fragment Life Cycle in Android App Development Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss an important topic in Android development, which is the fragment lifecycle. Just like activities, fragments have their own lifecycle, which is closely tied to the lifecycle of the parent activity. Understanding the fragment lifecycle is crucial for managing UI components efficiently and ensuring a smooth user interface. In this session, we will explore the various phases of lifecycle of the fragment, from creation to destruction. By the end of this session, you will have a clear understanding of how to manage fragments through their lifecycle phases. Let's start with an overview of fragment lifecycle. Similar to an activity, a fragment has its own lifecycle, which includes various stages that represent different states from creation to destruction. These stages are important because they dictate how fragment behaves, how it interacts with the parent activity, and how resources are managed. Understanding these stages helps you create fragments that are responsive, maintain state across configuration changes, and manage resources effectively. The lifecycle of a fragment is divided into several phases, each with specific lifecycle methods that can override to control the fragment's behavior. Now let's look at the lifecycle phases from when a fragment is created to when it starts running. These phases include OnAttach method. This is the first method called when the fragment is associated with the parent activity. It's where you get a reference to the activity and perform any setup required before the fragment is created. Then we have the OnCreate method called. This method is called next, where you initialize essential components of the fragment that need to persist through the lifecycle, such as data models. Then we have onCreateView. In this phase, you create and return the view hierarchy for the fragment. This is where you define the UI for the fragment. Then we have the OnActivityCreated. After the activity's onCreate method has completed, this method is called. It's a good place to complete the final initialization, such as binding data to views. Then we have onStart method. The fragment becomes visible on this stage, but it is not yet ready for user interaction. Then we have onResume. Finally, the fragment enters the running state and is ready to interact with user. At this point, the fragment is fully active and can handle user input. These phases ensure that your fragment is initialized properly and ready for interaction when it reaches the running state. Next, let's discuss the phases from when a fragment is running to when it ends. These phases include onPause method. This method is called when the fragment is about to stop interacting with the user. It's a good place to save any changes or stop processes that should not continue when the fragment is paused. Following onPause method, we have onStop method. The fragment is no longer visible at this stage. You should release any resources that are not needed when the fragment is not visible. Then we have onDestroyView. This method is called when the view hierarchy associated with the fragment is being removed. Here you can clean up resources related to the UI such as binding

objects. Then we have OnDestroy method. This method is called to perform the final cleanup of the fragment state. It is used to release any remaining resources. And finally we detach the fragment using OnDetach method. Finally the fragment is detached from the activity on this method. This is the last phase in the fragment lifecycle and it is where you clean up any references that the fragment holds to the activity. Understanding these phases helps you manage the fragment's lifecycle effectively, ensuring that your app remains responsive and the resources are used efficiently. To summarize, understanding the fragment lifecycle is essential for managing the UI and resources efficiently in Android applications. We covered the various phases of the fragment lifecycle starting from when the fragment is created to when it is running and then from when it is running to when it ends. By mastering these concepts, you can ensure that your fragments perform optimally and enhance the overall experience in your Android apps. Thank you for your attention throughout this session. I hope you now have a better understanding of fragment lifecycle and how to manage it effectively. Thank you.

Replacing the Fragment in the Activity with Another Fragment Video• .

Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss an important aspect of Android development, which is fragment replacement. Fragments are versatile components that allow you to build dynamic user interfaces by replacing one fragment with another within the same activity. In this session, we'll explore what fragment replacement means, why you might want to replace fragments in an activity, and how to set up this functionality. By the end of this session, you will understand how to implement fragment replacement to create more dynamic and responsive Android applications. Let's start with understanding the fragment replacement in Android. Fragment replacement refers to the process of dynamically changing one fragment to another within the same activity. This technique is commonly used to create dynamic user interfaces where the content on the screen changes based on user actions, such as clicking a button. For example, you might want to have a screen that displays different sections of content like an overview and a detailed view, and you want to switch between these sections without leaving the current activity. Fragment replacement allows you to do this seamlessly, making your app more interactive and responsive to the user input. So why do you want to replace fragments within an activity? Fragment replacement is useful in scenarios where you want to create a dynamic user interface and changes based on user actions. This approach allows you to manage different sections of content within the same screen, providing a more fluid and cohesive user experience. Some common use cases for fragment replacement include switching between different content sections like showing a list of items and then displaying the details of selected item, creating a multi-step form where each step is shown in a different fragment, and creating a tabbed interface where each tab displays a different fragment. By using fragment replacement, you can design flexible and interactive UIs that respond to user actions in real time. Let's move on to setting up of fragments for replacement. The first step is to create fragments that you want to switch between. For this example, let's assume we have two fragments, fragment 1 and fragment 2. Next we will add two buttons in the activity layout to allow the user to switch between these fragments. The buttons will have, next we will add two buttons in the activities layout to allow the user to switch between

these fragments. The buttons will have click listeners that trigger the fragment replacement. And here is how you can set it up. So first we have the button 1, `setOnClickListener`, calling the method `replaceFragment` and the parameter to it is fragment 1. And then we have the fragment 2 button for the second fragment, `setOnClickListener`, defined with `replaceFragment` with fragment 2. In this code, we defined click listeners for each button. When the user clicks a button, the corresponding fragment is displayed by calling `replaceFragment` function, which we will explore next. To implement fragment replacement, you will need to define the `replaceFragment` function. This function uses the fragment manager to begin a transaction, replace the current fragment with the new one, and then commit the transaction. The code snippet, which is given in the slide, demonstrates the same. In this code, `supportFragmentManager.beginTransaction` starts a new fragment transaction. The `replace` method replaces the existing fragment in the specified container, which is `r.id.fragmentContainer` with a new fragment passed to the function. Finally, `commit` method applies the changes. By setting up the `replaceFragment` function, you enable your activity to dynamically switch between different fragments, creating a more interactive user interface. Let's take a look at the emulated output to see how fragment replacement works in practice. In the emulator, when you click `showFragment1` button, fragment 1 is displayed in the activity. When you click `showFragment2` button, fragment 2 replaces fragment 1 and the new content is shown. This demonstrates how fragment replacement allows you to change the content on the screen without starting a new activity or reloading the entire UI. It's a powerful technique that enhances the user experience by providing smooth transitions between different sections of your app. To summarize, fragment replacement is a key technique in Android development that enables you to create dynamic and responsive user interfaces. We discussed the concept of fragment replacement, why is it useful for building flexible UIs and how to set up and implement fragment replacement using the Fragment Manager. By mastering fragment replacement, you can develop Android applications that are more engaging and user friendly. Thank you for your attention throughout this session. I hope now you have a better understanding of fragment replacement and how to implement it effectively in your Android projects.

Fragment Communication Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will discuss an essential aspect of Android development, which is fragment communication. Fragments are often used in Android to create modular and reusable UI components. However, there are many situations where these fragments need to communicate with each other either within the same activity or across different activities. In this session, we will explore the purpose of fragment communication, the methods to achieve it are live data and view model, and the best practices to ensure efficient and maintainable code. By the end of this session, you will have a clear understanding of how to implement effective fragment communication in your Android projects. Let's start by understanding the purpose of fragment communication. Fragment communication is crucial when you have multiple fragments within an activity or across different activities that need to exchange data or respond to user interactions. For example, in a multi-pane UI, one fragment might display a list of items, while another fragment shows details of the selected item. The two fragments need to communicate to update the UI based on the user selections. Fragment communication ensures that these interactions happen smoothly and efficiently, providing a seamless user

experience. Some of the common use cases for fragment communication are passing data between fragments. In a multi-pane UI, as told earlier, you might need to pass data from one fragment to another. For example, a list fragment selected item's data to a detail fragment to update its content. And the next one is updating UI components. Sometimes, events in one fragment require updating the UI components in another fragment. For instance, a settings fragment might change a preference that affects the display in another fragment. These scenarios highlight the importance of effective fragment communication to maintain consistency and responsiveness in your app's user interface. To implement fragment communication effectively, Android provides tools like LiveData and ViewModel. Let's explore how these work together to facilitate communication between fragments. First let's talk about Shared ViewModel. A ViewModel can be shared between multiple fragments to store and manage UI-related data in a lifecycle-aware manner. This decouples the fragment logic from activity and supports efficient data handling across configuration changes such as screen rotations. Let's see what LiveData is. LiveData is a lifecycle-aware data holder that allows fragments to observe changes to the data and automatically update the UI when data changes. This ensures that fragments remain updated with the latest data without requiring direct communication between them. In the code snippet, you actually see an example of a Shared ViewModel class. In this example, fragments can observe the selected data, LiveData and react to changes ensuring that the UI remains consistent and up to date. Now let's discuss some of the best practices for fragment communication. First one is to minimize direct fragment-to-fragment communication. Avoid having fragments communicate directly with each other as this can lead to tightly coupled code that is difficult to maintain and debug. We have to favor LiveData and ViewModel for consistency. Using LiveData and ViewModel ensures that your fragments are decoupled and the data is handled in a lifecycle-aware manner. This approach helps maintain a clean architecture and improves code reusability. And as told earlier, we have to avoid tight coupling between fragments can make your code more rigid, harder to maintain. By using ViewModel and LiveData, you can keep your fragments independent and make your app more modular. And finally, testing and debugging. Ensure that your communication logic is easy to test and debug. Using ViewModel and LiveData makes it easier to write unit tests and track down issues since these components are designed to work with lifecycle-aware data. To summarize, the effective fragment communication is essential for creating responsive and modular Android applications. We discussed the purpose of fragment communication, common use cases where fragments need to exchange data and update UI components and how to implement communication using LiveData and ViewModel. By mastering these techniques, you can create applications that provide seamless user experience. Thank you for your attention throughout this session. I hope now you have a better understanding of fragment communication and how to implement it effectively. Thank you.

Clickable Images in Android Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will explore the significance of clickable images, how to implement them in Android, and the steps to handle click events efficiently. By the end of this session, you will have a clear understanding of how to add clickable images to your Android projects. Let us start by understanding what clickable images are, and their significance in Android development. Clickable images allow users to interact

with images in the same way they interact with buttons. This feature is particularly useful in creating more engaging and user-friendly interfaces where images can serve as interactive elements like icons, buttons, and even game controls. Clickable images are widely used in various applications, such as in galleries, where clicking an image might open it in full view, or in e-commerce apps, where clicking a product image might take the user to the detailed product page. This versatility makes clickable images a valuable tool for enhancing user interaction and providing a seamless experience. Now let's move on to the implementation steps for adding a clickable image in Android Studio. The first step is to add the image you want to make clickable to the resources folder under Drawable subfolder in your Android project. This folder is where all your app's drawable resources such as images and icons are stored. For example, if you have a logo image named logo.jpg, you would place this into the resource folder under Drawable subfolder. This allows the image to be referenced to you in your layout files and used in your app. Once you have added the image to the drawable folder, the next step is to define the layout in which the image will be displayed. This is done in the activity underscore main dot xml file. Here you have to add a image view element in order to display the image, as you can see in the code snippet. In this code, we set the android source attribute to the image file located in the drawable folder. The android clickable attribute is set to true to make this particular image clickable and the android content description is provided for accessibility purposes. This ensures that screen readers can convey the image's purpose to visually impaired users. Now that the image is clickable, the next step is to handle the click events. This is done in the main activity dot kt file. You will use the find view by id method to get a reference to the image view and then implement a setOnClickListener listener to define what happens when the image is clicked. In the code snippet, the user clicks on the image, a toast message saying image click will be displayed. This is a simple way to test the click event is working correctly or not. Let's now see the output of a clickable image in the emulator. After implementing the steps we have discussed, you should be able to run your application in android emulator. When you click the image, the toast message image click will appear on the screen, confirming that the image is clickable and the click event is being handled properly. This output demonstrates how clickable images can be integrated into your app, providing an interactive element that enhances user engagement. To summarize, clickable images are powerful feature in android that allow you to create more interactive and engaging user interfaces. We discussed the significance of clickable images, how to add an image into a drawable folder, define the layout using image view and handle click events in the main activity.kt file. We also saw how clickable images work in the emulator. By following these steps, you can implement clickable images in your android projects to create more dynamic and user-friendly applications. Thank you for your attention throughout this session. I hope you now have a better understanding of how to implement clickable images in android and how to use them effectively in your projects. Thank you.

Checkbox in Android Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to the session on mobile app development. In this session, we will discuss an essential component in Android user interaction, that would be the checkbox. The checkbox is a simple yet powerful UI element that allows users to make a selection or choose multiple options from a set of choices. In this session, we will explore the significance of

checkboxes, their key features, and how to implement them in Android Studio. By the end of this session, you will be able to create and manage checkboxes in your Android applications effectively. Let's begin by understanding what a checkbox is, and its significance in Android development. A checkbox is a user input control that allows users to select one or more options from a set of choices. It is a versatile UI element commonly used in forms, settings, or any interface where multiple selections are needed. For example, in a registration form, you might use checkboxes to let users choose their preference or agree to the terms and conditions. Checkboxes are simple to use and integrate into your app's UI, making them an essential tool for enhancing user interaction and improving the overall user experience. Checkboxes have several key features that make them useful in Android applications. One of it is user input control. Checkboxes provide a straightforward way for users to make selections or indicate preferences. And it is also versatile. They can be used in a variety of contexts, from settings menu to survey forms, making them highly adaptable to different app needs. It is also a simple UI element, i.e. the checkbox is easy to understand and use, contributing to a clean and user-friendly interface. It also provides a visual feedback. The checkboxes provide immediate visual feedback showing users whether an option is selected or not, which enhances the clarity of the interaction. These features make checkboxes a fundamental element in Android UI design. Now, let's move on to the implementation steps for adding a checkbox in Android Studio. The first step is to define the checkbox in your layout file, typically activity underscore main dot xml file. Here, in the code snippet, you can see as to how to add a checkbox in the activity underscore main dot xml file, using the checkbox tag. In this code, we have created a checkbox with id my checkbox. The android text attribute is used to label the checkbox and the android layout center in parent attribute centers the checkbox in parent layout. This is a straightforward way to add a checkbox to your Android application's UI. After adding the checkbox to your layout, the next step is to handle its state, which involves responding to user interactions and this is done in main activity dot kt file using Kotlin. In the code snippet, you see find view by id is used to get references to the checkbox and a text view that displays the checkbox status. The set on check change listener method listens for changes in the checkbox state and updates the text view accordingly. When the checkbox is checked, the status is updated to checked and when it is unchecked, the status changes to unchecked. This provides immediate feedback to the user based on their interaction. Let's now see the output of the checkbox implementation in the emulator. After following the steps we have discussed, you should be able to run your application in Android emulator. When you check the accept terms and conditions box, the status below will be changed to checked. When you uncheck it, the status will revert back to unchecked. This output demonstrates how the checkbox works in practice, providing users with clear feedback based on their selections. It's a simple yet effective way to enhance the interactivity of your Android application. To summarize, checkboxes are a vital UI element in Android that allows users to make selections and provide input in a straightforward manner. We discussed the significance of checkboxes, their key features and how to add them to your Android layout file. We also explored how to handle checkbox state changes using Kotlin and we saw the practical output in the emulator. By implementing checkbox in your Android projects, you can create more interactive and user-friendly applications that enhance the overall user experience. Thank you for your attention throughout this session. I hope you now have a better understanding of how to implement checkbox in Android and how to use them effectively. Thank you.

Radio Box in Android Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will explore a fundamental user interaction component in Android, and that would be radio buttons. Radio buttons are commonly used in Android applications to allow users to select one option from a group of choices. They provide a simple and intuitive way for users to make single selections. In this session, we will discuss the significance of radio buttons, their key features, and how to implement them in Android Studio. By the end of this session, you will understand how to use radio buttons effectively in your Android applications. Let's begin by understanding what a radio button is and its significance in Android development. A radio button is a UI element that allows users to select a single option from a set of choices. It is a common component used in forms, surveys, and settings where users need to choose only one option from several possibilities. The key feature of radio buttons is that they are mutually exclusive within the same group, meaning that selecting one radio button automatically deselects any previously selected one in the same group. This makes radio buttons ideal for scenarios where only one choice is allowed. Radio buttons are widely used in various applications due to their simplicity and clarity. Some of the common use cases include user preferences, for example, choosing a language preference in an app setting, surveys and forms where respondents are asked to select one option from a list of possible answers, or it could be a payment option, allowing users to select one payment method, such as a credit card or a debit card or a PayPal. These examples demonstrate how radio buttons help in guiding users to make a clear and single selection, ensuring that app's logic follows the user's intent. Now let's move on to implementation steps for adding radio buttons in Android Studio. The first step is to define the radio buttons in your layout file, typically ActivityMain.xml file. Here is how you can add a group of radio buttons using the radio group tag. In the radio group tag, you have to define the radio button tag as well, with different options. In the code snippet, you also see that there is a text view tag which is also added. In this code, we have created a radio group that contains multiple radio button elements. The radio group ensures that only one radio button can be selected at a time. A text view is added to display the selected option. After adding the radio buttons to your layout, the next step is to handle the selection logic. And this is done in the MainActivity.kt file using Kotlin. Here is how you can manage the selection and display the result. First we have to get the reference of the radio group by using findViewById. And on this radio group reference, we have to call the method. The setOnCheckedChangeListener method listens for changes in the selected radio button and updates the text view with the selected option's text. This provides immediate feedback to the user based on their selection. Radio buttons are widely used in various applications due to their simplicity and clarity. Some common use cases include user preferences, for example, choosing a language preference in an app setting, or surveys and forms, where respondents are asked to select one option from a list of possible answers, or it could be payment options, allowing users to select one payment method, such as credit card or debit card or PayPal. These examples demonstrate how radio buttons help in guiding users to make a clear and single selection, ensuring that app's logic follows the user's intent. Let's now see the output of radio button implementation in the emulator. After following the steps we have discussed, you should be able to run your application in Android Emulator. When you select an option, the text below the radio buttons will display the selected option's text, such as Selected Option 1. This output demonstrates how radio buttons work in practice, providing users a clear feedback based on their selection. It's a simple yet effective way to enhance the interactivity of your Android application. To summarize, radio buttons are a useful UI

element in Android that allow users to make single selections from a group of options. We discussed the significance of radio buttons, common use cases, and how to add them to your Android layout. We also explored how to handle radio button selections using Kotlin, and we saw the practical output in the emulator. By implementing radio buttons in your Android projects, you can create more interactive and user-friendly applications that guide users to make clear and precise selections. Thank you for your attention throughout this session. I hope you now have a better understanding of how to implement radio buttons in Android, and how to use them effectively in your projects. Thank you.

Introduction to Menus in Android Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will be discussing a important aspect of Android development that would be menus. Menus are essential part of any Android application, providing users with options and actions that can be performed in different contexts. In this session, we will explore the different types of menus available in Android, the process of implementing them in Android Studio, and practical use cases where menus enhance user experience. By the end of this session, you will understand how to create and use menus effectively in your Android applications. Let's begin by understanding what a menu is in Android and its significance. A menu in Android is a set of options or actions that are available to users. These menus are typically accessed through a button or an icon, providing users with additional functionality within an app. There are different types of menus in Android, including options menus, context menu, and pop-up menu. Each serves a unique purpose and is used in different contexts within an app. Menus are vital for enhancing user experience by organizing actions and options in a structured manner, making it easier for users to interact with the app. Now let's focus on pop-up menu, which is a type of menu that appears as a floating list of items anchored to a specific view. A pop-up menu is typically used for providing contextual actions that are related to a specific element on the screen. For example, in a file manager app, a pop-up menu might be used to display actions like rename, delete, or share when a user taps on a file. The key advantage of a pop-up menu is that it provides relevant options without cluttering main UI, making it an excellent tool for improving user experience. Now let's see as to how to implement pop-up menu in Android Studio. The first step is to set up the layout in the activity underscore main dot xml file. Here is how you can add a button that will trigger the pop-up menu. In this code, we have created a button with id show pop-up button. This button will be used to anchor the pop-up menu, meaning the menu will appear relative to this button when clicked. After setting up the layout, the next step is to implement the code for pop-up menu in the main activity dot kt file, and in the code snippet you can see as to how to do it. In this code, the show pop-up menu function creates and displays the pop-up menu when the button is clicked. The menu items are defined in a separate xml file, which is main menu dot xml, and set on menu item click listener method handles the user selection. The menu items that appear in the pop-up menu are defined in another file, which is main menu dot xml file, located in the resources folder, under which you have a menu subdirectory. Here is how you can define the menu items, using a menu tag. Under the menu tag, you define various items using the item tag, which has an id and title. In this code, three menu items are defined with unique ids and titles. These items correspond to the actions that the user can choose from when the pop-up menu is displayed.

Pop-up menus are commonly used in various applications to enhance user interaction. Some practical examples include list item actions, displaying actions related to a specific item in a list such as edit, delete, or share, or contextual editing, where providing options to edit or modify content directly from the UI element being interacted with. These use cases demonstrate how pop-up menus can be used to provide a clean and efficient way for users to access additional functionality without cluttering the main interface. Finally, let's see the output of the pop-up menu implementation in the emulator. When you run the application and click on show menu button, the pop-up menu will appear with options option 1, option 2, and option 3. Typing any one of these options will trigger the corresponding action as defined in your code. This output shows how pop-up menus can be integrated into your Android application to provide users with contextual options and enhance the overall user experience. To summarize, menus are vital component in Android applications, providing users with easy access to various actions and options. We discussed the concept of menus, focused on pop-up menu, and walked through the implementation steps in Android Studio. We also explored real-world use cases where pop-up menus are commonly used and saw the practical output in the emulator. By mastering the use of pop-up menus, you can create more interactive and user-friendly Android applications that cater to the needs of your users. Thank you for your attention throughout this session. I hope you now have a better understanding of how to implement pop-up menus in Android and how to use them effectively in your projects. Thank you.

Introduction to Navigation Component Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will be discussing an essential topic in Android development, and that would be navigation. Navigation is a critical aspect of any mobile application, guiding users through the app and helping them find the information or features they need. In this session, we will explore the different types of navigation in Android, the importance of effective navigation, and the common navigational UI components used to create a seamless user experience. By the end of this session, you will have a solid understanding of how to implement navigation in your Android applications effectively. Let's begin by understanding what navigation in Android means. Navigation refers to the way users move through an app screen and interact with its features. Effective navigation is crucial because it helps users find what they need quickly and intuitively, improving their overall experience. In Android, navigation can be implemented in various ways, depending on the app's structure and users' need. We will discuss four common types of navigation. That would be linear, hierarchical, lateral, and backstack navigation. Each type serves a different purpose and is used in different contexts within an app. Let's explore the different types of navigation in Android. First one is a linear navigation. This type of navigation follows a straightforward path, where users move from one screen to the next in a sequence. It's commonly used in onboarding flows or step-by-step processes. Then we have the hierarchical navigation. This structure allows users to move between screens that have a parent-child relationship. It's useful in apps where content is organized in a hierarchy, such as Settings menu. Then we have the lateral navigation. Lateral navigation enables users to switch between sibling screens or content of the same level, often using swipe gestures or tabs. This type is common in apps that require quick access to related content. Finally, we have the backstack navigation. Backstack navigation is a system that allows users to return to previous screens by

pressing the back button. This type of navigation is automatically managed by Android, ensuring that users can retrace their steps through the app. Each of these navigation types plays a unique role in guiding users through an app and ensuring that they can access the information they need efficiently. Effective navigation is critical for several reasons. The first one is the user experience. Good navigation contributes to a positive user experience by ensuring consistency, efficiency and logical flow between screens. Then we have the user retention. An app with intuitive navigation keeps users engaged and reduces the likelihood of them abandoning the app outside of frustration. When navigation is easy to understand, users can move through the app without confusion, leading to a more satisfying experience. Finally, we have the ease of use. A well-designed navigation system makes it easier for users to accomplish tasks, whether it's finding information, completing a purchase or accessing settings. By focusing on these aspects, you can create an app that not only meets user needs but also provides a pleasant and efficient user experience. Let's discuss the difference between using a single activity and multiple activities for navigation. Single Activity Model In this model, the entire app's navigation is managed within a single activity using fragments. This approach is efficient and keeps the app's structure simple, especially when using modern navigation components. Then we have the Multiple Activities Model. This traditional approach involves creating a new activity for each screen. While it's straightforward, it can lead to a more complex app structure and may require more management of backstack and transitions. Both models have their advantages and can be chosen based on the complexity of the app and the developer's preference. Then we have the Fragment-Based Navigation. This is commonly used in the Single Activity Model, where different fragments represent different screens or sections of the app. To implement navigation effectively, Android provides several UI components. First one is the Action Bar or Toolbar. This component provides a consistent place to put the app's main navigation options, such as the back button, title and overflow menu. Then we have the Navigation Drawer. This is a panel that slides in from the side, typically used for app-wide navigation. It's useful in apps with multiple sections or categories. Then we have the Bottom Navigation Bar. This component displays the primary navigation destinations at the bottom of the screen, allowing users to switch between top-level views quickly. Then finally we have the Tabs. Tabs provide lateral navigation between sibling content, often used with swipe gestures to navigate between sections of the same level. These components help create a structured and intuitive navigation system that enhances the overall user experience. To summarize, navigation is a fundamental aspect of Android development that guides users through an app's features and content. We discussed the importance of effective navigation, the different types of navigation structures and the key UI components used to implement navigation. By understanding these elements, you can create Android applications that are not only functional, but also easy and enjoyable to use. Thank you for your attention throughout this session. I hope now you have a better understanding of using navigation in Android. Thank you.

Implementing Navigation Video• . Duration: 4 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we will be discussing an essential aspect of Android development, that is implementing navigation. In this session, we will walk through the process of implementing basic navigation in Android, specifically moving from one activity to another using a button click. By the end of this session, you will be able

to set up navigation in your Android applications effectively. Navigation in Android allows users to move between different screens or activities within an app. This is a fundamental feature that contributes to the overall usability of the application. For example, when a user clicks a button in main activity, they must be taken to a new screen like second activity, where they can access additional content or features. Implementing navigation is straightforward, but crucial for creating a seamless and intuitive user experience. Let's begin by setting up main activity class 5. In this code, activity will include a button labeled Go to second activity. When the user clicks this button, they will be navigated to the second activity. Set on click listener is called using the button object and an intent with reference to the target activity is the input parameter to the intent object. The intent specifies second activity as the destination. The start activity method with the intent object is called to start the new activity. This simple setup is all you need to navigate from main activity to second activity. Now let's create second activity, which is the screen that the user will be navigated to after clicking the button in main activity. The user interface for second activity will be simple, containing just a text view with a welcoming message. This layout centers the text in the middle of the screen, providing a clear and straightforward welcome message to the user upon navigating to the second activity. To ensure that both main activity and second activity are properly recognized by the Android system, we need to declare them in the androidmanifest.xml file. In this configuration, main activity is set as a launcher activity, meaning that it will be the first screen displayed when the app starts. Second activity is also registered, ensuring that the system knows it exists and can be navigated to. Let's now see the output in the emulator. When you run the application and click the Go to second activity button in the main activity, you will be navigated to the second activity where the message, welcome to the second activity is displayed. This simple navigation example demonstrates how easy it is to move between activities in Android, creating a smooth and intuitive user experience. In summary, navigation is a fundamental part of Android applications, allowing users to move seamlessly between different screens. We covered the basics of implementing navigation by setting up main activity with a button to trigger navigation and also creating second activity. We also configured androidmanifest.xml file by registering both the activities. By understanding these steps, you can enhance the usability of your Android app, making them more intuitive and user-friendly. Thank you for your attention throughout this session. I hope now you have a better understanding of how to implement basic navigation in Android. Thank you.

Back Button Navigation Video• . Duration: 4 minutes

[Notes]

Hello everyone, and welcome to this session on mobile app development. In this session, we'll be discussing an important aspect of Android development, that would be the back button navigation. Any Android application, navigating back to a previous screen is a fundamental feature that enhances user screen. In any Android application, navigating back to a previous screen is a fundamental feature that enhances user experience. In this session, we will explore how to implement and customize the back button navigation in Android, ensuring that users can smoothly return to the previous activity. By the end of this session, you will understand how to effectively manage back button behavior in your Android application. Navigation in Android is not just about moving forward between screens. It's equally important to allow users to move back smoothly. The back button, which is a standard feature on Android devices, lets users return to the previous

screen. This feature is critical for maintaining a logical flow within your app, which helps users to retrace their steps without confusion. While the default behavior of the back button is usually sufficient, these are situations where customizing this behavior can significantly improve the user experience. In this session, we will focus on how to enhance navigation by customizing the back button in second activity to return to main activity. To customize the back button behavior in Android, you need to overwrite the `onBackPressed` method in your activity. This method is called when the user presses the back button on their device. By overriding this method, you can define what happens when the back button is pressed. For example, in second activity, you might want to ensure that pressing the back button closes the current activity and returns to the main activity. Here is how you can do it. In this code, calling the `finish` method inside `onBackPressed` method closes second activity effectively taking the user back to the main activity. This simple customization ensures that the navigation flow remains smooth and intuitive. Customizing the back button behavior is useful in various scenarios. For instance, if your app has complex navigation with multiple nested activities, you might want to ensure that pressing the back button returns the user to a specific activity, rather than simply closing the current one. This is especially important in apps where the user needs to revisit a particular screen before exiting the app or continuing with other tasks. By properly managing the back button, you can prevent users from getting lost or frustrated, thereby improving the overall user experience. Let's now see the output of this implementation in the Android emulator. When you run the application, navigate to the second activity by clicking on the appropriate button in main activity. Then press the back button on your device. You will notice that second activity closes and you are returned to the main activity. This simple demonstration shows how effective back button customization can be in managing the user's navigation experience, ensuring that they can easily return to the main screen without any confusion. To summarize, back button navigation is a critical feature in Android that allows users to move smoothly between activities. We discussed the importance of enhancing navigation by customizing the back button, how to override `on back press` method in second activity and the practical output of this customization. By understanding back button customization, you can create Android applications that provide a more intuitive and user-friendly navigation experience. Thank you for your attention throughout this session. I hope you now have a better understanding of how to implement and customize back button navigation in Android. Thank you.

Hierarchical Navigation Patterns Video• . Duration: 5 minutes

[Notes]

Hello everyone, welcome to this session on mobile app development. In this session, we will be discussing an important topic in Android development and that is the hierarchical navigation. Hierarchical navigation is commonly used in applications where users need to navigate through layers of related content such as settings menu, file directories or category based apps. In this session, we will explore what hierarchical navigation is, real-time examples of its use and the benefits and considerations of implementing this navigation pattern. By the end of this session, you will have a good understanding of how to effectively use hierarchical navigation in your Android applications. Hierarchical navigation is a navigation pattern where users move through layers of related content in a structured way. This type of navigation is ideal for apps that require users to drill down through categories, subcategories and details. For example, in a settings menu, users might

start at a top-level category like display and then navigate through subcategories like brightness or font size to specific settings. Similarly, in file manager app, users navigate through folders and subfolders to locate a file. The structure provided by hierarchical navigation helps users easily find and manage content within the app. Let's take a look at some of the real-time examples where hierarchical navigation is commonly used. First one is the settings menu. Most settings menu in Android devices use hierarchical navigation. Users start with broad categories like network and internet or system, then drill down to more specific settings. Then we have file managers. File management apps also heavily rely on hierarchical navigation. Users navigate through directories and subdirectories to organize or find their files. Then we have the e-commerce apps. In e-commerce apps, hierarchical navigation is used to help users browse through product categories. For instance, a user might start with top-level category like electronics, then narrow down to mobile applications, further into smartphones. These examples show how hierarchical navigation is an effective way to organize content and help users find what they need in a structured manner. Hierarchical navigation offers several benefits that make it an excellent choice for organizing content. First, it provides a clear structure. This navigation pattern provides a clear and logical structure, making it easier for users to understand where they are within the app. Then scalability. Hierarchical navigation scales well as your app grows. You can easily add new categories or subcategories without disrupting the overall navigation flow. Good user control. By allowing users to navigate through categories and subcategories, hierarchical navigation gives them greater control over how they explore the content. These benefits contribute to a more intuitive and user-friendly experience, especially in apps with complex content structures. While hierarchical navigation is powerful, it's important to implement it carefully to avoid common pitfalls. Here are some considerations and best practices. First one is to avoid using deep nesting. Too many levels of nesting can confuse users and make it difficult to navigate. Try to keep the hierarchy as shallow as possible. Then it is to provide context. Always provide clear labels and breadcrumbs to help users understand where they are in the navigation hierarchy. This ensures they don't get lost. Then is to maintain consistency. Good design and behavior across all levels of hierarchy helps users learn how to navigate your app quickly. By following these best practices, you can create a navigation system that is both efficient and easy to use, enhancing the overall user experience. To summarize, hierarchical navigation is a widely used pattern in Android applications that allow users to navigate through layers of related content. We discussed what hierarchical navigation is, looked at real-world examples like settings, file managers, and e-commerce apps, and explored the benefits such as clear structure, scalability, and user control. We also covered some best practices to ensure your implementation is effective and user-friendly. By understanding hierarchical navigation, you can create apps that are organized, intuitive, and easy to navigate. Thank you for your attention throughout this session. I hope now you have a better understanding of hierarchical navigation and how to implement it. Thank you.

Introduction to Fragment Using XML Video• . Duration: 7 minutes

[Notes]

In this demo, we are going to see how to work with fragment using XML. So what I have done now is created an application and this is the XML file which you are seeing and I have the main activity which is defined over here. So first what I have to do is in my layout file, I will have to make space

for me to put in the two fragments inside. So for which I'm going to make use of the fragment pack, okay, and define the content as wrap content and wrap content for width and height. And let me close this for now. And I'm going to add one more fragment. Okay. And here also, I will specify it as wrap content. And let me give ID to each of the fragment. Fragment 1 and this one will be my Fragment 2, okay. It is showing me that these two are not constrained. So let me go to the design. Let me select Fragment 1, rather, let me select Fragment 2, the one which is selected, then I'm going to define the constraint for it, okay. So let me, okay, let me not define that. I will have to enlarge this. So now let me pull this one. So I'm going to align it horizontally to the parent as well as vertically, let me do it manually. And I will constrain it to this, okay. Let me pull this one over here as well. I'll expand it. And Fragment also, I'm going to increase the size, okay. So this is how it will look on the screen. So if I go back to the design, you will see that the error is gone, okay. So what I'll do is, I'll create two Fragments now, right click on your package and go to new and click on Fragment and a blank Fragment, of course, there are different types of Fragments, but we would just require a blank Fragment. Let me name this as First Fragment, okay, by default, it's going to create a layout also for us, that's fine for us. So let me click on finish. So you will see that there is a Fragment first layout which is created and there is also the Fragment file which is created for us, okay. So here, what I'll do is, I'll go to the layout file of the Fragment and let me change this layout from Frame layout to Constraint layout, okay. So that it is easier for me to move around this and let me add the text over here as Fragment 1, okay. Similarly, what I'll do is, I'm going to go to create another Fragment which is again a blank Fragment and this will be named as Fragment 2, okay, or rather second Fragment, okay. Let me click on finish. So you can see that my second Fragment is also created along with its XML layout file. So here also, let me change this Frame layout to Constraint layout, okay. So I will have to add the Constraints to this, which I'm going to do in a bit. So I will say Fragment 2, I can even increase the text size to say like 20sp, let me do it here as well, I'll use text size 25sp, okay. So now what I have to do is, I'll go back to my main activities layout file. So in this first Fragment, I will have to define the class, okay, there are two Fragments and you can see that they have already popped out. So let me put the first Fragment over here and in the second Fragment, I'm going to place the second Fragment, okay. Let me quickly check as to how my first Fragment is going to look like, I think I will have to add the, okay, so it is occupied the whole thing. So it is matching the parent, that's fine. So here also it should match the parent. So this will go into that Fragments, right? So it should be fine. So let me run the application. So Gradle is going to build this application, install and it's going to run for us. You can see that Fragment 1 is present over here and Fragment 2 is also present over here. Basically what we have done is, we have placed both the Fragments in the layout file of the main activity. So this is how we add Fragments to the main activity file using XML. Of course, this is not the preferred way to do it. It's better that we program it and programmatically we add these Fragments so that it gives us flexibility to add, replace or remove the Fragments.

Add Fragments to Activity Programmatically Video• . Duration: 6 minutes

[Notes]

In this demo, we are going to see as to how to work with fragments by adding them programmatically to our activity. So what I have done is, I have continued from our previous

example. So main activity was always created. So in the its layout file, I have created two buttons, okay, named it as fragment one and fragment two. So when I click on fragment one button, the fragment one will be displayed on the main activity or it will be added to the main activity. When I click on fragment two, then fragment two will replace fragment one in the main activity, okay. And I have created a fragment container using the frame layout. And I have specified the width and height of it. So this particular fragment container will be used to load my fragments. Like in previous case, I have created the two fragments that is fragment one and fragment two. In fragment one and two, there is a factory method which is called as companion object. So this particular companion object will be used, this particular method will be used to create a new instance of any fragment, okay. So that is present in fragment two as well. So you can see that there is a new instance method which is here as well. In fragment one, I just have a text view which names this particular fragment as fragment one. Similarly, I have the same in fragment two as well. So let me take you to the main activity. So here, as you can see, what I have done over here is I have called the fragment manager and I have begun a transaction. And I am going to call the apply, which is going to basically call the specified function block. So this is the function block which I have defined over here that is replace and commit, okay. And it is going to return. So basically using this, what I am doing is I am basically adding the fragment one by creating a new instance of it into the fragment container, and I am committing that particular transaction, okay. So let me start with this. So as soon as this activity gets created, I should be able to see the text fragment one. So let me run this. So you can see that the two buttons are there, F1 and F2, and I can see the fragment one loaded over here, okay. Now what I have to do is I have to define the onClickListener on button one and button two, okay. So let me do that. So I have defined the onClickListener for button one. And I am also displaying a pop up which says F1 for a short duration, and I am replicating the same code which I did when the initial layout file was being displayed. So using the fragment manager, I am beginning a transaction, and I am creating the object of the new in, I am creating the object of fragment one, and I am asking it to get loaded in the fragment container, okay. Similarly, when I click on fragment two button, okay, same thing happens, but only changes instead of fragment one, I'm creating the instance of fragment two, and that will get loaded into the container. Okay, so let me run this particular example for you. So if you see the application is loaded, fragment one is already there, when I click on fragment two, fragment two is loaded, okay, but what happened to fragment one, is it gone to the back stack might not be okay, because when I click on the back button, okay, what should happen is this particular text which is showing fragment two should be replaced by fragment one, that is fragment two will be destroyed and fragment should get displayed to me, let me click on the back button and see what happens. So when I click on the back button, I see that the whole application, okay, it's destroyed, including the activity. Okay, the reason is, as we saw in activities, back stack is not automatically maintained for fragments. So we need to specify in the code to tell that our respective fragment when it gets added to the container, we should add that to the back stack. So for that, we have this method called as add to back stack. So this will be called within this particular apply call, same I'm going to do it over here as well. Okay, so I'm going to add this particular fragment to the back stack, whenever there is a new fragment which comes over this, okay. So let me run this particular example. So we will see a small change in the behavior now, whenever we click on the back button. Okay, so we see that fragment one is already loaded, I'll click on fragment two. So fragment two is as replaced fragment one now, and fragment one is added to the back stack. So when I click on back button, what's going to happen is this fragment two will be destroyed and fragment one which was added to the back stack will be displayed to the user. So when I click on the back button, you

see that fragment two was destroyed, and fragment one is getting displayed to the user. So when I click on the back button, again, even fragment one gets destroyed and only activities there. So when I click on the back button again, the activity which was by default added to the back stack now is getting destroyed, okay?

Networking Basics Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this module on networking and web services in Android development. I'm Professor Chandrasekhar RK, and in this session, we'll be covering the fundamental aspects of networking in Android applications. Networking allows mobile apps to connect to remote servers, APIs, and databases to send and retrieve data. This is essential in modern apps where data from internet is used to provide services such as loading images, fetching user details, or submitting form data. In this session, we will discuss the basics of networking in Android. Networking is a crucial part of mobile apps. It allows the app to interact with web servers or remote databases to either send or retrieve it. For example, when a user opens a news app, it fetches the latest news from a server and displays it. Similarly, when someone submits a form or logs into an app, that data is sent to the server for processing. Some of the common use cases of networking include fetching and displaying data from web services, sending data like user details to a server, loading images or media from the internet. Networking enables the app to provide dynamic real-time content to the user, making it more functional and interactive. Now let's discuss about the key components of networking in Android. The most important protocols are HTTP, which stands for Hypertext Transfer Protocol. This is the standard protocol used to send and receive data over the internet. Then we have HTTPS, which is Hypertext Transfer Protocol with security. This secure version of HTTP, where data is encrypted for privacy and security. The basic structure of network requests involves a request. The app sends a request to a server, such as a GET request to fetch data or a POST request to send data. Then we have the response. The response is from the server side. The server responds with the requested data, often in formats like JSON or XML, which the app processes to display or use it. Now let's see how we can handle network requests in Android. Here is an example of how to make a GET request to retrieve data from a URL. To handle a network request, first we define the URL. Now this is the address you want to connect to. Then, using HTTP URL connection, we open a connection to connect to the server. Then we have to set the request method. This could be GET or POST, depending upon what you want to do. Then we have to handle the response. After connecting, you read the data from the response and process it, often by parsing the JSON or XML data which is received. Now that we know how to make network requests, it is important to follow some best practices to ensure that your app works efficiently and securely. First, it's about efficient networking. Minimize the number of network requests by using caching to store data that does not need to be repeatedly fetched. Also, try to group smaller requests into one larger request to save battery life. Second one would be to handle the error. Always handle network failures or slow connections properly. Show appropriate user feedback like a retry button or an error message to inform the user when the network is not available. And the third thing is to look into the security. Always use HTTPS when sending or receiving sensitive data to ensure privacy and security. This encrypts the data so that it cannot be easily intercepted. By following these practices, you ensure that your app performs well under different network conditions and also keeps the user

data secure. To summarize this session, we looked into the networking part of it, the key components of Android networking, which includes protocols like HTTP and HTTPS. We learned how to handle network requests by opening a connection, making the request and processing the response. And finally, we even discussed the best practices like efficient request handling, proper error management and securing the sensitive data using HTTPS as the protocol. By understanding these concepts, you can implement effective networking strategies in your Android application. Thank you for joining today's session. I hope you now have a better understanding of networking in Android and how to handle network requests securely and efficiently. Thank you.

Including Permission in Manifest Video• . Duration: 4 minutes

[Notes]

Hello everyone, and welcome to this session on networking and web services in Android development. In this session, we will be discussing connecting to internet and include permissions in Android. Android applications often need to connect to the internet or access other system resources like the camera or location services. However, for security reasons, apps must request for the appropriate permission from the user. In this session, we will learn why permissions are necessary, how to declare them in Android manifest.xml file, and the importance of ensuring that permissions are requested properly. By the end of this session, you will understand how to manage permissions to ensure your app can access the required resources. Now, let's discuss why permissions are necessary in Android apps. Android uses permissions as a way to protect users from apps that might access sensitive data or resources without their knowledge. Permissions are required whenever an app needs to access certain system resources, such as internet for that matter, which is used for making network requests, camera for taking photos and videos, location for providing location-based services, external storage for saving or reading files, for Bluetooth or connecting with other devices wirelessly, SMS for sending or receiving messages. Without these permissions, apps would not be able to access these features. This helps ensure that users' privacy and security are protected. By granting permissions, users have control over what data or resources the app can access. To ensure that your app has necessary access, permissions must be declared in Android manifest.xml file. This file is located in your app's configuration and specifies all the permissions your app will need. For example, if your app requires access to internet, you need to declare the internet permission, which is shown in the slide. This declaration tells the system that your app will perform network operations, such as making HTTP requests. Without this permission, the app would not be able to connect to the internet. It's important to declare only the permissions your app truly needs to maintain user trust and avoid unnecessary access to sensitive data. Now let's talk about the importance of declaring permissions in Android. If you fail to declare the required permissions, such as internet, your app won't be able to perform the associated actions like making network requests. Properly declaring permissions is not only necessary for functionality but also ensures that users are aware of what data or resources your app needs to function. There are three main reasons why correctly declaring permissions is important. First one is the control. Users have control over the data they share with apps, which builds trust. Second one is transparency. Declaring permissions clearly shows the user what data your app will access and why. Third one is data minimization. You should only request the permissions necessary for the task your app needs to perform. This ensures your app respects user privacy and minimizes its data access. To

summarize this session, permissions are required in Android to access system resources like internet, camera, storage and more. And you must declare all necessary permissions in AndroidManifest.xml file. And declaring permissions properly ensures control, transparency and data minimization. By understanding these concepts, you can ensure your Android apps are functional, secure and respect user privacy. Thank you for your attention today. I hope this session has given you a clear understanding of permissions in Android and how to declare and request them properly. Thank you.

HTTP Basics Video• . Duration: 5 minutes

[Notes]

Hello everyone. Welcome to the module on networking and web services. In this session, we will focus on HTTP basics in Android development. HTTP or Hypertext Transfer Protocol is the foundation of data communication on the web. It allows apps to send and retrieve data from web services. In this session, we will discuss what HTTP is, how it works in Android and demonstrate an example of making an HTTP request. By the end of this session, you will understand how to use HTTP to fetch or send data in Android apps. Now let's talk about what HTTP is. HTTP or in other words, Hypertext Transfer Protocol is the standard protocol used for data communication on the web. It's a language that web browsers and web servers use to talk to each other. When you open a website, the browser sends an HTTP request to the server and the server responds with the requested data, such as web pages or images. There are two key operations in HTTP. The first one is the GET method. This method is used to retrieve data from the server. For example, when you open a web page, the browser uses a GET request to download the content. Then you have the POST method. This method is used to send data to the server for processing. For instance, when you fill out a form and submit it, the data is sent to the server using a POST request. In Android development, HTTP is used to communicate with web services, making it possible for apps to fetch and send data to the internet. Let's now see how HTTP is used in Android apps. Mobile applications often need to communicate with web services or APIs to fetch data like weather information, news articles, or even user details. HTTP is the protocol that allows this communication to happen. For example, when your app needs to load data from a web server, it makes an HTTP request, such as a GET request to retrieve data. Similarly, when an app needs to send information, like submitting a form or uploading a file, it uses POST requests. In Android, HTTP URL connection is commonly used to make HTTP requests. It helps you open a connection to a server, send the request, and process the server's response. Now let's take a look at an example. Here is an example of how you can make a GET request in an Android app using HTTP URL connection. Let's assume we want to fetch some data from a web server. In this example, we first define the URL we want to connect to. Then we open a HTTP connection using HTTP URL connection. Then we set the request method to GET to fetch user data. The response from the server is processed, and the data is read using a buffered reader. This allows the app to retrieve data from the server and display it or process it further. This process is essential when building apps that need real-time data from web services. Let's see the output in the Android emulator. After running the code, the app will make a GET request to a server and display the data. For example, a TOAST message can be used to show success of HTTP request or the data fetched from the server can be displayed in a text view or processed using a buffered reader. This demonstrates how HTTP requests can be integrated into

Android apps, allowing them to fetch or send data over the internet. You can apply this concept to create apps that require real-time data, such as weather applications, news applications, or e-commerce applications. To summarize this session, we have gone through what is HTTP, which is the standard protocol used for data communication, and we have seen in Android development as to how HTTP is being used in order to communicate with web services or APIs. We also saw as to how to use GET or TOAST methods, and there was a simple example which made a HTTP GET request in order to fetch an item from the server. By understanding how HTTP works and how to use it in Android apps, you can build applications that interact with web servers and provide dynamic content to the users. Thank you for your attention today. I hope this session has given you a clear understanding of HTTP in Android and how to use it to connect to web services and APIs. Thank you.

Managing the Network State Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to this module on networking and web services. In this session, we will focus on an important topic that is managing network state in Android. In mobile apps, the network state can change frequently, such as switching from Wi-Fi to mobile data or losing connection entirely. It is essential for Android apps to handle these network changes appropriately. In this session, we will explore how to check network state, manage it properly, and also look at best practices for dealing with network changes. By the end of this session, you will be able to handle network states effectively in your Android applications. Now let's discuss why it is important to manage network state in Android. The network state can change frequently as users move between different network connections, such as Wi-Fi, mobile data, or no connection at all. Apps that do not handle these changes properly may attempt to make network requests when there is no connection, leading to failed requests and potential data loss. For example, think of a situation where a user is watching a video or downloading data. If the app does not detect that the network connection has dropped, it may continue attempting to download, leading to errors or wasted resources. This negatively impacts the user experience. By managing the network state effectively, you can ensure that the app only performs network-related tasks when the connection is available. Now let's learn how to check the network state in Android. You can use the `ConnectivityManager` class to check the device's current network state, such as whether it is connected to Wi-Fi or mobile data or if there is no connection. Here is an example as to how you can check the network status. In this code, we use `ConnectivityManager` to check the active network connection. The app checks if the network is connected using `networkInfo.isConnected` method. If the device is connected, the app can safely proceed with network operations. Otherwise, it can show an offline message or retry option. This simple check helps prevent failed network operations by ensuring the app only makes requests when the network is available. To ensure a smooth user experience, there are some best practices that you should follow when managing network state in Android. First one is to handle transitions gracefully. When the network state changes, such as switching from Wi-Fi to mobile data or losing the connection entirely, provide user feedback. For example, show an offline message or display a retry button. This prevents users from being confused if the app suddenly stops working. And second one is to avoid unnecessary requests. Safely perform network operations when a connection is available. This helps prevent failed requests or ensures that your app does not waste

data or battery power when the network is down. For instance, avoid downloading large files when the connection is weak or intermittent. Third one is to show retry options. If a network request fails due to connectivity issues, offer users the option to retry once the network is available again. This keeps users engaged with the app or prevents frustration. Following these best practices will help you create apps that handle network changes effectively, which improves the user experience as well as the app performance. To summarize this session, we covered a few key points, like need for managing network state. It is important to manage network state to prevent failed network requests. And we also saw as to how to check a network state using `ConnectivityManager` class. Then we also saw as to how to make use of best practices, which was discussed, which includes handling transition gracefully, avoiding unnecessary requests, and offering retry options. By following these guidelines, you can build Android apps that handle network changes efficiently, ensuring a smooth experience for users. Thank you for your attention. I hope this session has given you a clear understanding of managing network state in Android. Thank you.

Connectivity Manager Video• . Duration: 6 minutes

[Notes]

Hello everyone, welcome to the module on networking and web services. In this session, we will focus on an important component of Android which is Connectivity Manager. Connectivity Manager class provides access to network services and helps apps monitor and manage the device network connectivity, whether it is switching between Wi-Fi and mobile data or checking for a stable internet connection, Connectivity Manager allows developers to ensure that network-related tasks are handled efficiently. By the end of this session, you will be able to understand what Connectivity Manager is, how to monitor network changes, and explore an example use case for managing connectivity in Android apps. Now let's talk about what Connectivity Manager is. The Connectivity Manager class in Android provides access to network connectivity services. It allows your app to monitor the device network connections such as Wi-Fi, mobile, and even VPN connections. Connectivity Manager helps apps adapt to changes in the network status such as switching from Wi-Fi to mobile data, ensuring that apps can continue functioning smoothly without crashes when no network is available. For example, apps can check whether they are connected to the internet before making a request to download or upload data. This improves the user experience by avoiding failed operations or errors that occur when there is no network. One of the most important features of Connectivity Manager is the ability to monitor network changes. And here is how you can check if the device is connected to the internet or not. In this code, we use Connectivity Manager to get information about current network. We check if the network is active and connected by using the method `ActiveNetwork.isConnected` Depending upon whether the device is connected or not, the app can perform the required actions such as showing a message or retrying the network operation. By monitoring network changes, apps can handle connectivity efficiently and prevent failed operations. Let's now look at some example use cases of Connectivity Manager in Android apps. First one is checking network availability before making requests. Before fetching data from a remote server, the app should check if the device is connected to Wi-Fi or mobile data. This helps prevent the app from attempting network operations when there is no connection, avoiding crashes or failed requests. Second one is to detect network type. It could be either Wi-Fi or mobile data. Apps can detect whether the user is on Wi-Fi or mobile data and optimize their behavior

accordingly. For example, a video streaming app might reduce video quality on mobile data to save bandwidth. Third one is to react to network changes. Apps can update their behavior automatically when the network state changes. For example, a messaging app can queue messages for later when offline and send them automatically once the connection is restored. Fourth one is to handle background syncing. Apps that sync data in the background should only perform syncing when there is an active and stable network connection. For instance, a note-taking app might delay syncing notes with the server until a reliable Wi-Fi connection is available. Then we have showing custom alerts when network is unavailable. Apps can display custom alerts to inform users when there is no network connection. For example, an e-commerce app might show no internet connection message when trying to load products without a network. Then we have limiting data-intensive operations based on network type. To prevent high data charges, apps can restrict data-intensive operations such as auto-uploads or downloads to Wi-Fi connections only. A cloud storage app might allow auto-uploads only when connected to Wi-Fi to save mobile data and costs. To summarize this session, we saw as to what ConnectivityManager is, which is an essential class in Android that provides access to network connectivity services. We also saw as to how to make use of ConnectivityManager to monitor network changes. Then we also explored certain example use cases where ConnectivityManager can help ensure smooth app functionality. By understanding ConnectivityManager, you can ensure that your Android apps are robust, responsive and provide a seamless user experience even in varying network conditions. Thank you for your attention throughout this session. I hope you now have a better understanding of how to use ConnectivityManager and to monitor and manage network changes in Android apps. Thank you.

Introduction to Broadcasts Video• . Duration: 6 minutes

[Notes]

Hello everyone. Welcome to this module on networking and web services. In this session, we will focus on a key topic in Android, which is broadcasts. Broadcasts are a mechanism that allows apps to send or receive messages from the system or from other applications. These messages help apps respond to various events like the battery being low, network status changing, or even app-specific events like data being updated. In this session, we will explore how broadcasts work in Android, the types of broadcasts, and some important use cases as well. By the end of this session, you will have a clear understanding of how to implement and manage broadcasts in Android applications. Let's begin by discussing what broadcasts are. In Android, a broadcast is a message that the system or an app sends to other apps or components when certain events happen. These events could be system-related, like when the device boots up or the battery is low, or app-specific, like when a certain task is completed. For example, if an app needs to know when the phone battery is low, it can register to listen for a battery-low broadcast. When the system sends that broadcast, the app can respond accordingly, such as by saving battery or reducing background activity. Similarly, apps can also send their own broadcast to notify the other apps or system components about important events. Now let's discuss the types of broadcasts available in Android. There are two main types. One, the system broadcasts. These are broadcasts sent by the Android system when certain events occur. Examples like, battery low. When the device battery is running low, you get a broadcast from the system to the apps. Next one, it could be like network status change. This would come in when the device connects to or disconnects from a network. Device boot completed.

When the device has finished starting up. Next one is custom broadcasts. These are broadcasts sent by apps to communicate specific events to other components or apps. For instance, an app might send a custom broadcast when a new data is available for other apps to use. By understanding these two types of broadcasts, you can decide when and how to use them in your Android applications. Now let's look at how broadcasts work in Android. Broadcasts are sent using intent objects. These intents contain an action string that identifies the type of event, such as `android.intent.action.battery_low`. The intent filter class is used to specify which broadcasts an app want to receive. Here is an example of how to register a broadcast receiver for battery low broadcast. In this code, we create an intent filter for battery low action and register the broadcast receiver to listen for that specific broadcast. When the system sends the broadcast, the app will be notified and it can take appropriate action, such as reducing battery usage or saving the state. Now let's explore some use cases for broadcasts. First one, monitoring battery status. Apps can listen for battery low broadcast to take action when the device battery is running low. This is especially useful for apps that perform background tasks like syncing data or downloading files. Then we could also use it to detect connectivity changes. Apps can listen for network status changes to ensure that they only make network requests when the device has an active connection. For example, a news app might stop refreshing data if the device is offline. We could also use it to notify when a new data is made available. An app can send a custom broadcast when new data is available. Other apps or services can listen for this broadcast and update their data accordingly. Broadcasts provide a simple way to allow apps to respond to both system level and app specific events, ensuring a better and more efficient user experience. To ensure that your apps work efficiently, here are some best practices for using broadcasts in Android. First, minimize the use of broadcasts. While broadcasts are powerful, using too many of them can put strain on system resources. Only use broadcasts when absolutely necessary. Then, prefer local broadcasts for app communication. If you are sending a broadcast to components within the same app, use local broadcasts instead of system-wide broadcasts. Local broadcasts are more efficient and don't need to be declared in the manifest file. Then you need to handle broadcasts efficiently. Make sure that your apps handle broadcasts efficiently by processing them quickly. Long-running tasks triggered by a broadcast should be handled in a background service or worker thread to avoid blocking the main thread. To summarize this session, broadcasts allow apps to send and receive messages from the system or other apps in response to events. So we discussed two types of broadcasts, system broadcasts sent by the Android system and custom broadcasts which are sent by apps. We explored how broadcasts work using intents and intent filters to deliver messages between apps. We also looked at common use cases such as monitoring battery status or detecting connectivity changes or reviewed some best practices to ensure efficient usage of broadcasts. Thank you for your attention. I hope this session has given you a clear understanding of broadcasts in Android and how to implement them in your apps. Thank you.

Broadcast Receivers Video• . Duration: 6 minutes

[Notes]

Hello everyone. Welcome to the module on Networking and Web Services. In this session, we will be discussing an important concept called as broadcast receivers. Broadcast receivers are a core component in Android that allow an app to listen for messages or broadcast intents sent by Android

system or other applications. These broadcasts can indicate events such as connectivity changes, low battery or app-specific triggers. By the end of this session, you will understand how broadcast receivers work, their lifecycle and how to implement them in Android applications. Let's start by understanding what broadcast receivers are. In Android, a broadcast receiver is a component that allows your app to listen and respond to broadcast messages sent by Android system or other applications. These broadcasts could include system-wide events such as network connectivity changes, low battery or even app-triggered events. For example, if your app needs to respond when device connects to Wi-Fi, you can use a broadcast receiver to listen for a network connectivity change broadcast. Broadcast receivers work by receiving intents, which are messages that indicate certain events or actions have occurred. They allow your app to respond to events even when app is not actively running. Now let's explore the types of broadcast receivers in Android. There are two main types. Static broadcast receivers. These receivers are declared in AndroidManifest.xml file and are registered at app installation. They can respond to system-wide broadcasts even when app is not running or is inactive. An example of this is a receiver that listens for the boot-completed event so that app can perform tasks after the device has started up. Now let's see the other type of broadcast receiver, which is the dynamic broadcast receiver. So dynamic broadcast receivers are registered programmatically while the app is running. They are automatically unregistered when the app is closed or the component that registered them is destroyed. Dynamic receivers are ideal for app-specific events that only need to be handled while the app is active, such as receiving push notifications or listening for media playback status. Understanding when to use static versus dynamic receivers is important for managing system resources efficiently. Let's now look at the lifecycle of a broadcast receiver. The broadcast receiver's lifecycle is quite simple compared to other Android components. It is active only when a broadcast event it has registered for occurs. Here is the basic lifecycle of a broadcast receiver. First we have to register the receiver. The app registers the broadcast receiver either statically in the manifest or dynamically in the code. The second step is to receive a broadcast. When a registered event occurs, the Android system delivers the broadcast as an intent to the receiver. As a third step, we have to implement the onReceive method. The onReceive method is called when you define the action the app should take in response to the event. Finally, we have to unregister the receiver. For dynamic receivers, the receiver is automatically unregistered when the app or the component is destroyed. Static receivers remain registered as long as the app is installed. The onReceive method should execute quickly since broadcast receivers have a limited amount of time to process the broadcast before the system considers them unresponsive. Now that we understand how broadcast receivers work, let's see how to implement a broadcast receiver in an Android app. The implementation involves the following steps. First we create a class that extends the broadcast receiver class. Then inside the class, we overwrite the onReceive method to define how the app should react when it receives a broadcast. Then we have to register the receiver in AndroidManifest.xml for static receivers or programmatically for dynamic receivers. By following these steps, you can effectively implement broadcast receivers to handle system-wide or app-specific events in your Android applications. To summarize this session, broadcast receivers allow your app to listen and respond to broadcast messages from the Android system or other apps. And there are two types of receivers, static and dynamic. Static broadcast receivers are registered in the manifest, work even when the app is inactive. And dynamic broadcast receivers, which are registered programmatically while the app is running. And the broadcast receiver lifecycle is straightforward. It registers, responds to the broadcast using onReceive method, and then unregisters itself. We also discussed the steps for

implementing a broadcast receiver, which includes creating a class, overwriting the onReceive method, and registering the receiver. By understanding broadcast receivers, you can create Android apps that efficiently respond to system events and improve the user experience. Thank you for your attention. I hope this session has helped you understand the concept of broadcast receivers in Android and how to implement them. Thank you.

Services Introduction Video• . Duration: 4 minutes

[Notes]

Hello everyone. Welcome to the module on networking and web services. In this session, we will focus on an important topic in Android development, which is services. Services are background components that help perform long-running operations in Android without user interaction. Unlike activities, services do not have a user interface. They are essential for ensuring smooth, uninterrupted background tasks like downloading files or playing music, even when the app isn't actively being used. By the end of this session, you will have a clear understanding of what services are, the different types of services, and when to use them. Now let's define what a service is. In Android, a service is a background component that performs long-running operations without user interaction. This means that unlike activities, services do not have a user interface. Why use a service then? Services are particularly useful for tasks like downloading files in the background, playing music while the app is minimized, handling network operations such as syncing data with the server, or performing computational tasks that need to continue running without interrupting the user interface. Services keep the app responsive by offloading these tasks to the background, allowing users to interact with other parts of the app smoothly. Let's now explore the types of services available in Android. There are three main types. Foreground services. These services run in the foreground and display a persistent notification to the user. An example of this is a music player where the user is aware that the service is running because of the notification. Then we have background services. These services run in the background without the user being aware of them. However, they can be stopped by the system to conserve resources. An example is syncing data with a server. Then we have bound services. These services are bound to another component such as an activity and provide an interface for interaction. For example, a bound service might be used to interact with a remote server and provide real-time updates to the user. Each type of service is designed for a specific purpose and choosing the right one depends on your app's requirements. Now let's talk about when to use a service in your Android app. Services are best suited in the following situations. When your app needs to play music or video in the background, such as in the media player app, we need to use service. Or it could be for handling long-running downloads. If your app needs to download large files, such as a file-sharing app, then service is a good choice. Then running tasks that need to continue even when the app is not in the foreground. For example, syncing data with a server or running background updates. It is important to note that services should be used wisely. Running too many background tasks can drain the device battery and overload system resources. Therefore, always ensure that services are optimized and run only when necessary. To summarize this session, a service in Android is a background component used to perform long-running operations without a user interface. And there are three types of services. Foreground services, background services, and bound services. These are tied to other components and provide interactive interface. Services are useful for tasks like playing music,

downloading files, and running tasks in the background. Even when the app is not actively being used, by understanding these points, you will be able to implement services in your Android apps, ensuring smooth and efficient background operations. Thank you for your attention. I hope this session has helped you understand services in Android and how to use them to manage background tasks efficiently. Thank you.

Stated and Bound Services Video• . Duration: 5 minutes

[Notes]

Hello everyone, welcome to the module on networking and web services. And in this session, we'll be discussing started and bound services in Android development. Services are components that allow background processing without user interaction. Understanding the differences between started services and bound services is crucial for managing background tasks efficiently. In this session, we'll explain what started and bound services are, provide examples and highlight their differences as well. By the end of the session, you will be able to understand when and how to use these services in your Android applications. Let's begin with what started and bounded services are. A service is considered started when an app component like an activity calls `startService` method. It runs in the background indefinitely even if the user moves away from the app. It must be stopped manually using either `stopService` or `stopSelf`. Then we have the bound service. A bound service allows other components, such as activities, to bind to it and interact with it. The service continues to run as long as the components bound to it. It allows communication between the service and the components that are using it. These two types of services provide flexibility depending upon the requirements of the task you need to accomplish. Now let's look at an example of how to implement a started service in Android. A common use case is playing music in the background while the user navigates away from the app. In this scenario, the service continues to run until it is explicitly stopped by the user or the system. Here is the code example. In this example, we create an intent to start `myService` class, which handles the background task. The service will continue running in the background until it is manually stopped by calling `stopService` or `stopSelf`. A started service is ideal for tasks that need to run continuously without interaction from the user, such as playing music, downloading files, or performing periodic updates. Now let's explore an example of bound service. In this case, multiple app components, such as activities or fragments, can bind to the service to interact with it and share data. A typical use case is an app that tracks fitness data, where various components might need access to the step count or heart rate. Here is a code example of how to bind a service. In this example, we create an intent to bind the `myService` class to the current component. The `bindService` method allows us to connect the component with the service, creating a long-lasting interaction until the service is unbound. Bound services are ideal for tasks that require ongoing interaction between the service and the app, such as fetching data in real-time, managing user inputs, or maintaining long-running background tasks that rely on app interaction. Now let's discuss the differences between started and bound services. Started service, once started, the service runs independently and continues indefinitely until it is stopped. It does not require any interaction with other components once it starts running. Example, playing music or downloading files in the background. Whereas the bound service exists only while there are components bound to it. It allows for communication between the service and the components, enabling real-time data sharing. Example being fetching fitness data and allowing multiple activities to access and update

the data. Understanding the key differences helps you choose the right type of service depending upon your app's specific needs. To summarize today's session, we saw started services, which are services that are launched and continue to run in the background until explicitly stopped. They are ideal for tasks that need to run indefinitely, such as music playing or file downloads. Then we saw bound services that allow components to bind to them and interact with them, running only while these bound components are attached. They are used to tasks that require ongoing interaction, such as tracking fitness data or performing real-time updates. We also discussed how to implement both types of services. By understanding both started and bound services, you will be able to implement efficient background tasks in your Android applications. Thank you for your attention. I hope this session has helped you understand started and bound services in Android and how to implement them. Thank you.

Service Lifecycle Video• . Duration: 5 minutes

[Notes]

Hello, everyone. Welcome to this module on networking and web services. In this session, we will focus on an important concept, which is service lifecycle in Android development. Services are used to run tasks in background, but like any other Android component, they have a lifecycle that needs to be managed carefully. Understanding the lifecycle of services is crucial for building efficient and reliable apps. So in this session, we'll explore the lifecycle of both started service and bound service. Now, let's take a look at service lifecycle in Android. Like activities, services have a lifecycle, but it is focused on handling background tasks. The lifecycle involves different methods that manage the creation, execution, and destruction of a service. Here are some main lifecycle methods of a service. OnCreate method. This method is called when the service is first created. OnStart command. This method is called when the service is started. OnBind. This method is called when a component binds to the service. OnDestroy. This method is called when the service is no longer needed and is being destroyed. These methods help control the flow of a service from its creation to its termination, allowing you to manage background tasks efficiently. Let's first discuss the lifecycle of a started service. A started service is one that runs independently in the background once it is started. The service continues running until it is explicitly stopped. Here are the key stages in the lifecycle of a started service. First, we call OnCreate method. So this is where the service is created. Then we have OnStart command method. This method is called when the service starts running in the background. Then OnDestroy method. This method is called when the service is stopped and destroyed. Started services run in the background even if the app is closed, and they continue until explicitly stopped by app or the user. Next, let's talk about the lifecycle of a bound service. A bound service allows components like activities or fragments to bind to the service and interact with them. Unlike a started service, a bound service only runs while there are components bound to it. Here are some lifecycle methods of bound service. OnCreate method. The service is created after calling this method. OnBind, the service is bound to a component allowing interaction between the component and the service. OnUnbind, this method is called when all components have unbound from the service. And then we have OnDestroy. The service is destroyed when it is no longer needed. Bound services are ideal for tasks that require interaction between the app and the service, such as fetching the data from remote server or maintaining live updates. Let's now look at a simple application that shows the service lifecycle in action. This app will help you understand

how services behave when they are started and stopped. First, service created. The service is initialized by calling the `onCreate` method. Then the service is started. The service begins running when `onStart` command is called. Then the service is destroyed. When the service is stopped, the `onDestroy` method is called. In the log, you will see messages like service created and service started appear when you click the Start Service button. And service destroyed appears when you click Stop Service button. This confirms that the service lifecycle methods are being triggered as expected. To summarize this session, the service lifecycle consists of methods like `onCreate`, `onStart` command, `onBind`, and `onDestroy`, which manage the creation, execution, and destruction of services. The lifecycle of a started service involves running in the background until explicitly stopped. The lifecycle of a bound service involves running only while components are bound to it, and it is destroyed when no longer needed. The app showed how these lifecycle methods are triggered during the creation, running, and stopping of a service. By understanding the service lifecycle, you can ensure that your Android apps manage background tasks efficiently and avoid unnecessary resource consumption. Thank you for your attention. I hope this session has given you a clear understanding of a service lifecycle in Android and how to manage started and bound services. Thank you.

Using Retrofit for HTTP Requests Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to the module on networking and web services in Android development. In this session, we'll be focusing on using Retrofit for HTTP requests. Retrofit is a popular library that makes it easier to work with REST APIs by simplifying network calls and automatically handling JSON conversion. In this session, we will cover what Retrofit is, how to set it up, how to define API calls, and finally, how to make a network request. By the end of this session, you will understand how Retrofit works and how to implement it in your Android apps. Now let's start with an introduction to Retrofit. Retrofit is a popular library for making HTTP requests in Android apps. It simplifies working with REST APIs by converting responses, usually in JSON format, into Kotlin or Java objects automatically. This means that developers don't need to manually parse the JSON data, making the process faster and error-free. Why use Retrofit? Retrofit simplifies the entire network call process by automatically handling the JSON conversion, supporting both synchronous and asynchronous requests, and providing a clean and easy-to-use API for making network requests. This makes Retrofit a powerful tool for Android developers working with web services. Let's now look at how to set up Retrofit in Android Project. First, you need to add the required dependencies to your `build.gradle` file. For Retrofit, you will use something which is shown in the snapshot. Then we need to configure the Retrofit. So, you need to set up Retrofit with a base URL and a converter factory to handle JSON conversion. And here is how you can configure it. The base URL is where your API requests will be sent. The JSON is used as a converter to automatically handle JSON data. By setting up Retrofit in this way, you can quickly connect your Android app to REST APIs. Once Retrofit is set up, the next step is to define the API calls using an interface. This interface will contain all the API endpoints that your app needs to communicate with. Here is an example of how to define an API interface. We have at get users, this annotation tells Retrofit that you want to perform a get request to the user's endpoint. Suspend function get users, this defines an asynchronous function call that returns a list of users. By defining API calls in an interface like

this, Retrofit handles the network request and converts the API response into a list of user objects automatically. Now that API interface is ready, we can make a network request to fetch the data from the API. Here is an example of how you can make a network request using Retrofit. Lifecycle scope dot launch ensures that the network request runs in the background. Get users method makes the API call and fetches the user data. In this example, we are using Kotlin coroutines to make the network request asynchronously. If the request is successful, the user data is retrieved and in case of an error, it is caught and handled. Now let's briefly see how Retrofit works behind the scenes. First, the API interface which defines the HTTP methods and endpoints using annotations like `@get` or `@post`. Second, Retrofit processes these annotations and creates a proxy class that handles the actual HTTP request. Then the network request is executed and Retrofit manages the connection to the server. Once the server returns a JSON response, Retrofit uses a converter such as `JSON` to automatically map the JSON data into a Kotlin object and that your app can easily use. By managing the entire process, Retrofit allows developers to focus on building their app while simplifying the network calls. To summarize this session, we saw Retrofit, which is a powerful library that simplifies network requests and JSON conversion in Android. We covered how to set up Retrofit by adding the necessary dependencies and configuring the base URL and converter factory. We learned how to define API calls using our interface, how to make a network request using Kotlin coroutines. Finally, we explored how Retrofit works by processing annotations and managing HTTP requests behind the scenes. By using Retrofit, you can make HTTP requests in your Android apps more efficiently and focus on building a great user experience. Thank you for your attention. I hope this session has given you a clear understanding of using Retrofit for HTTP requests in Android. Thank you.

Introduction to RESTful APIs Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to the module on networking and web services. In this session, we will focus on an important concept in mobile app development, which would be RESTful APIs. A RESTful API allows apps to interact with servers over the internet using HTTP methods like GET, POST, PUT, and DELETE. In this session, we will cover what RESTful APIs are, how HTTP methods work in REST, the request-response cycle, and the structure of RESTful APIs. By the end of this session, you will have a clear understanding of how RESTful APIs work and how they can be used in Android applications. Let's begin by defining what RESTful APIs are. A RESTful API, which is Representational State Transfer, is a way for mobile apps to communicate with a server by using the HTTP protocol. REST is stateless, meaning each request from the client to the server must contain all the necessary information because the server does not store client context. Some of the key features of RESTful APIs are First one is stateless. No client data is stored on the server in between requests. And when it comes to client-server architecture, the client, which is the app, and the server communicate separately over HTTP. Then we have multiple HTTP methods. REST uses standard methods like GET, POST, PUT, and DELETE to interact with resources. RESTful APIs are commonly used because they are simple, scalable, and compatible with almost all platforms, making them perfect for mobile app development. Now let's look at common HTTP methods used in REST. GET method used to retrieve data from the server. For example, getting a list of users or fetching a post from a blog. POST method used to send new data to the server, such as submitting

a new post or creating a user account. PUT used to update existing data on the server. For example, editing a user's profile information. DELETE used to remove data from the server, such as deleting a user account or removing a post. Each HTTP method performs a specific action on the resources available on the server. Understanding these methods is crucial for interacting with RESTful APIs in your app. Let's now discuss the REST request-response cycle and how it works. First, the client app sends an HTTP request to the server. This request contains HTTP method, such as GET or POST, and the endpoint, which is a URL, that specifies the resource being requested. The server processes the request and performs the necessary action, such as retrieving data, creating a new entry, or updating an existing information. The server responds by sending back a response to the client, typically in JSON format. The response contains the requested data or confirmation of the action, such as success or failure. For example, when a user logs into an app, the app sends a POST request to the server with the user's credentials. The server verifies the information and sends back a response indicating whether the login was successful or not. Finally, let's see the basic structure of a RESTful API. First, we have a base URL. The base URL is the starting point for all API requests. For example, in the case of a REST API hosted at `https://api.example.com`, this would be the base URL. Then we have endpoints. Endpoints are the paths added to the base URL to specify the resource or action. For instance, `/users` could be endpoints for fetching user data. Other examples include `/posts` for blog posts or `/comments` for user feedback. Then we have the response. The server's response is usually sent in JSON format, which can then be converted into objects in Android using libraries like Retrofit. This makes it easier to work with the data in your app. By structuring your API calls this way, your app can efficiently send requests to and receive responses from a server, ensuring smooth interaction with web services. To summarize this session, RESTful APIs allow mobile apps to interact with servers using HTTP methods like GET, POST, PUT, and DELETE. We learned about common HTTP methods and how each method performs a specific action on the server, such as retrieving or modifying data. The REST request-response cycle involves sending a request from the client and receiving a response from the server, typically in JSON format. The structure of a RESTful API includes the base URL, endpoints, and responses, which are essential for making network requests in mobile apps. By understanding these concepts, you'll be able to integrate RESTful APIs into your Android apps, making them more dynamic and connected. Thank you for your attention. I hope this session has helped you understand RESTful APIs and how they can be used to make HTTP requests in mobile applications. Thank you.

Parsing JSON Responses Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to the module on networking and web services. In this session, we will focus on an essential topic, which would be parsing JSON responses in Android development. JSON, JavaScript Object Notation, is a lightweight format used for structuring data and is commonly used to exchange data between client and a server. In this session, we will discuss what JSON is, how to parse JSON data in Android using JSON, and how to integrate Retrofit for handling API responses efficiently. By the end of this session, you will have a clear understanding of how to work with JSON data in your Android apps. Let's begin by defining what JSON is. JSON stands for JavaScript Object Notation, and it's a lightweight format for structuring data. It's easy to read and

write, making it a popular choice for exchanging data between a client like Android app and a server. JSON data is structured in key value pairs, making it simple to understand and interpret. You can see an example in the snapshot. In this example, ID, name, and email are the keys, and their respective values are 1, john, and john at example.com. JSON represents data in a way that can be easily interpreted by programs, including your Android app. To work with JSON in Android, we often use the JSON library. JSON is a powerful tool that automatically converts JSON data into Kotlin or Java objects. Here is how you set up JSON in your project. First, we need to add JSON as a dependency in build.gradle file. Then we need to parse the JSON data using JSON. Let's look at a basic example of how to convert JSON into a Kotlin object. In this example, fromJSON is a JSON function that takes JSON string and converts it into a user object. Now the data from JSON string is available in the user object, which can be used throughout your app. To JSON data effectively, you need to define a data model that represents the structure of your JSON. In Kotlin, we do this using data classes. Let's define a data model for JSON example. This Kotlin class defines how JSON data will be mapped to a Kotlin object. Each key value pair in the JSON data such as id1 is mapped to the corresponding property in the user class val id int. By defining a data model, you can easily convert JSON data into Kotlin objects using JSON. Let's now look at how Retrofit and JSON work together to simplify JSON parsing in Android apps. Retrofit is a popular networking library that automatically handles network requests and responses, while JSON takes care of converting JSON data into Kotlin objects. Here is an example of how to integrate Retrofit with JSON. First, we start with defining an API interface. Then we need to configure Retrofit, which is used to parse the JSON object. In the example, Retrofit will make a HTTP request to fetch data, get the JSON response from the server, use JSON to automatically convert the JSON response into a list of user objects. This approach makes it easy to work with API data as Retrofit and JSON handle most of the heavy lifting for you. To summarize this session, JSON is a lightweight format used to structure data and exchange information between clients and servers. JSON is a powerful library that simplifies JSON parsing in Android, automatically converting JSON into Kotlin or Java objects. A data model is needed to define how JSON data maps to Kotlin objects, making it easier to work with structured data. And Retrofit works seamlessly with JSON to handle network requests and automatically parse JSON responses into Kotlin objects. By understanding these concepts, you can easily manage and work with JSON data in your Android applications. Thank you for your attention. I hope this session has given you a clear understanding of how to work with JSON and JSON in Android, as well as how to integrate Retrofit for seamless API communication. Thank you.

Check Network Connectivity Status Video• . Duration: 6 minutes

[Notes]

In this demo, we shall see as to how to check the network connectivity of our device. I have created an application. This application, I have defined a tag as well, which says network connectivity status, and I'm using the view binding in order to bind to my view elements of the layout file. This we have already seen in the previous contact session. Then I'm created a local method overhand a private method, which is check network connection, and I'm passing the binding element to it so that I can access the different other views which are present in the layout. Let us see as to what is defined in the layout. There are two views which are defined, you can see the design over here. I have a text view, which is currently displaying halo word. Later on, it when I check the connectivity of the device,

connection status will be displayed over here in the text view. This will be triggered or this means one to work when you click on the button, which is named as check network connection. Let us go to the code part and complete the method which is check network connection. As already seen for this, we have to make use of two important classes. One is connectivity manager. The other one is network info. First, we'll create a variable for connectivity manager. I'll call it as CM. This basically extends connectivity manager. I need to provide a reference to the connectivity service, so for which I'm going to call the method, get system service, and I will have to provide the connectivity service context, as connectivity manager. Once I have created that, now using this particular instance, I will find out the active network of the device. Active network will be CM, which is my instance of the connectivitymanager.activeNetworkInfo. Then I will check there is connection or not, for which I will create one more variable, which I will name it as isConnected. Using this active info first, I will allow to check if active network is not null and active network is connected or connecting. This will get the information of whether the device is either connected or it is the process of connecting. Then is connected. Connected. If it is connected, then I'm going to specify the connection set as connected or not. For which I can make use of the binding.tvConnectionStatus.text = CONNECTED. Else, I can say bindingelement.tvConnectionStatus.text is not connected. This is all what is required. Later on, we will see as to how I can check whether it is connected over mobile or Wi-Fi. But first, let us see the connection status. Let me run this on my emulator. The application is installed and running. When you click for this particular button, it is going to call the meth check Network connection, and it is going to see if my device is connected or not. Let me click on it. You see that it is connected, but I do not know whether it is connected over Wi-Fi or through mobile. What I do is I will set it to airplane mode. I'll set it to airplane mode. Now let me click on the button check the network connection. Now you can see that it is not connected because I put it in airplane mode and in airplane mode, we know that none of the connections actually work. I'm going to remove this airplane mode. I'm going to append my code to check if it is connected, what type of connection it is, for which I can again use of the active network instance. If activeNetwork.type, if this is equal to ConnectivityManager.TYPE_MOBILE. If this is true, I am going to basically pop a toast message. Here the context is this, and the text is connected over mobile. Then this define and it is short. This is fine. Then if it is not, if or what I can do is, I can quickly copy paste these two lines, and change the type to Wi-Fi. Likewise, I need to change my text as well. It's going to check as to what type of connectivity is there in the mobile. Let me run this application. My application is built and installed. When I get on the check network connection, it is going to see as to which one is currently active. You can see disconnected and connected over Wi-Fi. What I can do is, I'll quickly remove the Wi-Fi or disable the Wi-Fi. Now only mobile data is there. When I click on, check Network connection, it will show that it disconnected and it disconnected over mobile. Again, I can set it to airplane, and when I click on Check Network connection, you will see that it is not connected.

WebView - Display a Webpage Video• . Duration: 5 minutes

[Notes]

This demo, we shall see as to how to make use of WebView in order to display a web page. So I have created an application and in this application I'm going to call this method which is displaywebpage in which I'm going to implement the web view. So in order to display the web page, okay, first we have to add a WebView element to our layout file. So let me show you my layout file.

You can see this is the webview element which is added to the layout file. It has an ID and width and height. It is matching to the parent. Okay, so basically it is going to capture the whole screen, okay? Then in our AndroidManifest.xml file we have requested for permission to make use of Internet. In my main activity, now I'm going to make use of the element which was defined in my layout file and I'm going to load a URL. So what is the URL which will be loaded? Let me first define the URL. So I'm going to define a URL string which will be `http://www.google.co.in`. So this is the URL which I want to load, okay? For which I'll make use of the element which is defined for `WebView.loadurl` is the method which I need to use and the parameter which I need to pass to it is the `urlstring`, okay? So let us run this application and see as to how it works. So application is getting built and it is getting installed now. You can see that the application immediately it loads and it tries to start the Chrome browser. And once this Chrome browser opens, it will try to open the URL string. But what we want is we want to show the output of this particular web page in my app itself. So for which I will have to create a `WebView` client, okay? So `webViewDisplayhtml.setWebViewClient` so `.WebViewClient`, okay? Within which I will have to pass on `webView` client instance, okay? So once I do that, then the URL which I have given will get loaded inside my `WebViewClient` itself. Along with that what I will do is I'm also going to make use of `Settings` and enable the `JavaScript` display settings. `JavaScript` enabled, I will set it to `true`. Okay? Then if at all you want the images to be automatically loaded for that also we can make use of `settings` and enable it loads images automatically. This also I will set it to `true`. Likewise, there are multiple options in `Settings`. I suggest you explore them. Now we shall see as to how this particular program will execute. Let me Rerun this app. It is built and it is installed now. You can see now my app did not try to start the Chrome browser which is installed in the device. Okay, but the whole page that is `google.co.in` it got loaded within the `WebViewClient`, okay? I can click on some search over here. Let me type `bitspilani`. Okay, so when I select this, it's going to display the contents related to that particular search string. And when you click on the back button you will see that the app gets destroyed. But what is that we would want to have is that in case I have multiple pages which are opened or which are navigated, when I click on the back button, if there is a page which I have navigated before the current page, then it should go back to the previous page for which what we'll have to do is we will have to implement or override the `onBackPressed` method. Okay, so we will do that now `onBackPressed` method is what we are going to override. Okay, so first what we will have to do is I will have to first check if my `WebView` as a page which is already navigated for which I will have to check by making use of the element. Can it go back? If it can go back, then I will ask it to go back by making use of the method `goBack`, okay? Else I will make use of the `super.onBackPressed` where it is going to exit the app. Okay, so let me run this particular application again. Okay, now, let me type in okay, `Bits Pilani`. It's displaying the page. Now, I have the search page as my previous. So when you click on the back button, it should basically go to my search page again. So that's what happened over here. When I click on the back button again, my app will be destroyed.

Get Resource from Internet Using `HTTPURLConnection` Video• .

Duration: 10 minutes

[Notes]

In this demo, we are going to see an example which will use `HTTP URL` connection in order to get a resource from Internet. This is the URL which I'll be passing, and the response, I will receive it in the

form of JSON. I'm going to pass this particular JSON object and get the name out of this object. I have created an application. In this application, we are going to make use of this JSON URL, which I just showed to you in the browser and using which we are going to retrieve a JSON object. Let us see the layout file. In the layout file, I have a text view which is going to show the output of the connection whether this device is capable of meter connection or not. Whether it is connected or not. Then I have a button which is named as Fetch when I click on it, the URL will be sent to the server and getting the response and in that the name value will be taken out and it will be displayed. In the Android manifest.xml file, we need to have access to Internet. Let me place the users permission tag for Internet and there is one another thing which we'll have to know here. That is starting from Android 9, that is API level 28. This usesClearTextTraffic by default, it is disabled. In case you are making use of the URL, which basically targets the sites which support HTTP, then we know that there could be used which will happen. Basically, my request will go in the form of plain text. It will not be encrypted. If it is HTTPS by default, it will be encrypted. But since I'm using HTTP, I'm asking the Android system to allow this particular traffic to go through. Let me explain you as to what is happening in the main activity. I'm defining the set on clear link or the bt Fetch, which is the name of the button. First, I'm checking for the network connection, which we have already done in a previous session. If this returns true, then I'm going to execute a AsyncTask card to which I am passing on the URL which I prepared, that will be passed on so that this AsyncTask can execute and get the result. If the connection fails, that is if checkNetworkConnection, if it returns false, then there will be a toast message which would be pop asking me to check the network connection. So let me prepare some methods. Here, I am preparing a method which is called a getHttp, which will be called in AsyncTask method. Here what is happening is we are preparing the HTTP URL connection. We are making a connection to the URL which is provided, getting the response, passing the response, and sending the result. Finally, this result will be in the screen. First, what we have to do is we have to obtain a new HTTP URL connection, for which we have seen that we have to create a URL, and we have to create a connection object by making use of url.openConnection, which will return as HTTP URL connection. So we have to typecast it to HTTP URL connection. Then we have to prepare the request. When I prepare the request, I can give the connectionTimeout, readTimeout. What is a request method? How should I send my request, so all these details can actually be prepared in the request. Then I am making a GET connect call. I'm making a GET request to the given URL. Once it makes a connection, and the server sends back the response, first, I will have to look into its responseCode. If the responseCode matches HTTP_OK then I would know that it has given me a response. Otherwise, we know that HTTP handles other response codes as well if the server is not able to serve my request. This particular response code will also be logged over here. When I check the response code with HttpsURLConnection.HTTP_OK. That means it would have either written 200 or two not one. Then it assures us that it has sent back a proper response. Using the input stream on the connection instance, we receive the response, and we check if it is not null. Then I'm converting the input stream into a normal string. I will use this particular string to convert into a JSON object. This is what is happening in these two. I'll explain the methods as to what these methods basically do. If the input stream turns out to be null, then we will say that it did not work. Finally, we are going to disconnect the connection which is made to the server. After that, whatever the result object is there, that will be returned back. Now, here is a simple method which is going to read the JSON object, and it is going to get the value which is associated with the name and it will return it. That's what it basically does. This accepts the input, which is the string. The parameter is the string, that string, I will have to parse it as a JSON object

and then using that particular object, I can call the method `getString`, pass on the key value for that particular object. Once I do that, I'll be able to get the value which is assigned with the name, and this name is returned back. I have one more method which is called this basically accepts the input string which is coming in the form of res pulse, and it converts it into the normal string, by making use of buffered reader library. Now let us look in to the class which is defined for the `AsyncTask`. This is the `AsyncTask` which is defined, which takes in three parameters. That is the string, void, and the string. Here, class name is `HTTPAsyncTask` and as you saw, we are going to call this particular name with `execute` method in order to execute the `AsyncTask`. We have already seen that it has different methods, which are callbacks, which are over ridden. So `onPreExecute`, what am I doing over here is, I'm showing a progress dialogue over here and this creates I can set the title to this. I can set the message to this, and I can specify whether it has to wait indeterminate or it has to stop after a specific time. Once these values are set, I'm basically calling `show` in order to show the `ProgressDialog`. Then the next method is `doInBackground`. `DoInBackground` basically accepts the URL. This URL is passed on to this HTTP an task with the `execute` method. It takes this URL, and it calls the `getHttp` method which we just saw. This `getHttp` method is going to take the URL which is present in the index of URLs, which is a array. Going to make a connection, get the response, and it is going to return a result. Now, that result will be taken over here in the `post execute`, that is `onPostExecute` method, and it will be assigned to a text view which is named as `tvResult`. One more thing what we are doing `onPostExecute`, the `ProgressDialog` bar, which was used will be dismissed. This is what will happen while making use of HTTP URL connection and executing using the `AsyncTask`. The prime reason for using `AsyncTask` is it is going to internally make use of a worker thread in order to make a network call. Otherwise, network calls on main UI thread is prohibited. Our app is going to crash in case we make a network call on the main UI thread. Let us run this application and see as to how it functions for us. The app is installed now. When I click on the Fetch button, this Hello World should be replaced with the status of the connection with respect to this particular device and along with that, if I have got proper response, the response code also will be shown in this logcat. You can see that it shows the status is connected. It has given the response code as 200 and from the JSON object, I was able to retrieve only the name and display it over here.

HTTPURLConnection Using Coroutine Video• . Duration: 4 minutes

[Notes]

In this demo, we will see the HTTP URL connection example using coroutines. We are making use of coroutines to handle the network call, which might turn out to be a long running task and can block the main thread. Along with that, it can also make our app unresponsive. Coroutines will help us make our apps responsive, not turn into unresponsive. If we see my Android file, it consists of same set of permissions which we have used earlier, and it also has attribute for the application, which uses clear text traffic is equal to true since I'm using just HTTP. Even the layout file is also exactly the same. In main activity, in our previous example, we saw that it was implemented using `async task`. But here, instead of `async task`, we'll be making use of `coroutine`. In order to identify the main thread and the coroutines, I have put in certain log messages. The first log message, it will get printed in the main thread. By making use of `Thread.currentThread.id`, I'll be printing the ID of the main thread. Similarly, when I come into the `coroutine`. `Coroutine`, this is where it actually starts.

When it calls `launch`, which is nothing but the coroutine builder, a coroutine will be called, and my code will execute the coroutine. When it is executing in the coroutine, what is the ID with which it is working will be displayed. Similarly, when it changes the dispatcher. That is when it changes the context from `main`, two `IO`, it'll be put onto a new thread, and that thread ID will be displayed over here. `Main` basically indicates that the code will get executed in the main thread itself. Now with context, I'm changing the scope to `dispatchers.IO`. `IO` basically is used when I want to make a network call or in order to make a disk access. Since I'm calling the network or I'm calling a URL using `HTTP` method, this is placed in the scope `coroutines IO` scope. Let us run this application and see as to how it works. My application is installed, and you can see that there is a first log message which says `main thread`. This is the idea of the main thread. When I click on "Fetch," it is going to first check for the network connection. If the connection is on, it is going to get displayed over here as `connected`, then it is going to fetch the data which is present in the URL which I have passed, and it is going to display the data over here in the `JSON` format in a form of string. Of course, as we saw in the previous case, you could even pass that `Day JSON` object and print those individual or make use of those individual items which are present in the `JSON` object. I will click on `Fetch` now. When I click on "Fetch," you see that from `main thread`, the next one, it still executes in the `main thread`. It is a coroutine, but it still executes in the `main thread` itself. Later on when I'm making or when I'm calling this `getHTTP` using the `JSON URL`, it is put on to another worker thread with the ID `246`. Now I've got the response, and you can see the response on the screen over here. Of course, this particular response, which is there in the `JSON` format can be passed, and individual items can actually be picked.

Create a Start Service Video• . Duration: 12 minutes

[Notes]

In this demo, we are going to see as to how to create a simple start service and run it. What I've done is I've created a project for the service, I've named it as `new service`. I have created a service file as well. This particular file, which is a class, it extends `service`. Now, how do you do that it's quite simple. You just have to go to the package, right click on the package. If you go to `new` and create a `Kotlin class`, give a name over here as `my service` or `your service`, whatever you feel is appropriate. Once it is done, you can come back to the file and extend it to `service` by passing `or`. Likewise, you can extend it.. Once you have extended, then here, you can write over the method, which is on `start` command. Let us start from the `main activity`.. In `main activity`, there are multiple click listeners which are set, but before that, we have to see as to what is our layout file. Layout file looks like this, let me show you the design of it. Here, as you can see, there is a button to start the service, there is a text view which will indicate the service status, whether the services started or whether it is stopped, so that information is showed over here. Then we have a button to stop the service. Whatever the service which is start services can be stopped. That is done using the `stop service` button. Then along with that, what I intend to do is I need to pass some particular text value to this service, and the service has to use this text value to do a job, I want this particular text value as such. For which I have a edit text field, and I also have a button which is `send data`. When I click for the `send data`, whatever is entered in this particular text field will be taken and it will be used for some purpose in the `on start` command method inside the service class. You can even see the code as well, so I have the button, which is for start service, this is for stop service, a text field to display

the status, and this is to accept the input, another button to send the data. Using the constraints, I have defined the layout proper. What I'll do now is I'll go back to my main method, and here I will have to define the on click listeners for the buttons once it is click. Here I am defining the listener for the start button. What I'm going to do is I'm going to create a variable called as service intent and call the intent instructor and pass on the arguments, which is the package context, and I will call the My service. Which is a Java class. Once I have created the intent, I will have to pass it on to the method start service so that my service can be started using this particular intent. Along with that, I will also use the display text to say that service has start. If I have defined the on-click listener for the start service or the button start service. Likewise, I have the button stop services on which the On Click listener is set. Here, again, I'm creating a intake with the same information that is the two arguments when it is a package context, the other one is the service class. Then this is passed on to the stop service so that the service can be stop, and I'm displaying the status of it. Let us see this particular part in working before we go further. You can see that the button, start service is there, stop service is there. When I click on Start Service, you will see some display messages over here in the log which says, our service was initialized and running. Let's see as to where that particular message came from. It is saying, nothing is received in on Start Command because I did not send anything to the on Start command so that's the reason this particular message is being shown. Let me stop running this application and go back. I have started the service. I've stopped the service over here. But let us see in my service what is happening. My service, when this particular service is initialized or instantiated, this unit block is going to get executed, and this is where I'm logging the message that the service was initialized and it was and it is running. This is where the message came from. Then, on bind has to be overridden, even though it returns null. On bind is generally used when I'm bonding other component to this particular service, but here nothing of that sort is applied. But still, I will have to do that. If I do not do, you will see that it could say that the on bind is not implemented here. We will have to implement it and keep it as return null. Then I'm implementing the on start command over here, which accepts the intent, the flag, and the start ID, and it returns the integer. From the intake, the data which is sent, I'm getting it over here. If this particular data is not null, then whatever the data which is received, that will be log, else, it is going to log saying nothing is received in on Start command. This is basically implemented in on Start command. It is in this particular on start command where we are going to define our logic for long running operations. Now let us go back to the main method or main activity. Implement to send the data. Here, again, we will define a variable for the service intent and call the intent Constructor package context, and my service boolean Java class. Using this, I'm going to start the service again. But before that, let me collect the data which was entered in the data field. So I'm going to create a variable with var, four letter as data string. From ET data, I am getting the text, what is entered, and converting it to string before it is assigned. Then I will call the service intent, add this particular data string into the intent. Do it. Put extra. Then I need to give in the key value, same as what is present over here. So basically, here, I've given it as RIC asc data. The same thing, I will put it as the key value over here. The data string will be passed on as a object. Let me delete this one [inaudible]. This is the name and this is the value. Now what I have to do is have to start the service, for which I'm going to pass the intent. So once I do this, what's going to happen is when I click on the button, whatever the data which is entered, that will be accepted over here using this particular intake. Using intent, using the get string extra by passing on the key value, the data will be received. Since it is not null, then the data, whatever is send, it will be displayed. I'm just displaying here, but you could make use of that particular data to perform any operation. Let us see this function in action

now. Let me run the app again. so there is a start service top service, but before starting the service, let me enter something over here and click on send data. It says our service was initialized and running, and it sent hello. I did not click on the start service, I did not click on stop service, I did not initiate the service, but still the survey got started. The reason for that is, if you observe in button send data, here also we are calling the start service. We are calling the start service. Well, of course, I can stop this particular service. But when I click on something else over here, Again, the service will be started. So I'm going to click on Stop service, and delete the whole over here, and I'm going to even clear the Logcat, and I'll click the start service. When you click on the Start service, you will see that the service is initialized and running, nothing is received in on-start command. Nothing is received. But let me enter something over here. When you click on the send data, you just see that AOM find is received, but you will not see this our service was initialized. If the service is already started, Android is going to notice that the service of this particular name is already started, but it's not going to restart that particular service. It's going to continue with the on Start command. Another thing which I wanted to discuss over here is, say for example, if I have a very long running activity, say, I have one while loop, which is basically a infinite loop. Let us see as to how it behaves. This while loop is presenting on-start command, meaning to say that I have sent something to on-start command and it is doing something. So I'll click on or Start Service our service has started, but our app becomes non responsive, I cannot enter anything, I cannot stop service. Basically, it is saying that the service is not responding. Do you want to close it? I would say, the reason is that this while loop is also running inside the service and service is running in the main thread of the application, which is not advice. What we have to do is we have to make use of threads I'm going to create a thread over here, and internally, I'm going to run the while loop. What the application does is it's going to when it is running, it is going to create a separate thread, I will run the while loop inside that particular thread. Let me show you this. You can see that I can click on Start service. My services started, I can even pass on data over here now. This is still function, this I loop, it is running inside or thread. No, it is not running in the main thread applications main thread.

Intent Service - implementation Video• . Duration: 7 minutes

[Notes]

In this demo, we are going to see the implementation of Intent service. I have created a project named as Service Start Stop. In this, I will be implementing the Intent service. In the main activity, what I have done is initially, I have put in a log message which will say that the IntentService and the service has started with the thread and what is a thread ID it is going to print. Basically, this will be the Mainthread. Let me just rename this as so Mainthread, thread ID, it's going to print, so that we can identify the main thread from the worker thread. In the XML file, if you see the layout file, there are two buttons which are defined. One is the start service, the other one is a stop service. Both of these are implemented in the main activity. On start service, I have defined a OnClickListener and stop service also have defined a OnClickListener. Let me show you as to what is defined OnClickListener. This part we have already seen, so I have created a variable Intent service, and using the constructor of Intent, I am creating the intent for this, and I'm calling the start service method in order to start this particular IntentService. Once I call the startService, what's going to happen is, it's going to instantiate this particular MyIntentService class. How do you create the service class? We have already seen in the previous session example. Likewise, I'm going to create

a new `MyIntentService` class which will extend the subclass of service which is `IntentService`. So here, compared to the service, I will have to pass on a name to this particular `IntentService`, while instantiating the `MyIntentService` class. Then I have a `net` block where this particular `IntentService` will be initialized by creating an instance and by logging a message saying that the service is initialized. This is the tag, and this is the message which will be displayed. Here, in this particular class, instead of `OnStart` command will be overriding `OnHandleIntent`. `OnHandleIntent` is this particular method which will be executed in a worker thread. What is it going to accept? It's going to accept the intent which will be sent by the service. It'll check if the service is running. While if the service is running, it is going to randomly generate a few random numbers between zero and hundred, and that `randomNumber` will be displayed in the log message. This display will happen every two seconds. I'm asking the thread to sleep for, that is this particular thread refers to the worker thread. This particular thread will sleep for two seconds before showing the next `randomNumber`. Since this is being worked in the worker thread, it is better to put it in a `try-catch` block, just to make sure that in case this gets interrupted, we will be able to catch that particular interrupt and display it. This is what happens inside the `OnHandleIntent`. Along with us, we have also created a companion object. This companion object, the purpose of this creation of the companion object is here, I can define different members or functions. When they are written inside this particular block, they can be called without having the instance of the class. I don't have to create the instance of a class, but I can call these members or functions without creating the instance of a class. That is the purpose of the companion object. Here, what we have done is here, we have created instance of `MyIntentService`, which will be initialized later. That's what this late in it actually says. Then the variable `ServiceRunning` is set to `false`. I've also defined a function, which is named as `IntentServiceStop`. This particular method will be called, to stop this service. In order to stop the service, first I am going to set this `ServiceRunning` to `false`, instance object dot here and call in the `stopSelf`. The service is going to stop itself. This particular method, that is `IntentServiceStop` will be called when I click on the button, `Stop Service`. If you see here, `MyIntentService`, which is the class the `IntentServiceStop`. I'm directly calling that particular method, which is part of the companion object without creating the instance as such. Along with that, the new service which is created that is also added or declared in the `AndroidManifest.xml` file, so with the name, add exported being `false` because this service will not provide the services outside the app. Now let us see this particular service in action. I'm going to run this application. You can see the application is launched, and it is saying that in the main thread, thread ID is one. Once I click on the "Start Service," it is going to go inside `OnHandleIntent`, and random numbers will be displayed. The service is started, and you can see that the service is initialized. The service is running with the thread ID 232, and it is generating random numbers every two seconds. Again, click called the "Stop Service," and it is going to call the method which is defined in the companion object, and the service is going to do a `SelfStop`. The service is stopped or stopping. I can call `StartService` again, it is going to again start a service on a different thread. It's not going to make use of the same worker thread because that thread will be. On a new thread, which is 233, now it is showing me the random numbers which are generated. Likewise, I can stop and I can start again. A new thread will be used to create it.

Foreground Service Video• . Duration: 9 minutes

[Notes]

In this demo, we are going to create a foreground service and see it's working. I have created a project by name foreground service. Here is the MainActivity, which extends AppCompatActivity. Let me show you the layout file of this. I have created two buttons. One, which will help me start the foreground service. The other one will stop the foreground service. I also have edit text where I will send some data to this particular foreground service, and it will be shown in the notification. In order to create the service itself, I have created a new service file or class, which extends service. If you remember, when we started learning about the service, in the started service example, we created something like this. Here, also, we are going to create a class with ForegroundService, which will extract service. Here, I'm going to create a method for starting the service. I'm going to create a method for stopping the service that I'm doing it in the companion app object. As you see, I have created function deemed as startFGService for which I'm passing on the context and the string. I'm creating intent variable over here, using the Intent constructor while passing on the context and the service class. Inside the intent, I'm adding this particular message and calling the startService using that context. Similarly, even I'm creating a stopService as well. I'm creating a method for stopping the service. These two methods will be called in my MainActivity for the two buttons. Let me go and define the buttons there. My first button will be start, so I'm going to define that OnClickListener. Here, I'll be calling the ForegroundService class and the method without creating the instance of the ForegroundService class. I would want to start the foreground service as such, and I need to pass on the context. Then the string which I have accepted from the edit text. That is being passed on to the data string. I'm going to pass it over here. Since it is saying that it is basically expecting a string, but what it found was editable string. In order to change it to string, I'm going to call the method to string. This is all. I create a toast message to indicate that the services started. I have to specify the toast length and call the show method. Similarly, I'm going to define for the Stop button as well, copy paste the code. It is btStop. Here, instead of startFG, it will be stopFG. I don't have to pass any string over here. Only the context is required for me to stop this particular service. That's all I have to do it in the MainActivity. Now, going back to the ForegroundService plus, so here in the companion object, I've created the functions for starting the service and stopping the service. Now, as we have already discussed, I need to call or I need to make this particular service as a foreground service. We said that for me to do that, I will have to start on foreground service method inside the onStartCommand method inside this. If you see my last statement is start for ground with the ID, and I need to pass the notification which I have created. This will basically put the service as a ForegroundService. Now, before going to this, there are a couple of things which I need to do. One, I need to create the intent for the activity. Now, this particular intent is taken, and it is passed on too so that this can be placed in the parentstack in the backstack so that whenever I click on the notification, this activity is going to start again. Using this TaskStackBuilder, we are going to create add the intent, which was created with the MainActivity as the second argument to class. This particular intent will be added to the parent stack, and using the pendingIntent, I'm going to create the pendingIntent. This part will help me create the pendingIntent. This particular pendingIntent will be passed on to the notification. I'm going to create a notification object over here using NotificationCompat.Builder, so I'm basically using a building method over here in order to build the notification. I have to pass on the context as well as Channel ID for the NotificationCompat.Builder. I can set the content title, so I'm setting the content title over here, and I'm setting the content text. This particular text is basically coming in from here. Whatever the intent which was passed on, so that intent is taken, the extra item is taken, the field is taken, and that is passed on over here. I can even create icon. In order to create icon, what you have to do is you can go to the drawable. In the

resources you have a drawable, you can right click on it. You can go to the Image Asset. In the Image Asset, you can click on "Clip Art," which is going to show different other icons, and you can select any icon which you want, and you get applied to it. Then the content intent for this particular notification is coming from the pendingIntent. Finally, I'll call the build in order to build this particular notification. This notification will be passed on as one of the argument to the services to specify that this service have to be placed in the foreground service. This is the onDestroy method of the service. Here, I'm going to call stopForeground with a log message stating that the notification is removed. This is the method where we are going to build the notification channel. This we have already seen, so I'm going to skip it. Let me run this particular example for you and show as to how it's going to work for us. Let me click on the "startForegroundService". When a click on "startForegroundService," basically, it is going to call this particular method, and I can pass on any string to it. I'll click on "ForegroundService". This string will be accepted. It will be passed on to the notification, and it will be placed in the status bar as a notification when I click on, "startForegroundService". If you see that there is a notification with a star indicating that there is a notification as such, and whatever I have typed that it is being part of the notification over here. Now, when I click on "stopForegroundServices," you will see that the notification is also removed. Now I'm going to pass on a new string. When I click on "startForegroundServices," it's going to again, start and you can see that the string is passed on to the notification over here. Now, I will have to check if this particular my application or this activity is added to the backstack or not. What I'll do is I'm going to destroy this particular activity because whenever we see that when the notification comes, when I click on the "Notification," it should basically start the application. That is the intent over here. That is a use of the pendingIntent as well. When I click on this, you can see that my application has started again. When I click on "stopForegroundService," the service will be stopped.

Local Bounded Service Video• . Duration: 9 minutes

[Notes]

In this demo, we are going to create local bounded service. I have created a application named as Service Local Bound, which created the main activity for me as well as the layout file. If we look into the layout file, it has one text view which is going to display the time. Initially, it's going to display all over world but current data and time, and there is also a button. On click of this particular button, we are going to display the local timers. Now, this time will be the time and the date information we will be fetching it from this service. The client in this application, is going to bind to the service which is running, and using the public functions which are present in that particular service, we are going to get the local date and time. I'll come to the main activity later because implementation of main activity will happen later in different other activity lifecycle callback. But before that, let me show you as to how to create a local bound service. I have created a service file or a service class, which is named as local bound service, which extends service. You must notice one thing over here. Unlike the previous cases, almost all the examples which we have done till now, we have basically returned a null value on call of this particular method onBind because in one of the previous examples, we were trying to bind our client to a service as such. But here, we would want to do that. What it does is this particular function that is onBind, it returns a myBinder. MyBinder is basically local binder, which is actually return. This myBinder object, it basically refers to the local binder. This

myLocalBinder is basically implemented as an inner class, which extends binder. What this does is, it returns the instance of a local service so that the clients can call the public methods. Using this get service, it's going to get an instance of the local service, and once it gets that particular instance, then it will be able to call the methods which are defined in my local these bounded services. What is a public method which I've defined over here? That is to get the current time. By making use of this particular simple date format, in this format, I'm going to get the date information, and this date information will be written back to the client. This is what happens in this local bound service. Now let us go back to the implementation of. In main activity, in the on life cycle callback, we are just setting the layout file, which is going to display the text view as well as the button. Inflate the layout file. Here, we would want to restrict the lifetime of the local bound service between the start, the stop of the life cycle of the activity. Meaning to say, if I minimize the activity, my binding should go away. When I start the activity again, my binding should come back and I should be able to access the services. That is why on start method, we are going to, in order to start the local bound service. While creating the intent, I'm passing the package context as well as the local bound service class. I need to call the bind service with instead of start service, I'm calling your bind service arguments. One is the intent which we created. The other one is an anonymous object which we have created for the service connection class, which I'm going to explain in a while. Then the third parameter is Context.BIND_AUTO_CREATE. What does it will automatically create the service as long as the binding exists. That's what it means by having this particular context defined. Then I'm going to log the message saying that the service is called that. What will I do in onStop? Since I want to restrict the usage of the service in the lifetime of the activity between start and stop, in order to unbind the service, I'm passing the same anonymous object, which is not type service connection. Then I'm going to specify a post message as well as put it in the log. Now, let us see as to how this service connection is actually defined. In this service connection, we are going to define the callbacks for service binding, which will be used. So let us see the implementation of it. So this particular service connection, it's going to return an anonymous object, and that's what is called as a myConnection, which has been passed on to bind service as well as unbind service. In this, we are going to implement two methods. One is onServiceConnected, what should be done, and onServiceDisconnected, what should be done. OnServiceConnected, basically, I'm getting the reference of the binder. Here you can see that I'm creating a value binder and using the LocalBoundService by calling the myLocalBinder class, I'm getting the instance of it. By calling the getService, here is where I get the instance of the binder. Using this particular reference, I'll be able to call the public methods which are defined in the LocalBoundService. I'm also having a Boolean check over here or a flag which indicates that whether I have bonded to it or not. Once I have come into this onServiceConnected, that means client is bonded to the server. Letting it know to the user that the service is bonded, and I'm also putting it in the log message so that I can keep note of it. Then the next method which needs to be defined is onServiceDisconnected. Here I will have to just make the flag as false, and I'm going to specify that the service is disconnected, so this will be called when over here. That is, while unbind service, so by passing on, is myConnection during this particular time onServiceDisconnection is going to be okay. I have a method over here, which is called to show the time. This showtime basically by making use of this myService, which has that connection object to the server, I'm checking whether it is null or not, if it is null, it just returns null. Otherwise, it is going to call the method get current time, which is defined in the LocalBoundService, which will be assigned to this variable, and that will be displayed in the text field of the display time. Now, let us run this particular example and see it in action. We see that as soon as the activity is

created, displayed a dosed message. Since it vanishes, that's the reason I put it in the log. If you actually see in the log messages, it said that service onCreate that is onStartMethod, so service is actually created. The service is so where am I displaying this particular message? Service is connected. Here is where it is getting [inaudible]. That is onServiceConnected. What I do now is I will show the disconnection part. When I minimize this, onStop is going to get called, onStop of the activity is going to get called, so I'm going to minimize this. You will see that the service is stopped. The client is no more bound to the service. When I start this again, you will see that the service is connected back. Now, what I'll do is I'm going to click on the display time button which is going to call the public method, which is defined in the service. When I click on display time, it shows the current date and time for me. I can click multiple times and it is going to change the date and time form. This is when the service is connected. One small thing which I have to show here is, so probably you will be wanting to know as to where the showtime method is called. Basically, this particular method is called in the onClick parameter of the button. The onClick parameter of the button, I'm calling the method showtime. Even in Android manifest.xml file, the service is declared the name of the classes passed on to the name, and it is enabled, and it is not exported because the service is not available to the outside clients.

Remote Bounded Service Video• . Duration: 5 minutes

[Notes]

In this demo, we are going to see the implementation of services which are remotely bound. So I have created a project named as Service Remote Bound, which by default, gives me the activity and the layout file. In the layout file, I have defined a text editor. Basically, I'm going to use this in order to type in a text, and that will be sent to my remote service. When I click on this particular button. But in the main activity, what is implemented, I'll come to that later. Before that, I have created remote service class, which extends service. Here, as I've already stated, the service, it implements a handler that receives a callback for each client from the over here end the service uses the handler to create a message object, which is a reference to the handler. In this line 14, as you can see, we are creating a messenger object, which uses the handler. The messenger then it creates a iBinder that the service returns to the clients from on onBind. In this onBind method, we are retaining the iBinder using the messenger. Client. My main activity is the client, which I'll show the implementation. Basically, here we are creating a intent, and class which will be called is the remote service, and I'm calling the bind service with the intent. This myConnection is an anonymous object which is implemented on the service connection. In service connection, we have implemented onService connected, what should happen and onService disconnection, what should happen? In onService connected, once the connection is established, then this messenger, it returns client will use the binder, so this service is the binder over here. It'll use the binder to instantiate the messenger, which will be used to send the messages to the service. I also have a Boolean over here which specifies that whether I'm bounded or not, since the service connection is established, I'm going to have it as true over here. Finally, what happens is I'm implementing a send message function. This send message function, it makes use of a message which basically implements per syllable. What it does is it defines a message which contains description, arbitrary data object. Here I create a message object, I create a bundle. I put in my text value which I want to send to the service, as well as the key to it into the bundle and assign it to the message. Then using this

particular myMessage object, I'm going to, so when I send the message to my service, it is going to get handled in this handle message function. The message will be received over here, the [inaudible] part will be retrieved. From here, I'm going to take out the value which is assigned to my string, and this will be log. Now, let us see this particular application in action. We run the application. My application is launch. Whatever the message which I want to send to my remote service, I can type it over here. When I click on the send message, it's going to call this particular function, which is send message. In the XML file itself, that is the layout file on click attribute, I called this particular function which is send message. When I click on this, it's going to basically create a bundle of it, put it into a parcelable message, and then send it across to the remote message using this particular object, which is my service. We have the bind up, which will basically bind it to the remote service over here. In the remote service, this particular message will be displayed. Let me show you the rocket as well. When I run this program, so [inaudible] remote service, and you can see in the messages, it is said that bind it to the remote service, hello, remote service. Whatever message which I've sent, it is handled by the remote service, so I will check again. You will see that it is. When you click on the back button, this particular service will be destroyed.

Implement Broadcast Receiver - Static Registration Video• . Duration: 6 minutes

[Notes]

In this demo, we are going to implement a broadcast receiver by registering the receiver in AndroidManifest.xml file. I've created a project, named it as BroadcastReceiver. It has given me the MainActivity.kt which extends AppCompatActivity. Along with that, it also has given a layout file. I'm not going to use a layout file for anything. Here, what I have to do first is I register my receiver in AndroidManifest.xml file. I will have to create my own BroadcastReceiver class. For which what you can do is you can Right click on the package, go to New, and create a New Kotlin class. I will name it as myNewBroadcast. This is basically a class. You can see that it has created. Now I can extend it to implement the broadcast. It says that I will have to implement its members. I'm going to say onReceive, I'll implement this. This is the method which needs to be implemented inside the BroadcastReceiver. I'm going to rename this p0 to context and p1 to intent, so that identify them clearly. What I need to do over here is to implement as to how I will receive the intent. After I have created this particular BroadcastReceiver, what I need to do is I will have to register this particular receiver in AndroidManifest.xml file. Let me show it to you. I'll Manifest.xml file. Within the application element, I am going to have a receiver with a name. I have two BroadcastReceivers over here. I'm going to put in as MyBroadcastReceiver and Android exported, I will keep it as because it will have to receive it from outside system as well. Then here inside, I will have to define intent filter. I will have to define Intent filter, which will help the system to filter out on the based on the different intents, and it's going to only root the intent, for which I'm registering. Android name, so here I will have to pass from there. This is my first filter. I will add one more filter. The first filter will help me identify whenever there is a airplane mode, setting changed. The second one is going to help me identify whenever there is a change in the date. I have defined the intent filters over here. This is how my receiver is actually registered. Now I'll go back to MyBroadcastReceiver and I'm going to implement, so on receiver method, what is that I would want to do. First, I'm going to check if I receive a intent action over here. I will check as to what is the intent which is received, for which I'm

going to have, or if block. Inside, I'm going to check for `intent.action` if it is equal to `Intent.ACTION_DATE_CHANGED`. Basically, I'm going to check if the date is changed or not. You can see that I will have to check for non-safety over here, so I will append a question mark operator over there. Then if I have received that particular intent. What I'm going to do is I'm going to create a message and display the message, which will say that the broadcast which is received is and I'm going to add the `intent.action` over here, and I'm going to display it in the toast message. I have defined on receive as well. What is going to happen is whenever there is a date change, since I have registered my application to get to know that particular event, the Android system will route that particular intent to my application, and in here, I'll be able to handle it. When I say handle it on receive method, I'm basically displaying that I have received this particular intent. This is what I have to do. That's it. Let me run this particular application. What I'll do is, I'll go to the Settings and change the date. I will set the date. It is wrongly set to February 1. I'm going to change the date to January 29. When I click on Done, you should observe that a small pop up is going to come. I'm going to check for the `intent.action` item for airplane mode change, so `Intent.ACTION_AIRPLANE_MODE_CHANGED`. I'm going to pop out a message which will show if I have received this particular intent or not. Let me run this particular application. My application has started. I'm going to minimize the application and go to the Settings, add in Settings here. There is this airplane mode, so I'm going to set it. We see that the intent is received over here. When I uncheck it, again, the intent is received. I can even look into the state information of this particular action field or the intent which I have received and specify whether the airplane mode is enabled or not.

Implement Broadcast Receiver - Dynamic Registration Video• .

Duration: 3 minutes

[Notes]

In this demo, we are going to register our broadcast receiver dynamically in the code and see as to how it works. So I have created this project, which is named as dynamic broadcast receiver. And I am going to define a function which is configure receiver. First, here in this configure receiver, I will have to create a intent filter, add action to it. Then I will have to create a receiver, then using the register receiver method, I need to send my receiver as well as filter object to it and register my receiver. So let's do it. I'm going to create a filter first, okay, by calling the `intent filter pass`. Then I'm going to add action to this, okay, add action. So here I can specify anything, I will specify it as `pgp.fsc.certificate`, okay. So this is the action which I would want to check for, then I'm going to create a receiver. So receiver and so this, I will have to call my receiver which I have implemented over here, okay. So this is the my receiver which I have implemented. So I will be calling my receiver and I will have to register the receiver now, okay, for which I will have to first pass the receiver and then my filter, intent filter, okay. So registration will be done, then I will display a toast message, okay, which will say that the registration is done, okay. So that's it, that is what is required for configuring the receiver. Then in `onStop`, I'm going to call `unregister receiver`, okay, then I will have to use this particular function `broadcast intent` in order to send the broadcast, okay, for which I will have to create an intent, okay, of type `intent` and I will have to specify the action over here, okay. So the action should be matching with this one. to receive anything over here, okay. So let me run the application, okay. So my broadcast is registered, okay. But when I click on the broadcast, will I

receive any message is what I need to check. So I did not receive any pop-up over here, okay. So what I will do is let me kill this application, I will go back to this, I will copy the action string and try to match it inside my receiver, okay. So let me run the application again, okay, my broadcast is registered. Now when I click on broadcast, I should be able to receive this particular message, okay, which will say broadcast intent received with the action. So I click, you can see that broadcast intent received, VGPF certificate is actually getting popped up. So this is how you register your broadcast receivers dynamically in the code.

Network Request Using Retrofit Library Video• . Duration: 9 minutes

[Notes]

In this demo we will see as to how to make use of retrofit library in order to make network request. So this is the URL or the website which we are targeting. And this API lists all the different comments. This is publicly available website, okay? It lists all the comments which are available. In case I pass on a ID to it, okay, it is going to respond with only that particular JSON object for me. So this is what we are going to try. So I have created this particular application. Now, in AndroidManifest.xml I need to make use of users permission for Internet so this is the only thing which we will have to do in AndroidManifest.xml. Now let me show you the layout file. In the layout file I have two text views. One for the name, the other one for the email. So from the JSON object I'm going to retrieve the value which is associated with name and email and I'm going to display it over here, okay? And I can pass on the id. So this is a edit text, okay, where I can pass the ID and that respective IDs name and email will be displayed on the screen. And this is going to happen when I click on the fetch button. Okay, so before we see as to how the whole thing is implemented, I request you people to look into the organization of this particular package. Inside the package I have created three folders. One, the first one which is named as utils. Here I have created a constants class, okay? So this is a constants class which is created under utils, okay? This constants it is defined a companion object in which we are going to define the base URL. So this particular base URL should match the URL which you are targeting. Okay, so this is the first step. Second, you will have to define a models folder. Okay, in this model folder what you have to do is you need to define the data class. This class should match the API's JSON response, okay? So we saw the API's response. It had the ID, the name, post ID, email body. So the name of the variables over here should be exactly same as the JSON object key. So this particular post, it will be used for sending and receiving the requests. Okay, so in order to define this particular class or create this class, we can just right click on the model, go to new click on kotlin class or file. So you will find a data class. So select this and give a name over here. So once you give a name it is going to create the data class for you. Later you will have to define all these values, okay? The third one is you have to create a API directory. So the API directory is the one which is going to hold the API request, the interface as well as the retrofit instance. So first we will have create a interface. Okay, let me show you how to create an interface. I should right click on this, go to new kotlin class or file. So here you will find the interface. So select this and you can give a name for the interface. So once you create, you will get the interface file and in this you will have to define the function which will be invoked, okay? The function which you are going to invoke in your main activity. So here we are defining the function which is getRequest. And you should see that it is not just function, but it is a suspend function. Since this particular function is going to get called within a coroutine, the function is marked

as suspend. Only suspend functions can be called within the coroutine. Normal functions cannot be called. So this getRequest, okay, whatever the id which we pass, that id will be sent as an argument, okay, for this get request, okay? So for the base URL, it is going to append comment/ id and it is going to send it as a request through the retrofit instance, okay? And the return type of this will be the response and the structure of that will match your post. So it is going to match this particular post. That's the reason whatever you define over here should exactly match what is there in the response object, okay? Finally, you will have to create a retrofit instance inside the API folder. So this retrofit instance is of type object. So in order to create that you can right click on API, go to new click on kotlin class or file. You can select a object and give the name over here in order to create the retrofit instance. So this retrofit instance, it's going to implement simple API interface, okay, by making use of the retrofit builder, okay? So I'm going to create a client out of the simple API which will be of type Simple API making use of retrofit buildup. So for this I will have to pass on the base URL which was defined in constants kt. Then I need to add the converter factory. So here I am making use of JSON converter factory and by calling build it is going to build this particular retrofit instance and it going to create by making use of the simple API class. Okay, so once we are done with all this, then we will have to call the first method in the Main activity. So now let us see as to what is implemented in the main activity. So in the main activity you can see that I am making use of coroutine. Since I am making a network core over the Internet, this coroutine, the dispatcher is IO indicating that I'm making a network call and I'm calling the .launch builder of the coroutine which is going to build the coroutine for us, okay? Inside this I am calling the getRequest method of the simple API using the retrofit instance to which I am passing the id, okay? So whatever the value which is entered in the id, it is consumed as a argument over here to be passed, okay? So once you get the response, now you need to switch over from I would to main within the coroutine because now you will have to get the response object details from it and you have to append the user interface. Okay, so user interface can be accessed only through the main thread. So by making use of with context, okay, I'm switching over to the main and inside the main, the body of it I am checking whether the response is successful and I'm calling connecting the body, okay, inside the data object and within that I can get the value which is assigned to name. I can get the value which is assigned to the email and that will be assigned to TB name and tb email of my layout file, okay? If there is an error I am going to display it. And since this is part of the try catch block in the catch also we are trying to capture the exception in case it arises, okay? So let us run this particular application and see as to how it works. My app is installed so I will have to enter a ID here. Let me give the id as 2 and when I click on fetch it should basically target the URL which I have given and fetch the details of id2 and get the name and email and present it over here. So you can see that it has got the name and the email ID of the object which I was targeting.

Introduction to Location Services Video• . Duration: 7 minutes

[Notes]

Hello everyone, welcome to this module on location services in Android. And I am Professor Chaturvedi Shekhar and in this session, we will learn about Android apps which can determine a device physical location using GPS, Wi-Fi and mobile networks. In this session, we will explore the basics of location services, the architecture behind these services, key components and how to use them to get the current GPS location of a device. By the end of this session, you will be able to

understand how location services work and how to implement them in your Android applications. So let's start with the question, what are location services? Location services are essential for apps that require location-based features such as maps, navigation and location tracking. For example, apps like Google Maps use location services to provide directions, while weather apps use location data to deliver weather updates based on your current position. Location services in Android use GPS, Wi-Fi and mobile networks where the app, they help enhance user experiences by allowing apps to offer personalized content based on where the user is. Location-based apps also help users navigate to their destination or find nearby places like restaurants and shops. Now let's discuss why location services are important. First, they allow apps to offer personalized experiences based on users' location. For example, a restaurant app can recommend nearby dining spots, while a shopping app can show deals available in the user's area. Secondly, location services enable location-based features such as finding nearby places, providing directions or triggering actions based on proximity. For instance, a fitness app can track a user's run using GPS, while a navigation app can provide turn-by-turn directions based on user's current location. Let's now explore the architecture and key components that make up location services in Android. The `LocationManager` class is responsible for managing location providers and giving the app access to location data. It interacts with different location providers such as GPS, network providers, and passive providers to determine the best possible location. Some of the key components are `LocationManager`, which manages location providers and provides location updates. `LocationListener`, which receives notifications when the location changes. `LocationProvider`, which provides the actual location data. And we have `LocationObject`, which stores details like latitude, longitude, altitude, and speed. These components work together to deliver accurate and real-time location data to your app. Android uses location providers to determine the device location. The three main providers are GPS, which is a global positioning system that provides the most accurate location but consumes more battery and may not work well indoors. Then we have `NetworkProvider`, which uses Wi-Fi and mobile network data to estimate the location. It is less accurate than GPS but more power efficient. Then we have `PassiveProviders` that listen from location updates when other apps or services request them, helping conserve battery. Additionally, the `FusedLocationProvider` client from Google Play Services combines data from GPS, Wi-Fi, and cell towers to provide the most accurate location, while the `GeoFencing` API triggers actions when the device enters or exists a specific geographic area. The `LocationObject` stores the device geographic information such as latitude, longitude, altitude, speed, and accuracy. This object has several methods that help retrieve important location details. Some key methods include `GetLatitude` and `GetLongitude`, which return the device latitude and longitude in degrees. We have `GetAltitude` that returns the altitude from sea level in meters. `GetSpeed` tells the device speed in meters per second. `GetAccuracy`, which returns the estimated accuracy of the location in meters. These methods allow apps to access specific location details and use them for navigation, tracking and other purposes. Here are some more methods used for setting the bearing, longitude, latitude, and speed, etc. Let's now look at an example of how to retrieve current GPS location using the `LocationManager`. Here is a simple code snippet. In this example, we use the `LocationManager` to get the last known location from `GPSPProvider`. The app retrieves the longitude and latitude and can then use values to display the user's current location on a map or perform other location-based tasks. To summarize this session, we saw location services in Android that helps apps determine the device physical location using GPS, Wi-Fi, and mobile networks. The `LocationManager` class, along with `LocationProviders`, helps apps access location data. Android offers several location providers, including GPS, network providers, and passive providers to

provide location details. Then we add `LocationObject`, which stores important geographic information like latitude, longitude, and speed. We also explored an example of how to retrieve the current GPS location using `LocationManager`. By understanding these concepts, you can create location-based Android apps that deliver a personalized and enhanced user experience. Thank you for your attention. I hope this session has given you a clear understanding of location services in Android and how to use them in your apps. Thank you.

Using FusedLocationProviderClient Video• . Duration: 6 minutes

[Notes]

Hello, everyone, and welcome to the module on Location Services in Android. In this session, we'll be focusing on using the Fused Location Provider Client is part of Google Play Services Location API. And it provides an easy and efficient way to obtain devices location. This service combines data from GPS, Wi-Fi, and mobile networks to offer accurate and power efficient location results. In this session, we will cover how to set up Fused Location Provider Client, request location permissions, retrieve the last known location, and handle location updates. By the end of this session, you will have a solid understanding of how to use this service in your Android applications. So what is Fused Location Provider Client? The Fused Location Provider Client is a part of Google Play Services that allows apps to access the device's location in a simple and optimized manner. It combines data from multiple sources, including GPS, Wi-Fi, and mobile networks, to provide accurate location information. Fused Location Provider Client offers several advantages, first one being power efficiency. It optimizes battery usage by selecting the most appropriate location provider based on the situation. It has a simplified API, which offers a straightforward way to request location updates without the need for complex configurations. And it is accurate, that is, by combining data from multiple location sources, it provides more accurate location results compared to using only GPS or network providers alone. Now let's see how to set up Fused Location Provider Client in our Android project. First step is to add dependencies. We will need to include the Google Play Services dependency in our `build.gradle` file. In the second step, we need to initialize Fused Location Provider Client in your activity or fragment. Here is an example. By setting up Fused Location Provider Client, we can now access location data from our app in an optimized and power-efficient way. Before accessing the device location, it is important to request location permissions. Since location data is sensitive, the app must ask the user for permission at runtime if it has not already been granted. First one is to check for runtime permissions. Ensure that your app requests location permissions in the manifest and at runtime using the following code. Next is to handle permission results. After requesting permissions, we must handle the user response. If the permission is denied, the app should either disable location-based features or prompt the user again with an explanation. Let us now see how to get last known location of the device. The last known location is the most recent location fix that is available on the device. Here is a simple way to retrieve it using Fused Location Provider Client. Keep in mind that the last known location could be null if the device hasn't received a location update yet, or if the location services are turned off. Always ensure you handle this case properly. Once you start receiving location updates, it's important to handle them efficiently and stop the updates when they are no longer needed to conserve battery. As a first step, handle the location updates. You can receive regular location updates using Location Callback. This allows your app to update the user's location in real-time. As

a next step, stopping location updates. To avoid unnecessary battery drain, stop location updates when there is no longer needed. This can be done as shown in the following code snippet. By managing location updates properly, you can ensure that your app remains battery efficient while still delivering accurate location data. Let's walk through an example using Fused Location Provider Client to get the user's current location. Here is a simple code implementation using Kotlin. In this example, the Fused Location Provider Client retrieves the user's last known location and the latitude and longitude are extracted and used as required by the app. This simple approach ensures that your app can access accurate location data with minimal battery usage. To summarize this session, Fused Location Provider Client combines data from GPS, Wi-Fi and cellular networks to provide accurate and power-efficient location results. We learned how to set up Fused Location Provider Client and request location permissions at runtime. We also saw as to how to use last known location and also efficiently handle and stop location updates which conserve battery life while still providing up-to-date location information. By understanding these steps, you can integrate location services into your Android apps and offer personalized and location-based experiences to your users. Thank you for your attention. I hope this session has provided you with a clear understanding of using Fused Location Provider Client for location services in Android. Thank you.

Introduction to Google Maps API Video• . Duration: 5 minutes

[Notes]

Hello everyone, and welcome to the module on location and maps. In this session, we will focus on the Google Maps API. Google Maps API allows developers to integrate Google Maps into their Android applications, providing users with interactive maps, real-time location tracking, and navigation features. In this session, we will cover what Google Maps API is, how to set it up, and how to add and initialize Google Maps in your app. By the end of this session, you will understand the basics of integrating Google Maps into your Android projects. So let's start with what is Google Maps API. The Google Maps API is a service that allows developers to embed Google Maps into their mobile applications. With the Google Maps API, you can provide users with features like displaying locations on a map, geocoding, which converts addresses into geographic coordinates like latitude and longitude, route planning and navigation, which offers directions and guiding users to their destination, adding custom markers, lines, and polygons to the map. The Google Maps API is widely used in apps that need to show locations, offer directions, or provide location-based services such as finding nearby places or real-time tracking. Let's now look at how to set up the Google Maps API in your Android project. The first step is to get an API key from the Google Cloud Console. So you have to go to Google Cloud Console at console.cloud.google.com. Create or select a project where you want to enable the API. Then navigate to the APIs and Services. Go to the Credentials section. Create an API key. This key will allow your app to use the Google Maps API. Then enable the Maps SDK for Android. This is necessary to allow your app to display Google Maps. Once you have API key, you can proceed to the next set of steps. The second step is to add API key to your Android project. To do this, you have to first open your Android project in Android Studio. Add the API key to the AndroidManifest.xml file as shown in the code snippet. This will allow your app to use the Google Maps services provided by the API key. The third step is to add the Google Play Services dependency to your project. Google Maps is a part of Google Play Services. So you need to ensure that it is included in your app's build configuration. So you need to right-click

on your project's app folder and select Open Module Settings. Go to Dependencies section. Click on Plus in order to search for `com.google.android.gms:play-services-maps`. Add the following dependency to the `build.gradle` file. After adding the dependency, sync your project to include the library. After setting up the API key and dependencies, the next step is to add Google Maps to your app's layout and initialize the map. First step is to add a map fragment or support map fragment in your layout file. In the snapshot, you can see an example. Then we need to initialize the map in the Kotlin code. This is also shown in the code snippet. This sets up the Google Map in your app and allows you to customize it with markers, lines, or routes as needed. Let's quickly review some common use cases of Google Maps API. First one is to display locations, which will show specific locations on the map, such as a store, a landmark, or a user's current location. Then we can use it even for geocoding, that is to convert addresses into geographic coordinates to display them on the map. It is also used for route planning and navigation. Custom marks and overlays can be put on it. So these features make Google Maps API a powerful tool for building location-based apps. To summarize this session, the Google Maps API allows you to integrate interactive maps to your Google apps. We also discussed how to set up the API by obtaining a key. We learned how to add and initialize Google Maps in the app using support map fragment. By understanding these steps, you can easily implement map features in your Android apps, providing users a rich and more interactive experience. Thank you for your attention. I hope this session has given you a clear understanding of Google Maps API and how to integrate it into your Android apps. Thank you.

Displaying a Map Video• . Duration: 5 minutes

[Notes]

Hello everyone, welcome to the module on Location and Maps. In this session, we will focus on displaying a map in Android. Google Maps API allows developers to integrate interactive maps into their Android apps, which can be used to display user locations, places of interest, and even navigation. In this session, we will cover how to add map to your app layout, initialize the map, and add features such as markers and camera movements. By the end of this session, you will be able to effectively display and customize maps in your Android applications. The Google Maps API makes it easy to display interactive maps in Android apps. Maps are essential for various use cases such as showing user locations, highlighting places of interest, providing navigation for users to reach their destinations. With Google Maps, developers can build location-based apps that offer users real-time updates and detailed geographic information. Whether it's a travel app or a navigation tool, maps can greatly enhance the user experience by offering personalized and interactive data. To display a map in your Android app, two key components are required. Google Maps API. This API allows you to integrate Google Maps features into your app. Second one is `SupportMapFragment`. This component is used to add a map to your layout. It provides the map interface and handles the integration between your app and Google Maps. With these components, you can embed a fully functional map that interacts with user's action, such as tapping on markers or zooming in on locations. The first step in displaying a map is to add a `SupportMapFragment` to your layout XML file. In this code, the `SupportMapFragment` is embedded into the layout, which will display the map when initialized. This provides the container where Google Maps features will be displayed, such as markers, zoom controls, and camera movements. After adding the map to the layout, the next step is to initialize the map in your Kotlin or Java code. You can do this by using

getMapAsync method, which is called when the map is ready for use. This is shown in the score snippet. In this code, the getMapAsync method provides a reference to the Google Map, which you can use to interact with the map. After initializing, you can start adding markers and adjusting the camera position based on user's location or other factors. Once the map is initialized, you can make it more interactive by adding markers and camera movements. And here is how it is done. In this example, a marker is added at the coordinates for Mumbai, Bangalore, and Chennai. The camera is then moved to the focus on marker's location. You can add multiple markers, move the camera, and zoom in or out to create a dynamic map experience for users. In addition to single markers, you can also add multiple markers to your map. This is useful for showing several points of interest, such as multiple stores or landmarks. This code adds markers for San Francisco and Los Angeles.

Displaying multiple points on the map, you can add as many markers as needed, allowing users to see several important locations at once. To summarize this session, the Google Maps API allows you to display interactive maps in Android apps, enhancing the user experience by offering real-time geographic information. You can easily add a map to your app by adding or by using support map fragment in your layout. After adding the map, you must initialize it using your Kotlin code. Markers and camera movements can make the map interactive, allowing users to view specific locations and zoom in on areas of interest. You can also add multiple markers. By understanding these concepts, you can create dynamic and useful map-based features in your Android apps. Thank you for your attention. I hope this session has provided you with a clear understanding of displaying a map in Android. Thank you.

Location Data Video• . Duration: 6 minutes

[Notes]

Hello everyone, and welcome to the module on location and maps. In this session, we'll focus on handling location data in Android. Many Android apps require access to location data to provide personalized services like navigation, local recommendations, or location-based alerts. In this session, we will cover the steps for handling location data, requesting permissions, and using the Fused Location Provider client to get the last known location. By the end of this session, you will be able to efficiently manage location permissions and retrieve location data in your Android apps. Let's begin by understanding what location data is. Location data refers to geographic information that allows an app to determine where the device is located. This information is crucial for apps that rely on location-based services such as navigation, weather, and local business directories. Location data can be obtained from different sources. GPS, which provides highly accurate location data but consumes more battery. Wi-Fi and mobile networks, which provide less accurate location data but are more power efficient. By handling location data, apps can offer users more personalized and context-aware services. Now, let's see why we need to handle location data. Handling location data is important for several reasons. First one being personalized user experience. Apps can use location data to provide users with content, services tailored to their location. For example, weather apps show local forecasts and map apps provide nearby landmarks or navigation instructions. Then, proximity-based actions. Location data allows apps to trigger actions when users enter specific geographic areas such as receiving notifications when near a store or an event venue. You can even have it as location-based features. Apps like food delivery or taxi services rely on accurate location data to provide seamless services. By properly managing location data, you can ensure

that your app delivers a smooth and reliable experience for users. To access location data, your app must request location permissions from the user. There are two main types of location permissions in Android. Access underscore find underscore location, which provides precise location data, usually from GPS. Source underscore course underscore location, which provides less precise location data, typically from Wi-Fi or mobile network. You need to declare these permissions in the androidmanifest.xml file as shown in the code snippet. Without the correct permissions, your app will not be able to access the device location. In addition to declaring permissions in the manifest, Android requires you to request permissions at runtime for sensitive data like location. Here is an example of how to request location permissions in your app dynamically in runtime. This code checks whether location permissions have already been granted. If not, it requests the permission from the user. By declaring runtime permissions, ensure that your app respects the user privacy and complies with Android's permission policies. After requesting permissions, your app needs to handle user's response. This is done by overriding the onRequestPermissionsResult method as shown in the snapshot. If the user grants the permission, the app can proceed to access location data. If the permission is denied, the app should either show a rationale explaining why permission is necessary or provide alternative features that do not require location access. To retrieve location data in Android, we use FusedLocationProviderClient. This client combines location data from multiple sources, GPS, Wi-Fi, and mobile networks to provide accurate and efficient location results. Here is an example of how to use FusedLocationProviderClient to get the last known location. This code retrieves the most recent location fix which can be used to display the user's location on a map or perform other location-based actions. To summarize this session, location data which provides geographic information used for location-based services in Android apps was looked into. We also saw as to how to request for location permissions. Runtime permission requests ensure that users can control whether the app can access their location. Finally, the FusedLocationProviderClient is used as an efficient tool for retrieving location data which combines data from GPS, Wi-Fi, and mobile networks. By following these steps, you can effectively manage location data and permissions in your Android apps. Thank you for your attention. I hope this session has helped you understand handling location data in Android. Thank you.

Displaying Location on Map Video• . Duration: 5 minutes

[Notes]

Hello everyone. Welcome to the module on location and maps in Android development. In this session, we'll focus on displaying location on a map using Google Maps API. Many Android apps such as navigation, tracking and mapping apps require the user location to be displayed on a map. In this session, we will cover the prerequisites, key components and steps for showing the user's current location on a map. By the end of this session, you will understand how to display location data on a map in your Android application. Displaying the user's current location on a map is a common feature in many Android applications. This is particularly useful in navigation apps, which provide directions to the user based on their current location. And also by location tracking apps, where tracking the real time movement of the users is done. Mapping apps, which show nearby places or landmarks based on user's current position. In all these scenarios, the user's location is updated in real time and displayed on a map. This functionality enhances the user experience by offering personalized and location based services. Before displaying the user's location on a map,

there are some prerequisites that needs to be set. First one is to set up Google Maps API. You need to obtain our API key from the Google Cloud Console and enable Maps SDK for Android. Then in the second step, we need to add permissions in AndroidManifest.xml file, which will request the necessary permissions to access the location data. Third step is to add support map fragment in the layout file. This fragment allows you to embed Google Map into your app's user interface. Fourth one is to initialize map in Kotlin. Fifth one is to request location permissions at runtime. Finally, we need to handle permission result. Let's now walk through the steps for setting up the Google Maps API in the Android project. Go to the Google Cloud Console and create a project. Enable the Maps SDK for Android and obtain a API key. Add the API key to your project by updating your AndroidManifest.xml file as shown in the diagram. This will allow your app to access the Google Maps API and display the map with location features. Next, you need to add a support map fragment in your layout to display the map. Here is how it is done in the layout.xml file. Once the fragment is added, you need to initialize the map in the Kotlin code as shown in the below diagram. This initializes the map and prepares it for adding features like markers and displaying the user's current location. To access location data, you need to request location permissions from the user. There are two types of location permissions in Android. One is a find location, which gives precise location data using GPS. The other one is course location, which gives approximate location data using Wi-Fi or mobile networks. At runtime, you also need to request these permissions from the user and handle the result accordingly. Finally, to display the user's current location on the map, use the fused location provider client to retrieve user's last known location and update the map accordingly. Here is how it is done in the example. In this code, the user's location is retrieved, a marker is placed at their position, and the camera is moved to focus on the current location. To summarize this session, we saw as to how to display the user location on a map before showing the location, you need to set up Google Maps API, add permissions, and configure the map. You also need to use fused location provider client to retrieve the user's current location and always handle location permissions carefully to respect the user's privacy and to ensure app works as expected. By understanding these steps, you can build powerful location-based apps that provide real-time location for tracking users. Thank you for your attention. I hope this session has given you a clear understanding of how to display user's location on a map. Thank you.

About Firebase Video• . Duration: 6 minutes

[Notes]

Welcome to the module on Firebase integration. I am Professor Chhatha Shekhar and in this session, we will be introducing Firebase, a powerful platform from Google that provides a variety of cloud-based tools to help developers build, improve, and grow their mobile and web apps. In this session, we will cover what Firebase is, its history, and why developers should use it in their projects. By the end of this session, you will understand the basics of Firebase and its advantages for modern app development. Let's begin with what Firebase is. Firebase is a platform developed by Google that provides a range of tools and services for building and improving mobile and web applications. Some of its key offerings include Real-Time Database. Firebase allows you to store and sync data in real-time across all clients, making it easier to build collaborative apps. Authentication Firebase provides simple, secure authentication methods like Google, Facebook, and email password login. Analytics You can track user behavior and engagement using Firebase

Analytics, which helps improve app performance. And Cloud Functions This allows developers to write server-side logic that can be executed in response to events triggered by Firebase features or HTTPS requests. Firebase integrates smoothly with both mobile and web applications, making it a popular choice for developers looking for an all-in-one solution. Now let's explore the brief history of Firebase. Firebase was initially launched as a real-time backend platform by its founders, James Tamplin and Andrew Lee, in 2011. It gained popularity for its ability to synchronize data in real-time, which made it ideal for chat applications and collaborative tools. In 2014, Firebase was acquired by Google, which expanded its offerings beyond just real-time databases. Today, Firebase includes a wide range of services that support the entire app development lifecycle, from database management to cloud hosting and analytics. Its growth has been phenomenal, becoming one of the most preferred platforms for developers worldwide due to its easy integration with Google Cloud Services and comprehensive suite of tools. Now let's discuss why you should use Firebase in your projects. There are several reasons why Firebase stands out as a valuable tool for developers. First, it is an all-in-one platform. Firebase provides everything you need for app development in one place. From authentication to analytics, Firebase covers all aspects of the app lifecycle. Firebase's real-time database is highly useful for apps that require instant updates, such as messaging, live data streaming, or collaborative editing tools. It provides scalability as well. Firebase is built on Google's cloud infrastructure, making it highly scalable. Whether you have a small app or an app with millions of users, Firebase can handle the load effortlessly, and it is also cross-platform. Firebase supports both Android and iOS, as well as web apps, which makes it easier for developers to build cross-platform applications. And it provides easy integration. Firebase integrates seamlessly with popular tools like Google Analytics, AdMob, and other Google Cloud Services, making it a versatile platform for app development. Firebase provides a wide range of tools that developers can use to enhance their applications. Let's briefly discuss some of the key tools. First one is the Firestore. This is a NoSQL cloud database that allows developers to store, sync, and query data for mobile and web apps at global scale. Firebase Hosting. It offers secure, fast, and reliable hosting for web apps, landing pages, and progressive web apps. Cloud Messaging. Firebase Cloud Messaging allows you to send notifications and messages to your users across platforms. And then there is Crashlytics. This tool helps you track and fix app crashes in real-time, allowing for faster issue resolution and a more stable user experience. Each of these tools is designed to simplify and streamline the development process, making it easier to manage various aspects of app development. To summarize, Firebase is an incredibly powerful platform that provides a range of cloud-based tools for both mobile and web application development. Some key points covered included Firebase, which is a Google platform offering tools like real-time databases, authentication, analytics, and cloud functions. The history of Firebase began as a real-time back-end system, and after being acquired by Google, it has grown into a fully featured app development platform. Developers should use Firebase because it offers an all-in-one solution, real-time data syncing, and scalability, all backed by Google's robust infrastructure. By using Firebase, developers can create highly interactive real-time applications while simplifying the back-end management. Thank you for your attention. I hope you have a better understanding of Firebase and its advantages for app development. Thank you.

Firebase Products Video• . Duration: 6 minutes

[Notes]

Hello, everyone. Welcome to the module on Firebase integration. And in this session, we will discuss Firebase products. Firebase is a platform developed by Google that offers various tools to help developers build, release, monitor, and engage with mobile and web applications. Firebase provides a wide range of products that cover different stages of app development, from building the app to engaging users. By the end of this session, you will understand the core products of Firebase and how they can help streamline your app development process. Firebase offers several key products that cover every stage of app development. Here is a quick overview of Firebase products. First, you have Firebase Realtime Database. This is a NoSQL cloud database that syncs data in real time across all connected clients. Then we have the Firestore, another cloud database, but with more advanced querying capabilities and better integration with Google Cloud. Then we have Firebase Authentication. It provides secure and easy authentication for users via email, Google, Facebook, and other login methods. Then we have the Firebase Cloud Storage. This allows you to store large files like images, videos, and documents in the cloud securely. Then we have Firebase Cloud Functions. These are serverless functions that run in the cloud in response to events like user authentication or database changes. Now let's focus on products that help you build your app using Firebase. Let's first look into Realtime Database. Firebase Realtime Database helps you build collaborative apps by syncing data in real time across all users. Firestore is similar but more powerful than Realtime Database. It offers advanced querying features and better performance for more complex apps. That is, Firebase Authentication helps you manage user authentication in a secure and simple way, supporting login via multiple ways, either through email, social media accounts, or anonymous sign-in. Then it also provides storage in the cloud. That is, Firebase Cloud Storage is ideal for storing large files such as images, user-generated content, et cetera. It also provides functions in the cloud. With Cloud Functions, you can write server-side logic without managing servers. For example, you can trigger actions like sending emails or updating databases when certain events happen in your app. Once your app is built, Firebase also helps you release and monitor it. Firebase Crashlytics. This tool helps you monitor your app for crashes and issues in real time, allowing you to fix problems quickly. Firebase Performance Monitoring Tool. It helps you track the performance of your app, such as loading times and network requests, so you can optimize it for better user experience. Firebase App Distribution. This tool lets you easily distribute your app to testers before launching it publicly. Firebase Test Lab. Firebase Test Lab allows you to test your app on various real devices in the cloud to identify potential bugs and issues across different environments. These products help you ensure that your app is ready for users and is monitored effectively after launch. Firebase also provides tools to engage with users and improve their experience. Firebase Analytics. This tool helps you track user behavior and gain insights into how users interact with your app. With this data, you can make informed decisions on how to improve your app. Firebase Cloud Messaging. The FCM allows you to send push notifications to your users, keeping them engaged with your app by informing them about updates or new content. Firebase Remote Config. With Firebase Remote Config, you can change the appearance or behavior of your app without publishing an update to the Play Store or App Store. This allows for quick adjustments based on user feedback or data insights. Then we have Firebase Dynamic Links. Dynamic Links are smart URLs that work across multiple platforms, allowing you to guide users to specific content in your app, whether they are new or existing users. Firebase In-App Messaging. This tool lets you engage users while they are using your app by displaying relevant messages based on their actions, like offering a discount or tutorial. To summarize, Firebase provides a wide range of products for building, releasing, monitoring, and engaging apps. Some key products for

building include Firebase Realtime Database, Firestore, Authentication, Cloud Storage, and Cloud Functions. For release and monitoring, Firebase offers Crashlytics, Performance Monitoring, App Distribution, and Test Lab. For user engagement, tools like Analytics, Cloud Messaging, Remote Config, Dynamic Links, and In-App Messaging help keep users engaged and improve their experience. By leveraging these Firebase products, you can build more efficient, scalable, and engaging applications. Thank you for your attention. I hope you now have a clear understanding of Firebase products and how they can help you in building, releasing, and engaging with your apps. Thank you.

Firebase Android Video• . Duration: 7 minutes

[Notes]

Hello everyone, welcome to the module on Firebase integration for Android. And in this session, we will learn about Firebase for Android and how it can enhance your Android app development experience. Firebase is a platform developed by Google that provides a variety of tools and services to help developers build, improve and grow their apps. In this session, we will focus on the key features of Firebase, how to set up Firebase in an Android app and how Firebase authentication works in Android. By the end of this session, you will understand how to integrate Firebase into your Android projects. Let's begin by discussing the key features of Firebase for Android. Firebase provides several useful tools that make it easier to develop, deploy and monitor Android applications. And here are some of the key features. **Firebase Realtime Database** This is a cloud-based NoSQL database that syncs data in real-time across all connected clients, making it ideal for collaborative apps. **Firebase Firestore** is a NoSQL database. Firestore offers powerful query and better performance for complex applications. **Authentication provided by Firebase** This feature simplifies user authentication by providing secure login methods, including email password, Google and Facebook logins, the FCM, which is **Firebase Cloud Messaging**. FCM allows you to send push notifications and messages to users across different platforms, keeping them engaged with your app. **Analytics by Firebase** Firebase Analytics helps you track user interactions and gather insights to improve app performance and user engagement. These features make Firebase an all-in-one solution for developing Android apps that are scalable, secure and user-friendly. Next, let's walk through the process of setting up Firebase in your Android app. Setting up Firebase is simple and involves the following steps. First step, create a project in Firebase Console. Go to the Firebase Console, create a new project and then click Add App, choosing Android as the platform. Second step is to register your app. Enter the package name of your Android project to register the app with Firebase. Third step is to download GoogleServices.json. After registering the app, download the GoogleServices.json file and place it in the app folder of your Android project. Continuing with setting up of Firebase in an Android app. As a fourth step, add the Firebase SDK dependencies to your build.gradle file. For example, for Firebase Analytics, you would add the dependency which will implement com.google.firebase Firebase Analytics. Then finally, you need to sync your project with Gradle to complete the Firebase setup. By following these steps, you can easily integrate Firebase into your Android app allowing you access to all its powerful features. Now, let's discuss how to implement Firebase Authentication in Android. Firebase Authentication makes it simple to add secure authentication to your app using popular login methods such as Google, Facebook, and email password authentication. Here is how Firebase Authentication works. First, to

use `signInWithEmailAndPassword` method. This method is used to sign users in with email and password. Firebase automatically handles the authentication process and returns a user object if the login is successful. Second method is through `SocialLogin`. Firebase also supports social login methods like `GoogleSignIn` and `FacebookLogin` which you can integrate with minimal setup. Firebase Authentication not only simplifies the login process but also ensures that user data is stored securely giving users confidence in your app's security. Let's take a deeper look at some of the key features of Firebase Authentication. Firebase Authentication provides multiple authentication methods. Firebase supports email password login as well as third-party providers like Google, Facebook, and Twitter. This makes it easier for users to sign up and login. It also provides secure authentication. Firebase takes care of securely handling user credentials so you don't have to worry about securing passwords or tokens manually. It also provides anonymous sign-in. Firebase allows users to sign in anonymously letting them try out your app without creating an account. Later, they can upgrade to a permanent account without losing any data. With these features, Firebase Authentication provides a flexible and secure way to manage user accounts in your Android app. To summarize this session, Firebase is a versatile platform that offers several key features for Android development including real-time database, Firestore, authentication, cloud messaging, and analytics. Setting up Firebase in an Android app involves creating a project in Firebase Console adding the `google-services.json` file and configuring the SDK in your project's `build.gradle` file. Firebase Authentication simplifies the process of adding secure login methods to your app supporting email password and social logins like Google and Facebook. By integrating Firebase into your Android app you can take advantage of these powerful features to build scalable, secure, and engaging applications. Thank you for your attention. I hope now you have a good understanding of Firebase for Android and how to integrate its key features into your project. Thank you.

Introduction to Firebase Realtime Database Video• . Duration: 7 minutes

[Notes]

Hello everyone, welcome to the module on Firebase integration. And in this session, we will explore Firebase Realtime Database is a powerful NoSQL database from Google. It is designed to store and sync data in real time, making it ideal for apps that require frequent updates, like chat applications or collaborative tools. In this session, we will cover the key features of Firebase Realtime Database, its use cases, how it works, and how to set it up in an Android app. By the end of this session, you will have a solid understanding of how Firebase Realtime Database can help you build dynamic and responsive apps. Now, let's discuss what Firebase Realtime Database is. Firebase Realtime Database is a cloud-hosted database provided by Google that allows you to store and sync data in real time. It offers three key features. First one is the real-time synchronization. Firebase ensures that data is updated across all devices in real time. Whenever data is changed, it is instantly reflected on all connected devices. Second, it provides offline support. Firebase Realtime Database can also work offline. This means users can continue using the app and making changes even without an internet connection, and once the device goes back online, the data syncs automatically. It also provides cross-platform compatibility. Firebase supports Android, iOS, and web apps, making it easier to build apps that work across multiple platforms with real-time data sync. These features

make Firebase Realtime Database ideal for building dynamic applications that require constant data updates and offline functionality. Now, let's take a look at some ideal use cases for Firebase Realtime Database. This database is well suited for applications that require continuous updates. Some of the most common use cases include chat applications. Firebase Realtime Database is perfect for building chat applications where messages need to be updated in real-time for all users in a conversation. Collaborative applications, apps that require multiple users to work together, like collaborative document editing or real-time drawing, can benefit greatly from Firebase Realtime Data Sync. Live data applications, apps that provide real-time updates, such as live sports scores, stock market data, or news feeds, can use Firebase Realtime Database to keep users updated without delay. These use cases show how Firebase Realtime Database excels in managing real-time collaborative data environments. Let's now understand how Firebase Realtime Database works. Firebase stores data as a large JSON tree. The data is structured in nodes, and each node can contain key-value pairs or even other nodes. First, let us look into the data structure. Firebase uses a hierarchical JSON structure to store data. For example, a node for user profiles could have subnodes for each user's details. Second one is real-time sync. Firebase automatically syncs data across all clients connected to the database. If one user makes changes to the data, these changes are instantly reflected for all other users. Third one is offline access. Even if the device loses internet connectivity, Firebase Realtime Database continues to function. Any changes made while offline are queued and synced with the server once the device is back online. This design ensures that your app provides a seamless user experience, even when internet connectivity is inconsistent. Let's now look at steps for setting up Firebase Realtime Database in your Android app. As the first step, we need to add Firebase to your Android project. First, go to the Firebase console and either create a new project or use an existing one. Add an Android app to the project and register it using app's package name. Then, download the google-services.json file and place it in the app directory of your Android project. Then, we need to add Firebase Realtime Database SDK to build.gradle file. Open your app's build.gradle file and add the dependency which will be for Firebase database. And finally, we need to sync the Gradle. After adding dependency, you have to sync your project with Gradle to apply the changes. These steps will enable Firebase Realtime Database in your Android project, allowing you to start integrating real-time data into your app. To summarize this session, Firebase Realtime Database is a powerful tool for storing and syncing data in real-time across multiple platforms. It provides real-time synchronization, offline support and cross-platform compatibility, making it ideal for dynamic and collaborative applications. And we also saw some common use cases that include chat applications, collaborative tools and live data apps. And Firebase Realtime Database works by storing data as a JSON tree and syncing it across all clients, even supporting offline changes. Setting up Firebase Realtime Database in Android is easy, requiring just a few steps to integrate it into your project. By understanding and implementing these concepts, you can build responsive and real-time applications using Firebase Realtime Database. Thank you for your attention. I hope you now have a better understanding of Firebase Realtime Database and how it can enhance your Android app development. Thank you.

Adding Data to Database Via Firebase Console Video• . Duration: 6 minutes

[Notes]

Hello everyone, welcome to the module on Firebase integration. In this session, we will learn how to add and manage data in Firebase real-time database using the Firebase console. Updating Firebase database using console is a very useful feature for developers who want to quickly set up and manage data in their apps without having to write code. By the end of this session, you will understand how to access Firebase console, add data, edit data, and import JSON files into the database. The Firebase real-time database console is a user-friendly interface that allows developers to manage, view, and modify data in real-time, providing instant updates and seamless integration with Firebase-powered apps. Using the Firebase console to add data is simple and efficient, offering a user-friendly interface for real-time updates, testing, debugging without code. It allows secure management of database, easy data organization, and immediate reflection of changes, making it ideal for development and testing purposes.

How to access and configure Firebase real-time database? Here are the set of steps. First, we have to open the Firebase console in your browser and select your project from the project dashboard or create a new project if you don't have one. Navigate to the real-time database. On the left-hand side of the console, click on build and then select real-time database from the menu. Third step is to create the database. If you haven't set up your database yet, click on create database. You will be prompted to choose a location for the database and configure its security mode, either lock mode or test mode. Lock mode is a secure one and test mode is open for all users. Once the database is set up, you will see an empty JSON tree in the real-time database section ready for you to start adding data. Let's now look at how to add data to Firebase real-time database via console. As a first step, we need to add manually the data. To manually add data, you will see JSON structure in the Firebase console. Click on the plus button or add child button to add new key value pairs. As a second step, we need to define key value pairs. The key represents the node or field in the database and the value represents the data you want to store. For example, you might have a key for name and the value like John or Jane. Third one is real-time updates. After adding data, it is instantly saved and synced across all clients connected to the Firebase real-time database, allowing users to access the most up-to-date information. This method is ideal for quick manual additions or for testing purposes during app development.

Next, let's learn how to edit data in Firebase real-time database directly from console. As the first step, we have to locate the data. To edit existing data, locate the specific node or field you want to modify. Click the pencil icon. Next to each node, you will see a small pencil icon. Clicking this icon allows you to edit the value. Then we have to save the changes. After making your changes, click save to update the database. These changes are immediately reflected in real-time across all clients. Firebase simple editing interface makes it easy to update data without writing any code, which is helpful during the early stages of app development or testing. Let's now talk about how to import or export data using JSON in Firebase real-time database. You can import data into Firebase by using JSON files. For example, if you have a JSON file that contains user data, such as shown in the code snippet, you can upload this JSON file into the Firebase real-time database to quickly populate your database with data. In order to export, similarly, you can use existing Firebase data into the JSON format by clicking export JSON option. This allows you to backup and migrate your database easily. This feature is especially useful when you need to bulk upload data or create backups. To summarize, Firebase real-time database can be easily managed through the Firebase console. You can access and configure your database via the Firebase console, where you can add, edit and delete data without needing to write code. Manually adding data is simple using key-value pairs. Editing and managing data is also easy with Firebase console. Importing and exporting data using JSON files allows for bulk uploads and backups. By

understanding these features, you will be able to manage your Firebase real-time database efficiently, especially during development and testing. Thank you for your attention. I hope you now have understanding of how to add, edit and manage data in Firebase real-time database using the Firebase console. Thank you.

Adding Data to Database Via Android App Video• . Duration: 5 minutes

[Notes]

Hello, everyone. Welcome to the module on Firebase integration. In this session, we will explore how to add and manage data in Firebase Realtime Database directly through an Android app. Firebase Realtime Database is a powerful tool that allows data to be stored and synced across all connected clients in real time, making it ideal for applications like messaging, collaboration, or live data updates. By the end of this session, you will know how to set up Firebase Realtime Database in an Android project, initialize the database, and add both simple and complex data to it. Firebase Realtime Database helps developers store and synchronize data across all connected clients in real time, ensuring that all users receive instant updates. To achieve this, we use Firebase SDK, which simplifies the process of interacting with database. The overall process involves setting up Firebase, initializing the Realtime Database, and then writing data from your Android app. As soon as the data is updated, it is instantly available to all connected clients in real time, improving user experience. Let's begin with the steps to set up Firebase Realtime Database to your Android project. Add Firebase to your Android project is the first step. Open the Firebase console, create a new project or select an existing one, and add your Android app to the project. Next step is to add Firebase dependencies to build.gradle file. In the project level build.gradle file, add the Firebase plugin. In the app level build.gradle file, add the Firebase Realtime Database dependency. Finally, you need to sync your project. That is, after adding the necessary dependencies, sync your gradle files to apply the changes. Once these steps are complete, your Android project is ready to use Firebase Realtime Database. Now let's learn how to initialize and add data to Firebase Realtime Database from your Android app. As a step one, we need to initialize the database. To initialize the database, you first need to create an instance of database reference in your activity, as shown in the snapshot. In the second step, we need to add data. To add simple data like username, you can use setValue method to update the database. This adds a new node to the database under users with key user1 and sets the username to John Doe. These basic steps allow you to add and update data in real time across all devices connected to the database. Let's now explore how to add complex data to Firebase Realtime Database. Sometimes you need to add more detailed or structured data to the database. Such as user object with multiple fields. As a first step, you need to create an object. You can create a data class to represent your complex data, for example, as shown in the snippet. Then we need to add object data. To add this object to Firebase, you can use setValue method. This will create a new node for user2 with fields for name and email. Firebase automatically converts the object to JSON format for storage in the database. Next, let's learn how to add data with auto-generated keys. Auto-generated keys are useful when you need to store multiple entries like chat messages or comments, and you don't want manually specifying unique key for each entry. As a first step, you need to generate a unique key for each new entry using the push method. Once you have the key, you can add data to it. To add data, we can use setValue method. Firebase generates a unique key for each message and stores it in messages node. This method is commonly used in

chat apps and other scenarios where you need to store multiple items with unique identifiers. To summarize, we explored how to set up Firebase real-time database in Android app by adding dependencies and syncing the project with Gradle. You learned how to initialize the database and add simple data like username using `setValue`. We could also add complex, that is, objects using the same `setValue` method. Finally, we covered how to add data with auto-generated keys, which is useful for storing multiple entries like messages or comments. By understanding these techniques, you can effectively manage data in Firebase real-time database from your Android app. Thank you for your attention. I hope you now have a clear understanding of how to add data to Firebase real-time database through an Android app. Thank you.

Introduction to Firebase Authentication Video• . Duration: 4 minutes

[Notes]

Hello everyone, welcome to this module on Firebase integration. In this session, we'll explore the importance of Firebase authentication, which simplifies the process of adding secure login and identity management to your Android apps. Firebase authentication supports various methods such as email password, Google sign-in, Facebook login and even anonymous authentication. By the end of this session, you will understand how Firebase authentication works, its key features and how to set it up in an Android app. Let's begin by discussing what Firebase authentication is. Firebase authentication is a service that helps developers easily manage user authentication for their mobile and web apps. It provides a range of features such as easy integration. Firebase authentication is easy to set up and integrate into your Android app, making it a go-to choice for developers. It provides secure authentication. It provides secure methods for signing in users and protects user data with encryption. And it also provides cross-platform support. That is, Firebase authentication works across Android, iOS and web platforms, allowing for a consistent login experience across all devices. With Firebase authentication, you can enable your users to sign in quickly and securely using methods they are familiar with, such as Google, Facebook or just email. Now, let's explore the different Firebase authentication methods that are available. First one is the email password authentication. This is the traditional method where users sign up with their email and password. Then we have social logins. Firebase supports several social login providers, including Google Sign In, Facebook Login, Twitter and even GitHub. Then there is phone number authentication as well. Users can log in using their phone number, which is validated via an SMS code. Then it also allows anonymous authentication. Firebase also supports anonymous authentication where users can use the app without creating an account. This is useful for apps where users might want to try out features before committing to registration. These different methods give you flexibility in offering multiple login options based on your app's needs. Let us now look at the steps for setting up Firebase authentication in your Android app. First, open the Firebase console and either create a new project or select an existing project. As step 2, in the Firebase console, register your Android app by entering the package name. In step 3, we have to download the `google-services.json` file and we have to add it to the app directory in Android Studio. As the fourth step, in the root level `build.gradle` file, add Firebase plugin and in app level `build.gradle` file, add the following dependencies, which is Firebase then. In the fifth step, go to the Firebase console, enable the desired authentication methods such as email, Google or Facebook login. Once these steps are completed, Firebase authentication will be integrated into your Android project and you can start

implementing login methods. To summarize, Firebase authentication simplifies the process of adding secure login systems to your Android apps. It supports various authentication methods including email password, Google sign-in, Facebook login, phone number authentication and even anonymous login. And setting up Firebase authentication in an Android app involves configuring Firebase console, adding dependencies and enabling authentication methods. Firebase ensures that user data is securely managed, providing developers with a reliable platform for user authentication. By integrating Firebase authentication into your Android app, you can offer users a seamless and secure login experience. Thank you for your attention. I hope you now have a clear understanding of Firebase authentication and how to set it up in Android apps. Thank you.

Implementing User Authentication Video• . Duration: 4 minutes

[Notes]

Hello everyone. Welcome to the module on Firebase integration. In this session, we will learn how to implement user authentication in mobile applications using Firebase. Authentication is the process of verifying a user's identity before granting access to an app. Firebase authentication simplifies this process by supporting various login methods including email and password, Google sign-in, phone authentication and more. By the end of this session, you will understand the basics of user authentication and how to implement it using Firebase. Let's introduce Firebase authentication. Firebase authentication simplifies the process of adding secure user login to your Android apps. It supports different methods such as email password login, Google sign-in, phone authentication and more. Firebase handles authentication logic securely, allowing developers to focus on building other parts of their app. This service also integrates with other Firebase features, making it an efficient solution for managing user identity and security across multiple platforms. So what is user authentication? User authentication is the process of verifying a user's identity before granting them access to an application. It ensures that only authorized users can interact with the app's features, protecting sensitive data and resources. This is important for both security and user management. With Firebase, implementing user authentication becomes simple, secure and flexible as it offers different authentication methods tailored to different use cases. Now let's see as to how email and password authentication is done. This is one of the most common methods of user authentication. Users create an account with their email address and a password, which is then stored securely in Firebase. When a user logs in, Firebase verifies their credentials and provides access to the app. To implement this, you first need to enable email password authentication method in the Firebase console. Then you can use Firebase SDK to register new users, log in existing users and manage password resets. This method is simple to implement and offers strong security features for app developers. Let's now look at how to implement email and password authentication in Android. First, we need to integrate Firebase authentication into our Android app. After Firebase setup and adding Fire dependencies, as a third step, you need to do sign-up code. Here is an example of how to implement user sign-up with email and password. We have to use the method create user with email and password. Next is to log in. Similarly, for user login, we have to use the method sign in with email and password, where the parameters will be email and password. This process enables users to securely register and log in into your app using email and password. Firebase authentication simplifies user login with various methods like email, password, Google and phone book authentication. User authentication is crucial to secure app resources and verify user identity.

Email and password authentication is one of the most popular methods ensuring security while being easy to implement. Implementing Firebase authentication in Android is straightforward, requiring you to configure Firebase console, add dependencies and use simple code to handle registration and login. By using Firebase authentication, you can enhance the security and usability of your Android apps. Thank you for your attention. I hope you now have a clear understanding of Firebase authentication and how to implement email password authentication in Android apps. Thank you.

Push Notifications with Firebase Video• . Duration: 6 minutes

[Notes]

Hello everyone. Welcome to the module on Firebase integration. In this session, we will explore how to implement push notifications in your mobile applications using Firebase Cloud Messaging. Push notifications are a powerful way to engage users by delivering timely information directly to their mobile devices. By the end of this session, you will understand how Firebase push notifications work, the types of messages supported in FCM, and why FCM is a great choice for sending notifications. Let's begin by discussing what Firebase Cloud Messaging is. Firebase Cloud Messaging is a cross-platform messaging service provided by Google that allows developers to send notifications and data messages to their mobile apps. FCM supports Android, iOS, and web platforms, ensuring a seamless experience for app users. Using FCM, you can send two types of messages. 1. Notification messages. These are simple push notifications that appear in the notification tray of the device. Then it can also send data messages. These are more complex and can contain custom key-value pairs for advanced interactions. Firebase Cloud Messaging allows you to engage users with timely and relevant notifications, improving user retention and app engagement. Now let's see why we need to use FCM for push notifications. There are several benefits of using Firebase Cloud Messaging. First one is a cross-platform support. FCM works on Android, iOS, and web apps, allowing you to reach users on multiple platforms with a single solution. Then it provides real-time messaging as well. FCM delivers messages in real-time, which is crucial for applications that need to send timely alerts such as breaking news or chat notifications. It also is supported by a scalable infrastructure. Firebase is built on Google Cloud, making it highly scalable. Whether you are sending messages to a few users or millions of users, FCM handles the load effectively, and it is free to use. Firebase Cloud Messaging is free, which makes it an excellent choice for developers who want to implement push notifications without worrying about costs. With these advantages, FCM becomes a valuable tool for sending notifications to your users. Let's now explore how Firebase push notifications work. Here is a basic process. First step is to register the app. When a user installs your app, the app registers with FCM to receive messages. FCM generates a unique token for each device. Second step is to send a message. You can send notifications from Firebase console or programmatically using the FCM API. The message is sent to FCM servers, which route it to the targeted devices. Third step is notification delivery. Once the message reaches the FCM server, it is delivered to the device where it appears in the notification tree. The user can tap the notification to open the app or perform other actions. This flow ensures that messages are sent quickly and reliably to user's device, keeping them engaged with your app. Firebase Cloud Messaging supports three types of messages. 1. Notification Messages These are messages that are simple and contain basic elements such as a small icon, the icon that appears in

the notification bar, the app name, i.e. name of your application, time stamp, when the notification was received, title, a short title for the message, text, which is the body of the notification message, and a large icon, an optional image to display. Then it also supports data messages. These messages contain custom key-value pairs that you can use to deliver more complex data to your app. Data messages are not displayed directly in the notification tree, but can trigger background processing in the app. 2. Mixed Messages These are a combination of notification and data messages. They contain both notification elements and additional data, offering more flexibility for handling notifications. Understanding the different types of messages helps you choose the best option based on your app's requirements. To summarize, Firebase Cloud Messaging is a powerful tool for sending push notifications and data messages to your app users. It supports two main types of messages, i.e. notification messages and data messages, with an option to combine both as mixed messages. It is highly scalable, supports multiple platforms, and provides real-time delivery of notifications. Understanding the different types of messages and how Firebase push notifications work enables you to engage users with timely and relevant information. By implementing push notifications, you can keep your users engaged and informed, improving app retention and overall user experience. Thank you for your attention. I hope now you have a clear understanding of how to implement push notifications with Firebase Cloud Messaging. Thank you.

Introduction to Firebase Cloud Messaging Video• . Duration: 3 minutes

[Notes]

Hello everyone, welcome to the module on Firebase integration. In this session, we'll explore how to use Firebase Cloud Messaging to send push notifications in mobile applications. Firebase Cloud Messaging is a cross-platform solution that allows developers to send messages and notifications to their users on Android, iOS, and web platforms. By the end of this session, you will understand the key features of FCM, how it works, and how to set up FCM to send notifications in Android apps. Some of the key features of Firebase Cloud Messaging are cross-platform support, that is, FCM works seamlessly across Android, iOS, and web apps, allowing developers to reach a wider audience with a single platform. It also supports real-time notifications, that is, FCM delivers notifications in real-time, making it ideal for applications that need to send timely updates, such as news apps or messaging platforms. Then, it also has targeted messaging. FCM allows you to send notifications to specific users or a group of users, based on criteria such as user preferences or geographic location. It also supports multiple message types. So, it supports both notification and data messages and also mixed messages, giving developers flexibility in how messages are delivered and handled by the app. Now, let's have a look at how FCM works. The FCM workflow involves three main components. First, we have the FCM server. The FCM server is responsible for sending messages via the FCM backend. It can process requests from your app server or directly from Firebase console. Then, the second component is app server. Your app server can send custom messages or data directly to FCM. This is useful when you want to send notifications based on specific events within your app. Then, the third one is the FCM SDK on the device, which receives messages and displays them as notifications or triggers custom actions within the app. The overall flow starts with FCM server receiving a message request from the developer server or Firebase console. FCM processes the request and delivers the message to the targeted client devices. The FCM SDK on the client side then handles the notification in the device. To summarize

this session, Firebase Cloud Messaging is a versatile tool for sending push notifications to Android, iOS and web users. It supports two main types of messages, notification messages and data messages. The FCM workflow involves FCM server, the optional app server and the FCM SDK on the client side. By understanding and implementing FCM, you can keep users engaged and informed through timely notifications. Thank you for your attention. I hope you now have a better understanding of how to use Firebase Cloud Messaging to send and receive push notifications in Android apps. Thank you.

Sending and Receiving Messages Video• . Duration: 4 minutes

[Notes]

Hello everyone, welcome to this module on Firebase integration. In this session, we will explore how to implement Firebase Cloud Messaging in Android for sending and receiving messages. FCM is a powerful tool that enables developers to send notifications or messages to users' devices, keeping them engaged with the app. We will cover the types of messages supported by FCM, how to set up FCM in Android, and how to receive messages in your app. By the end of this session, you will have a complete understanding of how to use FCM in your Android projects. Let's first understand the concept of sending and receiving messages with FCM. Firebase Cloud Messaging is a cross-platform service that allows developers to send notifications or messages to devices running on Android or iOS or even web applications. And it supports two different types of messages, the notification messages and the data messages. FCM makes it easy to engage users by sending timely notifications and messages directly to their devices. Next, let's look at the steps to set up Firebase Cloud Messaging in your Android project. As the first step, we need to add Firebase to your Android project. So, you need to go to Firebase console, create a project and register your Android app. Then, you need to download GoogleServices.json file and place it in the app directory of your Android project. We need to open the build.gradle file and add the Firebase messaging dependency as shown. Then finally, after adding the dependency, we need to sync the project and apply the changes. These simple steps will allow you to integrate FCM into your Android app. Now, let's learn how to receive messages in an Android app. To handle incoming messages from FCM, you need to extend `FirebaseMessagingService` class in your app. In this code, `onMessageReceived` handles both notification and data messages, while `onNewToken` method manages FCM tokens for device registration. Let's now explore how to send notifications from Firebase console. You can easily send a push notification to users directly from Firebase console by following these steps. First, you need to go to Cloud Messaging. In the Firebase console, navigate to Cloud Messaging section. Then, click on Send your first message to create a new notification. Then, choose the target audience for the notification. You can send it to all users, specific users, or users with specific conditions like location or app version. Then, you need to enter notification title and body. You can also add an image or action buttons if needed. It also has advanced options where you can set additional parameters such as message priority, time to live, and custom data. Once the message is composed, click Send to deliver the notification. This allows you to easily send push notifications to your app users without needing any code. Firebase Cloud Messaging is a versatile tool for sending and receiving messages in mobile apps, supporting Android, iOS, even web applications. FCM supports notification messages, which are handled by the system, and data messages, which are handled by the app itself. Setting up FCM in Android app involves adding Firebase dependencies

and syncing the project with Gradle. To receive messages in Android, you need to extend the Firebase Messaging service class and handle incoming notifications and data messages. You can also send push notifications from Firebase console, allowing you to target users based on various criteria and send personalized messages. By implementing FCM in your Android projects, you can enhance user engagement and improve app retention. Thank you for your attention. I hope now you have a better understanding of how to implement Firebase Cloud Messaging for sending and receiving messages in Android apps. Thank you.

Image Selection from Gallery Video• . Duration: 4 minutes

[Notes]

Hello everyone, welcome to the module on Firebase integration. And in this session, we will learn how to allow users to select images from their device gallery in an Android app. Image selection is a common feature in mobile apps, especially in applications like social media, messaging, or photo editing. By the end of this session, you will understand the steps needed to implement image selection in Android. The permissions required and how to write code in Kotlin to select an image from the gallery. Let's see as to what image selection is. Image selection allows users to choose images from their device's gallery and use them in the app. For example, in social media apps, user can select images from their gallery to upload as profile pictures, or in messaging apps, they can send images as attachments. Implementing image selection improves the user experience by enabling users to interact with your app using their own images, making the app more personal and dynamic. Let's discuss why it is important to implement image selection. There are several reasons to include this feature in your app. First one is the user engagement. Allowing users to select images from their gallery makes the app more engaging, especially in apps like photo editing, messaging, or social media. Then you can have custom content. Users can personalize their experience by choosing images from their gallery, making the app feel more tailored to their needs. Then you have enhanced functionality. Apps that deal with images such as photo sharing or e-commerce platforms benefit significantly from image selection functionality. Implementing this feature is crucial for providing a seamless user experience in apps that involve user-generated content. Now, let's look at permissions required for image selection. In Android, accessing the gallery requires specific permissions to ensure user privacy and security. We have to add permission in androidmanifest.xml file. To access external storage and the gallery, you need to declare read external storage permission in the androidmanifest.xml file, as shown in the snapshot. Runtime permissions for Android 6.0, that is API 23 and above. Starting from Android 6.0, which is Marshmallow, you also need to request runtime permissions. You can check if the permission is granted or not. Request it from the user at the runtime. Ensuring the correct permissions are in place is essential to successfully access the gallery and selecting images. Now, let's move on to writing the Kotlin code to select an image from the gallery. Here are the key steps. First one is to open the gallery and allow users to select an image. You can use an intent with intent.action underscore pic and the type will be image and you need to use startActivityForResult method. The next step is to handle the selected image. Once the user selects an image, you need to handle the result in onActivityResult method. The code enables the app to open the gallery, allow the user to choose an image, and handle the image once selected. Image selection from gallery is a useful feature that allows users to interact with app by choosing images from their device. The permissions

required for accessing the gallery include `read_externally_stored_media_metadata`, and for Android 6.0 and above, you need to have runtime permissions as well. The code in Kotlin to open the gallery involves using intent and handling the selected image in `onActivityResult` method. By implementing these steps, you can successfully allow users to select images from their gallery, enhancing the functionality of your app. Thank you for your attention. I hope now you have a good understanding of how to implement image selection from the gallery. Thank you.

Other Operations on Image in Cloud Storage with Firebase Video• .

Duration: 5 minutes

[Notes]

Hello everyone, welcome to the module on Firebase integration. In this session, we'll explore how to perform different operations on images stored in Firebase Cloud Storage. Firebase Cloud Storage is a powerful tool that allows developers to store and manage media files such as images, audio, and video in the cloud. In this session, we'll focus on how to upload, download, and delete images in Firebase Cloud Storage. By the end of this session, you will have a good understanding of how to manage image files efficiently in your Android apps using Firebase. So, what is Firebase Cloud Storage? Firebase Cloud Storage is a Google service that provides a powerful, secure, and scalable solution for storing user-generated content such as photos, videos, and other media files. It integrates seamlessly with Firebase authentication, allowing developers to store and manage files with minimal effort. Using Firebase Cloud Storage, you can easily upload images from your app, store them in the cloud, and retrieve them when needed. It provides a secure way to handle large files without worrying about storage limits or performance issues. Let's now see why Firebase Cloud Storage is ideal for storing images. There are several benefits using Firebase Cloud Storage for handling images in mobile apps. First and foremost is scalability. Firebase Cloud Storage scales automatically, so whether you are handling a few images or millions of images, Firebase can manage the load seamlessly. The next one is security. It integrates with Firebase authentication, ensuring that only authenticated users can upload or access files. And it also supports cross-platforms. Firebase Cloud Storage works across Android, iOS, and web platforms, allowing users to access their stored files from any device. By using Firebase Cloud Storage, developers can easily manage images in a secure, scalable, and reliable way. Let's now look at the key operations you can perform on images stored in Firebase Cloud Storage. First one is that you can upload images from user's device to Firebase Cloud Storage, ensuring they are stored safely in the cloud. Then, once uploaded, images can be retrieved as well from Firebase Cloud Storage and displayed in the app. You can even delete images from the cloud storage, which are no longer needed, freeing up storage space. Additionally, you can perform operations like resizing, compressing images before upload, etc. Let's now go through the steps to upload an image to Firebase Cloud Storage. Here is how you can do it. First, you need to get reference to the Firebase Storage. Then, you need to get the image URI from the device. Then upload it to the Firebase using `putFile` method. This code allows you to upload the selected image to Firebase Cloud Storage, where it is stored securely. Let's now learn how to download an image from Firebase Cloud Storage. You can retrieve the image stored in Firebase and display it in your app using the following steps. First one is to get the storage reference. The next step is to use `getDownloadUrl` method to retrieve the image URL. This allows you to download and display the image in your app whenever needed. Finally, let's see

how to delete an image from Firebase Cloud Storage. Deleting an image is straightforward and helps free up storage when an image is no longer needed. First step is to get reference to the image. Second one is to use delete method to remove the image. This code allows you to delete any unwanted images from Firebase Cloud Storage. To summarize, Firebase Cloud Storage allows you to store, manage, access images from your app securely and reliably. The key operations for managing images include uploading, downloading, and deleting images. You can perform image resizing and compression before upload to optimize storage usage. Firebase Cloud Storage provides seamless cross-platform support, scalability, and security making it a reliable solution for image management in your app. Thank you for your attention. I hope now you have a good understanding of how to perform image operations in Firebase Cloud Storage. Thank you.

Introduction to Performance Optimisation Video• . Duration: 7 minutes

[Notes]

Welcome to this module on Advanced Topics in Android Development. I am Professor Chhathasekar R.K. and in this session, we will focus on Performance Optimization in Android Development. Performance Optimization is critical for ensuring that your app runs efficiently, loads quickly and provides a smooth user experience. In this session, we will explore what Performance Optimization means, why is it important to us and how to achieve it through key metrics and best practices. By the end of this session, you will have a solid understanding of how to make your Android app perform at its best. Performance Optimization refers to the techniques used to improve speed, efficiency and responsiveness of an Android app. An optimized app loads quickly, runs smoothly and conserves resources like memory and battery. Why does it matter? Poorly optimized app leads to slow load times where users may become frustrated and uninstall the app. And it can also result in laggy and unresponsive UI. This can lead to a negative user experience. Along with this, we could even have high memory usage where it can cause crashes or slow performance on lower end devices. Or it could also result in excessive battery consumption. This can lead to users avoiding your app, especially on mobile devices where battery life is crucial. To effectively optimize an Android app, it's important to measure performance using certain key metrics. Let's go over some of the most important ones. First one is the CPU usage. This measures how much processing power your app consumes. High CPU usage can slow down the app and drain the device battery. Next one is frames per second. This measures the smoothness of your app's animations. A low FPS results in stuttering, causing a janky and unpleasant user experience. Then the memory management. Ensuring efficient memory use avoids memory leaks and prevents the app from consuming excessive memory, which can lead to slowdowns and crashes. Monitoring these metrics helps developers identify performance bottlenecks and areas that need improvement. Continuing with key performance metrics, we have network performance. Minimizing network requests and optimizing data transfer reduces the time users spend waiting for content to load. It also helps reduce data usage, which is critical for users with limited data plans. Uptime and latency. Uptime measures how responsive your app is and latency measures how fast your app loads content. Keeping load times low is key to maintaining a smooth and responsive user experience. By monitoring these key metrics, you can ensure that your app performs well across different devices and networks. Now let's discuss some best practices for optimizing your app's performance. First one is lazy initialization. Delay the creation of objects until they are actually needed. This helps

reduce memory usage and improves app load times. Then efficient layouts. Use lightweight UI components like RecyclerView to handle large datasets efficiently. Avoid complex and deeply nested layouts as they can slow down the rendering process. Continuing with more best practices. Asynchronous operations. Use background threads or coroutines to handle time-consuming tasks like network requests or database operations. This ensures that your app's UI remains responsive even when heavy operations are happening in the background. Then coming to the memory leak prevention, one of the most common causes of performance issue is memory leaks. To avoid this, use weak references wherever necessary to prevent objects from staying in memory longer than needed. Then optimize battery usage. Avoid unnecessary background processes and reduce the frequency of network requests. Optimizing battery usage ensures that your app runs smoothly without draining the user's device battery. By following these additional practices, you can avoid common performance pitfalls and ensure that your app delivers a better user experience. To properly optimize your app, you need to use profiling tools that help measure performance in real-time. Let's look at some of the essential tools available in Android Studio. First one, we have Android Profiler. This tool monitors your app's real-time memory, CPU, and network performance. It helps identify performance issues during development. Then we have the CPU Profiler. This allows you to analyze tasks that consume too much processing power and optimize them accordingly. Then we have Memory Profiler. This tool detects memory leaks and helps monitor memory allocation to ensure efficient memory usage. Then we have Battery Profiler. This tool identifies operations that consume too much energy, helping you optimize battery usage in your app. Using these tools, you can get detailed insights into your app's performance and make the necessary adjustments. To summarize the key points from this session, we looked into the performance optimization, which involves improving the speed, efficiency, and responsiveness of an Android app. We also looked into key metrics such as CPU usage, FPS, memory management, and network performance, which are crucial for measuring app performance. We also looked into some of the best practices like lazy initialization, using efficient layouts, asynchronous operations, and memory leak prevention, which help in optimized performance. We also looked into certain profiling tools like Android Profiler, CPU Profiler, and Memory Profiler. By applying these techniques and using these tools, you can significantly improve the performance of your Android apps. Thank you for your attention throughout this session. I hope this session has given you a good understanding of performance optimization in Android development. Thank you.

Profiling Tools and Techniques Video• . Duration: 9 minutes

[Notes]

Hello everyone, welcome to this module on advanced topics in Android development. In this session, we will focus on profiling tools and techniques. Profiling tools are essential in Android development because they help developers measure the performance of an app and identify any issues that may slow it down, such as high memory leak, CPU bottleneck, or network delays. In this session, we will discuss various profiling tools, their importance, and how to use them to optimize your app's performance. By the end of this session, you will be able to confidently use these tools to improve your app's speed and efficiency. Profiling tools help developers measure and identify performance issues in Android apps, such as excessive memory usage, CPU bottlenecks, or network delays. By using profiling tools, developers can make data-driven decisions about how to

optimize their app for better performance. Without proper profiling, it is hard to know what parts of your app are causing performance slowdowns or draining too much battery. Profiling is an essential step to ensure that your app runs smoothly, especially on lower-end devices. One of the most important tools in Android development is the Android Profiler. This tool allows you to track memory usage, CPU load, network activity, and battery consumption in real-time while your app is running. The Android Profiler offers Memory Profiler to detect memory leaks and track how much memory is used during runtime, CPU Profiler to identify heavy tasks or methods that slow down the app's execution, then Network Profiler to monitor network requests and make sure data is being transferred efficiently. To access the Android Profiler in Android Studio, we have to follow the following steps. First we have to go to View, Tool Windows, Profiler, and run your app in Debug mode. The Profiler tab will display real-time performance metrics showing where you can improve. This is a screenshot of Android Studio Profiler tool, which is used to monitor app performance in real-time. The Profiler displays key performance metrics like CPU usage, memory usage, and more for selected activity, in this case, main activity. Some of the key information from the screenshot are,

1. We have the CPU usage. The CPU usage shows 0% at the current time marker, suggesting that there is little to no CPU activity at this point.
2. It shows the memory usage. The memory usage is approximately 53.9 MB. The blue section represents the memory consumption and it shows that app's memory usage has been relatively stable at around 53.9 MB.
3. Profiling as debuggable. A message indicates that profiling is being done in a debuggable mode, which is typical during development. However, it warns that this does not represent production performance. For accurate performance measurements in production, you should enable profilable mode.
4. Optimizing CPU and Memory Usage. If CPU usage spikes during certain operations, consider optimizing those processes. If memory usage increases over time, check for memory leaks.
5. Profile in Production Mode. For more accurate representation of app performance, consider enabling the profilable flag in your release builds.

This snapshot is useful for evaluating how efficiently your app is using system resources during development and debugging. Now let's look at some of the common profiling techniques that help you dig deeper into performance issues. First one is the method tracing. This technique measures the execution time of each method in your app, helping you identify slow methods that need optimization. Method tracing provides insight into which parts of app are consuming the most CPU time. Next one is the heap dumps. A heap dump captures a snapshot of your app's memory usage at a specific time. This helps you analyze memory leaks and track object creation so you can optimize memory management and prevent excessive memory consumption. Then we can even trace network. Network tracing tracks the number of network requests made by your app and ensures that data transfer is efficient. By reducing unnecessary network calls, you can speed up your app's performance and reduce data usage. This image is a screenshot from Android Studio's memory profiler, specifically showing options to capture a heap dump or record Java or Kotlin allocations. The profiling session is focused on memory usage in the main activity class. Certain key information from the screenshot are memory usage breakdown. The memory chart displays different memory categories such as Java, Native, Graphics, Stack, Code, and others. The current memory usage is hovering just under 48 MB. The majority of memory appears to be used by Java and Stack as shown by the green and blue sections of the graph. Options for memory profiling. Capture heap dump. This option allows you to capture a heap dump, which is useful for analyzing which objects are using memory at a specific point in time. This helps in detecting memory leaks and identifying large memory allocations. It also records Kotlin allocations. This option enables you to record Kotlin objects are being allocated over time, which is useful for understanding the memory

allocation patterns. Then there is also a record button. The record button at the bottom is used to start recording memory usage. You can choose between heap dump or allocation tracking based on what you want to analyze. This profiling tool will help you optimize your app's memory usage and ensure better performance, particularly when dealing with resource-intensive activities. Now let's talk about some best practices you should follow when profiling your Android app. First one, profiling regularly. That is, perform profiling regularly throughout the development cycle. This will help you catch performance bottlenecks early and address them before they affect the user experience. Then we need to optimize it gradually. When profiling, tackle one issue at a time. It's better to focus on fixing one problem thoroughly, such as reducing memory leaks or CPU usage, before moving on to the next issue. Testing on real devices. Always test your app's performance on real devices, not just emulators. Real devices give you accurate performance data in real-world scenarios, and testing across different devices helps ensure that your app performs well for all users. To summarize, we discussed several profiling tools and techniques that can help you improve your app's performance. So we saw Android Profiler, which tracks memory, CPU, and network performance in real-time. We also saw Method Tracing, which is used to identify slow methods that are consuming too much CPU, Heap Dumps, which capture memory usage to detect memory leaks, Network Tracing, which tracks network requests and helps reduce unnecessary data transfer. By using these tools and following the best practices, you can optimize your app and ensure a better user experience. Thank you for your attention throughout this session. I hope now you have a good understanding of how to use profiling tools and techniques to optimize the performance of your Android apps.

Introduction to Background Processing Video• . Duration: 6 minutes

[Notes]

Hello everyone, welcome to this module on advanced topics in Android development. In this session, we will focus on background processing. Background processing is an essential concept in Android development that allows apps to perform tasks that don't block the main thread. In this session, we will discuss what background processing is, the types of tasks handled in the background, and the best practices for implementing it efficiently. By the end of this session, you will have a solid understanding of how to manage background tasks effectively in your Android applications. Now let's define background processing. Background processing refers to performing tasks in the background without blocking the main thread, which is responsible for handling the app's user interface. For example, tasks like downloading data or syncing files with a server are typically run in the background. Why is it important? The key reason for using background processing is to keep the app's UI responsive. If tasks that take longer than a few seconds run on the main thread, they will cause the app to freeze, leading to poor user experience and potentially causing app not responding errors. Background processing ensures that long-running tasks can be completed without freezing the app's interface. Tasks such as downloading files, fetching large files or data from a server should happen in the background to avoid blocking the main UI. Syncing data Syncing user data or app settings with a remote server also happens in the background, keeping the app functional for the user. By running these tasks in the background, the app remains responsive and users can continue interacting with it smoothly. Let's look at some of the examples of background tasks that should be handled away from the main thread, fetching data from a remote server. For example, loading new content in a social media feed or fetching data from a weather

service. Uploading files or syncing data The app could be uploading a photo to a cloud storage service or syncing app settings across devices. Processing images or performing large computations Any task that requires significant processing power, like image editing or complex calculations, should be handled in the background to prevent the app from freezing. These tasks take time and should be managed carefully to maintain a smooth user experience. Why is background processing necessary? If you run tasks that take more than a few seconds on the main UI thread, the app becomes unresponsive, leading to a poor user experience. This can result in ANRs, which are critical errors that occur when an app does not respond to user actions within a certain time. Background processing is required to ensure that the app remains interactive while these long-running operations are handled efficiently. Now, let's explore some best practices for background processing in Android. First, avoid UI blockage Always ensure that long-running tasks like downloading files or database queries are done off the main thread. This prevents the app from freezing and triggering ANR errors. And to use correct APIs It's important to use right API for the task. For example, use WorkManager or IntentService for long-running background operations that need to be scheduled or executed sequentially. Handling errors gracefully Always make sure to handle errors in background tasks properly. If a download fails, for instance, notify the user and retry the task instead of crashing the app. And for tasks that need to run even when the app is not in the foreground, consider using foreground services or job scheduler to handle them efficiently. And to conserve battery life, avoid frequent network requests and unnecessary tasks that consume too much power. Users expect apps to be battery efficient, so ensure that your background processing does not drain the battery unnecessarily. And limit the number of background tasks so that it can run simultaneously, which can slow down the device. Schedule tasks appropriately and use batching techniques to optimize performance. To summarize this session, background processing allows tasks like downloading files, syncing data and performing computations to be handled without blocking the main thread. It ensures that the UI remains responsive, enhancing the user experience and preventing ANRs. Examples of background tasks include fetching data from a server, uploading files and processing images. And best practices include avoiding UI blockage, using correct APIs, handling errors and conserving battery life. By following these guidelines, you can ensure that your app runs smoothly while managing background tasks efficiently. Thank you for your attention. I hope now you have a clear understanding of background processing in Android and how to implement it effectively. Thank you.

Using Work Manager Video• . Duration: 5 minutes

[Notes]

Hello, everyone. Welcome to this module on advanced topics in Android development. In this session, we'll be focusing on using WorkManager. WorkManager is an API in Android that handles background tasks, ensuring they're executed reliably, even if the app is closed or the device is restarted. In this session, we will discuss what WorkManager is, why is it important, and how to schedule tasks using it. By the end of this session, you will have an understanding of how to use WorkManager effectively for managing background tasks in Android. WorkManager is an API in Android that allows you to handle background tasks, ensuring they're reliably executed, even if your app is closed or the device restarts, WorkManager ensures that the task is completed. Why should we use WorkManager? We should use WorkManager because it ensures that tasks are completed,

even under specific conditions like low battery or limited network. You can even set tasks to run only under certain conditions, like when the device is charging or connected to Wi-Fi. Some of the common use cases of WorkManager for background tasks, such as syncing data with the server, uploading files, or performing regular backups are handled seamlessly. Let's now discuss the benefits of using WorkManager. First is guaranteed execution. One of the key advantages of WorkManager is its ability to guarantee that tasks will run regardless of app or device state, even if the app is killed or the phone restarts, WorkManager persists the task until it is completed. Then we have flexible scheduling. WorkManager provides flexible scheduling options. Tasks can be scheduled for one-time execution or on recurring basis. Additionally, you can specify conditions for when tasks should run, such as only when the device is charging or connected to Wi-Fi. WorkManager also ensures that tasks are executed without draining the battery unnecessarily. Now let's discuss task scheduling in WorkManager. There are different types of tasks you can schedule. One-time tasks. These are the tasks that are executed only once. For example, syncing data once when the user opens the app or periodic tasks. These are the tasks that are repeated at regular intervals, such as syncing data with a server every day or backing up data weekly. Some tasks may be immediate, requiring urgent execution, while others can be long running but still need to be executed reliably. The diagram usually shows how WorkManager manages one-time and periodic tasks, including immediate, long running, and expedited tasks. WorkManager handles both foreground work, which requires user awareness, and background tasks that can run without user interaction. One of the powerful features of WorkManager is task chaining. You can chain multiple tasks together so that one runs after another based on the success or failure. For example, you can schedule a sequence of tasks such as compressing a file, uploading it to a server, and then notifying the user. This ensures that tasks are executed in the correct order. Best practices for using WorkManager include using expedited work for urgent tasks, such as those that need to be completed under strict time constraints. Using foreground work for long running tasks that require user awareness, like downloading large files. Scheduling tasks only when specific conditions are met, such as running tasks only on Wi-Fi to save data. These practices ensure that your background tasks are handled efficiently without negatively impacting user experience. Task chaining with WorkManager is powerful to ensure tasks are completed in the right order. For instance, you can schedule tasks that depend on each other, such as processing data before uploading it. You can also set up tasks so that if one task fails, the next task does not run, ensuring reliability. This way, tasks are performed in a structured sequence, improving both the efficiency and accuracy of background processes. To summarize this session, WorkManager is an Android API for managing background tasks reliably, even if app is closed or device restarts. And WorkManager provides flexible scheduling options for one-time or periodic tasks. You can even chain multiple tasks together to ensure they run in a sequence based on success or failure. Then we looked into best practices, which included using expedited or foreground work for urgent tasks and scheduling tasks only under certain conditions like when the device is on Wi-Fi. By following these practices, you can efficiently manage background tasks using WorkManager, ensuring that your app performs well and remains responsive. Thank you for your attention. I hope you now have a clear understanding of using WorkManager for managing background tasks in Android. Thank you.

Async Task in Android Video• . Duration: 6 minutes

[Notes]

Hello, everyone. Welcome to this module on advanced topics in Android development. In this session, we will focus on AsyncTask in Android. AsyncTask is an API that allows you to run background operations and update the UI without blocking the main thread. It is useful for handling tasks like fetching data from a remote API or processing large files in the background. In this session, we will explore what AsyncTask is, how it works, and its use cases in Android development. By the end of this session, you will have a good understanding of how to use AsyncTask to improve your app's performance and responsiveness. Let's start with what AsyncTask is. AsyncTask is an Android API that allows you to perform background tasks and update the user interface without freezing the main thread. This is especially important because running long tasks on main thread can cause app to become unresponsive, leading to poor user experience. For example, AsyncTask is ideal for tasks like downloading data from an API, loading large files such as images, processing data-heavy operations like parsing or calculations. AsyncTask helps keep the app responsive while these time-consuming tasks are handled in the background. Now let's discuss why AsyncTask is important. Running long tasks on the main thread causes app's UI to freeze and users may feel that the app has crashed. AsyncTask allow background tasks to be performed without blocking the main thread, keeping the UI responsive. This improves user experience and ensures that app runs smoothly, even when performing tasks like fetching data from a remote server or processing large files. By using AsyncTask, you ensure that your app remains interactive, which is a key factor for user satisfaction. Now let us understand as to how AsyncTask works. It follows a simple lifecycle with three main methods. First one is `onPreExecute`. This method is called before the background task starts. It is typically used to set up the task and show a loading indicator such as a progress bar. Second method is `doInBackground`. This is the most important method where the actual background task is performed. Time-consuming operations like network requests or data processing happen here. `onPostExecute`, after the background task is finished, this method is called. You can use it to update the UI, such as displaying the fetched data or hiding the progress bar. These three methods allow you to manage background tasks and ensure that UI interface is updated once the task is completed. When using AsyncTask, it is important to follow certain best practices to avoid issues. First one is it is used for short tasks only. Use AsyncTask for tasks that last a few seconds. For longer tasks, it is better to use `WorkManager` or other background processing tools. AsyncTask is not suitable for long-running background operations. Second one is cancel tasks. Always ensure that task is canceled if the activity or fragment is destroyed. This helps avoid memory leaks, which can cause the app to slow down or crash. Third one is to handle errors gracefully. If something goes wrong during the background task, such as network failure, make sure to handle the error properly to prevent the app from crashing. Following these best practices will help you use AsyncTask effectively without negatively impacting your app's performance. Let's now look at a simple example of AsyncTask in action. In this demonstration, we will fetch a JSON object from a report API and display the data in text view. Now let's see what happens by using AsyncTask methods. On pre-execute method, the progress bar becomes visible, indicating that a background task has started. Second one is doing background. The JSON data is fetched from API URL. This is the core part of the background operation where data is downloaded. And the third one is on post execute. Once the data has been fetched, it is displayed in the UI and the progress bar is hidden. This is how AsyncTask allows background work to be done without interrupting the user's experience. To summarize this session, AsyncTask is a useful Android API for running background tasks and keeping the UI responsive. It's ideal for tasks like downloading data from an API, loading large files or performing data-heavy operations. And it has three main methods. That is on

pre-execute, doing background and on post-execute, which handle the tasks lifecycle. And follow best practices like using AsyncTask for short tasks and canceling tasks to avoid memory leaks. By understanding these concepts, you can improve the performance and responsiveness of your Android apps. Thank you for your attention. I hope you now have a clear understanding of how to use AsyncTask for background tasks in Android. Thank you.

Async Task Usage Video• . Duration: 6 minutes

[Notes]

Hello everyone. Welcome to the module on advanced topics in Android development. And in this session we'll focus on AsyncTask usage. AsyncTask is a powerful tool in Android that helps you run background tasks without blocking the main UI thread. It is commonly used for short-lived operations like fetching data from an API, processing images, or performing lightweight calculations. In this session, we will explore when to use AsyncTask, common patterns in AsyncTask usage, and some of the challenges associated with it. By the end of this session, you will have a good understanding of how to effectively use AsyncTask in Android development. Now let's understand when to use AsyncTask. AsyncTask is ideal for short-lived background tasks that need to interact with user interface. These are tasks that typically last only a few seconds and must update the UI once completed. Some common use cases include fetching data from an API if your app needs to load data from server without blocking the UI, processing images, for example, resizing or compressing images in the background before displaying them, or background calculations like parsing large datasets such as JSON or XML while keeping the app responsive. AsyncTask is not suitable for long-running tasks. For longer operations, it is better to use alternatives like WorkManager. Let's now discuss the common pattern when using AsyncTask. The lifecycle of AsyncTask follows the three main methods which are already discussed, that is, onPreExecute, where this method is called before the background task starts. You can use it to set up initial UI components like showing a loading spinner or progress bar. Second main method is doInBackground. This is where the actual background task happens. You can use this method to perform long-running operations such as fetching data from a server or processing files, making sure you keep this method focused on background tasks only. onPostExecute. Once the background task is complete, this method is called. It's used to update the UI with the results of background operations like displaying fetched data or hiding the progress bar. By following this pattern, you ensure that background tasks are handled smoothly without blocking the UI. It is also important to know how to cancel AsyncTasks when necessary. If the user navigates away from the task or activity is no longer needed, you should cancel the AsyncTask to avoid wasting resources. You can use the method isCancelled to check if the task has been cancelled and handled appropriately. This ensures that unnecessary operations are stopped in time, preventing memory leaks or inefficient use of resources. Cancelling tasks is a key part of managing AsyncTasks, especially in dynamic apps where users often move between screens. Now let's discuss the challenges with AsyncTasks. While it is a useful tool, there are some limitations and potential issues you should be aware of. First one is the memory leaks. If AsyncTask continues running after the activity is destroyed, it can cause memory leaks. Always check for changes in the activity state and ensure that tasks are cancelled when no longer needed, and it has limited use cases. AsyncTask is suitable only for tasks lasting a few seconds. For long-running tasks like file downloads or data

synchronization, we should be using WorkManager or other advanced background processing tools, which is appropriate. By understanding these challenges, you can better decide when to use AsyncTask and how to manage it efficiently. One of the main challenges with AsyncTask is that it does not handle activity lifecycle changes well. For example, if the screen is rotated or activity is recreated, AsyncTask might be terminated mid-execution. This can result in incomplete tasks and potential crashes. To handle such lifecycle issues, it is recommended to use lifecycle-aware components like ViewModel or Loader, which manage background tasks better in the face of configuration changes. By being aware of these limitations, you can avoid potential pitfalls and ensure your background tasks are executed properly. To summarize this session, AsyncTask is ideal for short-lived background tasks that interact with the UI, such as downloading small amounts of data or processing lightweight operations. The lifecycle of AsyncTask includes three main methods: `onPreExecute`, `doInBackground`, and `onPostExecute`, allowing you to set up, perform, and finalize background tasks effectively. You can cancel tasks when needed, using `isCancelled` method to prevent unnecessary processing. And we also saw the challenges with AsyncTask, which include memory leaks and activity lifecycle issues, which require more careful management. By following best practices, you can make efficient use of AsyncTask in your Android projects. Thank you for your attention. I hope you now have a clear understanding of how to use AsyncTask effectively for background processing in Android. Thank you.

Loaders Video • Duration: 6 minutes

[Notes]

Hello everyone, welcome to this module on advanced topics in Android development. In this session, we will focus on loaders in Android. Loaders are an important component in Android that allow your app to load data in the background, keeping the user interface responsive. This is especially useful when retrieving data from a database or content provider. In this session, we will cover what loaders are, how they work, and best practices for using them. By the end of this session, you will understand how to implement loaders in your Android projects to ensure smooth and efficient data loading. Let's begin by understanding what loaders are. Loaders are components in Android that allow you to load data asynchronously, which means fetching data in the background without blocking the main UI thread. This ensures that your app's user interface remains responsive even when retrieving large amounts of data. Its use cases could be, loaders are commonly used for tasks like fetching data from databases or content providers, for example, querying contacts from a device contact list or loading data from a SQLite database. By using loaders, you ensure that app's performance is optimized and the user experience remains smooth, especially when dealing with large datasets. Let's now look at the loader architecture. The architecture of loader is designed to ensure that data can be fetched in the background while keeping the UI responsive. First, let us look into the cursor loader. One of the most commonly used loaders is the cursor loader. It queries data in the background without blocking the main thread. The next one is the content resolver. This component interacts with content provider, which manages access to the data. The activity or fragment is responsible for managing the cursor loader and updating the UI once the data is retrieved. For example, updating a recycler view with fetched data. This architecture ensures that data loading is handled efficiently and the UI remains responsive during background operations. Cursor loaders operate in the background, allowing your app to continue running smoothly while

data is fetched. A cursor loader queries data from the content provider using the content resolver. Once the data is retrieved, the activity or fragment updates the UI with fetched data, such as displaying it in the recycler view or list view. This process ensures that data is retrieved efficiently and users can continue interacting with the app without any lags or delays. Now let's explore the key features of loaders. Persistent across configuration changes. One of the most important features of loaders is their ability to persist data across configuration changes. For example, if the screen is rotated, the loader automatically reconnects to the last result, preventing redundant data queries and ensuring smooth user experience. Lifecycle awareness. Loaders are managed by activity or fragment lifecycle. This means that loaders are automatically started and stopped based on the lifecycle events of activity or fragment. This makes loaders particularly useful for managing long running tasks without having to manually handle lifecycle changes. Let's now go over some best practices when using loaders in Android. Using cursor loader for databases or content providers. When dealing with large data sets from SQLite databases or content providers, cursor loader is highly efficient. It allows data to be fetched in the background without impacting the main thread and loader usage in legacy apps. While loaders are still valuable for legacy Android applications, modern apps often use view model or live data for better lifecycle management. Consider using these newer components if you are developing an app that requires better control over background tasks. By following these best practices, you can ensure that your app handles data loading efficiently, keeping the user interface smooth and responsive. To summarize this session, loaders are essential components in Android that allow apps to load data asynchronously without blocking the main thread. The loader architecture ensures that efficient data loading by querying data in the background using cursor loader and updating the UI using RecyclerView or similar components. Key features of loaders include persistence across configuration changes and automatic lifecycle management, ensuring that data loading is handled smoothly. Follow best practices like using cursor loader for handling large data sets and considering modern components like view model and live data for more complex background tasks in newer apps. By understanding loaders, you can create Android apps that efficiently load data while maintaining a smooth and responsive user experience. Thank you for your attention. I hope you now have a solid understanding of loaders in Android and how to implement them for efficient background data loading. Thank you.

Importance of Security in Applications Video• . Duration: 7 minutes

[Notes]

Hello everyone, welcome to the module on advanced topics in Android development. In this session, we will focus on the importance of security in applications. Mobile applications often handle sensitive information like personal details, financial data, and login credentials. This makes them a prime target for malicious attacks. In this session, we will explore why security is crucial for mobile apps, the risks associated with poor security practices, and why developers must prioritize security. By the end of this session, you will understand the importance of implementing strong security measures in your apps. Let's start by understanding why security matters in mobile applications. Mobile apps often deal with sensitive information including personal details, financial data, and login credentials. If this data falls into wrong hands, it can result in data theft, financial loss, or privacy violations. For example, a banking app that handles user financial transactions must be secure to protect users from hackers trying to steal money or access their accounts. Similarly, apps that store

personal data like contact lists or location information must ensure that this data is not exposed to malicious entities. Because mobile devices are constantly connected to the internet, they are vulnerable to attacks. This makes security a top priority in mobile app development. Let's now discuss some of the common security threats faced by mobile apps. First one is Data Leakage. This happens when sensitive user data is exposed through insecure connections or improper logging. For instance, if an app transmits user data over an unencrypted connection, attackers can intercept and steal it. Next one is Malware. Malware is code or apps that can steal data or even take control of user's device. Malware can infect an app through third-party libraries or insecure downloads. Man-in-the-Middle Attacks. These attacks occur when a hacker intercepts the communication between app and a server. If the communication is not encrypted, the attackers can steal sensitive data like passwords or financial information. Then we have Code Tampering. Attackers can decompile and modify the app's code to exploit vulnerabilities. This allows them to inject malicious code that can cause data theft or other harmful actions. Understanding these threats help us take necessary steps to secure our applications. Now let's discuss the risks of poor security practices. While apps that lack proper security are at high risk of being attacked, leading to significant consequences. Some of the common security threats are listed here. Data Breaches. Poor security can result in data breaches where sensitive information is stolen, leading to loss of user trust and legal penalties for developers. Reputation Damage. Apps with weak security can lose credibility. Users may stop using the app or worst, it could be removed from app stores due to non-compliance with security standards. Then Financial Loss. If financial data is compromised, users can suffer financial loss and developers may face legal action for failing to protect user data. By ensuring robust security measures which are in place, developers can avoid these risks and ensure the safety of user data. Now let's talk about why developers should prioritize security in mobile apps. First one, Increasing Attacks. According to statistics, 82% of Android devices are exposed to OS vulnerabilities. This shows the growing need for strong security measures to protect apps from being exploited. Then we have High Risk Flaws. One in four mobile apps contain high risk security flaws. This highlights how widespread security issues are in mobile development. Developers must focus on secure coding practices to minimize these risks. By prioritizing security, developers can ensure that their apps not only protect users' data but also comply with security standards, building trust with users and maintaining their app's reputation. Let's now discuss some key security measures that developers can implement. First one is the Encryption. Encrypt sensitive data to prevent it from being intercepted by attackers. Data encryption ensures that even if data is stolen, it cannot be read without the proper decryption key. Then Secure Coding Practices. Always follow secure coding practices to avoid common vulnerabilities like SQL injection, buffer overflow attacks. Use secure libraries and frameworks to safeguard your app. Third one is to Regular Update. Keep your app and its dependencies up to date by releasing regular updates. This helps patch security vulnerabilities and ensures the app remains secure over time. By following these security measures, developers can build apps that are more resilient to attacks and offer better protection for their users. To summarize this session, mobile apps often handle sensitive information making them targets for malicious attacks. Security is crucial for protecting users' data and maintaining trust. Some common security threats include data leakage, malware, man-in-the-middle attack and code tampering. The risks of poor security practices include data breaches, reputation damage and financial loss. And developers should prioritize security by using encryption, following secure coding practices and releasing regular updates to patch vulnerabilities. By implementing these measures, developers can ensure that their mobile applications are secure and trustworthy. Thank you for your

attention in this session. I hope now you have a good understanding of the importance of security in applications and the steps required to secure your mobile apps. Thank you.

Introduction to Security Best Practices Video• . Duration: 7 minutes

[Notes]

Hello everyone. Welcome to this module on advanced topics in Android development. In this session, we will focus on introduction to security best practices. Mobile applications face various security threats, including data leakage, malware attacks, and unauthorized access. To protect sensitive user data and ensure the integrity of the app, implementing security best practices from the very beginning is crucial. In this session, we will discuss important security principles and practices that developers should adopt when building Android applications. By the end of this session, you will have a clear understanding of how to apply these practices to secure your apps. Let's first explore the overview of Android security best practices. Android applications are vulnerable to various security threats such as malware, data leakage, and unauthorized access. Implementing best security practices early in the development helps protect user data and maintain the integrity of the app. Some of the key best practices include using authorized APIs to prevent unauthorized access to sensitive information. Implementing regular updates to patch known security vulnerabilities. Conducting penetration testing to identify and fix security weaknesses before they are exploited. By applying code obfuscation to make reverse engineering more difficult. By following these practices, developers can secure their apps from potential threats and ensure better user trust. Next, let's talk about the secure handling of app data. One of the most important aspects of app security is protecting sensitive user data. This can be achieved through several measures. First one is encryption. Encrypting local data storage is essential to protect sensitive data in case the device is lost or stolen. Encrypting files ensures that data remains unreadable without the correct decryption key. Next one is to avoid logging sensitive information. Logging sensitive information such as login credentials is risky. Developers should disable sensitive logs in production builds or ensure that only general information is logged. By taking these measures, developers can ensure that sensitive data remains secure and protected from unauthorized access. Another critical security practice is validating and sanitizing inputs. This ensures that user inputs are properly checked before being processed to avoid common vulnerabilities like SQL injection or cross-site scripting. Input validation helps verify that data entered by users match the expected format, preventing malicious inputs from causing harm. Then input sanitization ensures that potentially dangerous characters are removed or escaped before they are processed by the app. Developers should use certificate pinning to prevent man-in-the-middle attacks. Certificate pinning verifies the app communicates only with a trusted server by checking the server's certificate against a pinned certificate stored in the app. Now let's take a look at how to ensure secure communication and authentication. Secure network communication is to use TLS which is transport layer security to encrypt all network communications protecting data from being intercepted. Always validate SSL certificates and ensure that latest version of TLS is used for secure data communication. Continuing with secure communication and authentication, let's see what we need to do for authentication. Authentication mechanisms. Implement strong authentication mechanisms like biometric authentication which is fingerprint or facial recognition or two-factor authentication to enhance security. These methods add an extra layer of protection ensuring that only authorized users can

access sensitive parts of the app. Developers should also use token-based authentication to verify the authenticity of app requests further securing the communication between the app and the server. Now let's recap the key security best practices which are discussed so far. Use authorized APIs to avoid unauthorized access to sensitive data. Implement regular updates to fix known vulnerabilities and strengthen app security. Conduct penetration testing to detect and resolve potential security weaknesses. Encrypt sensitive data stored locally to protect it from unauthorized access. Validate and sanitize all inputs to prevent vulnerabilities like SQL injection and XSS. Use certificate pinning to protect against man-in-the-middle attacks and secure communication between app and the server. Apply strong authentication mechanisms such as biometrics and two-factor authentication to secure access. By following these best practices, developers can build Android applications that are more secure, resilient to attacks. To summarize this session, mobile applications are vulnerable to various security threats making security a critical concern. Best practices like using authorized apps, regular updates and encryption are essential to protect user data and maintain app integrity. Input validation and sanitation and secure communication through TLS and certificate pinning can prevent common attacks such as data leakage and man-in-the-middle attacks. Implementing biometric authentication and two-factor authentication enhances app security by ensuring only authorized users can access sensitive data. By applying these security best practices, developers can build apps that are both secure and trusted by users. Thank you for your attention in this session. I hope you now have a clear understanding of Android security best practices and how to implement them effectively. Thank you.

Implementing Secure Storage Video• . Duration: 6 minutes

[Notes]

Hello everyone, welcome to the module on advanced topics in Android development. In this session, we will focus on implementing security storage in Android applications. Many mobile applications handle sensitive information such as user credentials, tokens, and personal data. Ensuring this data is securely stored is crucial for protecting user privacy and preventing data breaches. In this session, we will discuss the importance of secure storage, how to use Android Keystore for secure data storage, and the best practices for ensuring your app's data remains safe. By the end of this session, you will have a clear understanding of how to secure data in your Android apps. Let's start by understanding the importance of secure data storage. Mobile applications often store sensitive information including user credentials, tokens, and personal data. However, if this data is not properly secured, it can lead to data breaches, financial loss, or even identity theft. For example, storing sensitive data like passwords or tokens in external storage without encryption can expose the data to other apps or users, posing significant risks. If the device is compromised or lost, unencrypted data can be easily accessed by malicious actors. This highlights the need to use secure storage techniques to ensure that your app is properly protected. Let's look at potential risks of insecure data storage. If data is stored improperly, the following issues can arise. First one is the data leakage. Sensitive data can be exposed to unauthorized apps or users if not encrypted properly. This is a common risk when data is stored in external storage without proper protection. Then we can have identity theft. Storing user credentials without encryption can result in identity theft if attackers gain access to the data. Financial loss. In cases where financial information such as credit card details is not stored securely, it can lead to significant financial loss for users. By

securing data storage, you can mitigate these risks and ensure that user data is kept safe from attackers. Now let's talk about Android Keystore. The Android Keystore is a secure system that allows developers to store cryptographic keys in a protected container, ensuring that sensitive information like encryption keys is kept safe from unauthorized access. Now let's see as to how it works. The Keystore securely generates and stores encryption keys in the hardware of the device, which apps can use to encrypt and decrypt data without exposing the key itself. For example, if you need to store user passwords or authentication tokens, you can use the Keystore to manage the encryption keys, making sure they are isolated and protected. By using the Android Keystore, you can ensure that sensitive data remains encrypted and secure, even if the device is compromised. Let's now discuss some best practices for secure storage in Android. Internal vs External Storage Always store sensitive data in internal storage as it is private to the app and much more secure than external storage. External storage can be accessed by other apps and users, so it should only be used for non-sensitive data. Next one is by using encryption. Always encrypt sensitive data at rest. You can use tools like SQL Cipher for encrypting SQLite databases or Jetpack Security for encrypted shared preferences to ensure that sensitive data is stored securely. Keystore for Cryptographic Keys Use the Android Keystore to generate and securely store cryptographic keys for encryption and decryption. This prevents keys from being exposed and ensures that sensitive operations are handled securely. Following these best practices will help you protect sensitive data in your Android apps and reduce the risk of data breaches. To summarize this session, secure data storage is critical in mobile applications to protect sensitive information like user credentials, tokens, and personal data. Storing data insecurely can result in data leakage, identity theft, and financial loss. The Android Keystore allows developers to store cryptographic keys securely, ensuring that sensitive data is encrypted and protected. Best practices for secure storage include using internal storage for sensitive data, encrypting data at rest, and using the Keystore for cryptographic operations. By following these practices, you can ensure that the data in your Android applications remains safe and secure. Thank you for your attention in this session. I hope you now have a solid understanding of how to implement secure storage in Android apps using the Android Keystore and other encryption techniques. Thank you.

Preparing an App for Publication Video• . Duration: 5 minutes

[Notes]

Hello, everyone. Welcome to the module on publishing and maintenance of Android apps. In this session, we will learn how to prepare an Android app for publication on Google Play Store. We will cover key topics like finalizing the app, optimizing its performance, handling permissions, and maintaining user privacy. By the end of this session, you will have a solid understanding of how to make your app ready for Play Store. Before publishing, it is important to finalize your app properly. One of the main steps is versioning. You need to maintain the app's version numbers carefully. There are two parts to versioning. Version code, which is an integer that increases with every release, ensuring users always get the latest version. Then there is a version name. This is a user-friendly version identifier like 1.0 or 2.1. Also, do not forget to set minSDKVersion and targetSDKVersion in your build configuration. These settings ensure that your app works smoothly across different Android versions. The minSDKVersion is the minimum version of Android that your app can run on. It tells the Android system that lowest version of the OS that supports your app's

functionality. For example, if you set `minSdkVersion` is equal to 21, your app will not work on devices running Android versions below API 21, which is lollipop. Choosing the right `minSdkVersion` ensures that your app works on older Android devices. But setting it too low can also limit the features your app can use. Then we have `targetSdkVersion`. The `targetSdkVersion` indicates the version of Android your app is specifically designed and tested for. It does not prevent your app from running on newer versions of Android. But it ensures that app behaves correctly on the target Android version. For instance, if you set `targetSdkVersion` to 30, your app is optimized to work with features and behavior changes introduced in Android 11, which supports API 30. Setting `targetSdkVersion` regularly helps you take advantage of new platform features and maintain compatibility with recent Android versions. Next, let's talk about optimizing the app before you publish it. Start by removing any unnecessary resources like unused images or code to improve the app's performance and reduce its size. You can use tools like Lint in Android Studio to detect inefficiencies in your code. Additionally, the Android Studio Profiler can help you monitor CPU, memory, and network usage to identify and fix performance issues. Your app should be able to run on a wide range of devices. This includes different screen sizes, resolutions, and Android versions. To ensure this, you need to configure device support in the `AndroidManifest.xml` file. Making sure that your app is compatible with various devices is important to reach a large audience. Now let's talk about permissions and privacy. It is essential to request only the permissions your app needs. Check the `AndroidManifest.xml` file to ensure that you are not asking for unnecessary permissions that could make users uncomfortable or create security risks. Additionally, you must prepare a privacy policy that clearly explains how user data is handled. This privacy policy must be linked to your app's Google Play listing. Make sure to follow Google's data protection guidelines and be transparent with your users about how their data is used. To summarize, here are the key points we covered today. Finalizing your app by managing version numbers and setting the correct SDK versions. Optimize your app by removing unnecessary resources and using tools like Lint and Android Studio Profiler. Ensure your app supports various devices and screen sizes. And carefully manage app permissions and prepare a transparent privacy policy for your users. Thank you for joining this session. I hope you now have a clear idea of how to prepare your app for publication on the Google Play Store. Thank you.

App Signing and ProGuard Video• . Duration: 3 minutes

[Notes]

Hello, everyone. Welcome to the module on publishing and maintenance. In this session, we will discuss how to publish and maintain your Android app. By the end of the session, you will understand the essential steps involved in publishing an app to the Google Play Store and securing it using important tools like App Signing and ProGuard. Now let's discuss about App Signing. So what is App Signing? In simple terms, App Signing is a process that ensures only the original developer of the app can provide updates. This is important to maintain the integrity and authenticity of your app when it's published to the Play Store. To sign your app, you need something called KeyStore. This is a secure file that stores private keys used to sign the app. Without App Signing, you won't be able to publish your app to the Play Store. Let's now see the steps for signing your app in Android Studio. First, open your project in Android Studio. Navigate to the Build menu. Select the option to generate a signed APK. A dialog box will appear where you can generate the signed APK.

These steps are critical before you upload your app to the Google Play Store. Next, let's talk about ProGuard. ProGuard is a tool used to make your app more secure by making the code unclear. Obfuscation makes it difficult for others to reverse engineer your app, which is important to protect your intellectual property. ProGuard also helps optimize your app by removing unused classes and methods. This reduces the size of your APK, making your app lighter and faster. Now, why should you use ProGuard? There are two main benefits. First one is that it provides better protection against reverse engineering by hiding the original code. Then, it also reduces the size of your APK, which improves performance and user experience. These two benefits make ProGuard a crucial tool when preparing your app for release. Now, let's see how to configure ProGuard in your Android project. To enable ProGuard, you need to modify the build.gradle file. Here is an example of how you can set it up for the release version of your app. In this, minifyEnabled should be set to true. This code enables ProGuard and specifies which ProGuard configuration files should be used. To summarize, we have learned about two important aspects of app publishing and maintenance. App signing, which ensures only the original developer can provide updates to the app, and ProGuard, which protects app from reverse engineering and optimizes it by reducing the APK size. Make sure to implement both of these steps before publishing your app on the Google Play Store. Thank you for your attention. I hope this session has given you a clear understanding of how to sign and secure your app before publishing it. Thank you.

Using the Google Play Console Video• . Duration: 4 minutes

[Notes]

Hello everyone, welcome to the module on Publishing and Maintenance. In this session, we will explore Google Play Console, a tool used by developers to manage their Android apps on the Google Play Store. By the end of this session, you will have a good understanding of its key features, how to publish apps and manage app versions. So let's start with what is Google Play Console. Google Play Console is a powerful tool that allows developers to upload, monitor and manage their apps directly on the Google Play Store. This tool helps developers track the performance of their apps, handle user feedback and ensure smooth app distribution to users around the world. In simple terms, it is a control center for Android developers managing apps on the Play Store. One of the most important tasks for developers is uploading and releasing new versions of their apps. The Play Console simplifies this by allowing you to easily publish new app versions. It provides detailed analytics on how many downloads your app gets, how it performs in terms of crashes and user engagement statistics. With these features, developers can make better decisions about their app updates. Google Play Console lets developers track the performance of their apps as well. You can monitor key metrics like downloads, installs, crashes and user engagement. If the app is crashing frequently or users are not interacting with certain features, these insights can help you make improvements and deliver better user experience. Regular monitoring is essential to ensure your app performs well and keeps your users satisfied. Google Play Console also offers tools to read and respond to users' reviews. Listening to user feedback is crucial to improve your app and address any key issues that users may face. You can engage directly with your users, showing that you value their opinions. In addition, the Play Console provides financial reports allowing you to track app earnings, downloads and other important sales metrics. Managing multiple apps can be difficult, but Play Console makes it easier by offering a

single dashboard for all your apps. You can view important health indicators like crash reports and ANR. This feature helps in maintaining the overall health of your apps and ensures that you can manage multiple projects seamlessly. When it comes to publishing, developers can upload APKs or AABs. APK stands for Android Package Kit and AAB stands for Android App Bundles. The Google Play Console also allows you to manage both beta and production releases. Beta releases are for testing among a small group of users and production releases are for general public. Version control is essential for maintaining your app's update cycle. Version control is critical for any app development process. Google Play Console helps you manage app versions effectively by letting you increment the version code and version name with every release. You must ensure that the uploaded version matches the version mentioned in your build.gradle file. This process keeps your releases organized and ensures smooth updates for users. Let's quickly summarize what we have learned. The Google Play Console is a tool to manage, monitor and publish Android apps on the Play Store. It offers key features like app publishing, performance monitoring, user feedback management and financial reporting. App management is simplified with a single dashboard, while version control helps in ensuring seamless updates for users. Thank you for your attention and I hope now you have a clear understanding of Google Play Console. Thank you.

Publishing Process Video• . Duration: 4 minutes

[Notes]

Hello everyone. Welcome to the module on publishing and maintenance. In this session, we'll be discussing through the process of publishing an app to the Google Play Store. By the end of this session, you will understand the steps involved in publishing an app, how to configure the store listing, the process of submitting the app for review. So let's begin by looking at the key steps in this process. When publishing an app to the Google Play Store, the first step is to upload the app file. There are two main formats for this, APK, which is Android Package Kit, and AAB, which is Android App Bundle. APK is typically used for individual releases. AAB allows Google to optimize the resources for different devices. Both formats are essential, but AAB is now preferred format because it optimizes how users download your app based on their device's configuration. Once the file is uploaded, the next step is to fill out app's information. You need to provide the app title, a detailed description, and images of the app. The short description should be concise and highlight the key features of your app helping to attract users. Next, you will need to set the pricing for your app. Choose whether the app will be free or paid, then select the regions or countries where the app will be made available. By carefully managing this information, you ensure your app reaches the right audience. Now let's discuss about configuring the store listing details. This is where you will need to upload a high resolution app icon, which is 512 into 512 pixels. Screenshots of your app that give users a preview of how it works. After this, choose the right category for your app. Example, games, education, or tools. Choosing the correct category helps improve your app's visibility in the Play Store. If your app collects any personal data, it is mandatory to upload a privacy policy. The privacy policy ensures that users know how their data will be handled and how, and it also helps build trust with your users. This is a very important step for compliance, so don't forget to include if it is necessary. After filling out all the required information and uploading the necessary files, it is time to submit the app for review. Google will review your app to ensure it complies with their policies and guidelines. This review process checks for any violations of Google Play Store policies. Once

approved, your app is ready for the next step, whether it is testing or full public release. Before releasing your app to everyone, you can choose how you want to roll it out. There are two options. First one is beta testing. This allows you to release the app to a limited audience for testing. This helps you gather feedback and make necessary improvements before the full launch. Second option is to production release. Once your app passes testing and you're confident with its performance, you can make it available to the general public. This controlled rollout approach ensures that your app is stable and ready for wider audience. To summarize, the key steps involved in publishing your app on the Google Play Store is to first upload the APK or AAB file, fill out all the information about the app, configure store listing, submitting it for review, and rolling it out to the users. Either it could be beta users or general public. With these steps, you'll be able to successfully publish your app on the Google Play Store. Thank you for your attention. I hope this session has given you a clear understanding of how to publish an app to the Google Play Store. Thank you.

Maintaining and Updating an App Video• . Duration: 3 minutes

[Notes]

Hello, everyone. Welcome to the module on publishing and maintenance. In this session, we will focus on how to effectively publish apps to Google Play Store and maintain them through regular updates. By the end of this session, you will understand the importance of regular updates, versioning, and strategies for rolling out updates. Updating an app regularly is essential for a great user experience. Regular updates help fix bugs, improve performance, and add new features. Additionally, they ensure that your app stays compatible with new devices and operating system updates. Keeping your app functional and relevant will enhance user satisfaction. Security is a critical aspect of app development. Updates help patch security vulnerabilities, protect user data, and prevent unauthorized access. Furthermore, they ensure that your app remains compliant with latest industry regulations and app store policies. Consistent updates show users that your app is well-maintained and responsive to their needs. Bug fixes and feature improvements keep users engaged and reduce the chances of users abandoning your app. Regular updates also optimize performance, addressing issues like slow loading, crashes, and battery drain. Every app update needs proper version management. This is done using version code and version name in your build.gradle file. The version code is an integer that must increase with every update. The version name, which is usually string like 1.0 or 1.1, reflects the app's release version. Effective version control helps you manage your app updates easily. Google Play Console is your tool for managing app versions and updates. It allows you to roll out updates through staged rollouts or beta testing. This ensures the stability of your update by testing it with a smaller group before releasing it to everyone. Now let's see as app is rolled out. With staged rollouts, you release your update to small percentage of users first. This allows you to monitor the performance and identify any issues before the full release. It is a great way to minimize the risk of bugs affecting all users at once. Beta testing involves testing new features and bug fixes with a select group of users before making the update public. Beta users provide valuable feedback and this process helps developers address potential issues early on, making sure broader audience does not face any problems. In summary, regular updates are essential for maintaining user satisfaction, improving security, and keeping your app relevant. Version control and use of Google Play Console allow you to manage updates smoothly. Strategies like staged rollouts and beta testing are effective in ensuring that updates are stable and

user-friendly. Thank you for your attention this session on publishing and maintaining mobile apps and giving regular updates on the apps. Thank you.

Handling User Feedback and Bug Reports Video• . Duration: 4 minutes

[Notes]

Hello everyone, welcome to the module on publishing and maintenance. In this session, we are going to learn about publishing mobile apps and handling user feedback and bug reports after publishing to the Google Play Store. These are essential steps in maintaining a successful app and ensuring user satisfaction. Why is user feedback so important? Feedback provides valuable insights into how well your app is performing and how users are experiencing it. By understanding user feedback, you can identify issues such as bugs or missing features that users want, helping you improve the app. Some of the key benefits include improved user satisfaction by addressing problems users face, better user retention by showing that you are continuously working to improve your app. When managing reviews in the Google Play Console, it is important to respond to users regularly. This shows that their input is valued and that you are committed to making the app better. You should also prioritize feedback. Look for recurring themes in reviews like common bugs or popular feature requests. Focus on issues that affect many users first and plan your updates accordingly. Let's now look into bug reports and crashes. Bug reports and crashes are critical issues that need attention. You can use the Google Play Console to monitor crash reports and application not responding events. These logs give detailed information about which devices and operating systems are more affected. When it comes to fixing bugs, start with critical ones, especially those causing frequent crashes. Once the issues are fixed, use staged releases to ensure the update is stable before rolling it out to everyone. Now let's look into some other tools for bug fixing. First, let's look into Firebase Crashlytics. Firebase Crashlytics is an excellent tool for bug fixing. It offers real-time crash reporting, showing the exact point of failure with breadcrumbs that trace user actions leading up to the crash. This tool also integrates with Firebase Analytics to give a complete picture of your app's performance. Bug Finder is another useful tool, especially for remote logging. It logs app events even when the user is offline and helps developers identify bugs without the need for users to manually report them. This tool is very useful in detecting bugs that may go unnoticed or when users don't report the problem themselves. Sentry is another popular error monitoring tool that captures runtime errors, allowing you to fix issues before they affect users. It integrates with issue tracking systems like Jira and GitHub, making it easier to manage errors and monitor app performance. Sentry also supports performance monitoring and provides real-time alerts to help you stay on top of any problems. Another tool for bug fixing is Instabug. Instabug is a comprehensive tool that offers in-app bug reporting, crash reporting, and user feedback collection. It allows users to submit bugs with rich media like screenshots or recordings, giving developers detailed insights into the issues. It also captures crash reports with track traces and logs, which helps in identifying and fixing problems faster. In summary, we have discussed the importance of user feedback in app development and how to manage it effectively through the Google Play Console. We have also covered how to track and fix bugs using tools like Firebase Crashlytics, Bug Finder, Sentry, and Instabug. By using these tools and processes, you can maintain your app's stability, improve user satisfaction, and ensure that your app evolves in line with user expectations. Thank you.

Introduction to Jetpacks Video• . Duration: 4 minutes

[Notes]

Hello, everyone. Welcome to this session on Android app development using Jetpack. In this session, we will dive into what Jetpack is, its key benefits, and how it contributes to modern Android development practices. By the end of this session, you will have a clear understanding of Jetpack's role and its impact on the app development ecosystem. So what is Jetpack? Jetpack is a collection of Android libraries provided by Google. These libraries are designed to make app development faster, easier, and more efficient. By using Jetpack, developers can focus more on building app features rather than worrying about common and repetitive coding tasks. Jetpack also ensures better performance and compatibility across various versions of Android, which is crucial for creating high-quality apps. Now let's look into some of the key benefits of Jetpack. Jetpack brings several key benefits to developers. First one is backward compatibility. It helps ensure that your app works seamlessly on both older and newer versions of Android, saving you from potential compatibility issues. It reduces boilerplate code. Jetpack simplifies common tasks, reducing the amount of repetitive code you have to write. This allows you to focus more on developing features. It also supports modular development. The libraries within Jetpack are modular, meaning you can choose and integrate only the components you need. This flexibility is a significant advantage for app development. Jetpack plays a crucial role in modern Android development by supporting various practices that make development smoother and more efficient. First, it supports Kotlin language. Jetpack is fully compatible with Kotlin, which has become the preferred language for Android development. Kotlin's concise syntax, along with Jetpack's tools, helps streamline the development process. Next one is lifecycle awareness. Jetpack offers lifecycle-aware components, which make managing UI and data synchronization easier, reducing bugs and crashes. MVVM architecture support. Jetpack encourages the use of model-view-viewmodel architecture by providing libraries like ViewModel and LiveData, which help in structuring code in a way that separates business logic from the UI. Jetpack has a substantial impact on Android development process. It brings in stability and reliability. Since Jetpack libraries are maintained by Google, developers can trust the tools they use are stable and reliable. It also provides faster development cycle. With Jetpack, you no longer need to reinvent the wheel for common development tasks, allowing you to build robust apps more quickly. It also has community adoption. Jetpack is widely adopted in Android community, meaning there is plenty of support, resources, and examples available online, making it easier for developers to solve problems and improve their skills. To summarize, Jetpack is a powerful set of libraries that simplifies Android app development. It offers key benefits such as backward compatibility, reduced boilerplate code, and modularity. Jetpack plays an important role in modern development practices, supporting Kotlin and lifecycle-aware components and encouraging the use of MVVM architecture. Jetpack is essential for building stable, reliable, and modern Android applications. Thank you for participating in this session on Jetpack. We hope now you have a good understanding of Android development using Jetpack. Thank you.

Components in Jetpack Compose Video• . Duration: 3 minutes

[Notes]

Welcome to this session on Android development using Jetpack, where we will explore Jetpack Compose components. In this session, we'll focus on various UI components in Jetpack Compose,

how they work and how they make Android development easier and more efficient. Jetpack Compose components are built using declarative programming approach. This means that instead of describing how the UI should update step-by-step, you describe how UI should look based on the current state. The building blocks of Jetpack Compose are called composable functions. These functions allow you to easily create user interfaces by simply describing their behavior. Let's take a look at some of the key components used in Jetpack Compose. Text, which displays a text on the screen. Button, which is a clickable button element. Image, used for showing images in your app. Modifier, a tool to apply layout styles and interaction behavior like padding or alignment to composable elements. These components are flexible, allowing developers to create dynamic and interactive UIs with minimal effort. Here we see the use of key components in Jetpack Compose with reactive state management. It shows how a button can update text dynamically, how the row layout places text elements horizontally and how modifier handles layout and padding. An example of code demonstrates these elements in action. Now let's see how these components work together in practice. In Jetpack Compose, we manage the state using `rememberMutableStateOf` method. For example, when a button is clicked, we can track how many times it's been clicked using this method. When the state changes, Jetpack Compose automatically re-renders the relevant UI components to reflect the new state. This is known as automatic recomposition, making the user interface responsive and dynamic. In Jetpack Compose, layouts like `row` and `column` help us organize UI elements. `Row` aligns items horizontally, `column` aligns items vertically. Using modifier, we can adjust the layout of elements by adding padding, spacing, and alignment. This gives developers more control over positioning and appearance of their UI components. Here is an example of how you can use `row` with text elements. In this example, the `spacer` with the modifier is used to add space between text elements, making the UI cleaner and more organized. To summarize, Jetpack Compose allows us to build user interfaces using a declarative approach. Key components like text, button, image, and modifier make UI creation straightforward and efficient. Reactive state management helps in creating dynamic and responsive UI. Layouts like `row` and `column`, along with modifier, offer flexible, organized design options for developers. Thank you for joining this session on Jetpack Compose components. Thank you.

User Interaction Using Jetpack Compose Video• . Duration: 3 minutes

[Notes]

Hello everyone, welcome to this session on user interaction in Jetpack Compose. In this session, we will discuss how Jetpack Compose handles user interactions such as clicks, text input, and gestures, all in a declarative way. This session will help you understand how to make your app interactive using different components provided by Compose. Jetpack Compose makes it easy to manage user interaction through its declarative approach. Components like button, text field, and slider let users interact with the app effortlessly. Compose uses state management to automatically update the UI when users provide input. This makes building interactive apps simpler and more intuitive. One of the basic ways to interact with an app is by clicking a button. In Compose, the button component handles click events with `onClick` lambda, which defines what happens when the button is clicked. Here is an example. This simple code demonstrates how a button can respond to user clicks and trigger actions in your app. Now, let's explore how Compose handles text input, sliders, and gestures. For text input, we use text field, which allows users to type text. Here is an

example of it. Similarly, sliders can be used for continuous input like adjusting volume or brightness. The snapshot of it is shown. Gestures like taps, long presses, and swipes are also supported in Jetpack Compose. Using the modifier.pointerInput function, you can detect gestures like double taps. The code snippet shows the same. This flexibility allows you to create interactive apps that respond to different types of user input. State management plays a key role in making user interaction possible. By using rememberMutableStateOf function, Compose keeps track of user input and automatically updates the UI when the state changes. This reactive state management ensures that the app responds quickly to user actions, providing a smooth and dynamic user experience. To summarize, Jetpack Compose provides a declarative and easy way to handle user interaction. Gestures like button, text field, and slider make it simple to manage input from users. State management ensures that UI updates dynamically as users interact with the app. Gestures like taps, long press, and swipes are all well supported, allowing for more complex interactions. Thank you for participating in this session on user interaction in Jetpack Compose. We hope this session has given a good understanding of how to handle user inputs and create interactive Android applications using Jetpack Compose. Thank you.

Navigation in Jetpack Compose Video• . Duration: 3 minutes

[Notes]

Hello everyone, welcome to this session on navigation in Jetpack Compose. In this session, we will explore how navigation is handled declaratively in Jetpack Compose, reducing the complexity involved in managing the backstack and screen transitions. Jetpack Compose provides a declarative approach to navigation. This method simplifies the process of managing screen transitions and the backstack, which can be challenging in traditional Android development. Using NavController and NavHost, developers can easily create and manage navigation graphs that handle the transitions between different Compose screens. Before we dive deeper into Jetpack Compose navigation, let's take a look at issues with traditional navigation methods. First one is complex lifecycle management. Handling lifecycle events manually, especially with fragments and activities, can be tricky and error-prone. Then we have boilerplate code. Traditional navigation often requires more lines of code to manage fragment transactions and backstack. If not managed properly, it could lead to crashes. Manual backstack management. Developers need to explicitly handle operations like add to backstack and pop backstack, making navigation management cumbersome, and it has got limited flexibility. Handling advanced navigation, like clearing the backstack with pop up to, is harder without Compose. In Jetpack Compose, every screen or destination is added to backstack as you navigate through the app. As you move between composables, the backstack keeps track of screens, allowing users to move back and forth efficiently. For example, when you navigate from a login screen to the home screen, the home screen is added to the backstack while the login screen remains underneath. The pop backstack function is used to remove the current screen from backstack and return to the previous screen. This is typically used for simple back navigation, such as going back from home screen to login screen. For example, calling pop backstack removes the home screen and the user is returned to login screen. Jetpack Compose also allows you to clear parts of backstack using pop up to function. The inclusive flag in pop up to determines whether the specified destination should be removed from backstack. If inclusive is equal to true, the destination is cleared. If inclusive is false, the destination

remains in the backstack. This feature is useful when you want to clear multiple screens from the backstack while retaining a specific one. To summarize, Jetpack Compose simplifies backstack handling with pop backstack and pop up to. The declarative approach reduces boilerplate code and improves lifecycle management. NavController and NavHost work seamlessly with Jetpack Compose, providing a flexible and efficient way to manage navigation between screens. Thank you for your attention in this session. We hope now you have a clear understanding of how navigation works in Jetpack Compose. Thank you.

List in Jetpack Compose Video• . Duration: 3 minutes

[Notes]

Hello everyone, welcome to the session on using lists in Jetpack Compose. In this session, we will explore why lists are essential in Jetpack Compose, and how to efficiently render large sets of data using lazy column, a powerful tool for managing lists. Lists are a common and essential UI component, especially for displaying large sets of data such as contact lists, newsfeeds, or item catalogs. Using Jetpack's Compose lazy column, we can render long lists without impacting app performance. Lazy column only composes the visible items, making it ideal for scenarios where efficiency and speed are crucial. Traditional Android list implementations like RecyclerView require custom adapters, view holders, and XML layouts. In contrast, lazy column in Jetpack Compose is much simpler to set up. You can easily define a scrollable list with minimal code and without the need for custom adapters or XML files. Let's see how lazy column works. We have developed a scrollable list using lazy column to efficiently display large sets of data. Each item in the list is represented by a custom layout containing an image and text. This setup is perfect for scenarios like showing a list of products, contacts, or news articles. One of the key advantages of Jetpack Compose is the ability to create modern and declarative UIs. With lazy column, you can create clickable list items, allowing for future extensions like navigating to detailed views or adding interactive elements. This ensures both flexibility and high performance, particularly for apps that handle dynamic content and data. Here is a screenshot showing the output of a list rendered using lazy column. The code used to create the list efficiently renders the visible items while keeping performance high. Each item in the list is represented by a custom composable function called list item, which displays an image and text. The list item function renders each item in the list with an image and text. And here is the code for it. This layout provides a clean and efficient way to display a list of items with images and text. To summarize, lists are an important UI component for displaying repeated data efficiently. Jetpack Compose simplifies list creation using lazy column, which ensures smooth performance by only composing visible items. The use of custom composables like list item allows for flexible and interactive UIs. Thank you for participating in this session on lists in Jetpack Compose. We hope you have a good understanding of how lists are implemented and how to efficiently render large datasets in Jetpack Compose. Thank you.

Other Frameworks Used in Android Video• . Duration: 7 minutes

[Notes]

Hello everyone, welcome to this session on Android app development. In this session, we will be discussing the various frameworks used in Android development. Let's outline our key learning

objectives for this session. First we will go through the introduction to cross-platform development frameworks. Then we will have a look into the networking frameworks which are used in Android. And we shall also see as to how dependency injection frameworks are used in Android. And there are many database management frameworks which are used to handle data. Along with that, there is image loading and cache frameworks for managing media. And there are testing frameworks to ensure your app works as expected. Each of these frameworks serve a different purpose in Android development ecosystem and is key to building modern mobile applications. By the end of this session, you will have a good understanding of different frameworks, their use cases, and how they can be implemented in Android development. Here we introduce certain key frameworks used in Android development. It covers frameworks like Retrofit for networking, Dagger for dependency injection, Glide for image loading, Room for database management, along with few others. These frameworks address common challenges developers face, helping to build more efficient and robust Android applications. By leveraging these tools, developers can create apps that are better optimized for performance and provide smoother user experience. Let's start with cross-platform development frameworks. These frameworks allow developers to create mobile apps that work across multiple platforms like Android and iOS using a single codebase. Two popular frameworks used for cross-platform application development are Kotlin Multiplatform. This is growing in popularity because it allows you to write code that can run on Android, iOS, and even web applications. Several popular apps are already adopting Kotlin Multiplatform for its flexibility. Another widely used cross-platform framework is Flutter. It uses the Dart programming language and allows fast, high-performance app development with a single codebase. Many big brands are using Flutter to deliver cross-platform experiences. Next, we have networking frameworks, which are essential for handling API requests and data transfers in Android apps. Two widely used frameworks are Retrofit. This is a high-level networking library that simplifies the process of making API requests. It handles REST APIs, converts data, and serializes responses, making it a go-to app for developers working with network requests. Apps like Uber and Netflix use Retrofit to manage their data communication with servers. Then we have OKHTTP. Often used with Retrofit, OKHTTP provides lower-level HTTP handling capabilities. It's great for managing connections, pooling, and caching. Popular apps like Pinterest and Slack use OKHTTP for real-time data updates and image handling. Now, let's see the frameworks related to Dependency Injection. Dependency Injection is a design pattern that makes your code more modular and easier to test by providing dependencies rather than hard-coding them. Two important frameworks used for Dependency Injection in Android are Dagger. Dagger is a fully static, compile-time framework that ensures efficiency and correctness during Dependency Injection. It's widely used in Android apps where performance is critical. Then we have Hilt. Hilt is built on top of Dagger. Hilt simplifies the use of Dagger in Android apps, reducing the amount of boilerplate code. It is designed specifically for Android and has quickly become a preferred Dependency Injection framework for modern Android apps. Handling persistent data is crucial in app development and Android provides several data management frameworks to simplify this task. The two most common ones are Room. Room is a part of the Android Jetpack library. Room provides an abstraction layer over SQLite, making it easier to work with databases by offering features like compile-time query verification. Then there is Realm. An alternative to SQLite, Realm is designed for high-performance mobile apps, especially those that need to operate offline. It simplifies data handling and synchronization of complex mobile applications. Modern apps often need to handle large amounts of media and efficient image loading is a must. Some of the most commonly used image loading and caching frameworks are Glide and

Picasso. Glide is a powerful image loading library that helps you load, display, and cache images efficiently. Glide is often recommended for Android apps because it handles image requests and caching with minimal effort on the developer's side. Many modern Android apps use it for managing media content. Then Picasso is another popular image loading library that is used in Android apps. This helps by simplifying the image handling in Android. Testing is a critical part of any Android development process, and Android provides excellent tools for unit testing and UI testing. Espresso. This is a UI testing framework that helps ensure that the user interface behaves as expected. It automates interaction with UI components, making it easier to catch issues during development. Then we have JUnit. Used for unit testing, JUnit allows developers to test individual units of logic, ensuring the correctness of apps' core functions. Both of these testing frameworks help create more robust and error-free Android applications. In summary, we have covered the essential frameworks used in Android development, including cross-platform tools, networking libraries, dependency injection frameworks, and more. Each of these play a crucial role in making your app more efficient, scalable, and maintainable. Using these frameworks is key to becoming a successful Android developer, allowing you to create modern apps that offer a seamless user experience. Thank you for your attention throughout this session.